

全栈自动化框架零基础到实战

版权声明：

本人上市公司测试开发在职，本课程依据多年工作笔记的细化总结，作为个人副业非机构性质

课程的课件源码，资料由
测开星辰（QQ1934433596）拥有知识产权的权利

只限于学习者在本课程使用，不得应用于商业用途

任何其他人或机构不得盗版、仿造、售卖

注：课程提供配套指导服务（QQ1934433596），框架源码持续更新服务

购课链接：<https://student-api.iyincaishijiao.com/t/6PGFYyW/>

第一章 概述

1.1什么是自动化测试

一、了解自动化测试

自动化测试，是把人对关键的测试行为，转换为由机器执行测试行为的一种实践。

二、自动化测试有哪些优势？

- 1.完成重复性的工作
- 2.提升回归测试的效率
- 3.利用非工作时间执行测试
- 4.长时间进行稳定性和压力测试
- 5.保证每次验证的一致性

三、自动化测试有哪些劣势？

- 1.不能完全取代手工测试
- 2.比手工测试更加脆弱
- 3.脚本执行次数必要大于等于5次
- 4.具备编程能力

四、什么样的项目适合自动化测试？

- 1.需求文档，不会频繁变更
- 2.研发和维护周期长，需要频繁的执行回归测试
- 3.需要多平台运行相同的场景的测试步骤
- 4.手工测试无法实现的或者成本高的
- 5.被测软件的开发规范，具有可测试性
- 6.测试人员已经具备编程能力

第二章 Python基础

2.1 【环境配置】Python的安装及配置

一、学习目标

1. Python 的安装 (3.6.8)
2. Python 的环境变量配置

二、知识点

1. 【Python的安装】

步骤一：python卸载



步骤二：python安装

- (1) 打开python官网: <https://www.python.org/>

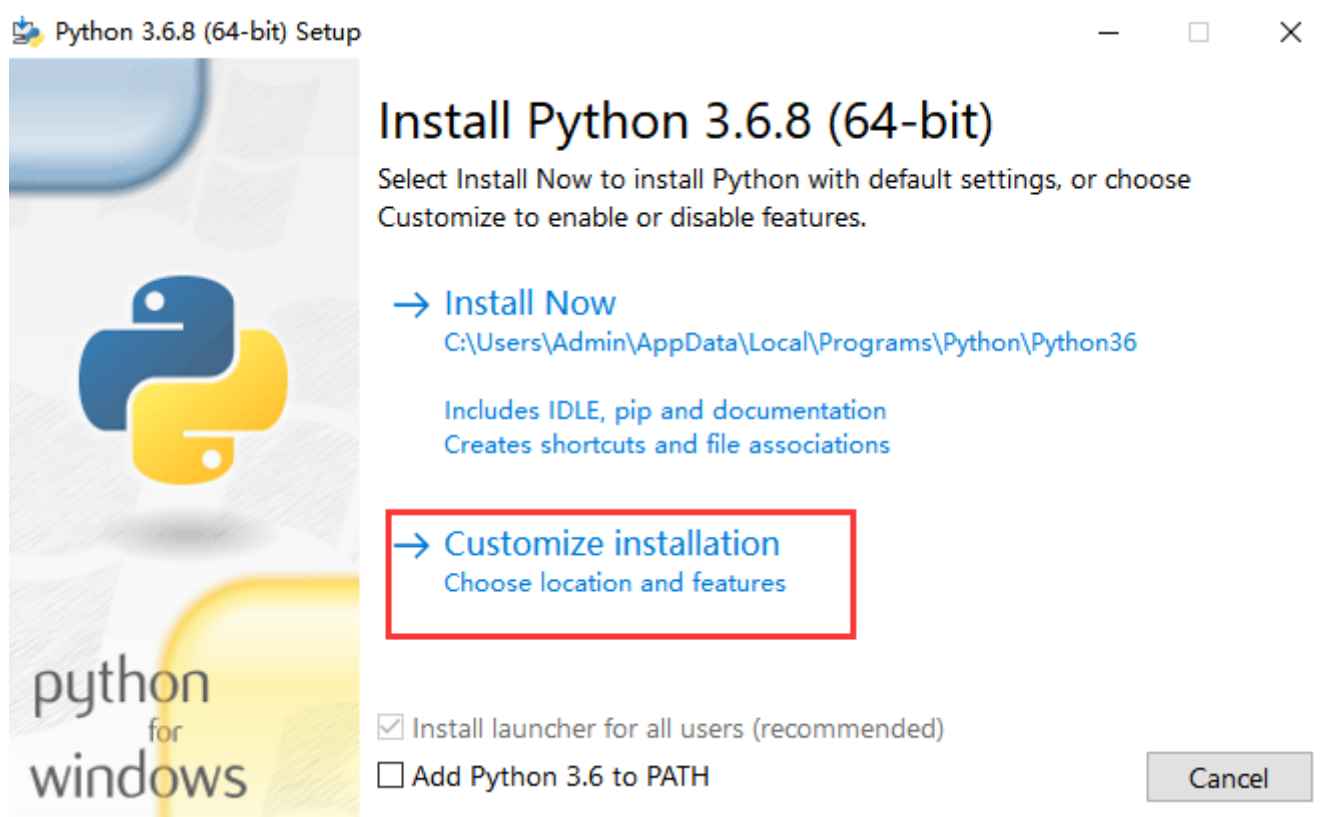
- [Python 3.6.8](#) - Dec. 24, 2018

Note that Python 3.6.8 cannot be used on Windows XP or earlier.

- Download [Windows help file](#)
- Download [Windows x86-64 embeddable zip file](#)
- Download [Windows x86-64 executable installer](#)
- Download [Windows x86-64 web-based installer](#)
- Download [Windows x86 embeddable zip file](#)
- Download [Windows x86 executable installer](#)
- Download [Windows x86 web-based installer](#)

(2)

(3) 安装python



Python 3.6.8 (64-bit) Setup



Advanced Options

- ☒ Install for all users
- ☒ Associate files with Python (requires the py launcher)
- ☒ Create shortcuts for installed applications
- ☐ Add Python to environment variables
- ☒ Precompile standard library
- ☐ Download debugging symbols
- ☐ Download debug binaries (requires VS 2015 or later)

Customize install location

D:\Python36

Browse

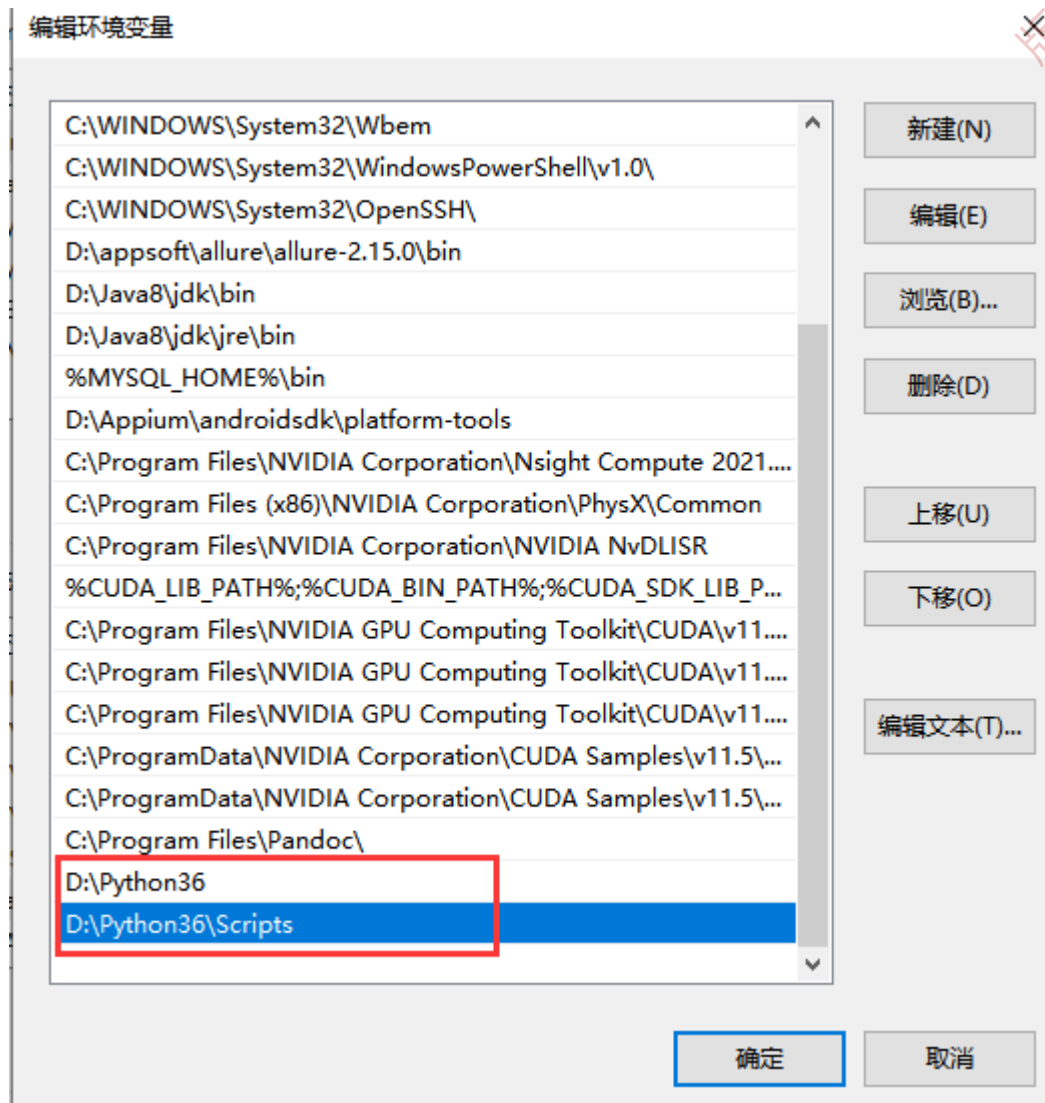
Back

Install

Cancel

2. 【Python的环境变量配置】

(1) 配置环境变量



(2) 验证python是否安装成功

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.19043.1466]
(c) Microsoft Corporation。保留所有权利。

C:\Users\Admin>python
Python 3.6.8 (tags/v3.6.8:3c6b436a57, Dec 24 2018, 00:16:47) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> exit()

C:\Users\Admin>
C:\Users\Admin>
C:\Users\Admin>
C:\Users\Admin>pip list
Package      Version
-----
pip          18.1
setuptools   40.6.2
You are using pip version 18.1, however version 21.3.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

C:\Users\Admin>

C:\Users\Admin>python -m pip install --upgrade pip
Collecting pip

```

2.2 【环境配置】Pycharm的安装及配置

一、学习目标

1. Pycharm 的安装
2. Pycharm 的开发环境配置
3. 使用 Pycharm 新建/打开项目

二、知识点

1. 【Pycharm的安装】

步骤一：下载安装包

<https://www.jetbrains.com/pycharm/download/#section=windows>

Download PyCharm

Windows

macOS

Linux

Professional

For both Scientific and Web Python development. With HTML, JS, and SQL support.

Download

Free trial

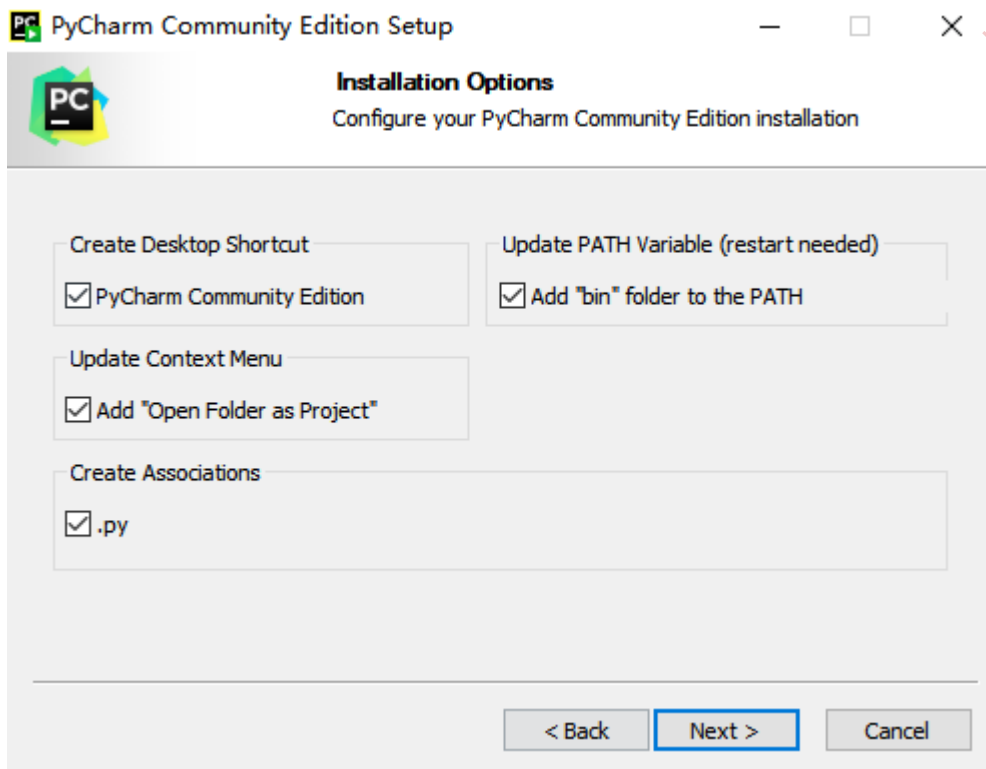
Community

For pure Python development

Download

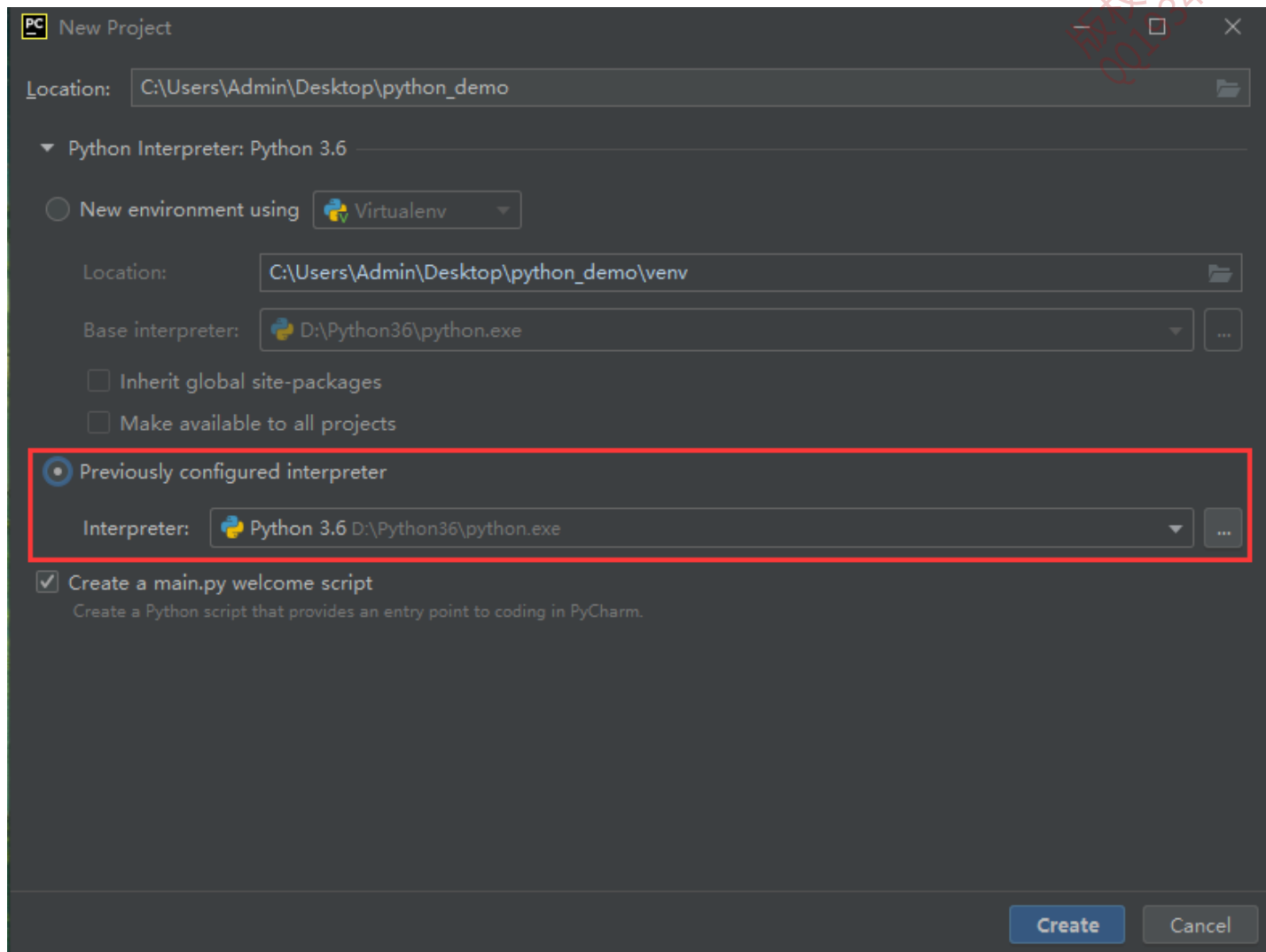
Free, built on open-source

步骤二：pycharm的安装

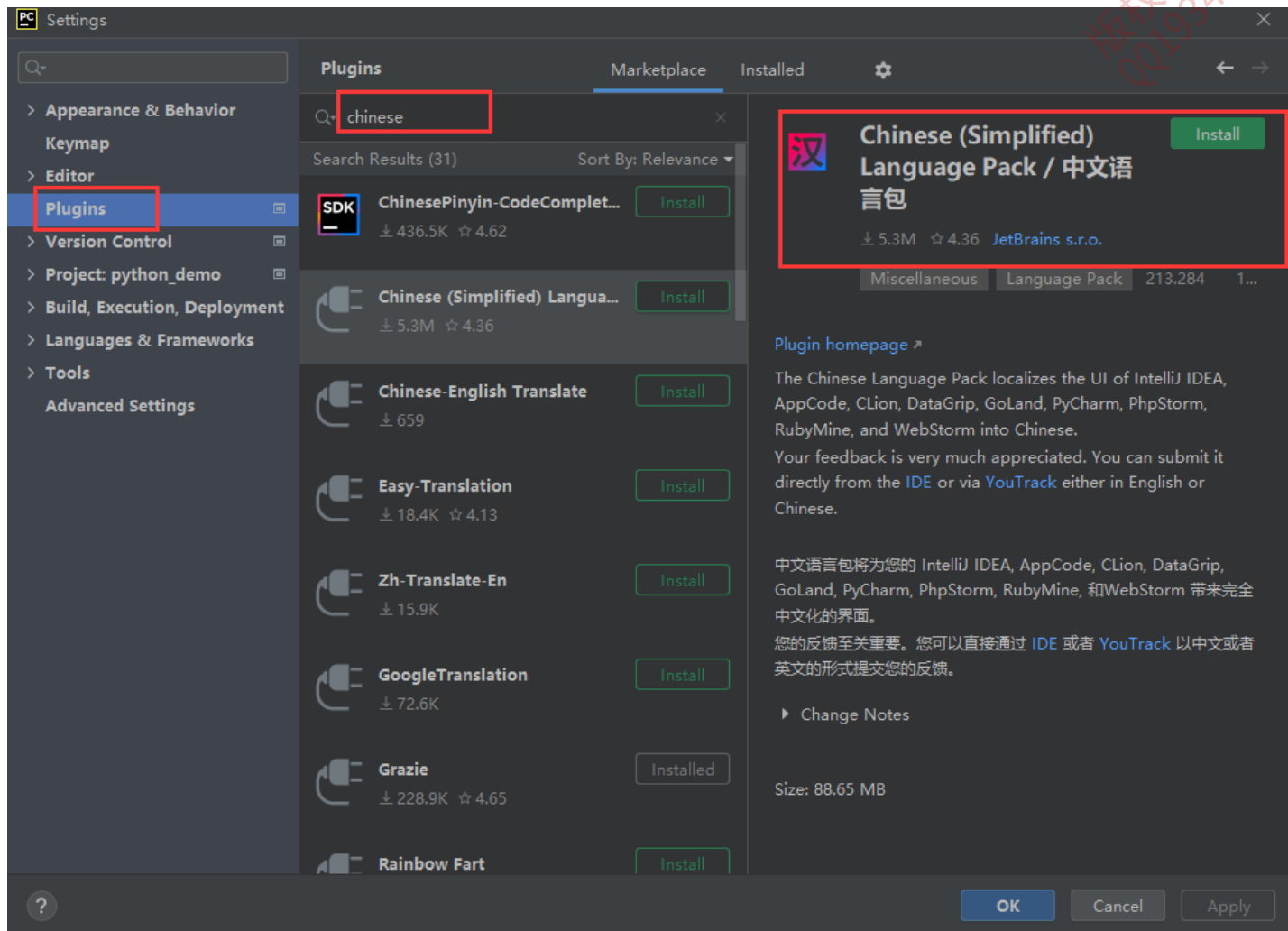


2. 【Pycharm的开发环境配置】

步骤一：选择本地的python环境

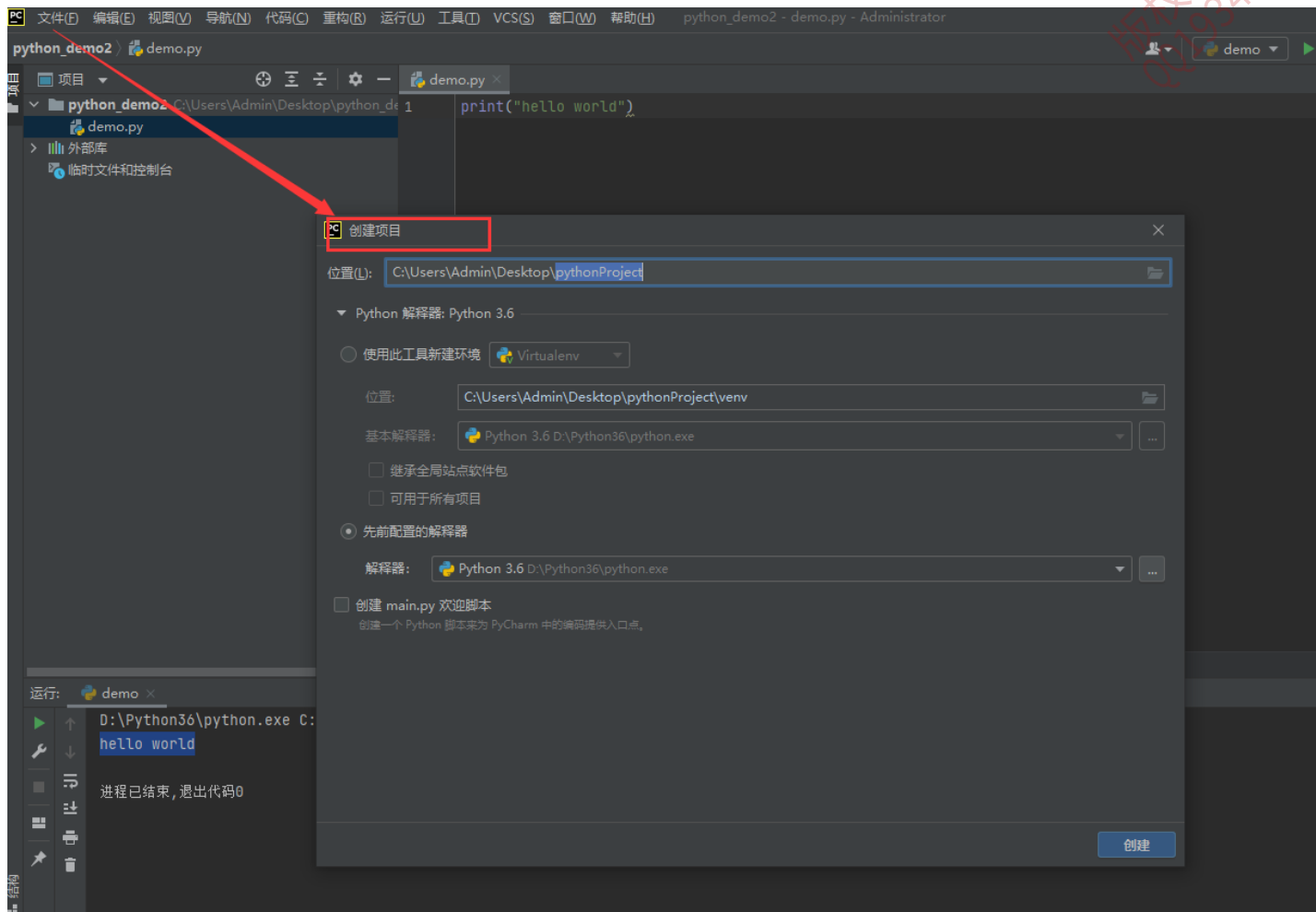


步骤二：中文插件安装

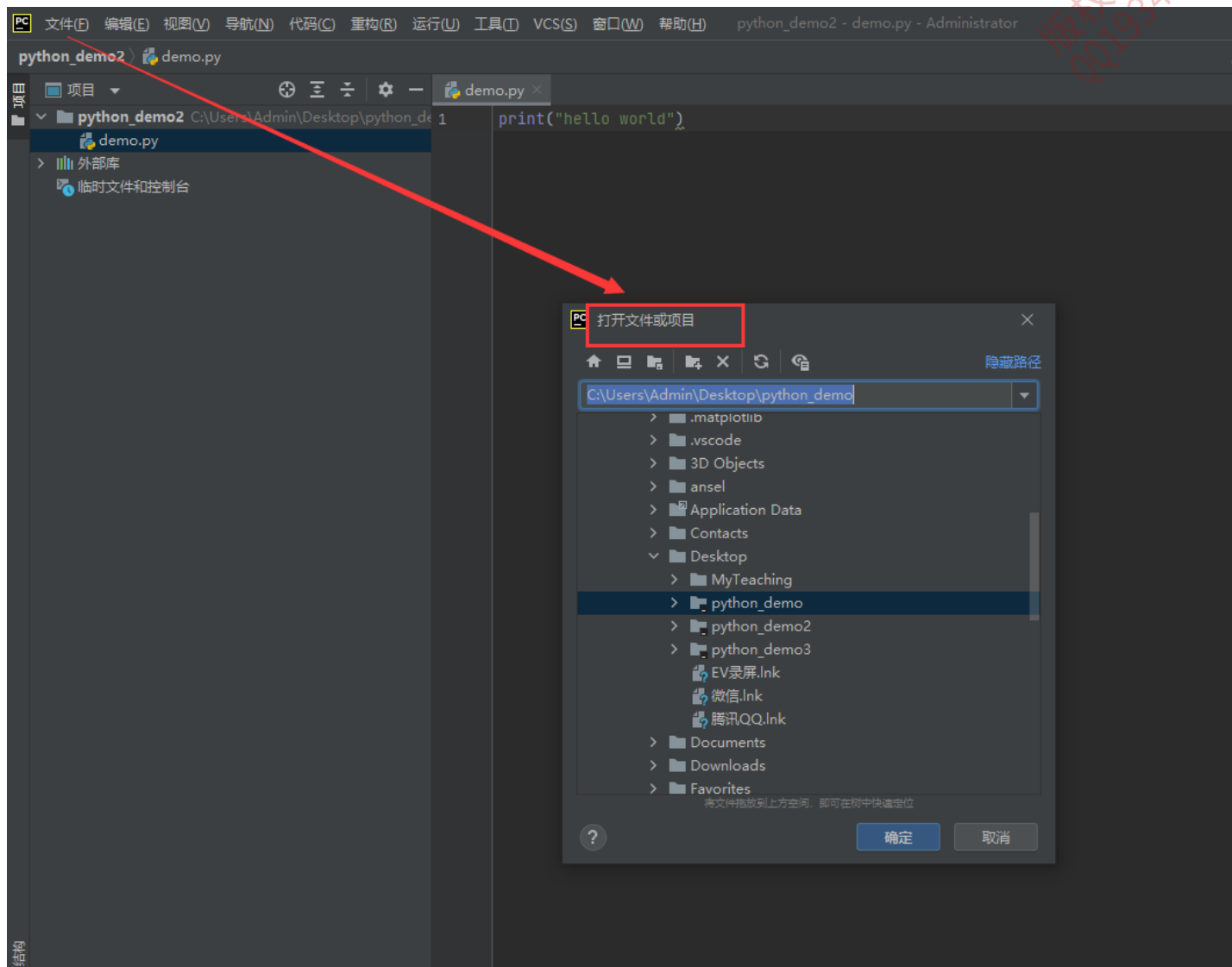


3. 【使用Pycharm新建/打开项目】

步骤一：新建一个项目



步骤二：打开已有的项目



2.3 【数据类型】Python数据类型概述

一、学习目标

- 1.python数据类型介绍
- 2.python数据类型的定义

二、知识点

1.【Python数据类型的介绍】

我们人类可以很容易的分清数字与字符的区别，但是计算机并不能呀，计算机虽然很强大，但从某种角度上看又很傻，除非你明确的告诉它，1是数字，“汉”是文字，否则它是分不清1和‘汉’的区别的，因此，在每个编程语言里都会有一个叫数据类型的东东，其实就是对常用的各种数据类型进行了明确的划分，你想让计算机进行数值运算，你就传数字给它，你想让他处理文字，就传字符串类型给他。

Number（数值）、String（字符串）、List（列表）、Tuple（元组）、Sets(集合)、Dictionary（字典）

不可变数据：Number（数值）、String（字符串）、Tuple（元组）

可变数据：List（列表）、Sets(集合)、Dictionary（字典）

2. 【python数据类型的定义】

• Number (数值)

Python3 支持 int、float、bool、complex (复数)。

数字类型是顾名思义是用来存储数值的，需要记住的是，有点和 Java 的字符串味道差不多，如果改变了数字数据类型的值，将重新分配内存空间。

整型(Int) - 通常被称为是整型或整数，是正或负整数，不带小数点。Python3 整型是没有限制大小的，可以当作 Long 类型使用，所以 Python3 没有 Python2 的 Long 类型。

浮点型(float) - 浮点型由整数部分与小数部分组成，浮点类型不精确存储，浮点型也可以使用科学计数法表示 ($2.5e2 = 2.5 \times 10^2 = 250$)

复数(complex) - 复数由实数部分和虚数部分构成，可以用 $a + bj$ 或者 `complex(a,b)` 表示，复数的实部 a 和虚部 b 都是浮点型。

```
#1.Number

a = 12
print(a)
print(type(a))

b = 12.0
print(b)
print(type(b))

c = 1 + 0.5j
print(c)
print(type(c))
```

• String (字符串)

创建字符串可以使用单引号、双引号、三单引号和三双引号，其中三引号可以多行定义字符串。

我们定义一个 `d='hello word'` 语句，它在计算机中的执行顺序是先在内存在中创建一个字符串 `hello word`，在程序栈寄存器中创建一个变量 `d`，最后把 Python 的地址赋给 `d`。

```
#2.string (字符串)
d = '''hello word'''
print(d)
print(type(d))
```

• List (列表)

Python 中的数据结构是通过某种方式组织在一起的数据元素的集合，这些数据元素可以是数字、字符、甚至可以是其他数据结构。在 Python 中，最基本的数据结构是序列（列表和元组），序列中的每个元素都有一个序号（元素的具体位置），这个序号叫索引，索引下标从 0 开始，以此类推...

列表是写在方括号 `[]` 之间、用逗号分隔开的元素列表，列表可以完成大多数集合类的数据结构实现。列表中元素的类型可以不相同，它支持数字，字符串甚至可以包含列表（所谓嵌套），列表中的元素是可以改变的。


```
#3.List (列表)
e = [1,2,3,1.1,'a']
print(e)
print(type(e))
```

- **Tuple (元组)**

元组 (tuple) 与列表类似 (戴了枷锁的列表)，不同之处在于元组的元素不能修改。元组写在小括号 () 里，元素之间用逗号隔开，组中的元素类型也可以不相同。

python 的元组与列表类似，不同之处在于元组的元素不能修改。

元组使用小括号()，列表使用方括号[]。

```
#4.Tuple (元组)
f = (1,3,5,'a')
print(f)
print(type(f))
```

- **Set (集合)**

集合 (set) 是一个无序不重复元素的序列，使用大括号 {} 或者 set() 函数创建集合，注意：创建一个空集合必须用 set() 而不是 {}，因为 {} 是用来创建一个空字典。

```
#5.Set (集合)
q = {'hello','world',1,2,3,'hello'}
print(q)
print(type(q))
```

- **Dictionary (字典)**

Python 中的字典提供了一种灵活的访问和组织数据的方式

字典是一种映射类型，它的元素是键值对，字典的关键字必须为不可变类型，且不能重复。创建空字典使用 {}。

字典是由很多值组成的集合

字典的索引可以是不同的数据类型，同样也不止是整数，也有字符串

字典的索引被称为“键”，键及键所关联的值叫键值对 (类似于Java中的Map集合)

字典是另一种可变容器模型，且可存储任意类型对象。

字典的每个键值 key=>value 对用冒号 : 分割，每个键值对之间用逗号 , 分割，整个字典包括在花括号 {} 中

```
#6.Dictionary (字典)
dic = {"username":"aaa","psw":"bbb"}
print(dic)
print(type(dic))
```

2.4 【数据类型】Python数据类型：字符串

一、学习目标

- 1.字符串的拼接
- 2.字符串的切片
- 3.字符串的格式化
- 4.字符串的常用函数

二、知识点

1. 【字符串的拼接】

```
#1. 字符串的拼接
s1 = 'hello'
s2 = 'python'

s = s1 + s2
print(s)

res = s1 * 3
print(res)
```

2. 【字符串的切片】

```
#2. 字符串的切片
s3 = 'hello xiaoming'
#通过索引取出对应的字符
print(s3[1])
#从1-4索引取出一段字符
print(s3[1:4])
#取出索引4之后的所有字符
print(s3[4:])
#取出索引4之前的所有字符
print(s3[:4])
#字符串的反转
print(s3[::-1])
```

3. 【字符串的格式化】

```
#3. 字符串的格式化
name = '小明'
age = 18
s4 = '我的名字是：{}, 我的年龄是：{}'
#%格式化字符串
s5 = '我的名字是：%s, 我的年龄是：%d'%(name,age)
print(s5)
#format() 格式化字符串
s6 = '我的名字是：{}, 我的年龄是：{}'.format(name,age)
print(s6)
```

4. 【字符串的常用函数】

```
#4. 字符常用函数
s7 = 'hello python'

#计算字符串的长度: len()
res1 = len(s7)
print(res1)

#统计某个字符出现的次数: count()
res2 = s7.count('l')
print(res2)

#第一个字符大写, 其他字符小写: capitalize()
res3 = s7.capitalize()
print(res3)

#全部变小写: casefold()
res4 = s7.casefold()
print(res4)

#字符串的替换: replace()
res5 = s7.replace('python', 'world')
print(res5)

#字符串转列表: split()
res6 = s7.split(' ')
print(res6)
```

2.5 【数据类型】Python数据类型：列表

一、学习目标

- 1.列表的增删改查
- 2.列表的切片
- 3.列表的常用函数

二、知识点

1. 【列表的增删改查】

下面以列表为例进行讲解。

```
l = [1,2,3, 'a', 'b', 'c']
```

- 列表新增: append()
参数: 传入要加入的值
返回: 无

#1. 列表的新增

```
l = [1,2,3,'a','b','c']
l.append('d')
print(l)
```

• 列表删除：

列表删除有三种方式：del 列表[索引值]、remove()、pop()

#2. 列表的删除

#(1)del 列表[索引]

```
l = [1,2,3,'a','b','c']
del l[2]
print(l)
```

#(2)remove()

```
l1 = [1,2,3,'a','b','c']
l1.remove('a')
print(l1)
```

#(3)pop()

```
l2 = [1,2,3,'a','b','c']
res = l2.pop()
print(l2)
print(res)
```

• 列表修改：列表[索引值] = 要修改的值

#3. 列表的修改

```
l3 = [1,2,3,'a','b','c']
l3[0] = 4
print(l3)
```

• 列表查询：列表[索引值]

#4. 列表的查询

```
l4 = [1,2,3,'a','b','c']
res = l4[1]
print(res)
```

2. 【列表的切片】

我们可以根据列表索引和冒号来获取列表中的片段，下面以l列表为例进行讲解。

```
l = [1,2,3,'a','b','c']
```

• 截取列表中间某些值：

#1. 截取1-4的值

```
r1 = l[1:4]
print(r1)
```

- 从第某个值截取到结尾：

```
#2.从第3截取到结尾
r2 = l[3:]
print(r2)
```

- 从开头切到某个值

```
#3.从开头截取到索引2
r3 = l[:3]
print(r3)
```

- 列表反转

```
#4.列表的反转
r4 = l[::-1]
print(r4)
```

3.【列表的常用函数】

```
l = [1, 4, 7, 2, 5, 8]
```

- 计算列表元素长度：len()

参数：传入列表对象

返回：返回长度数值

```
#1.计算列表的长度：len()
r1 = len(l)
print(r1)
```

- 列表排序，元素必须都是数字类型：sort()

参数：空

返回：空

```
#2.列表的排序：sort()
l.sort()
print(l)
```

- 统计元素在列表中出现次数：count()

参数：传入需要统计的值

返回：统计的数值

```
#3.统计元素在列表中出现的次数：count()
l1 = [1, 4, 1, 2, 1, 8]
r2 = l1.count(1)
print(r2)
```

- 拼接列表成字符串：join(),列表中的值必须都是字符串

参数：传入列表对象

返回：返回长度数值

```
#4. 列表的拼接:join()
l2 = ['a', 'b', 'c']
r3 = ''.join(l2)
print(r3)
```

2.6 【数据类型】Python数据类型：字典

一、学习目标

- 1.字典的新增
- 2.字典的删除
- 3.字典的修改
- 4.字典的取值（查询）
- 5.字典的常用函数

二、知识点

1. 【字典的新增】

字典的新增有两种方式：字典[key] = 新增的值；字典对象的setdefault()方法

```
#1. 字典的新增
d = {"name": "tom", "age": 18}

#(1) 方式一
d["friend"] = "timi"
print(d)

#(2) 方式二
d.setdefault('name', 'teacher') #key如果存在，则不修改value
print(d)
```

2. 【字典的删除】

字典的删除有四种方式：del 字典[key]、pop()指定key删除、clear() 清空字典、popitem() 删除字典的最后一个键值对

```
#2. 字典的删除
d1 = {"name": "tom", "age": 18}

# # (1) del 字典[key]
# del d1["age"]
# print(d1)

# # (2) pop()
# r1 = d1.pop('job', '没有')
# print(d1)
```

```
# print(r1)

# #(3)clear()
# d1.clear()
# print(d1)

#(4)popitem()
d1.popitem()
print(d1)
```

3.【字典的修改】

字典的修改和字典的新增用法一样：字典[key] = 要改的值

```
#3.字典的修改
d2 = {"name":"tom","age":18}
d2["age"] = 20
print(d2)
```

4.【字典的取值（查询）】

获取字典的key对应的值有两种方式：字典[key]、get()方法

```
#4.字典的查询
d3 = {"name":"tom","age":18}
#(1)字典[key]
r2 = d3["name"]
print(r2)

#(2)get()
r3 = d3.get("job","没有")
print(r3)
```

5.【字典的常用函数】

```
d = {'name': 'tom', 'age':18}
```

- 获取字典的键值对数：len()

```
#（1）获取字典的键值对数
r4 = len(d4)
print(r4)
```

- 两个字典更新：update()

```
d5 = {"name":"tom","job":"teacher"}
d4.update(d5)
print(d4)
```

- 获取字典的所有key：keys()

#(3) 获取字典中所有的key

```
r5 = d4.keys()
print(r5)
```

- 获取字典的所有value: values()

#(4) 获取字典中所有的value

```
r6 = d4.values()
print(r6)
```

- 获取字典的所有key、value: items()

#(5) 同时获取字典的key和value

```
r7 = d4.items()
print(r7)
```

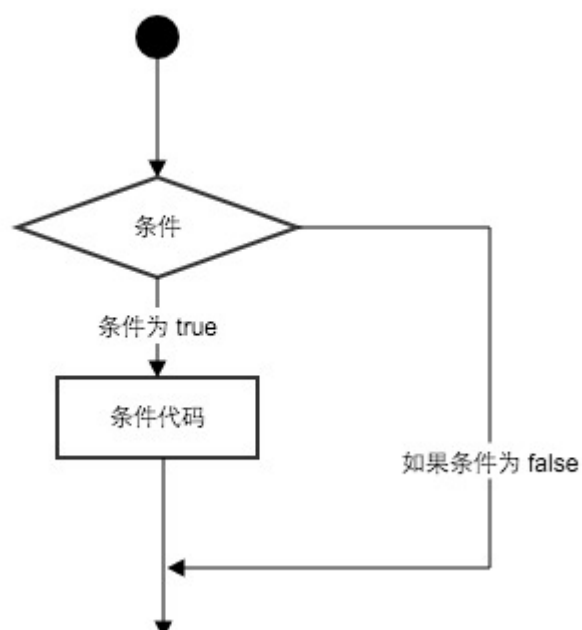
2.7 【流程控制语句】if 条件控制语句

一、学习目标

1. if 语句的执行流程
2. if 语句
3. if-else 语句
4. if-elif-else 语句

二、知识点

1. 【if语句的执行流程】



代码执行过程：



2. 【if语句】

- 写法：

```
if 判断条件:
    执行语句.....
```

- 代码实例：

#1.if语句

#需求：给出一个年龄，判断又没满18岁，不满18岁则打印：未满18岁，禁止观看！

```
age = 19
if age < 18:
    print("未满18岁，禁止观看！")
```

3. 【if-else语句】

- 写法：

```
if 判断条件:
    执行语句1.....
else:
    执行语句2.....
```

- 代码实例：

#2.if-else语句

#需求：给出一个年龄，判断又没满18岁，不满18岁则打印：未满18岁，禁止观看！，对18岁以上的打印：已满18岁，可以观看！

```
age = 19
if age < 18:
    print("未满18岁，禁止观看！")
else:
    print("已满18岁，可以观看！")
```

4. 【if-elif-else语句】

- 写法:

```
if 判断条件1:
    执行语句1.....
elif 判断条件2:
    执行语句2.....
elif 判断条件3:
    执行语句3.....
else:
    执行语句4.....
```

- 代码实例:

#3.if-elif-else语句

#需求：给出年龄，判断年龄段：1-14打印儿童；15-35打印青年；36-60打印中年；61及以后打印老年

```
age = -18
if 1 <= age <= 14:
    print("您的年龄段是：儿童")
elif 15 <= age <= 35:
    print("您的年龄段是：青年")
elif 36 <= age <= 60:
    print("您的年龄段是：中年")
elif age >= 61:
    print("您的年龄段是：老年")
else:
    print("年龄输入错误！")
```

5.【综合实例】

#猜拳游戏

```
import random
```

#1.从控制输入要出拳， --1石头 2剪刀 3 布

```
player = int(input("请输入1石头 2剪刀 3布: "))
```

#2.电脑出拳

```
computer = random.randint(1,3)
```

#3.判断玩家胜负

```
if (player == 1 and computer == 2) or (player == 2 and computer == 3) or (player == 3 and computer == 1):
    print("玩家胜利！")
elif player == computer:
    print("平局！")
else:
    print("电脑胜利")
```

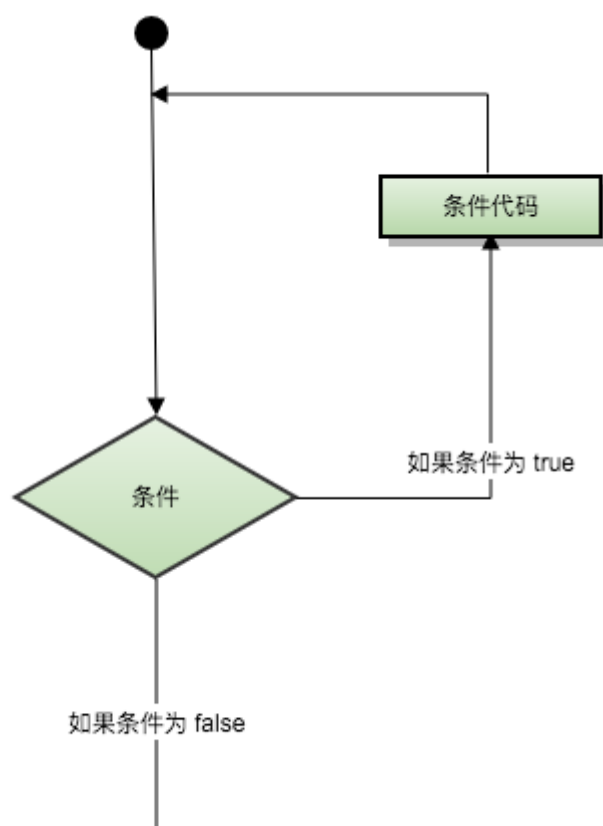
2.8 【流程控制语句】while 循环语句

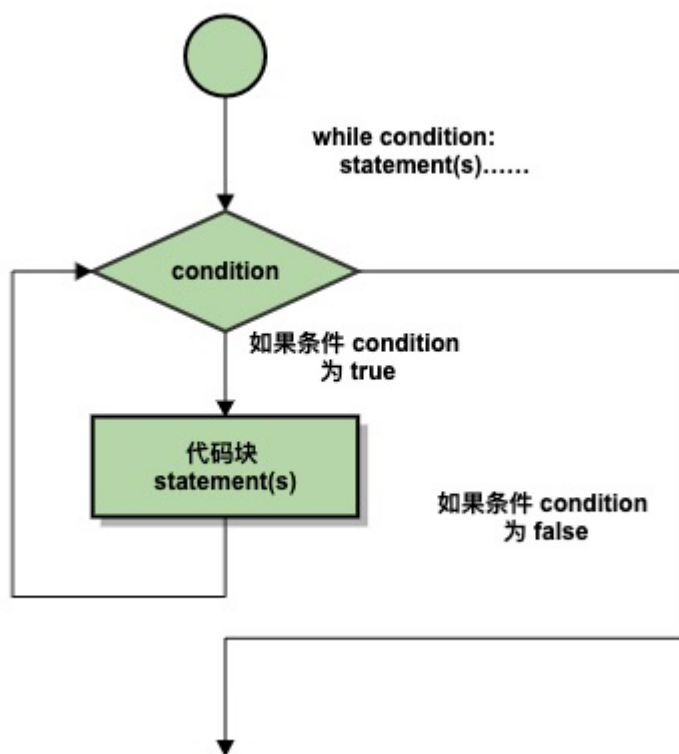
一、学习目标

1. while 语句的执行流程
2. 简单的 while 语句
3. while 循环使用 else 语句
4. while 循环使用 break 语句
5. while 循环使用 continue 语句

二、知识点

1. 【while语句的执行流程】





2. 【简单的while语句】

- 写法:

```
while 判断条件:  
    执行语句
```

- 代码实例:

```
#需求：打印1-10  
#1.简单的while语句  
  
#需求：打印1-10  
  
number = 1  
while number <= 10:  
    print(number)  
    number += 1
```

3. 【while 循环使用 else 语句】

在while循环中加入else语句，当循环条件不满足时则执行else语句中的代码，并跳出循环。

- 写法:

```
while 判断条件:  
    执行语句1  
else:  
    执行语句2
```

- 代码实例：

```
#需求：打印1-10，大于10时打印提示并跳出循环
#2.while 循环使用 else 语句
#需求：打印1-10，大于10时打印提示并跳出循环
number = 1
while number <= 10:
    print(number)
    number += 1
else:
    print("超过10次啦！")
```

4. 【while 循环使用 break 语句】

在while循环中加入break 语句，用于强行跳出循环。break后即时满足循环条件也不执行了。

- 写法：

```
while 判断条件:
    执行语句
    break
```

- 代码实例：

```
#需求：打印1-10，但打印到5时退出循环，即只打印1-4
#3.while 循环使用 break 语句
#需求：打印1-10，但打印到5时退出循环，即只打印1-4
number = 1
while number <= 10:
    if number == 5:
        break
    print(number)
    number += 1
```

5. 【while 循环使用 continue 语句】

在while循环中加入continue 语句，跳过该次循环，进入下一轮循环，换句话说，也就是本次不执行。

- 写法：

```
while 判断条件:
    执行语句1
    continue
    执行语句2
```

- 代码实例：

```
#需求：打印1-10，其中不打印5
#4.while 循环使用 continue 语句
#需求：打印1-10，其中不打印5
number = 1
while number <= 10:
    if number == 5:
        number += 1
        continue
    else:
        print(number)
        number += 1
```

6. 【综合实例】

```
#猜数字

num = 18
while True:
    guess = int(input("请输入一个数字: "))
    if guess > num:
        print("你输入的数字太大!")
    elif guess < num:
        print("你输入的数字太小")
    elif guess == num:
        print('恭喜你答对了! ')
        break
```

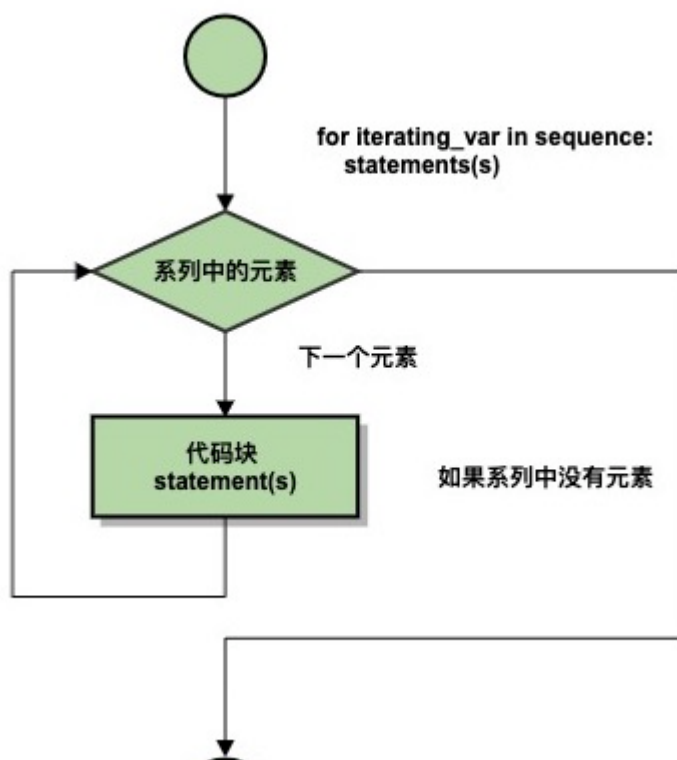
2.9 【流程控制语句】for 循环语句

一、学习目标

1. for 语句的执行流程
2. for 循环遍历序列
3. for 循环与 range 函数

二、知识点

1. 【for 语句的执行流程】



2. 【for循环遍历序列】

Python for循环可以遍历任何序列的项目，如一个列表或者一个字符串。

- 写法：

```
for 元素 in 列表/字符串:
    元素的处理
```

- 代码实例：

```
#1.for循环遍历序列
# # (1) for循环遍历字符串
#
# s = 'python'
# for a in s:
#     print(a+'*')
#     print("当前的字母是: {}".format(a))

# # (2) for循环遍历列表
# l = ["apple", "banana", "dog"]
# for b in l:
#     print(b+'*')
#     print("当前遍历出的元素是: {}".format(b))

# (3) for循环获取索引和值
l = ["apple", "banana", "dog"]
for i, v in enumerate(l):
    print("获取的索引是: {}".format(i))
    print("获取的值是: {}".format(v))
```

3. 【for循环与range函数】

for循环常与range函数一起使用

- range()函数介绍

格式：range(start, stop[, step])

参数说明：

- start: 计数从 start 开始。默认是从 0 开始。例如range (5) 等价于range (0, 5) ；
 - stop: 计数到 stop 结束，但不包括 stop。例如：range (0, 5) 是[0, 1, 2, 3, 4]没有5
 - step: 步长，默认为1。例如：range (0, 5) 等价于 range(0, 5, 1)
- 写法：

```
for 元素 in range(start, stop[, step]):
    元素的处理
```

- 代码实例：

```
#需求：获取0-10之间的偶数(含10)
#2.for和range()
#需求：获取0-10之间的偶数(含10)
for i in range(0,11,2):
    print("获取到的偶数是：{}".format(i))
```

4. 【综合案例】

打印99乘法表：

```
1*1=1
2*1=2 2*2=4
3*1=3 3*2=6 3*3=9
4*1=4 4*2=8 4*3=12 4*4=16
5*1=5 5*2=10 5*3=15 5*4=20 5*5=25
6*1=6 6*2=12 6*3=18 6*4=24 6*5=30 6*6=36
7*1=7 7*2=14 7*3=21 7*4=28 7*5=35 7*6=42 7*7=49
8*1=8 8*2=16 8*3=24 8*4=32 8*5=40 8*6=48 8*7=56 8*8=64
9*1=9 9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81
```

```
#行：第m行，第一个乘数m，第二个乘数1-m
#列：第n列，第一个乘数1-n，第二乘数n
```

```
for n in range(1,10):
    for m in range(1,n+1):
        value = n * m
        print("{}*{}={}".format(n,m,value),end=' ')
    print(' ')
```


2.10 【函数】函数的定义与调用

一、学习目标

- 1.函数的定义和调用
- 2.函数各要素详解

二、知识点

1.【函数的定义和调用】

其实我们在初中数学就接触过函数，如 $ax + b = y$ ，编程中的函数与其很类似，都是输入一个参数经过处理后返回一个值或结果

- 基本语法：

```
def 函数名 ([参数1], [参数2] ... [参数n]):  
    函数体
```

- 代码实例：

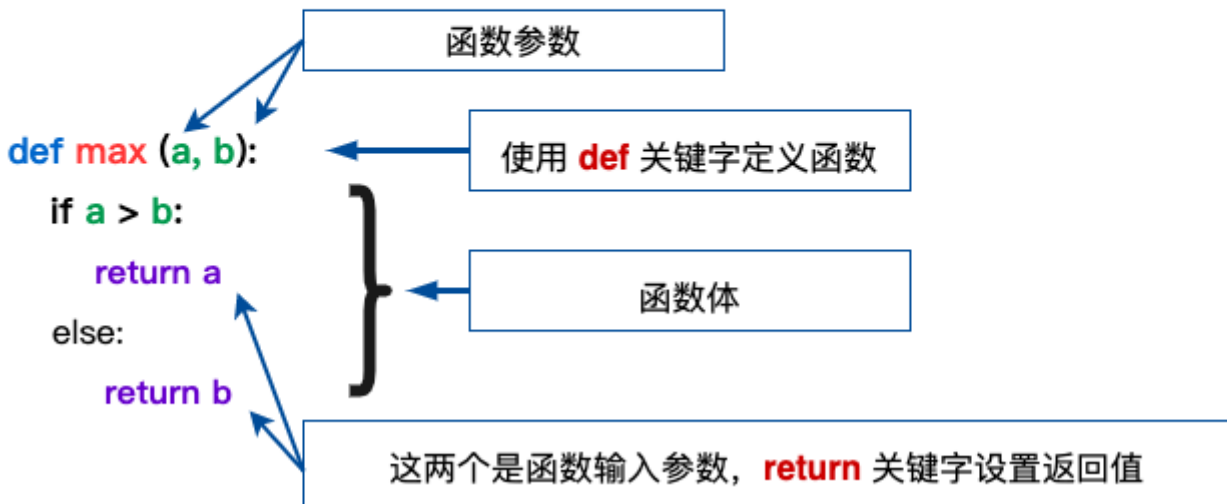
```
#定义一个函数  
  
def hello():  
    print("hello python!")  
  
#调用函数  
hello()
```

2.【函数各要素详解】

- 函数定义的规则：

函数

函数可以输入参数，并设置返回值



- 1.函数的代码块是以def关键字开头，后面跟函数名和括号。
- 2.函数可有入参，函数的入参必须放在括号的内部。
- 3.return返回返回值，如果没有return,默认返回None。return是结束语句。
- 4.函数是以：起始，必须缩进。

2.11 【函数】函数的参数传递

一、学习目标

1. 实参和形参
2. 位置参数
3. 关键字参数
4. 默认值参数
5. 可变参数

三、知识点

1. 【实参和形参】

- 形参：就是函数定义时小括号里的变量
- 实参：函数调用的时候，小括号里的表达式
- 函数可以没有形参和实参

```
#1.形参和实参
def add_number(a,b):
    print("结果输出: {}".format(a + b))

add_number(1,1)
```

2. 【位置参数】

位置参数，要求实参顺序必须和形参顺序完全一致，由形参顺序决定实参顺序,个数也必须一致

```
#2.位置参数,顺序必须一致,数量必须一致
def say_hello(name,age,home):
    print("大家好,我是{},我今年{}岁了,我来自{}".format(name,age,home))

say_hello('小明',18,'北京')
```

3. 【关键字参数】

关键字参数，函数调用时，实参可以是键值对，键就是形参名字，这样的调用，实参不必关心形参的顺序。

```
#3.关键字参数
def say_hello(name,age,home):
    print("大家好,我是{},我今年{}岁了,我来自{}".format(name,age,home))

say_hello(name='小明',home='北京',age=18)
```

4. 【默认值参数】

默认值，如果形参在定义的时候给定一个值，那么函数在调用时就可以不传实参，可以简化调用

- 默认值参数必须放到最右边
- 如果传了实参，那么实参优先，不会使用默认值
- 默认值只计算一次
- 默认值必须是不可变对象

#4. 默认值参数

```
def say_hello(name, age, home='北京'):
    print("大家好，我是{}", 我今年{}岁了，我来自{}".format(name, age, home))

say_hello('小明', 20, '上海')
```

5. 【可变参数】

场景：有的时候我们不确定需要传入函数的参数的数量，只要传入就接收并处理，这个时候需要用到不定长参数，不定长参数有如下两种形式：

- 使用*接收任意数量的位置参数

注意:*的不定长参数被当做元组处理

```
def demo1(my, *name):
    print("大家好，我是{}", 下面是我的朋友".format(my))
    for i in name:
        print(i)
demo1('小明', '张三', '李四', '王五')
```

- 使用**接收任意数量的关键字参数

注意:**的不定长参数被当做字典处理

```
def demo2(my, **name):
    print("大家好，我是{}", 下面是我的朋友".format(my))
    for v in name.values():
        print(v)

demo2('小明', f1='张三', f2='李四')
```

2.12 【函数】函数的返回值

一、学习目标

1. return 语句介绍

2. return 应用

二、知识点

1. 【return语句的作用】

- return的作用一个是终止函数的执行，所有执行了return后，其后的语句不会被执行
- 如果没有return语句，则默认返回的是None
- return可以返回一个值，可以返回多个值，也可以返回一个元组，也可以返回一个字典，这一点和函数入参类似

2. 【return的实例】

```
def add_number(a,b):  
    c = a + b  
    c1 = a + 1  
    c2 = b + 1  
    return (c,c1,c2)  
  
res1= add_number(1,2)  
print(res1)
```

2.13 【函数】lambda 匿名函数

一、学习目标

- 1.匿名函数介绍
- 2.匿名函数实例

二、知识点

1. 【匿名函数介绍】

- lambda 只是一个表达式，函数体比 def 简单很多。
- lambda的主体是一个表达式，而不是一个代码块。仅仅能在lambda表达式中封装有限的逻辑进去。
- 虽然lambda函数看起来只能写一行，却不等同于C或C++的内联函数，后者的目的是调用小函数时不占用栈内存从而增加运行效率。

2. 【匿名函数实例】

语法：

```
函数名 = lambda [arg1 [,arg2,.....argn]]:expression
```

实例：

```
# 函数名 = lambda [arg1 [,arg2,.....argn]]:expression  
  
def add_num(a,b):  
    return a + b  
  
res = add_num(1,2)  
print(res)  
  
add_num = lambda a,b:a+b  
res1 = add_num(1,2)  
print(res1)
```

2.14 【函数】变量的作用域

一、学习目标

1. Python 局部变量
2. Python 全局变量
3. global 关键字定义全局变量

二、知识点

1. 【Python局部变量】

在函数内部定义的变量，它的作用域也仅限于函数内部，出了函数就不能使用了，我们将这样的变量称为局部变量。

要知道，当函数被执行时，Python 会为其分配一块临时的存储空间，所有在函数内部定义的变量，都会存储在这块空间中。而在函数执行完毕后，这块临时存储空间随即会被释放并回收，该空间中存储的变量自然也就无法再被使用。

```
#1.Python局部变量

def demo1():
    var1 = '变量1'
    print("函数的内部: {}".format(var1))

demo1()
print("函数的外边: {}".format(var1))
```

2. 【Python全局变量】

除了在函数内部定义变量，Python 还允许在所有函数的外部定义变量，这样的变量称为全局变量。

和局部变量不同，全局变量的默认作用域是整个程序，即全局变量既可以在各个函数的外部使用，也可以在各函数内部使用。

```
#2.python的全局变量
var2 = '变量2'

def demo2():
    print("函数的内部: {}".format(var2))

demo2()
print("函数的外部: {}".format(var2))
```

3. 【global 关键字定义全局变量】

在函数体内定义全局变量。即使用 global 关键字对变量进行修饰后，该变量就会变为全局变量。

```
#3.global定义全局变量
def demo3():
    global var3
    var3 = '变量3'
    print("函数内部: {}".format(var3))

demo3()
print("函数外部: {}".format(var3))
```

2.15 【模块】Python三方模块（库）的安装

一、学习目标

- 1.在线安装三方模块
- 2.离线安装三方模块
- 3.项目依赖包整体迁移

二、知识点

1.【在线安装三方模块】

利用的目前常见的一些自动化安装工具，比如pip，easy_install等，安装完python之后会自带，这些工具可以自动帮你下载源码，并安装。

而且很多时候，由于要安装的包，还要依赖一些其他的别的包，而这类自动化安装工具，会自动帮你解决依赖关系，自动帮你下载并安装所缺少的那些包，所以相对来说，就省去了你的麻烦了。

注：在使用pip，easy_install之前需要配置环境变量：python目录下的Scripts

- PIP常用命令：

#1.显示版本和路径：

```
pip --version
```

2.获取帮助：

```
pip --help
```

#3.升级 pip：

```
python -m pip install --upgrade pip
```

```
easy_install --upgrade pip #如果上面升级命令出现问题，可以使用这个命令
```

#4. pip查看已安装包列表：

```
pip list
```

- PIP在线安装模块包(requests包为例)

#1.requests模块：

```
pip install requests # 最新版本
```

```
pip install requests==1.0.4 # 指定版本
```

```
pip install 'requests>=1.0.4' # 最小版本
```

#2.使用国内镜像安装（速度快）：

```
pip install -i https://pypi.douban.com/simple/ requests
```

#3.升级包：

```
pip install --upgrade requests
```

#4.卸载包：

```
pip uninstall requests
```

2. 【离线安装三方模块】

在环境部署过程中，往往会遇到没有外网而无法使用在线安装，可以在能联外网的电脑下载模块包，然后拷贝到内网机器，通过以下命令进行离线安装。

说明：python模块的离线包有两种格式，一种是.whl格式，一种是.tar.gz格式（压缩包）格式。

离线包下载地址：<https://pypi.org/>

Navigation

Project description

Release history

Download files

Project links

Homepage

Download files

Download the file for your platform. If you're not sure which to choose, learn more about [installing packages](#).

Filename, size	File type	Python version	Upload date	Hashes
requests-2.26.0-py2.py3-none-any.whl (62.3 kB)	Wheel	py2.py3	Jul 13, 2021	View
requests-2.26.0.tar.gz (104.4 kB)	Source	None	Jul 13, 2021	View

- 离线安装.whl格式模块包

步骤一：将.whl文件放入python安装目录的script目录下

步骤二：打开cmd,cd到script目录

步骤三：执行命令pip install 【文件名】

```
pip install requests-2.25.1-py2.py3-none-any.whl
```

- 离线安装.tar.gz格式模块包

步骤一：将.tar.gz文件解压

步骤二：打开cmd,cd到解压后的目录下

步骤三：执行命令

```
python setup.py install
```

3. 【python项目依赖包整体迁移】

说明：当我们把项目从A电脑，迁移到B电脑，为了确保开发环境一致，可以使用如下方法。

- 步骤一：在原项目下生成requirements.txt文件

```
pip freeze > requirements.txt
```

- 步骤二：依据requirements.txt文件下载离线包

```
pip download -r requirements.txt
```

- 步骤三：在B电脑安装所有离线包

```
pip install --no-index --find-links=C:\\Users\\lcy\\Downloads -r /xxx/xxx/site-packages/requirements.txt
pip install --no-index --find-links=./ -r requirements.txt    #都放在当前路径下执行
```

2.16 【模块】模块导入

一、学习目标

- 1.import 语句导入模块
- 2.from ... import 语句模块

二、知识点

1. 【import 语句导入模块】

想使用 Python 源文件，只需在另一个源文件里执行 import 语句，语法如下：

```
import module1[, module2[, ... moduleN]
```

导入安装的三方模块：

```
import requests

res = requests.get('https://www.baidu.com/')
print(res)
```

导入自己写的py模块：

```
import myfun
myfun.print_fun('小明')
```

引用导入模块中的函数：

```
myfun.print_fun('小明')
```

2. 【from ... import 语句模块】

Python 的 from 语句让你从模块中导入一个指定的部分到当前命名空间中，语法如下：

```
from modname import name1[, name2[, ... nameN]]
```

应用实例：

```
from myfun import print_fun

print_fun('小王')
```

from ... import * 语句：

把一个模块的所有内容全都导入到当前的命名空间也是可行的，只需使用如下声明：

```
from myfun import *  
print_fun('小李')
```

2.17 【模块】以主程序运行当前模块

```
if __name__=="__main__":  
    run()
```

一、学习目标

1. `__name__` 和 `__main__` 分别是什么？
2. `__name__` 和 `__main__` 的应用

二、知识点

1. 【name和main分别是什么？】

`__name__` 与 "`__main__`" 分别是什么？

- 1、`__name__` 是当前模块的一个属性，属性名称的前后各有两个下划线，表明它是一个内置属性(也称内置变量)
- 2、"`__main__`" 则是一个字符串对象，"xx"，一对双引号是创建字符串对象的快捷方式

`__name__ == "__main__"` 写在一起表示什么？

表示当前模块的 `__name__` 属性的值是否等于字符串 "`__main__`"，如果 `__name__` 的值是 "`__main__`" 则返回 True、如果 `__name__` 的值不是 "`__main__`"，则返回 False

`__name__` 属性的值什么时候等于 "`__main__`" 呢？

如果当前模块是程序的入口模块（也称顶级模块、脚本文件），则当前模块的 `__name__` 属性的值是 "`__main__`"

举个例子

- 1、首先定义一个模块：my_first.py
- 2、然后命令行执行语句：python my_first.py
- 3、此时 my_first 模块的 `__name__` 属性的值是 "`__main__`"

2. 【代码示例】

```
def demo():  
    print(__name__)  
  
# demo()  
  
if __name__ == '__main__':  
    demo()
```

2.18 【文件操作】文件读取与文件写入

一、学习目标

1.文件读取

2.文件写入

二、知识点

1. 【文件读取】

- 文件读取可分为以下步骤：
 - 打开文件
 - 读取文件内容
 - 关闭文件
- 打开文件要使用open内建函数：

```
open(file [, mode='r', encoding=None, errors=None])
```

参数说明：file：文件路径，可以是相对路径和绝对路径

mode：文件打开模式

encoding: 文件编码方式，不用于二进制文件，一般是utf-8,gbk

errors: 指定如何处理编码和解码错误，适用于文本文件

返回值：一个可迭代的文件对象

mode	解释
r	只读
w	只写，写之前会清空文件的内容，如果文件不存在，会创建新文件
a	追加的方式，在原本内容中继续写，如果文件不存在，则会创建新文件
r+	可读可写
w+	打开一个文件用于读写。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
a+	打开一个文件用于读写。如果该文件已存在，文件指针将会放在文件的结尾。文件打开时会追加模式。如果该文件不存在，创建新文件用于读写。
b	rb、wb、ab、rb+、wb+、ab+意义和上面一样，用于二进制文件操作

注意：二进制文件一般用于视频、音频、图片

读取文件常用函数：

函数	解释
<code>read([size])</code>	读取文件(读取size字符，默认读取全部)
<code>readline([size])</code>	读取一行,如果指定size，将读入指定的字符数
<code>readlines()</code>	把文件内容按行全部读入，返回一个包含所有行的列表

- 代码示例：

```
#1. 打开文件
f = open('./file.txt', mode='r', encoding='utf-8')

#2. 读取内容
#(1) read()
# content = f.read(20)
# print(content)

# # (2) readline()
# content = f.readline(5)
# print(content)

#3. readlines()
content = f.readlines()
print(content)

#3. 关闭文件
f.close()
```

```
#2. with 读取文件
with open('./file.txt', mode='r', encoding='utf-8') as f:
    print(f.readline())
```

```
#3. 循环读取
with open('./file.txt', mode='r', encoding='utf-8') as f:
    for line in f:
        print(line.strip())
```

2. 【文件写入】

- 代码示例：

```
path = 'file2.txt'

#1. 打开文件
f = open(path, mode='w', encoding='utf-8')

#2. 写文件
```

```
f.write("你好 Python! ")

#3.关闭文件
f.close()

#2.with写文件
with open(path,mode='w',encoding='utf-8') as f:
    f.write("你好 世界! ")
```

2.19 【文件操作】CSV文件读取与文件写入

一、学习目标

1.csv文件读取

2.csv文件写入

二、知识点

1. 【csv文件读取】

代码示例：

```
import csv

with open('file.csv') as f: #1.打开文件
    #2.获取csv读取器
    csv_reader = csv.reader(f)
    header = next(csv_reader)
    print(header)
    for line in csv_reader:
        print(line)
```

2. 【csv文件写入】

代码示例：

```
import csv

l1 = [[1,2,3],[4,5,6],[7,8,9]]

with open('file2.csv','w',newline='') as f: #1.打开文件
    #2.写入csv文件
    csv_writer = csv.writer(f,delimiter='*')
    for line in l1:
        csv_writer.writerow(line)
```

2.20 【异常处理】异常处理讲解（一）

一、学习目标

1. try-except 语句
2. 抛出异常
3. assert 断言

二、知识点

1. 【try-except语句】

基本语法：

```
try:
    【代码块A】 #可能会出错误的代码    异常检测
except Exception as e:    #异常处理
    【代码块B】 #检查到异常，如何处理
```

语法：

```
try:
    【代码块A】 #可能会出错误的代码    异常检测
except Exception1 as e1:    #异常处理
    【代码块1】    #异常处理代码
except Exception2 as e2:    #异常处理
    【代码块2】    #异常处理代码
....
except Exceptionn as en:    #异常处理
    【代码块n】    #异常处理代码
[else:]                    #可选，如果没有引发异常会执行
    处理语句
[finally:]                 #可选，无论如何都要执行的语句
    处理语句
```

- 注意事项：
 - except匹配顺序从上到下
 - except语句书写要求：精确的类型往前放，模糊的，不精确的往后放
 - except不带任何类型，则匹配所有类型异常，应该放到最后，吞掉异常
 - 可以将多种异常用元组的方式（异常类型1，异常类型2...异常类型n）书写，简化代码
 - except字句中e，是一个对象，打印它，会显示异常信息描述
 - try-except也可以捕获方法或函数中抛出的异常
 - 所有异常类型都继承自BaseException，使用BaseException可以将异常一网打尽
 - finally字句中可以进行一些清理工作，比如关闭文件，数据库等工作

2. 【抛出异常】

手动抛出一个指定类型的异常，无论是哪种异常类都可以带一个字符串参数，对异常进行描述。

- raise不带参数会把错误原样抛出

```
try:
    1/0
except ZeroDivisionError as e:
    print(e)
    raise ZeroDivisionError('分母不能是0')
```

3. 【assert断言】

语法：assert 条件 [,异常描述字符串]

执行流程：如果条件为假，则抛出AssertionError，条件为真，就当assert不存在

作用：对于可能出问题的代码，加上断言，方便问题排查

```
#2.assert断言
print('start')
num = int(input("请输入1-9的整数: "))
assert 0 < num <= 9, '你输入的整数不是1-9'
print('end')
```

2.21 【异常处理】异常处理讲解（二）

一、学习目标

1.自定义异常

二、知识点

1.【自定义异常】

- 自己定义的异常类必须继承BaseException或Exception
- 步骤：
 - 在自定义异常类的构造函数中，调用父类构造函数 `__init__`
 - 重写 `__str__` 方法输出异常信息
 - 编写异常处理方法处理异常（可省）

```
class CustomError(BaseException):

    def __init__(self,msg):
        super().__init__() #调用父类的初始化方法
        self.msg = msg

    def __str__(self):
        self.handle_exception()
        return self.msg + '!!!'

    def handle_exception(self):
        print("异常的处理")
```

```
try:
```

```
raise CustomError('自定义异常')
except CustomError as e:
    print(e)
```

第三章 面向对象

3.1 类和对象的概念

一、学习目标

- 1.面向过程和面向对象的区别
- 2.类和对象的概念

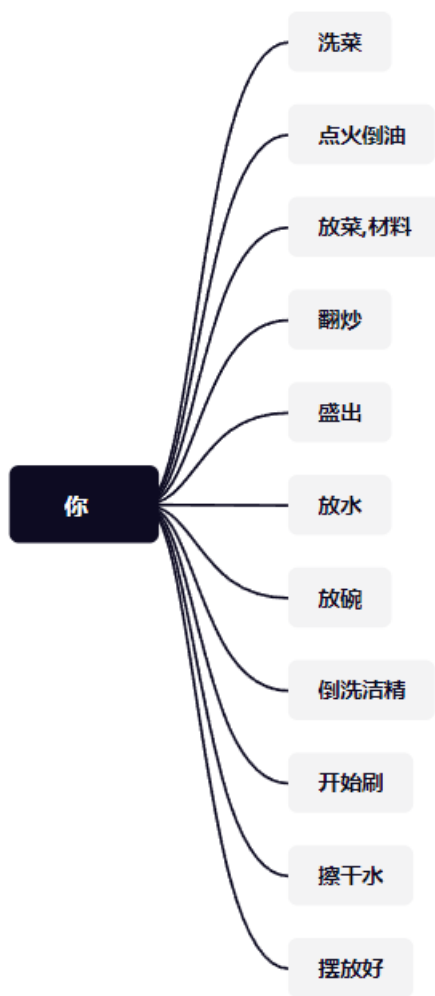
二、知识点

1.【面向过程和面向对象的区别】

- 面向过程

面向过程的程序设计的核心是过程（流水线式思维），过程即解决问题的步骤，面向过程的设计就好比精心设计好一条流水线，考虑周全什么时候处理什么东西。

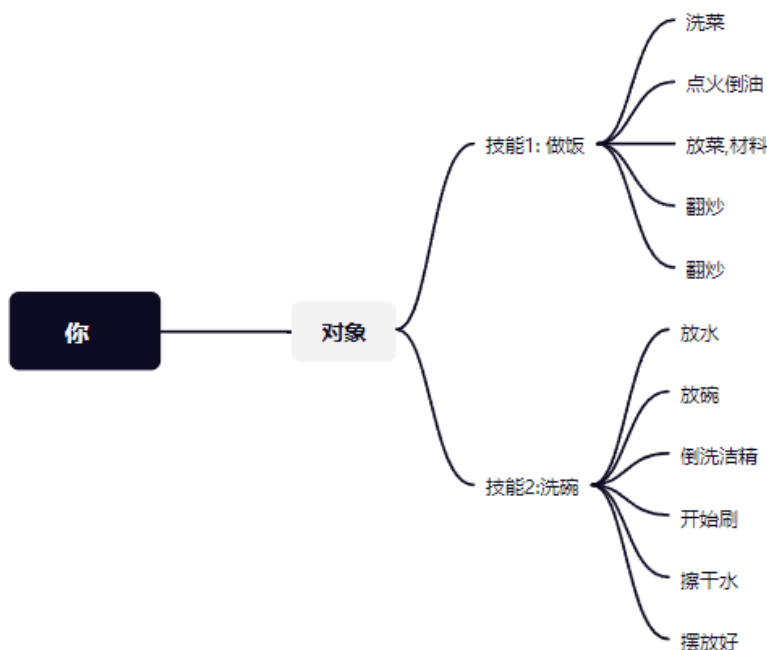
- 优点是：极大的降低了写程序的复杂度，只需要顺着要执行的步骤，堆叠代码即可。
- 缺点是：一套流水线或者流程就是用来解决一个问题，代码牵一发而动全身。
- 面向过程举例：(炒菜并洗碗)



- 面向对象

面向对象的程序设计的核心是对象（上帝式思维），要理解对象为何物，必须把自己当成上帝，上帝眼里世间存在的万物皆为对象，不存在的也可以创造出来。

- 优点是：解决了程序的扩展性。对某一个对象单独修改，会立刻反映到整个体系中，如对游戏中一个人物参数的特征和技能修改都很容易。
- 缺点：可控性差，无法像面向过程的程序设计流水线式的可以很精准的预测问题的处理流程与结果，面向对象的程序一旦开始就由对象之间的交互解决问题，即便是上帝也无法预测最终结果。
- 面向对象举例：(炒菜并洗碗)



2. 【类和对象的概念】

面向对象编程中必会的概念：类、对象、实例、实例化

- 生活角度解释

类：具有相同特征的一类事物(人、狗、老虎)，这是一个群类

对象 / 实例：具体的某一个事物（隔壁的老王、你家的旺财），这是一个个体

实例化：类——>对象的过程（这在生活中表现的不明显，我们在后面再慢慢解释）

- 程序角度解释

在编程语言中，不仅仅是python，面向对象的编程思想都是通用的。

类：用来描述具有相同的属性和方法的集合，是抽象的。对象是类的实例。（可以拿盖房子为例，房子的图纸就是类，房子就是对象，是一个具体的东西）

对象 / 实例：通过类定义的数据结构实例。对象包括两个数据成员（类变量和实例变量）和方法。

实例化：类名() 这个过程就是实例化。

- 面向对象编程最重要的是什么？

- 找对象：按功能将对象进行划分，确定对象的属性和行为
- 干活：调用对象中的属性或行为帮我们干活

3.2 类和对象如何编写

一、学习目标

1.类的创建

2.对象（实例）的创建

3.类属性、实例属性

4.类方法、实例方法、静态方法

二、知识点

1.【类的创建】

语法：

```
class 类名：  
    类体
```

注意：

- 类定义必须以关键字class
- 类名要符合标识符的规范
- 类名一般用大驼峰风格：每个单词首字母大写，其它小写，例如MyBook MyMoney
- 类体必须缩进
- 在python3中类默认继承object，所以可以这样写 class Dog:，它等价于class Dog(object)或class Dog():

```
class Dogs():  
    pass #占位用的
```

2.【对象的创建】

对象的创建过程也称之为对象的实例化过程。类通过实例化创建出来的就是对象。

```
class Dogs():  
    pass #占位用的  
  
wangcai = Dogs() #类的实例化  
  
print(wangcai)  
print(Dogs)
```

3.【类属性、实例属性】

给类添加的属性就是类属性，给对象添加的属性就是对象属性（实例属性）。类属性是所有的对象共有的体征；实例属性是每一个对象公用的，每一个对象特有的。

类属性可以被类和属性访问，而实例属性只能被实例访问。

```
class Dogs():  
    eyes = 2  
    legs = 4  
  
wangcai = Dogs()  
wangcai.color = 'black'  
  
print(Dogs.eyes)      #访问类中的属性  
print(wangcai.color)  #访问对象的属性  
print(wangcai.eyes)   #对象能够访问类属性  
  
# print(Dogs.color)   #类不能访问对象属性
```

4. 【类方法、实例方法、静态方法】

- 实例方法

实例方法就是类的实例能够使用的方法。

- 语法：

定义：第一个参数必须是实例对象，该参数名一般约定为“self”，通过它来传递实例的属性和方法；

调用：只能由实例对象调用。不需要传值，它会自动传值（实例对象）。

```
class 类名:

    def 实例方法名(self):
        方法体
```

- 方法和函数的区别：

- 方法作用域属于类，所以即便和普通函数重名，也不会被覆盖
 - 方法的第一个参数必须是self，但函数不要求
 - 方法必须通过对象调用，而函数不需要

- 代码示例：

```
class Dogs():

    def zaji(self):
        print("这是一个实例方法!", self)

wangcai = Dogs() #实例化
print(wangcai)
wangcai.zaji()
```

- 类方法

类方法是类本身作为对象进行操作的方法。假设有个方法，且这个方法在逻辑上采用类本身作为对象来调用更合理，那么这个方法就可以定义为类方法。另外，如果需要继承，也可以定义为类方法。

- 语法：

定义：使用装饰器@classmethod。第一个参数必须是当前类，该参数名一般约定为“cls”，通过它来传递类的属性和方法

调用：类和实例对象都可以调用。

```
class 类名:

    @classmethod
    def 类方法名(cls):
        方法体
```

- 代码示例

#2. 类方法

```
class Dogs():

    @classmethod
    def eat(cls):
        print('这是一个类方法：吃！',cls)

Dogs.eat()    #1. 类调用类方法
print(Dogs)

wangcai = Dogs()
wangcai.eat() #2. 实例调用类方法
```

- 静态方法

静态方法是类中的函数，不需要实例。静态方法主要是用来存放逻辑性的代码，逻辑上属于类，但是和类本身没有关系，也就是说在静态方法中，不会涉及到类中的属性和方法的操作。可以理解为，静态方法是个**独立的、单纯的**函数，它仅仅托管于某个类的名称空间中，便于使用和维护。

- 语法：

定义：使用装饰器@staticmethod。参数随意，没有“self”和“cls”参数，但是方法体中不能使用类或实例的任何属性和方法；

调用：类和实例对象都可以调用。

```
class 类名:

    @staticmethod
    def 静态方法名():
        方法体
```

- 代码示例

```
#3. 静态方法
class Dogs():

    @staticmethod
    def count():
        print("这是狗的数量！")

Dogs.count()    #1. 类调用静态方法
wangcai = Dogs()
wangcai.count() #2. 实例调用静态方法
```

3.3 面向对象的三大特性：封装

一、学习目标

- 1.什么是封装
- 2.封装的好处
- 3.封装的实际应用

二、知识点

1.【什么是封装】

将一些属性和相关方法封装在一个对象中。对外隐藏内部具体实现细节，内部实现,外界不需要关心，外界只需要根据"内部提供的接口"去使用就可以。

封装的概念来自电气电子元件，集成电路的一道工序，后来引用到面向对象的程序设计思想中。现实中处处可见封装的例子：比如你家的洗衣机。



把一台洗衣机看做洗衣机类的一个实例，洗衣机里有标准洗、速洗、精洗、桶干燥、洁桶等多种功能。作为用户不需要知道这些功能内部的具体实现，需要某项功能只要选择对应的功能键即可。这就是洗衣机封装了以上多种功能。

相似地，在面向对象中，封装，说白了就是一个类中抽象出来的静态数据（即属性）以及该类的一些功能（即方法）。这就是前面学习的对象的属性和方法。在一个对象内部，某些代码或某些数据可以是私有的，不能被外界访问。通过这种方式，对象对内部数据提供了不同级别的保护，以防止程序中无关的部分意外的改变

或错误的使用了对象的私有部分。这是前面学习的访问权限，在封装特性中，访问权限非常重要。

2. 【封装的好处】

- 使用起来更加方便
 - 因为已经把很多相关的功能, 封装成一个整体
 - 类似于像外界提供一个工具箱
 - 针对于不同的场景, 使用不同的工具箱就可以
- 保证数据的安全
 - 针对于安全级别高的数据, 可以设置成"私有"
 - 可以控制数据为只读, 外界无法修改
 - 也可以拦截数据的写操作, 进行数据校验和过滤
- 利于代码维护
 - 如果后期, 功能代码需要维护, 则直接修改这个类内部代码即可
 - 只要保证接口名称不变; 外界不需要做任何代码修改

3. 【封装的实际应用】

封装, 这是定义类的 准则, 单个类根据 职责 将 属性 和 方法 封装 到一个抽象的 类 中。

```
#1.输入用户名
#2.输入密码
#3.点击登录按钮

# user = 'xiaoming'
# psw = '123456'
# print("请输入用户名: {}".format(user))
# print("请输入密码: {}".format(psw))
# print("点击登录按钮! ")
# print("登录成功啦")
#
#
# user = 'xiaowang'
# psw = '1234567'
# print("请输入用户名: {}".format(user))
# print("请输入密码: {}".format(psw))
# print("点击登录按钮! ")
# print("登录成功啦")

from login import Login

lg = Login() #实例化
lg.user_login('xiaoming','123456')
lg.user_login('xiaowang','1234567')
```

3.4 面向对象的三大特性：继承

一、学习目标

- 1.什么是继承
- 2.继承的好处
- 3.单继承
- 4.多继承

二、知识点

1.【什么是继承】

继承概念：继承是类与类的一种关系，是一种子类与父类的关系，即爸爸与儿子，爸爸生个儿子，儿子继承爸爸的属性和方法。

如猫类，猫是动物；猫类继承于动物类。猫类就是子类，动物类就是父类。

Python有单继承与多继承。单继承即子类继承于一个类，多继承即子类继承于多个类，多继承会比较少遇到。

什么时候使用继承：假如我需要定义几个类，而类与类之间有一些公共的属性和方法，这时我就可以把相同的属性和方法作为基类的成员，而特殊的方法及属性则在本类中定义。这样子类只需要继承基类（父类），子类就可以访问到基类（父类）的属性和方法了。

- 语法

```
class Animal(object): #Animal: 父类、基类、超类
    pass
class Cat(Animal): #继承动物类, Cat:子类、派生类
    pass
```

- 代码示例

```
class Animal(object):

    def eat(self):
        print("我会吃！")

class Cat(Animal):

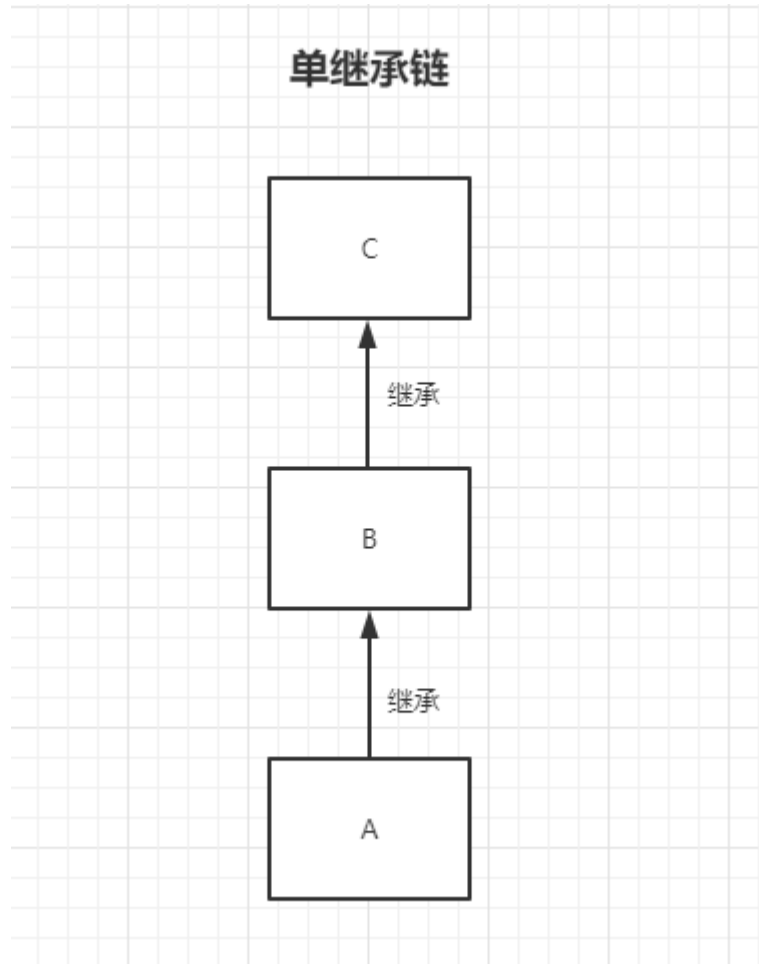
    def miao(self):
        print("我会喵喵喵！")

c = Cat() #实例化
c.miao()
c.eat()
```

2.【继承的好处】

- 提高代码的扩展性
- 提高代码的可复用性

3.【单继承】



- 概念
仅仅继承了一个父类
- 语法

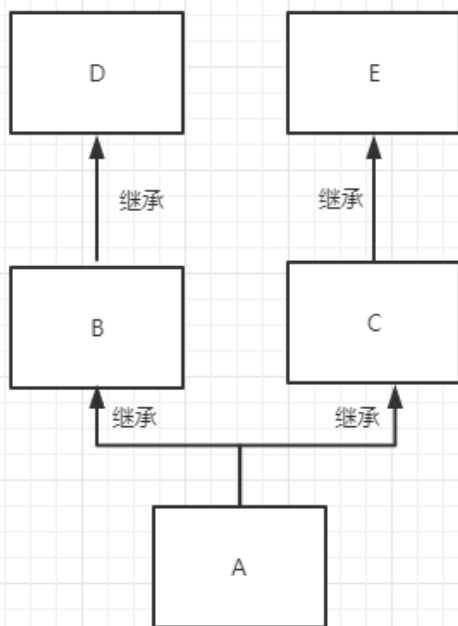
```
class A(B):  
    pass
```

- 代码示例

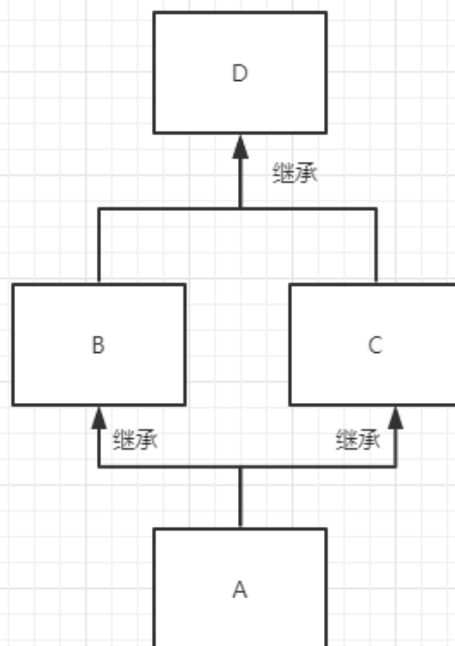
```
class C():  
    age = 18  
  
class B(C):  
    pass  
  
class A(B):  
    pass  
  
print(A.age) #资源的访问顺序：A-B-C
```

4. 【多继承】

无重复多继承链



有重复多继承链



- 概念

继承了多个父类，多继承又分为无重复多继承、有重复多继承。

- 语法

```
class A(B, C):
    pass
```

- 无重复多继承（单调原则）

```
class E():
    age = 16

class D():
    pass

class C(E):
    pass

class B(D):
    pass

class A(B,C):
    pass

print(A.age) #资源的访问顺序：A-B-D-C-E
```

- 有重复多继承（就近原则）

```
class D():
    age = 17

class C(D):
    pass

class B(D):
    pass

class A(B,C):
    pass

print(A.age) #资源的执行顺序：A-B-C-D
```

3.5 面向对象的三大特性：多态

一、学习目标

- 1.什么是多态
- 2.多态的实际应用

二、知识点

1.【什么是多态】

多态:不同的子类调用相同的方法，产生不同的结果

多态的关注点在“对象的行为和属性”而非类型。

2.【多态的实际应用】

```
class Animal():

    def talk(self):
        pass

class Cat(Animal):

    def talk(self):
        print("我会喵喵喵！")

class Dog(Animal):

    def talk(self):
        print("我会汪汪汪！")

def talk(obj):
    obj.talk()
```

```
c = Cat()
d = Dog()
talk(c)
talk(d)
```

3.6 面向对象 `__init__` 方法（构造方法）

一、学习目标

1. `__init__` 方法概念
2. `__init__` 方法实际应用

二、知识点

1. 【`__init__` 方法概念】

- 语法

```
class 类名:
    def __init__(self, arg1, arg2...):
        函数体
```

#参数: `arg1, arg2...` 根据需要自己定义，一个类只能有一个构造方法，如果定义多个，后面的会覆盖前面的
#如果没有定义构造方法，系统会生成一个无参构造方法，如果自己定义了构造方法，则系统不会自动生成

```
class 类名:
    def __init__(self):
        pass
```

- 目的

构造方法用于初始化对象，可以在这个方法里面，初始化一些实例属性和方法

- 时机

每个对象实例化的时候，都会自动执行这个方法

- 返回值

无返回

2. 【`__init__` 方法实际应用】

- 初始化实例属性和方法

```
# class Dog(object):
#
#     def bark(self):
#         print("我是{}犬，我的名字是{}，我的年龄{}".format(self.kind, self.name, self.age))
#
#
# d = Dog() #实例化
# d.kind = '金毛'
# d.name = '小金'
# d.age = 8
#
```

```
# d.bark()
```

#2.通过__init__()来初始化实例属性和方法

```
class Dog(object):

    def __init__(self,kind,name,age):
        self.kind = kind
        self.name = name
        self.age = age
        self.hello()

    def bark(self):
        print("我是{}犬，我的名字是{}，我的年龄{}".format(self.kind,self.name,self.age))

    def hello(self):
        print("大家好！")

d = Dog('金毛','小金',8)
d.bark()
```

- 子类调用父类的构造方法

```
class Animal(object):

    def __init__(self):
        self.eyes = 2
        self.mouth = 1

class Dog(Animal):

    def __init__(self,kind,name,age):
        super().__init__() #初始化父类的构造方法
        self.kind = kind
        self.name = name
        self.age = age

    def bark(self):
        print("我是动物类，我有{}只眼睛，{}张嘴！".format(self.eyes,self.mouth))
        print("我是{}犬，我的名字是{}，年龄是{}".format(self.kind,self.name,self.age))

d = Dog('金毛','小金',8)
d.bark()
```

3.7 面向对象 `__new__` 方法（单例模式）

一、学习目标

1. `__new__` 方法概念
2. `__new__` 方法实际应用

二、知识点

1. 【`__new__` 方法概念】

- 语法

```
class 类名:
    def __new__(cls, arg1, arg2...):
        函数体

#如果不自己定义__new__方法，系统会自动生成如下的__new__方法
def __new__(cls, *args, **kwargs):
    return super().__new__(cls, *args, **kwargs)
```

- 目的

Python中存在于类中的构造方法 `__init__()` 负责将类实例化，而在 `__init__()` 执行之前，`__new__()` 负责制造这样的一个实例对象，以便 `__init__()` 去让该实例对象更加的丰富（为其添加属性等）。简单的说，`__new__()` 是用来创建实例对象的方法,为对象分配内存空间。

- 时机

在执行 `__init__()` 之前执行

- 返回值

返回当前类的实例对象

2. 【`__new__` 方法实际应用】

- 创建实例对象

```
#1.__new__方法执行在__init__方法之前
#2.__new__返回当前类的实例对象，通过调用父类__new__方法来实现的

class A(object):

    def __new__(cls, *args, **kwargs):
        print("这是一个__new__方法！")
        return super().__new__(cls, *args, **kwargs)

    def __init__(self):
        print("这是一个__init__ () 方法！")

    def hello(self):
        print("大家好！")

d = A()
print(d)
```

- 单例模式（多次实例化只创建一个实例）

事实上，当我们理解了 `__new__()` 方法后，我们还可以利用它来做一些其他有趣的事情，比如实现 设计模式中的 单例模式。

因为类每一次实例化后产生的过程都是通过 `__new__()` 来控制的，所以通过重写 `__new__()` 方法，我们可以很简单的实现单例模式。

```
#单例模式：多次实例化，只创建一个对象。
class A():
    __instance = None

    def __new__(cls, *args, **kwargs):
        if not cls.__instance:
            obj = super().__new__(cls, *args, **kwargs)
            cls.__instance = obj
            return cls.__instance

    def hello(self):
        print("大家好！")

a1 = A()
a1.age = 8
print(a1)
a2 = A()
print(a2)
print(a2.age)
```

第四章 Pyautogui 客户端自动化模块介绍

4.1 Pyautogui模块介绍及环境安装

一、学习目标

1. Pyautogui模块介绍
2. Pyautogui模块安装

二、知识点

1. 【Pyautogui模块介绍】

鼠标功能模拟：可以模拟人对鼠标的操作，如鼠标移动，拖拽，单击左/右键，双击，鼠标滑轮滚动等

键盘功能模拟：可以模拟人对键盘的操作，如输入文本，键盘按键，组合键输入等

图像定位功能：可以定位截图的按钮在当前屏幕中的位置，并返回x,y坐标

弹窗功能：可以编写一个简单弹窗

2. 【Pyautogui模块安装】

- 在线安装：（当前电脑有网）

cmd执行：

```
pip install pyautogui
```

- 离线安装：（当前电脑无网）

去联网的电脑先下载模块包：<https://pypi.org/>

解压后cmd执行：

```
python setup.py install
```

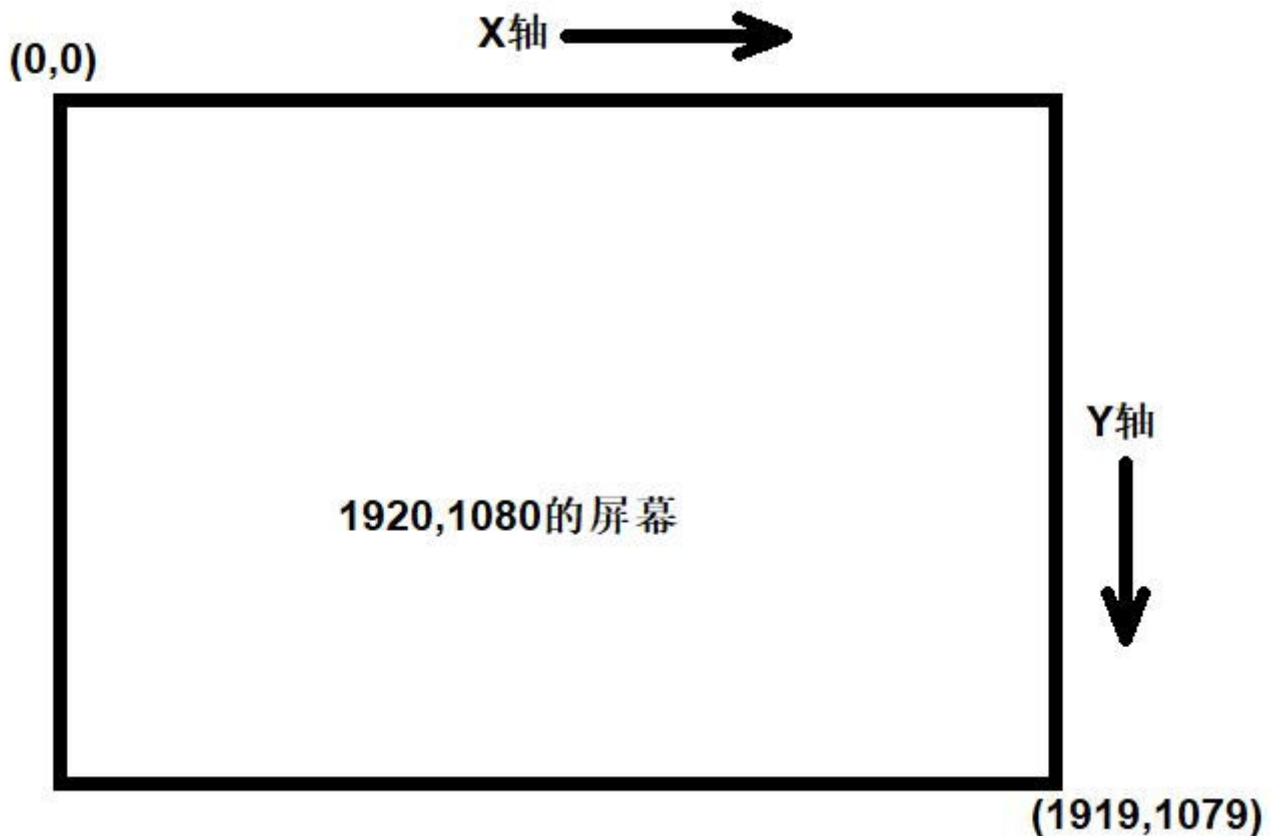
4.2 【鼠标操作】获取鼠标当前位置

一、学习目标

1. 获取鼠标当前位置

二、知识点

1. 【获取鼠标当前位置】



```
import pyautogui
import time

time.sleep(5)
#鼠标的当前位置
x,y = pyautogui.position() #返回x,y
print("鼠标位置的横坐标: {}".format(x))
print("鼠标位置的纵坐标: {}".format(y))
```

我们后面的课程要通过模拟鼠标行为要对位置坐标进行点击，所以我们提前写一个自动获取位置坐标的小工具：

```
import pyautogui
import os,time

try:
    while True:
        x,y = pyautogui.position()
        pos = "鼠标当前位置: " + str(x) + ',' + str(y)
        print(pos)
        time.sleep(0.2)
        os.system("cls")
except KeyboardInterrupt:
    print("end...")
```

4.3 【鼠标操作】模拟鼠标移动

一、学习目标

- 1.鼠标绝对位置移动
- 2.鼠标相对位置移动

二、知识点

1.【鼠标绝对位置移动】

- 语法

```
pyautogui.moveTo(x=None, y=None, duration=0.0)
```

- 参数

x：移动到的横坐标位置

y：移动到的纵坐标位置

duration：鼠标移动的快慢，浮点型，单位秒

- 返回值

无返回值

- 示例：

```
import pyautogui

#1. 鼠标的绝对位置移动
pyautogui.moveTo(x=239,y=107,duration=0.5)
```

2.【鼠标相对位置移动】

- 语法

```
pyautogui.moveRel(xoffset=None, yoffset=None, duration=0.0)
```


- 参数

xOffset：当前位置的横向偏移量（正数：向右偏移；负数：向左偏移；0不偏移）

yOffset：当前位置的纵向偏移量（正数：向下偏移；负数：向上偏移；0不偏移）

duration：鼠标移动的快慢，浮点型，单位秒

- 返回值

无返回值

- 示例

```
#2. 鼠标的相对位置移动
time.sleep(5)
# pyautogui.moveRel(xOffset=172, yOffset=0, duration=3)
# pyautogui.moveRel(172, 0, 0.5)
pyautogui.moveRel(172, 0)
```

4.4 【鼠标操作】模拟鼠标拖拽

一、学习目标

1.鼠标绝对位置拖拽

2.鼠标相对位置拖拽

二、知识点

1.【鼠标绝对位置拖拽】

- 语法

```
pyautogui.dragTo(x=None, y=None, duration=0.0, button='left')
```

- 参数

x：移动到的横坐标位置

y：移动到的纵坐标位置

duration：鼠标移动的快慢，浮点型，单位秒

button：使用鼠标的哪个键（"LEFT", "MIDDLE", "RIGHT", "PRIMARY", or "SECONDARY"）

- 返回值

无返回值

- 示例

```
import pyautogui
import time

#1. 模拟鼠标绝对位置拖拽
time.sleep(5)
# pyautogui.dragTo(x=431, y=267, duration=3, button='left')
pyautogui.dragTo(x=431, y=267, duration=1, button='left')
```

2. 【鼠标相对位置拖拽】

- 语法

```
pyautogui.dragRel(xoffset=None, yoffset=None, duration=0.0, button='left')
```

- 参数

xOffset：当前位置的横向偏移量（正数：向右偏移；负数：向左偏移；0不偏移）

yOffset：当前位置的纵向偏移量（正数：向下偏移；负数：向上偏移；0不偏移）

duration：鼠标移动的快慢，浮点型，单位秒

button：使用鼠标的哪个键（“LEFT”，“MIDDLE”，“RIGHT”，“PRIMARY”，or “SECONDARY”）

- 返回值

无返回值

- 示例

#2. 鼠标相对位置拖拽

```
pyautogui.dragRel(xOffset=-100, yOffset=0, duration=3, button='left')
```

4.5 【鼠标操作】模拟鼠标点击

一、学习目标

1.鼠标的点击

2.鼠标双击、三击

3.鼠标按下、松开

二、知识点

1. 【鼠标的点击】

- 语法：

```
pyautogui.click(x=None, y=None, clicks=1, interval=0.0, duration=0.0, button='left')
```

- 参数：

x：移动到的横坐标位置

y：移动到的纵坐标位置

clicks：点击次数

interval：点击间的耗时

duration：鼠标移动耗时

button：点击的按键（“LEFT”，“MIDDLE”，“RIGHT”，“PRIMARY”，or “SECONDARY”）

- 返回值：

无返回值

- 示例：

```
#1.模拟鼠标点击
# pyautogui.click(x=321, y=106, clicks=1, duration=3,button='left')
# pyautogui.click(321,106)
#当前位置点击
pyautogui.click()
```

2.【鼠标双击、三击】

- 语法：

```
pyautogui.doubleClick(x=None, y=None, interval=0.0, duration=0.0, button='left') #鼠标双击
pyautogui.tripleClick(x=None, y=None, interval=0.0, duration=0.0, button='left') #鼠标三击
```

- 参数：

x：移动到的横坐标位置

y：移动到的纵坐标位置

interval：点击间的耗时

duration：鼠标移动耗时

button：点击的按键（“LEFT”，“MIDDLE”，“RIGHT”，“PRIMARY”，or “SECONDARY”）

- 返回值：

无返回值

- 示例：

```
#2.模拟鼠标双击、三击
# pyautogui.doubleClick(321,106,interval=3)
# pyautogui.tripleClick(321,106,interval=3)
```

3.【鼠标按下、松开】

- 语法：

```
pyautogui.mouseDown(x=None, y=None, duration=0.0, button='left') #鼠标按下
pyautogui.mouseUp(x=None, y=None, duration=0.0, button='left') #鼠标松开
```

- 参数：

x：移动到的横坐标位置

y：移动到的纵坐标位置

duration：鼠标移动耗时

button：按下/松开的按键（“LEFT”，“MIDDLE”，“RIGHT”，“PRIMARY”，or “SECONDARY”）

- 返回值：

无返回值

- 示例：

```
#3.模拟鼠标按下、松开
pyautogui.mouseDown(321,106)
pyautogui.mouseUp(321,106)

# pyautogui.click()
```

4.6 【鼠标操作】模拟鼠标滑轮滚动

一、学习目标

1.模拟鼠标滑轮滚动

二、知识点

1.【模拟鼠标滑轮滚动】

- 语法：

```
pyautogui.scroll(clicks, x=None, y=None)
```

- 参数：

clicks：表示滚动的格数。正数则页面向上滚动，负数则向下滚动

x：移动到的横坐标位置

y：移动到的纵坐标位置

- 返回值：

无返回值

- 示例：

```
import pyautogui

#1.模拟鼠标滑轮的滚动
pyautogui.scroll(1000)
```

4.7 【鼠标操作】模拟鼠标无规则运动

一、学习目标

1.开始很慢，不断加速

2.开始很快，不断减速

3.开始和结束都快，中间比较慢

4.一步一徘徊前进

5.徘徊幅度更大，甚至超过起点和终点

二、知识点

1.【开始很慢，不断加速】

```
pyautogui.moveTo(100, 100, 2, pyautogui.easeInQuad)
```

2. 【开始很快，不断减速】

```
pyautogui.moveTo(100, 100, 2, pyautogui.easeOutQuad)
```

3. 【开始和结束都快，中间比较慢】

```
pyautogui.moveTo(100, 100, 2, pyautogui.easeInOutQuad)
```

4. 【一步一徘徊前进】

```
pyautogui.moveTo(100, 100, 2, pyautogui.easeInBounce)
```

5. 【徘徊幅度更大，甚至超过起点和终点】

```
pyautogui.moveTo(100, 100, 2, pyautogui.easeInElastic)
```

4.8 【键盘操作】模拟键盘输入

一、学习目标

- 1.不带中文的键盘输入
- 2.带中文的键盘输入

二、知识点

1. 【不带中文的键盘输入】

- 语法：

```
pyautogui.typewrite(message, interval=0.0)
pyautogui.write(message, interval=0.0) #在PyAutoGUI 1.0中，write()取代了typewrite()
```

- 参数：

message：键盘输入的文本（非中文）

interval：每个字符输入的间隔时间，单位秒

- 返回值：

无返回值

- 示例：

```
import pyautogui

#1.模拟键盘输入（非中文）
# pyautogui.typewrite('hello python!', interval=1)
pyautogui.write('hello python!')
```

2. 【带中文的键盘输入】

上面的方法只适用于不含中文的文本，如果输入文本中含中文则不会被输入，我们可以用pyperclip模块来实现。实现原理是讲要输入的字符先复制到电脑的剪切板中，然后再粘贴实现含中文文本的输入。

- 语法：

```
pyperclip.copy(message) #和键盘上的Ctrl+C作用一样，会把字符串传入到粘贴板；  
#pyperclip.paste() #和键盘上的Ctrl+V作用一样，会把粘贴板的内容粘贴出来。（有问题）  
pyautogui.hotkey('ctrl', 'v')
```

- 参数：

message：要输入的文本，可以含中文

- 返回值：

无返回值

- 示例：

```
#2. 模拟键盘输入（含中文）  
pyperclip.copy("你好！") #1. 复制文本到剪切板  
pyautogui.hotkey("ctrl", "v") #2. 粘贴
```

4.9 【键盘操作】模拟键盘按下、键盘松开

一、学习目标

1.模拟键盘完整的按下与松开动作

2.模拟键盘的按下或松开

二、知识点

1. 【模拟键盘完整的按下与松开动作】

- 语法：

```
pyautogui.press(keys, presses=1, interval=0.0)
```

- 参数：

keys：按键的名称,可以是列表

presses：重复按压的次数，默认1次

interval：每次按下之间的秒数，默认0s间隔

- 返回值：

无返回值

- 示例：

```
import pyautogui

#1.模拟键盘按下松开
# pyautogui.press('a', presses=3, interval=1)
# pyautogui.press('a')
pyautogui.press(["a","b","c"],presses=3, interval=1)
```

2.【模拟键盘的按下或松开】

- 语法：

```
pyautogui.keyDown(key) #按下
pyautogui.keyUp(key) #松开
```

- 参数：

key: 按键的名称

- 返回值：

无返回值

- 示例：

```
#2.模拟键盘的按下或松开
pyautogui.keyDown('a')
pyautogui.keyUp('a')

# pyautogui.press()
```

4.10 【键盘操作】模拟键盘组合键

一、学习目标

- 1.了解键盘按键的名称
- 2.模拟键盘组合键输入

二、知识点

1.【了解键盘按键的名称】

```
# pyautogui.KEYBOARD_KEYS数组中就是press(), keyDown(), keyUp()和hotkey()函数可以输入的按键名称
pyautogui.KEYBOARD_KEYS = ['\t', '\n', '\r', ' ', '!', '"', '#', '$', '%', '&', "'", '(', ')',
 '*', '+', ',', '-', '.',
 '/', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', ':', ';', '<', '=', '>', '?',
 '@',
 '[', '\\', ']', '^', '_', '`', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j',
 'k', 'l',
 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '{', '|', '}',
 '~',
 'accept', 'add', 'alt', 'altleft', 'altright', 'apps', 'backspace', 'browserback',
 'browserfavorites', 'browserforward', 'browserhome', 'browserrefresh',
 'browsersearch',
```

```
'browserstop', 'capslock', 'clear', 'convert', 'ctrl', 'ctrlleft', 'ctrlright',
'decimal',
'del', 'delete', 'divide', 'down', 'end', 'enter', 'esc', 'escape', 'execute', 'f1',
'f10',
'f11', 'f12', 'f13', 'f14', 'f15', 'f16', 'f17', 'f18', 'f19', 'f2', 'f20', 'f21',
'f22',
'f23', 'f24', 'f3', 'f4', 'f5', 'f6', 'f7', 'f8', 'f9', 'final', 'fn', 'hanguel',
'hangu1',
'hanja', 'help', 'home', 'insert', 'junja', 'kana', 'kanji', 'launchapp1',
'launchapp2',
'launchmail', 'launchmediaselect', 'left', 'modechange', 'multiply', 'nexttrack',
'nonconvert', 'num0', 'num1', 'num2', 'num3', 'num4', 'num5', 'num6', 'num7',
'num8', 'num9',
'numlock', 'pagedown', 'pageup', 'pause', 'pgdn', 'pgup', 'playpause', 'prevtrack',
'print',
'printscreen', 'prntscrn', 'prtsc', 'prtscr', 'return', 'right', 'scrolllock',
'select',
'separator', 'shift', 'shiftleft', 'shiftright', 'sleep', 'space', 'stop',
'subtract', 'tab',
'up', 'volumedown', 'volumemute', 'volumeup', 'win', 'winleft', 'winright', 'yen',
'command',
'option', 'optionleft', 'optionright']
```

2.【模拟键盘组合键输入】

- 语法：

```
pyautogui.hotkey(*args, **kwargs)
```

- 参数：

不定长参数：可以输入一个或多个按键名

- 返回值：

无返回值

- 示例：

```
import pyautogui

# #1. 模拟键盘的组合键
# #ctrl+a
# pyautogui.hotkey('ctrl', 'a')

# #win+r
# pyautogui.hotkey('win', 'r')
# pyautogui.press('enter')

#win+m
pyautogui.hotkey('win', 'm')
```


4.11 【图像定位】 屏幕截图

一、学习目标

1.全屏截图

2.区域截图

二、知识点

1.【全屏截图】

- 语法：

```
pyautogui.screenshot(pathName)
```

- 参数：

pathName：截图的保存路径+名字

- 返回值：

返回一个image对象

- 示例：

```
import pyautogui

#1.全屏截图
# pyautogui.screenshot('./img01.png')

img = pyautogui.screenshot()
img.save('./img02.png')
```

2.【区域截图】

- 语法：

```
pyautogui.screenshot(pathName, region)
```

- 参数：

pathName：截图的保存路径+名字

region：截取的区域，元组类型（左上顶点xy坐标，右下顶点xy坐标）

- 返回值：

返回一个image对象

- 示例：

```
#2.区域截图
pyautogui.screenshot('./img03.png', region=(33, 88, 505, 657))
```

4.12 【图像定位】获得图片在当前的屏幕上中心坐标

一、学习目标

- 1.获取截图在当前屏幕的位置
- 2.通过位置或者中心坐标
- 3.一步获取图片中心坐标
- 4.匹配屏幕所有的目标图片

二、知识点

1.【获取截图在当前屏幕的位置】

- 语法：

```
pyautogui.locateOnScreen(pathName)
```

- 参数：

pathName：截图图片的路径+名字

- 返回值：

返回的是一个位置元组(top, left, width, height)，如果匹配不到则返回None

- 示例：

```
#1. 获取截图在当前屏幕位置区域
res = pyautogui.locateOnScreen('./test01.png')
print(res)
```

2.【通过位置或者中心坐标】

- 语法：

```
pyautogui.center((top, left, width, height))
```

- 参数：

传入位置元组：(top, left, width, height)

- 返回值：

返回区域的中心坐标 (x,y)

- 示例：

```
#2. 获取区域的中心坐标
x,y = pyautogui.center(res)
print(x,y)

#3. 模拟点击
pyautogui.click(x,y)
```

3.【一步获取图片中心坐标】

- 语法：

```
pyautogui.locateCenterOnScreen(pathName)
```

- 参数：

pathName：截图图片的路径+名字

- 返回值：

直接返回图片的xy坐标，没有则返回None

- 示例：

```
#方式二：一步获取中心坐标
x,y = pyautogui.locateCenterOnScreen('./test01.png')
print(x,y)
pyautogui.click(x,y)
```

4.【匹配屏幕所有的目标图片】

- 语法：

```
pyautogui.locateAllOnScreen(pathName)
```

- 参数：

pathName：截图图片的路径+名字

- 返回值：

返回的是多个位置元组(top, left, width, height)，如果匹配不到则返回None

- 示例：

```
#匹配屏幕所有的目标图片
res = pyautogui.locateAllOnScreen('./test02.png')
print(res)
for i in res:
    print(i)
```

4.13 【图像定位】设置图像识别的超时时间、信任度、灰度

一、学习目标

- 1.设置超时时间
- 2.设置信任度
- 3.灰度匹配

二、知识点

1.【设置超时时间】

有时候当我们操作完上一个按钮后，需要等待一会才会加载新的界面，这种场景需要设置超时时间等待上一步操作完成。

- 语法：

```
pyautogui.locateOnScreen(pathName,minSearchTime=0)
```

- 参数：

pathName：截图图片的路径+名字

minSearchTime：默认为0秒

- 示例：

```
#1. 设置超时时间
res = pyautogui.locateOnScreen('./test01.png',minSearchTime=5)
print(res)
```

2. 【设置信任度】

Pyautogui默认是100%像素匹配，这样稍微有变化就会匹配不到，其实我们也可以设置信任度，只要像素匹配在这个信任度内则视为匹配成功。

注：在这之前需要安装opencv模块,否则使用这个参数会报错

```
pip install -i https://pypi.douban.com/simple/ opencv-python
```

- 语法：

```
pyautogui.locateOnScreen(pathName,confidence=0.95) #信任度达到95%则视为匹配成功
```

- 参数：

pathName：截图图片的路径+名字

confidence：设置大于0，小于1的数值

- 示例：

```
#2. 设置信任度
res1 = pyautogui.locateOnScreen('./test01.png',minSearchTime=5,confidence=0.5)
print(res1)
```

3. 【灰度匹配】

如果我们在图像定位的时候不需要考虑图片的颜色，只需要考虑内容，则可以设置灰度匹配参数为True。这样还能节省图片匹配的时间。

- 语法：

```
pyautogui.locateOnScreen(pathName,grayscale=True)
```

- 参数：

pathName：截图图片的路径+名字

grayscale：True为开启，False为关闭

- 示例：

```
#3.灰度匹配
```

```
res2 = pyautogui.locateOnScreen('./test01.png',minSearchTime=5,grayscale=True)  
print(res2)
```

4.14 【实战练习】简单实现QQ客户端登录自动化脚本

一、学习目标

1.脚本实践：QQ客户端登录

二、知识点

1.【脚本实践：QQ客户端登录】

我们写脚本的目的是模拟人工操作，所有我们首先需要梳理人工操作流程。

- 操作步骤：

步骤一：双击QQ图标(鼠标点击+图像识别)

步骤二：输入用户名（键盘操作）

步骤三：输入密码（键盘操作）

步骤四：点击登录（鼠标点击+图像识别）

- 代码实现：

```
#QQ客户端登录自动化脚本  
import pyautogui  
import time  
pyautogui.hotkey('win','m')  
time.sleep(2)  
# 步骤一：双击QQ图标(鼠标点击+图像识别)  
# (1)定位图片位置  
box1 = pyautogui.locateOnScreen(r'C:\Users\Admin\Desktop\stuCode\code0414\test01.png')  
print(box1)  
x,y = pyautogui.center(box1)  
print(x,y)  
# (2)模拟鼠标点击  
pyautogui.click(x,y,clicks=2,duration=0.3)  
  
# 步骤二：输入用户名（键盘操作）  
# (1)点击输入框  
box2 =  
pyautogui.locateOnScreen(r'C:\Users\Admin\Desktop\stuCode\code0414\test02.png',minSearchTime=5)  
x,y = pyautogui.center(box2)  
pyautogui.moveTo(x,y)  
pyautogui.moveRel(-100,0)  
pyautogui.click()  
# (2)模拟键盘输入  
pyautogui.write('2549583718',interval=0.2)  
  
# 步骤三：输入密码（键盘操作）
```

```
#(1)点击输入框
box3 = pyautogui.locateOnScreen(r'C:\Users\Admin\Desktop\stuCode\code0414\test03.png')
x,y = pyautogui.center(box3)
pyautogui.moveTo(x,y)
pyautogui.moveRel(-100,0)
pyautogui.click()
#(2)模拟键盘输入
pyautogui.write('xc12345678',interval=0.2)
# 步骤四：点击登录（鼠标点击+图像识别）
#(1)定位按钮位置
box4 = pyautogui.locateOnScreen(r'C:\Users\Admin\Desktop\stuCode\code0414\test04.png')
x,y = pyautogui.center(box4)
#(2)模拟鼠标点击
pyautogui.click(x,y)
```

第五章 Selenium Web自动化模块介绍

5.1 Selenium模块介绍及环境安装

一、学习目标

1.selenium模块介绍

2.selenium模块安装

二、知识点

1. 【selenium模块介绍】

- 浏览器操作模拟：如打开请求地址，浏览器最大化，关闭浏览器等操作
- 支持八种元素定位方式：id、name、class name、tag name、link text、patial link text、xpath、css selector
- 支持各种对元素的操作：点击、输入、获取属性信息等
- 支持鼠标和键盘操作
- 支持滚动条的操作
- 支持弹出框操作
- 支持对iframe的切换

2. 【selenium模块安装】

- 在线安装：（当前电脑有网）

cmd执行：

```
pip install selenium==3.141.0
```

- 离线安装：（当前电脑无网）

去联网的电脑先下载模块包：<https://pypi.org/>

解压后cmd执行：

```
python setup.py install
```

- 浏览器驱动下载（驱动的版本==浏览器的版本）

谷歌：<https://chromedriver.storage.googleapis.com/index.html>

火狐：<https://github.com/mozilla/geckodriver/releases>

5.2【浏览器操作】模拟浏览器相关操作

一、学习目标

- 1.如何创建浏览器对象
- 2.关闭浏览器
- 3.浏览器的其他方法

二、知识点

1.【如何创建浏览器对象】

浏览器对象你可以把它抽象的理解为一个浏览器，先有浏览器我们才能做后续的操作。

```
from selenium import webdriver

#1.创建浏览器对象
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\stuCode\code0515\chromedriver.exe')
print(driver)
```

2.【关闭浏览器】

```
#2.关闭浏览器
# driver.quit()
driver.close() #关闭页签
```

3.【浏览器的其他方法】

```
import time

from selenium import webdriver

#1.创建浏览器对象
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\stuCode\code0515\chromedriver.exe')
print(driver)

time.sleep(3)

#2.关闭浏览器
# driver.quit()
# driver.close() #关闭页签

#3.浏览器的其他方法
#（1）浏览器最大化
driver.maximize_window()

#（2）获取浏览器尺寸
s = driver.get_window_size()
```

```
print(s)

#(3) 设置当前浏览器尺寸
driver.set_window_size(400,400)

#(4) 获取浏览器位置
p = driver.get_window_position()
print(p)

#(5) 设置浏览器位置
driver.set_window_position(500,10)
```

5.3 【页面操作】模拟页面相关操作

一、学习目标

1. 打开请求地址
2. driver的常用属性
3. 页面的其他方法

二、知识点

1. 【打开请求地址】

```
from selenium import webdriver

driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#1. 如何打开网站
driver.get('https://www.baidu.com/')
```

2. 【driver的常用属性】

我们经常用到的driver对象的属性：title、current_url、page_source

```
#2. 获取当前页面的标题
t = driver.title
print(t)

#3. 获取当前页面的地址
u = driver.current_url
print(u)

#4. 当前页面的源码
p = driver.page_source
print(p)
```

3. 【页面的其他方法】

```
#其他方法
#1. 页面刷新
time.sleep(3)
```



```
# driver.refresh()
#
# #2. 页面回退
# time.sleep(2)
# driver.back()
#
# #3. 页面的前进
# time.sleep(2)
# driver.forward()

#4. 页面截图
# 方式一
data = driver.get_screenshot_as_png()
with open('baidu.png', 'wb') as f:
    f.write(data)

# 方式二
driver.get_screenshot_as_file('baidu2.png')
```

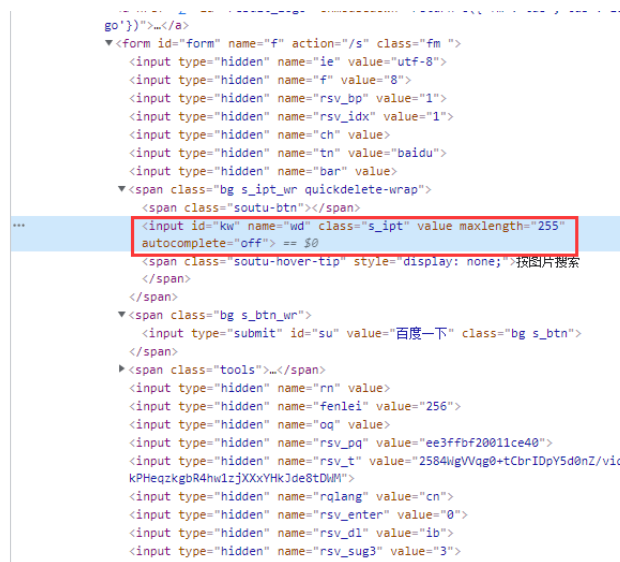
5.4 【元素的定位】id属性定位

一、学习目标

1. 通过id属性定位元素

二、知识点

1. 【通过id属性定位元素】



• 语法:

```
driver.find_element_by_id('value')
```

• 参数:

传入标签属性中id的值

• 返回值:

返回一个元素对象

- 代码示例：

```
from selenium import webdriver

#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#定位元素
e11 = driver.find_element_by_id('kw')
print(e11)
```

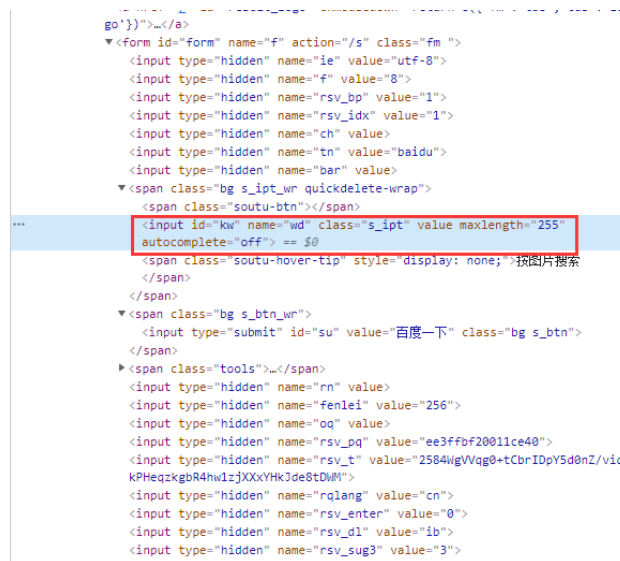
5.5 【元素的定位】name属性定位

一、学习目标

1.通过name属性定位元素

二、知识点

1. 【通过name属性定位元素】



- 语法：

```
driver.find_element_by_name('value')
```

- 参数：

传入标签属性中name的值

- 返回值：

返回元素对象

- 代码示例：

```
from selenium import webdriver

#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#定位元素
el2 = driver.find_element_by_name('wd')
print(el2)
```

5.6 【元素的定位】class属性定位

一、学习目标

1.通过class属性定位元素

二、知识点

1.【通过class属性定位元素】



```
... go'}}"></a>
▼<form id="form" name="f" action="/s" class="fm">
  <input type="hidden" name="ie" value="utf-8">
  <input type="hidden" name="f" value="8">
  <input type="hidden" name="rsv_bp" value="1">
  <input type="hidden" name="rsv_idx" value="1">
  <input type="hidden" name="ch" value="">
  <input type="hidden" name="tn" value="baidu">
  <input type="hidden" name="bar" value="">
  <span class="bg_s_ipt_wr quickdelete-wrap">
    <input id="kw" name="wd" class="s_ipt" value maxlength="255"
    autocomplete="off"> == $0
    <span class="soutu-hover-tip" style="display: none;">找图片搜索
  </span>
  </span>
  <span class="bg_s_btn_wr">
    <input type="submit" id="su" value="百度一下" class="bg_s_btn">
  </span>
  <span class="tools"></span>
  <input type="hidden" name="rn" value="">
  <input type="hidden" name="fenlei" value="256">
  <input type="hidden" name="oq" value="">
  <input type="hidden" name="rsv_pq" value="ee3ffb20011ce40">
  <input type="hidden" name="rsv_t" value="2584WgVqg0+tCbrIDpY5d0nZ/vi
  kPHeqzkgr4hw1zjXXxYHk7de8tDWM">
  <input type="hidden" name="rqlang" value="cn">
  <input type="hidden" name="rsv_enter" value="0">
  <input type="hidden" name="rsv_d1" value="ib">
  <input type="hidden" name="rsv_sug3" value="3">
  ...
```

• 语法:

```
driver.find_element_by_class_name('value')
```

• 参数:

传入标签属性中class的值（必须保class值唯一）

• 返回值:

返回元素对象

• 代码示例:

```
from selenium import webdriver

#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#定位元素
el3 = driver.find_element_by_class_name('s_ipt')
print(el3)
```

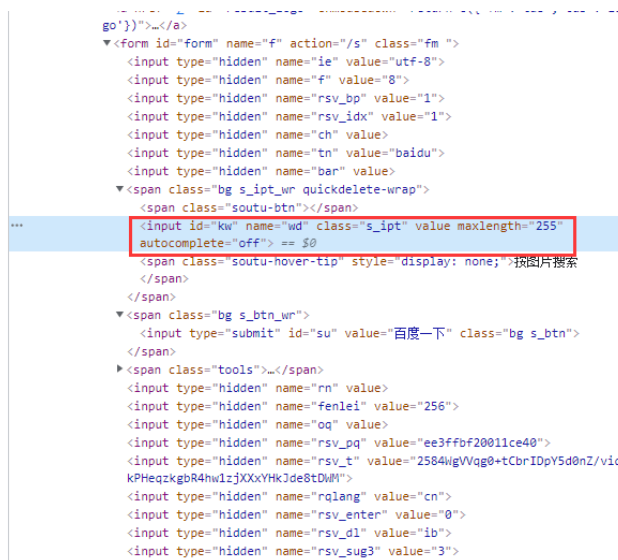
5.7【元素的定位】标签名定位

一、学习目标

1.通过标签名定位元素

二、知识点

1.【通过标签名定位元素】



• 语法：

```
driver.find_element_by_tag_name('tagName')
```

• 参数：

传入元素的标签名（必须保证标签名唯一）

• 返回值：

返回元素对象

• 代码示例：

```
from selenium import webdriver

#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#定位元素：标签名
e14 = driver.find_element_by_tag_name('form')
print(e14)
```

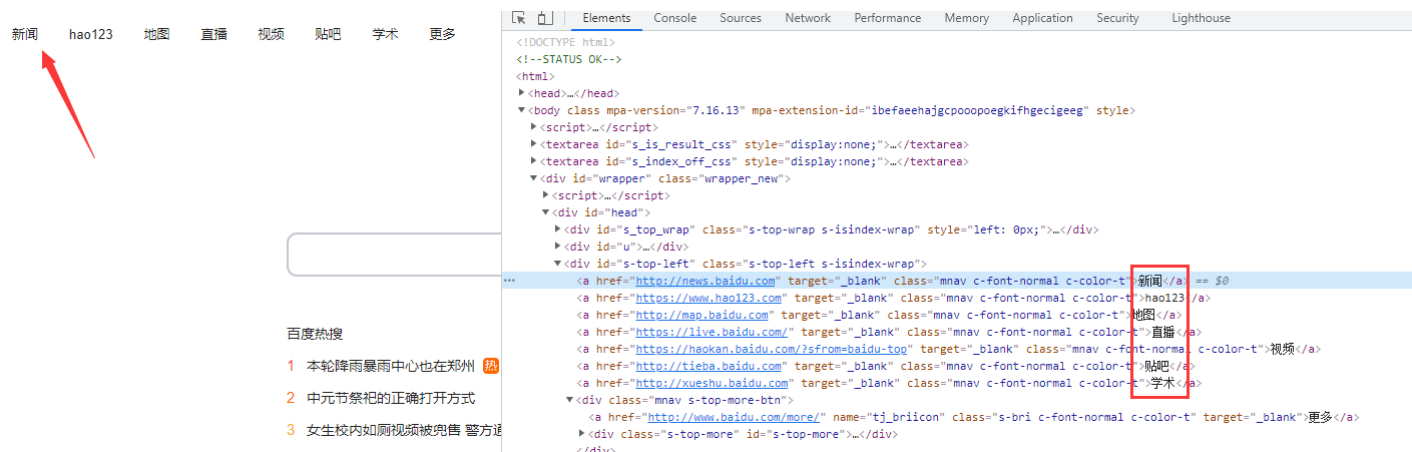
5.8【元素的定位】链接文本定位

一、学习目标

1.通过链接文本定位

二、知识点

1.【通过链接文本定位】



说明：如上图，标签的><之间存在文本，而且是**标签**，这种元素则可以通过链接文本定位的方式定位。

- 语法：

```
driver.find_element_by_link_text(text)
```

- 参数：

传入标签文本

- 返回值：

返回元素对象

- 代码示例：

```
from selenium import webdriver

#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#定位元素：链接文本
e15 = driver.find_element_by_link_text('新闻')
print(e15)
```

5.9【元素的定位】部分链接文本定位

一、学习目标

1.通过部分链接文本定位

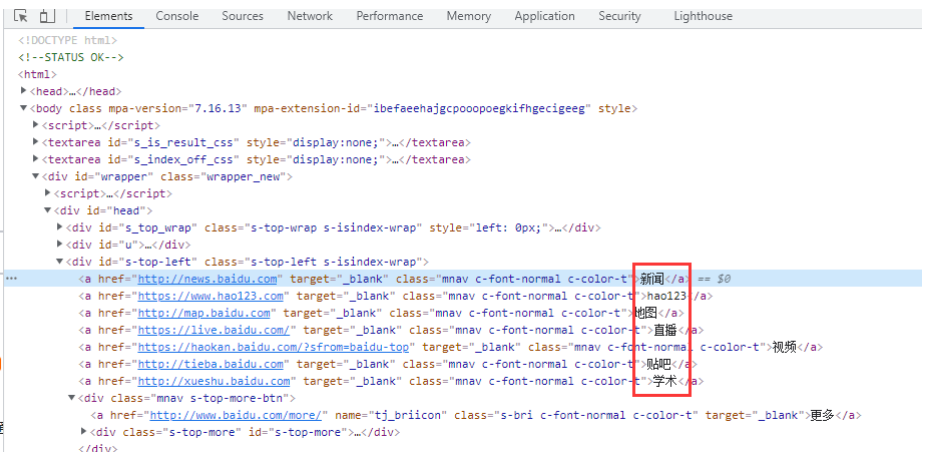
二、知识点

1.【通过部分链接文本定位】

新闻 hao123 地图 直播 视频 贴吧 学术 更多

百度热搜

- 1 本轮降雨暴雨中心也在郑州
- 2 中元节祭祀的正确打开方式
- 3 女生校内如厕视频被兜售 警方送



• 语法：

```
driver.find_element_by_partial_link_text(text)
```

• 参数：

标签文本的部分（也可以全部）

• 返回值：

返回元素对象

• 代码示例：

```
from selenium import webdriver

#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#定位元素：部分链接文本
el6 = driver.find_element_by_partial_link_text('hao123')
print(el6)
```

5.10【元素的定位】css选择器定位

一、学习目标

- 1.css结合基本属性定位 (id,class,标签)
- 2.css结合其他属性定位
- 3.css标签结合其他属性定位
- 4.css层级定位
- 5.css索引定位
- 6.css模糊匹配

二、知识点

1.【css结合基本属性定位 (id,class,标签)】



```
<input type="hidden" name="ie" value="utf-8">
<input type="hidden" name="f" value="8">
<input type="hidden" name="rsv_bp" value="1">
<input type="hidden" name="rsv_idx" value="1">
<input type="hidden" name="ch" value="
<input type="hidden" name="tn" value="baidu">
<input type="hidden" name="bar" value=
▼ <span class="bg s_ipt_wr quickdelete-wrap">
  <span class="soutu-btn"></span>
  ... <input id="kw" name="wd" class="s_ipt" value maxlength="255" autocomplete="off"> == $0
  <span class="soutu-hover-tip" style="display: none;">按图片搜索</span>
</span>
  > <span class="bg s_btn_wr"></span>
  > <span class="tools"></span>
```

前面几节课讲了直接通过标签的基本属性定位，同样，css选择器也支持基本属性定位方式。

• 语法：

```
driver.find_element_by_css_selector("#id值") #css+id定位
driver.find_element_by_css_selector(".class值") #css+class定位
driver.find_element_by_css_selector("标签名") #css+标签定位
```

• 参数说明：

#：井号代表id (类似与id=id值)

.：点代表class(类似与class=class值)

• 返回值：

返回元素对象

• 代码示例：

#1.css结合基本属性定位

```
e11 = driver.find_element_by_css_selector('#kw') #css+id定位
print(e11)
e12 = driver.find_element_by_css_selector('.s_ipt') #css+class定位
print(e12)
e13 = driver.find_element_by_css_selector('form')
print(e13)
```

2. 【css结合其他属性定位】

- 语法：

```
driver.find_element_by_css_selector('[属性名=属性值]') #css+其他属性
driver.find_element_by_css_selector('[属性名1=属性值1][属性名2=属性值2]') #css+多个其他属性
```

- 参数说明：

[]：用方括号来指定属性

- 返回值：

返回元素对象

- 代码示例：

#2.css结合其他属性定位

```
e14 = driver.find_element_by_css_selector("[name='wd'][id='kw']")
print(e14)
```

3. 【css标签结合其他属性定位】

同样，我们还可以指定标签的属性来定位。

- 语法：

```
driver.find_element_by_css_selector('标签名#id值') #标签+id属性定位
driver.find_element_by_css_selector('标签名.class值') #标签+class属性定位
driver.find_element_by_css_selector('标签名[属性名=属性值]') #标签+其他属性定位
```

- 参数说明：

标签名后面直接跟属性

- 返回值：

返回元素对象

- 代码示例：

#3.css标签结合属性定位

```
e15 = driver.find_element_by_css_selector('input#kw')
print(e15)
e16 = driver.find_element_by_css_selector('input.s_ipt')
print(e16)
e17 = driver.find_element_by_css_selector("input[name='wd']")
print(e17)
```


4. 【css层级定位】

有时候会碰到这种场景：当我们定位某个元素时没有简洁的标签属性，我们可以先定位到它的上一级或上上一级...，然后再通过层级关系到当前要定位的元素位置。

- 语法：

```
driver.find_element_by_css_selector('标签1>标签2')
```

- 参数说明：

标签1>标签2: 表示父级定位到子级

- 返回值：

返回元素对象

- 代码示例：

#4.css层级关系定位

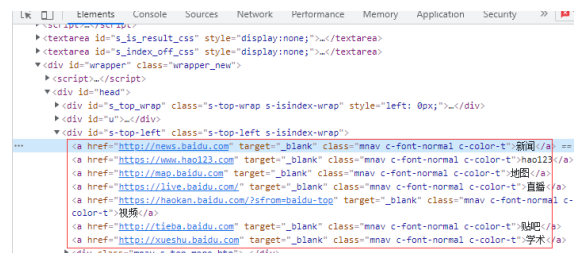
```
e18 = driver.find_element_by_css_selector("form[id='form']>span>input")
print(e18)
```

5. 【css索引定位】

新闻 hao123 地图 直播 视频 贴吧 学术 更多



百度一下



如上图所示，我们如果想要定位“新闻”按钮，发现同级路径下有6个与他一样的a标签，像这种有多个相同标签名的元素，可以使用索引定位。通过索引来指定要定位第几个a标签。

注：与python列表索引的概念不同，此处的标签索引是从1开始；python列表的索引是从0开始。

- 语法：

```
driver.find_element_by_css_selector('标签:nth-child(n)')          #正着数第n个标签
driver.find_element_by_css_selector('标签:nth-last-child(n)')     #倒着数第n个标签
```

- 参数说明：

n: 指代索引，也就是第几个

- 返回值：

返回元素对象

- 代码示例：

#5.css索引定位

```
e19 = driver.find_element_by_css_selector("div[id='s-top-left']>a:nth-child(3)")
print(e19.text)
e110 = driver.find_element_by_css_selector("div[id='s-top-left']>a:nth-last-child(5)")
print(e110.text)
```

6. 【css模糊匹配】

模糊匹配适合属性值太长，我们想要通过属性值的部分内容来匹配元素。



```

<input type="hidden" name="tn" value="baidu">
<input type="hidden" name="bar" value">
▼<span class="bg s_ipt_wr quickdelete-wrap">
  <span class="soutu-btn"></span>
  <input id="kw" name="wd" class="s_ipt" value maxlength="255"
    autocomplete="off">
  <span class="soutu-hover-tip" style="display: none;">按图片搜索</span>
</span>
▼<span class="bg s_btn_wr">
  <input type="submit" id="su" value="百度一下" class="bg s_btn" == $0
</span>

```

• 语法：

#1. 属性值由多个空格隔开，匹配其中一个值的方法

```
driver.find_element_by_css_selector("[属性名~='部分属性值']")
```

#2. 匹配字符串开头

```
driver.find_element_by_css_selector("[属性名^='属性值开头']")
```

#3. 匹配字符串结尾

```
driver.find_element_by_css_selector("[属性名$='属性值结尾']")
```

• 参数说明：

~：匹配部分

^：匹配开头

\$：匹配结尾

• 返回值：

返回元素对象

• 代码示例：

#6.css模糊匹配

```
e111 = driver.find_element_by_css_selector("input[class~='s_btn']")
```

```
print(e111)
```

```
e112 = driver.find_element_by_css_selector("input[class^='bg']")
```

```
print(e112)
```

```
e113 = driver.find_element_by_css_selector("input[class$='s_btn']")
```

```
print(e113)
```

5.11 【元素的定位】xpath表达式定位

一、学习目标

1.xpath结合属性定位

2.xpath文本定位

3.xpath层级定位

4.xpath索引定位

5.xpath模糊匹配

二、知识点

符号说明：

XPath 使用路径表达式在 XML 文档中选取节点。节点是通过沿着路径或者 step 来选取的。下面列出了最有用的路径表达式：

表达式	描述
/	从根节点选取（取子节点）。
//	从匹配选择的当前节点选择文档中的节点，而不考虑它们的位置（取子孙节点）。
.	选取当前节点。
..	选取当前节点的父节点。
@	选取属性。

1. 【xpath结合属性定位】



```

<input type="hidden" name="rsv_idx" value="1">
<input type="hidden" name="ch" value="">
<input type="hidden" name="tn" value="baidu">
<input type="hidden" name="bar" value="">
<span class="bg_s_lpt_wr quickdelete-wrap">
  <span class="soutu_btn" style="display: block;"></span>
  <input id="kw" name="wd" class="s_lpt" value="maxlength=255" autocomplete="off" == $0
  <span class="soutu-hover-tip" style="display: none;">按图片搜索</span>
</span>
<span class="bg_s_btn_wr"></span>
<span class="tools"></span>
<input type="hidden" name="rn" value="">
<input type="hidden" name="fenlei" value="256">
<input type="hidden" name="oq" value="">
<input type="hidden" name="rsv_pq" value="0071054b002a515a">
  
```

• 语法：

```

driver.find_element_by_xpath("//*[id='kw']")           #单个属性定位
driver.find_element_by_xpath("//*[id='kw' and @name='wd']") #多个属性定位
  
```

• 参数：

- [] ： 表示要根据属性找元素
- @ ： 后边跟属性的key,表示要通过哪个属性定位

• 返回值：

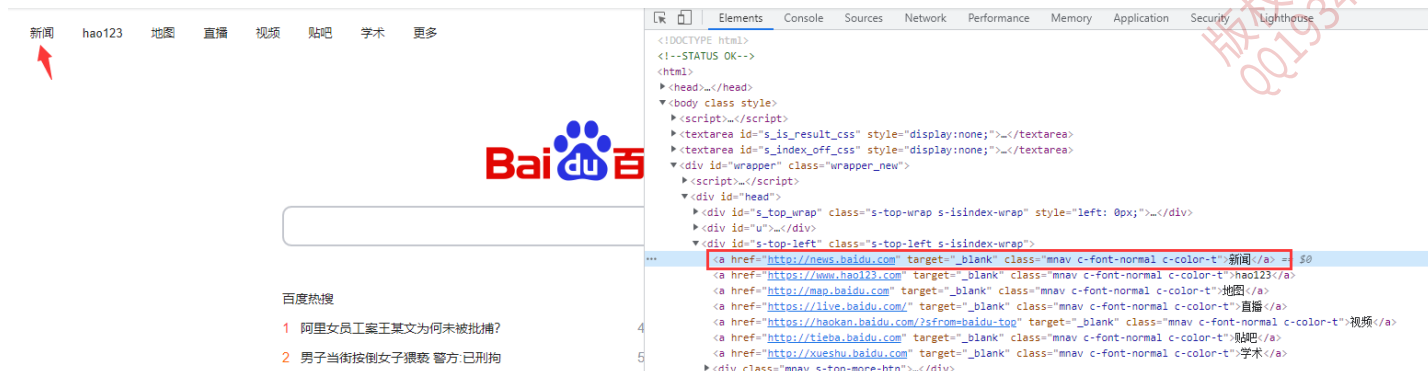
返回元素对象

• 代码示例：

```

#1.xpath结合基本属性定位
e11 = driver.find_element_by_xpath("//*[id='kw']")
print(e11)
e12 = driver.find_element_by_xpath("//*[id='kw' and @name='wd']")
print(e12)
  
```

2. 【xpath文本定位】



说明：标签文本是><直接的字符串，他不是键值对！！

- 语法：

```
driver.find_element_by_xpath("//*[a[text()='新闻']]")
```

- 参数：

[]：表示要根据属性找元素

text()：获取文本（注意没有指定属性@）

- 返回值：

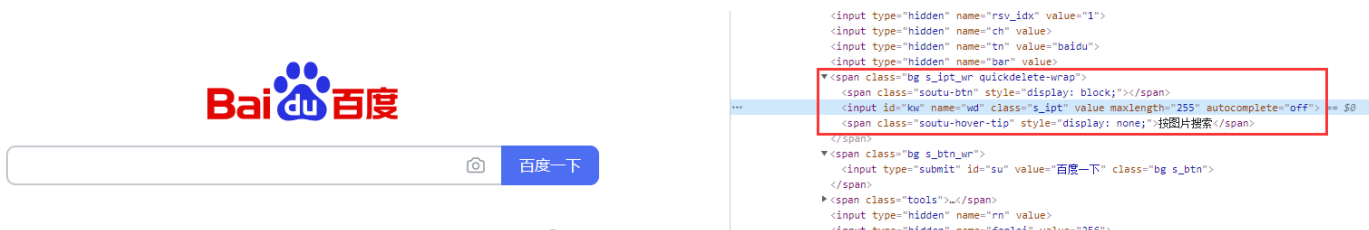
返回元素对象

- 代码示例：

```
#2.xpath文本定位
e13 = driver.find_element_by_xpath("//*[a[text()='新闻']]")
print(e13)
```

3. [xpath层级定位]

我们在做项目的时候，需要定位的元素没有像id,name,class这种基本属性，但是相邻的标签中有，我们可以先定位到相邻标签，然后通过层级关系来定位我们要定位的元素。



- 语法：

```
element_input1 = driver.find_element_by_xpath("//*[form[@id='form']/span/input")
#由上到下的层级关系
element_input2 = driver.find_element_by_xpath("//*[input[@class='s_ipt']/parent::span]")
#父子元素定位
element_input3 = driver.find_element_by_xpath("//*[input[@class='s_ipt']/preceding-sibling::span]") #哥哥元素定位
element_input4 = driver.find_element_by_xpath("//*[input[@class='s_ipt']/following-sibling::span]") #弟弟元素定位
```

- 参数：

parent::span : 向父级定位, 父级的标签名为span

preceding-sibling::span : 向哥哥级定位, 哥哥级的标签名为span

following-sibling::span : 向弟弟级定位, 弟弟级的标签名为span

- 返回值:

返回元素对象

- 代码示例:

#3.xpath层级定位

```
e14 = driver.find_element_by_xpath(".*//form[@id='form']/span/input") #由上到下
print(e14)
e15 = driver.find_element_by_xpath(".*//input[@class='s_ipt']/parent::span") #父级定位
print(e15)
e16 = driver.find_element_by_xpath(".*//input[@name='wd']/preceding-sibling::span") #哥哥定位
print(e16)
e17 = driver.find_element_by_xpath(".*//input[@name='wd']/following-sibling::span") #弟弟定位
print(e17)
```

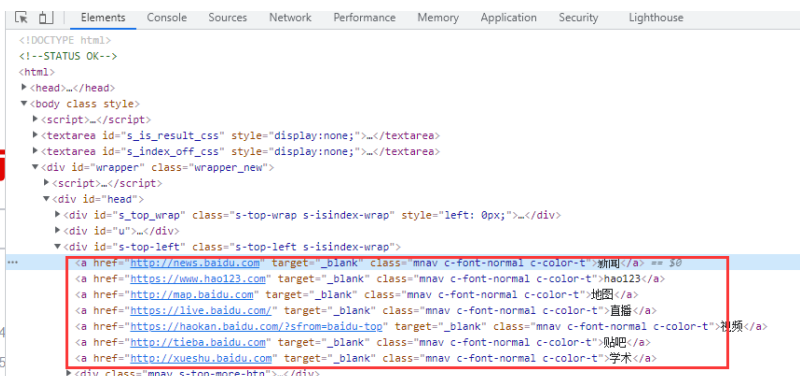
4. [xpath索引定位]

新闻 hao123 地图 直播 视频 贴吧 学术 更多



百度热搜

- 1 阿里女员工案王英文为何未被批捕?
- 2 男子当街按倒女子猥亵 警方:已刑拘



- 语法:

```
driver.find_element_by_xpath(".*//div[@id='s-top-left']/a[1]") #根据索引选择
标签
driver.find_element_by_xpath(".*//div[@id='s-top-left']/a[last()]") #最后一个标签
driver.find_element_by_xpath(".*//div[@id='s-top-left']/a[last()-1]") #倒数第二个标
签
```

- 参数:

1 : 指同名标签的第一个标签

last() : 指同名标签的最后一个标签

last()-1 : 指同名标签的倒数第二个标签

- 返回值:

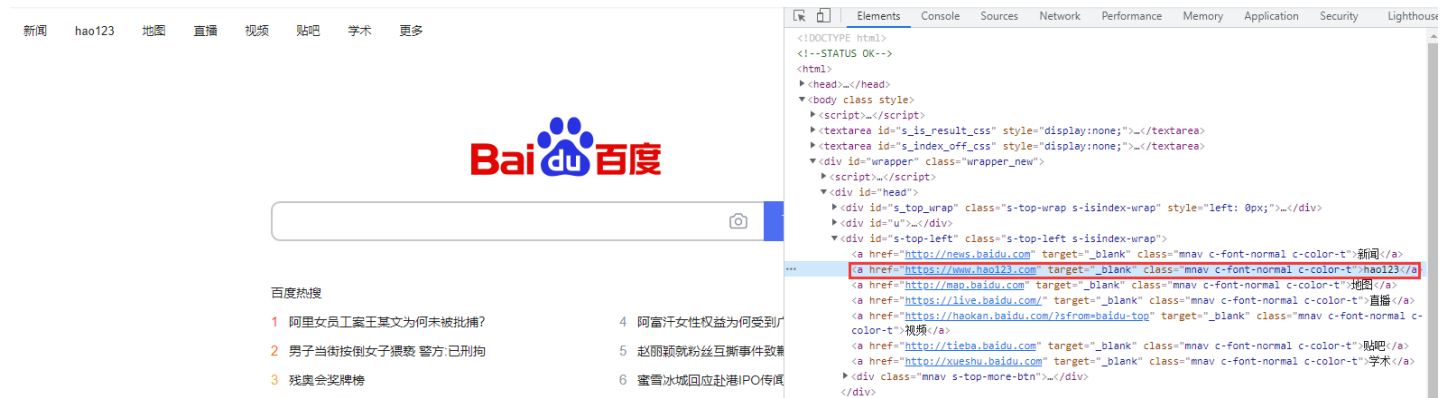
返回元素对象

- 代码示例:

#4.xpath索引定位

```
e18 = driver.find_element_by_xpath("//*[div[@id='s-top-left']/a[1]]") #索引数值定位同级标签
print(e18)
e19 = driver.find_element_by_xpath("//*[div[@id='s-top-left']/a[last()]]") #倒数第一个同级标签
print(e19.text)
e110 = driver.find_element_by_xpath("//*[div[@id='s-top-left']/a[last()-1]]") #倒是第二个
print(e110.text)
```

5. 【xpath模糊匹配】



• 语法:

```
driver.find_element_by_xpath("//*[span[starts-with(@class, 'soutu-hover')]]") #匹配开头
driver.find_element_by_xpath("//*[span[ends-with(@class, 'hover-tip')]]") #匹配结尾,注: ends-
with是xpath2.0的语法,可能你的浏览器还只支持1.0的语法
driver.find_element_by_xpath("//*[span[contains(text(), '图片')]]") #包含匹配
```

• 参数:

starts-with(): 匹配开头部分字符

ends-with(): 匹配结尾部分字符

contains(): 包含匹配部分字符

• 返回值:

返回元素对象

• 代码示例:

#5.xpath模糊匹配

```
e111 = driver.find_element_by_xpath("//*[input[starts-with(@value, '百度')]]") #开头匹配
print(e111)
e112 = driver.find_element_by_xpath("//*[input[contains(@value, '度一')]]") #包含匹配
print(e112)
```

5.12 【元素操作】模拟操作页面元素

一、学习目标

1.按钮点击操作

2.文本框操作

3.获取元素文本

4.获取元素属性

二、知识点

1.【按钮点击操作】

- 语法：

```
element.click() #对象.方法
```

- 参数：

无

- 返回值：

无

- 代码示例：

```
from selenium import webdriver

#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#元素定位，返回元素对象
el1 = driver.find_element_by_xpath("//*[id='s-top-left']/a[1]")
print(el1)

#模拟操作
el1.click()
```

2.【文本框操作】

- 语法：

```
element.clear() #清空文本框
element.send_keys('python') #输入
```

- 参数：

传入要输入的内容

- 返回值：

无

- 代码示例：

```
import time

from selenium import webdriver

#创建浏览器
```

```
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')

#定位元素
el1 = driver.find_element_by_id("kw")

#操作元素
el1.send_keys("python")
time.sleep(3)
el1.clear()
```

3. 【获取元素文本】

- 语法：

```
element.text    #text不是方法，不加括号
```

- 参数：

无

- 返回值：

返回标签文本字符串

- 代码示例：

```
#元素定位，返回元素对象
el1 = driver.find_element_by_xpath("//*[div[@id='s-top-left']/a[1]")
print(el1)

#模拟操作
# el1.click()
a = el1.text
print(a)
```

4. 【获取元素属性】

- 语法：

```
element.get_attribute(属性名)
```

- 参数：

传入要获取属性的名字

- 返回值：

属性名对应的值

- 代码示例：


```
#元素定位，返回元素对象
e11 = driver.find_element_by_xpath("//*[id='s-top-left']/a[1]")
print(e11)

#模拟操作
# e11.click()
# a = e11.text
# print(a)

b = e11.get_attribute("href")
print(b)
```

5.13 【鼠标和键盘操作】模拟鼠标操作页面元素（了解）

一、学习目标

- 1.鼠标点击
- 2.鼠标拖拽
- 3.鼠标悬停

二、知识点

首先需要了解 `ActionChains` 的执行原理，当你调用 `ActionChains` 的方法时，不会立即执行，而是会将所有的操作按顺序存放在一个队列里，当你调用 `perform()` 方法时，队列中的方法会依次执行。

`ActionChains` 方法列表：

```
click(on_element=None) --单击鼠标左键

click_and_hold(on_element=None) --点击鼠标左键，不松开

context_click(on_element=None) --点击鼠标右键

double_click(on_element=None) --双击鼠标左键

drag_and_drop(source, target) --拖拽到某个元素然后松开

drag_and_drop_by_offset(source, xoffset, yoffset) --拖拽到某个坐标然后松开

key_down(value, element=None) --按下某个键盘上的键

key_up(value, element=None) --松开某个键

move_by_offset(xoffset, yoffset) --鼠标从当前位置移动到某个坐标

move_to_element(to_element) --鼠标移动到某个元素

move_to_element_with_offset(to_element, xoffset, yoffset) --移动到距某个元素（左上角坐标）多少距离的位置

perform() --执行链中的所有动作

release(on_element=None) --在某个元素位置松开鼠标左键
```

`send_keys(*keys_to_send)` --发送某个键到当前焦点的元素

`send_keys_to_element(element, *keys_to_send)` --发送某个键到指定元素

1. 【鼠标点击】

示例地址：<http://sahitest.com/demo/clicks.htm>

- 语法：

```
ActionChains(driver).click(click_btn).perform()      # 单击按钮
ActionChains(driver).double_click(doubleclick_btn).perform() # 双击按钮
ActionChains(driver).context_click(rightclick_btn).perform() # 右键单击按钮
```

- 参数：

传入元素对象

- 返回值：

无

- 代码示例：

```
import time
from selenium import webdriver
from selenium.webdriver import ActionChains
#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('http://sahitest.com/demo/clicks.htm')

time.sleep(2)
#定位元素
e11 = driver.find_element_by_xpath(".*//form[@name='f1']/input[@value='click me']")
print(e11)
e12 = driver.find_element_by_xpath(".*//form[@name='f1']/input[@value='dbl click me']")
e13 = driver.find_element_by_xpath(".*//form[@name='f1']/input[@value='right click me']")

#操作元素
# ActionChains(driver).click(e11).perform() #鼠标的单击
# ActionChains(driver).double_click(e12).perform() #鼠标的双击
ActionChains(driver).context_click(e13).perform() #鼠标右键单击
```

2. 【鼠标拖拽】

示例地址：<http://sahitest.com/demo/dragDropMooTools.htm>

- 语法：

```
ActionChains(driver).drag_and_drop(起点元素, 终点元素).perform() #鼠标拖
住到元素位置
ActionChains(driver).drag_and_drop_by_offset(起点元素, 终点x坐标, 终点y坐标).perform() #鼠标拖
住到坐标位置
ActionChains(driver).click_and_hold(起点元素).release(终点元素).perform() #按住+松
开实现拖拽
```

- 参数：
传入元素对象
- 返回值：
无
- 代码示例：

```
import time
from selenium import webdriver
from selenium.webdriver import ActionChains
#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('http://sahitest.com/demo/dragDropMooTools.htm')

time.sleep(2)
#定位元素
e11 = driver.find_element_by_id("dragger")
print(e11)
e12 = driver.find_element_by_xpath("//*[text()='Item 1']")
print(e12)
e13 = driver.find_element_by_xpath("//*[text()='Item 2']")

#操作元素
# ActionChains(driver).drag_and_drop(e11,e12).perform() #鼠标的拖拽
# ActionChains(driver).drag_and_drop_by_offset(e11, 400, 150).perform() #移动到指定坐标拖拽
ActionChains(driver).click_and_hold(e11).release(e13).perform() #鼠标按下松开拖拽
```

3. 【鼠标悬停】

示例地址：<http://sahitest.com/demo/mouseover.htm>

- 语法：

```
ActionChains(driver).move_to_element(目标元素).perform() #鼠标悬停到元素上
ActionChains(driver).move_by_offset(10, 60).perform() #鼠标悬停到距离当
前位置的坐标点上（了解）
ActionChains(driver).move_to_element_with_offset(目标元素, 10, -40).perform() #鼠标悬停到距离指
定元素位置的相对坐标点上（了解）
```

- 参数：
传入元素对象
- 返回值：

无

- 代码示例：

```
import time
from selenium import webdriver
from selenium.webdriver import ActionChains
#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('http://sahitest.com/demo/mouseover.htm')
driver.maximize_window()

time.sleep(2)

#定位元素
e11 = driver.find_element_by_xpath(".*//form[@name='f1']/input[1]")
print(e11)

#操作元素
# ActionChains(driver).move_to_element(e11).perform() #鼠标悬停到某个元素
# ActionChains(driver).move_by_offset(10,60).perform() #鼠标悬停到坐标
# ActionChains(driver).move_to_element_with_offset(e11,10,20).perform() #鼠标悬停到目标元素的相对位置
```

5.14 【鼠标和键盘操作】模拟键盘操作页面元素（了解）

一、学习目标

- 1.输入文本
- 2.键盘按下与松开

二、知识点

1.【输入文本】

示例地址：<https://www.baidu.com/>

- 语法：

```
ActionChains(driver).send_keys(文本1).perform() #发送某个键到当前焦点的元素
（一般不用）
ActionChains(driver).send_keys_to_element(元素信息, 文本2).perform() #发送某个键到指定元素
```

- 参数：

send_keys(): 既可以输入文本，也可以输入按键模拟鼠标按键
send_keys_to_element(): 输入输入框的元素对象，输入框内容

- 返回值：

无

- 代码示例：

```
import time
from selenium import webdriver
from selenium.webdriver import ActionChains
#创建浏览器
driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')
driver.maximize_window()

time.sleep(2)

# #键盘输入
# ActionChains(driver).send_keys("python").perform()

#定位元素
el1 = driver.find_element_by_id("kw")
ActionChains(driver).send_keys_to_element(el1, "python").perform() #指定元素输入
```

2. 【键盘按下与松开】

Selenium中的Key模块为我们提供了模拟键盘按键的方法。它不仅可以模拟键盘输入，也可以模拟键盘的操作。

注：这里需要区分开我们的元素对象也有send_keys方法，ActionChains模块也提供了send_keys方法。两种是通用的，但组合键必须要使用元素对象的send_keys方法。

模拟键盘按键	说明
send_keys(Keys.BACK_SPACE)	删除键 (BackSpace)
send_keys(Keys.SPACE)	空格键(Space)
send_keys(Keys.TAB)	制表键(Tab)
send_keys(Keys.ESCAPE)	回退键 (Esc)
send_keys(Keys.ENTER)	回车键 (Enter)
send_keys(Keys.CONTROL+'a')	全选 (Ctrl+A)
send_keys(Keys.CONTROL+'c')	复制 (Ctrl+C)
send_keys(Keys.CONTROL+'x')	剪切 (Ctrl+X)
send_keys(Keys.CONTROL+'v')	粘贴 (Ctrl+V)

- 语法：

```
ActionChains(driver).key_down(Keys.SHIFT).perform() #键盘按下
ActionChains(driver).key_up(Keys.SHIFT).perform() #键盘抬起
ActionChains(driver).send_keys(Keys.BACK_SPACE).perform() #键盘按键
```

- 参数：

Key模块为我们提供了模拟键盘按键的方法

- 返回值：
无
- 代码示例：

```
import time
from selenium import webdriver
from selenium.webdriver import ActionChains
#创建浏览器
from selenium.webdriver.common.keys import Keys

driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

#打开页面
driver.get('https://www.baidu.com/')
driver.maximize_window()

time.sleep(2)

# 键盘输入
# ActionChains(driver).send_keys("python").perform()

#定位元素
e11 = driver.find_element_by_id("kw")
ActionChains(driver).send_keys_to_element(e11,"python").perform() #指定元素输入

time.sleep(2)
# ActionChains(driver).send_keys(Keys.CONTROL+'a').perform() #ActionChains模拟键盘按键
# e11.send_keys(Keys.CONTROL+'a') #元素对象模拟键盘按键

ActionChains(driver).send_keys(Keys.BACK_SPACE).perform()
```

5.15 【弹出框操作】模拟操作系统弹窗

一、学习目标

1. Alert 类型弹框
2. Confirm 类型弹框
3. Prompt 类型弹框

二、知识点

示例网页：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>测开星辰：全栈自动化 - 弹出对话框</title>
</head>
<script>
```

```

function appendEle(info) {
    var node = document.createElement("LI");
    var textNode = document.createTextNode(info);
    node.appendChild(textNode);
    document.getElementById("add").appendChild(node);
}

function clickResponse() {
    if (confirm("你确定要和星辰一起学自动化吗? ") == true) {
        appendEle("确定");
    } else {
        appendEle("取消操作");
    }
}

function clickPrompt() {
    var PythonCourse = prompt("请输入你想学习的Python课程", "Python基础");

    if (PythonCourse == null ) {
        appendEle("取消操作");
    } else {
        appendEle("你想学习:" + PythonCourse);
    }
}

</script>
<body>

<br>
<br>
<button id="b1" onclick='alert("现在开始和星辰一起学自动化!");'>alert</button>

<br>
<br>
<button id="b2" onclick='clickResponse()'>confirm</button>

<br>
<br>
<button id="b3" onclick='clickPrompt()'>prompt</button>

<div id="add"></div>
</body>
</html>

```

1. 【Alert 类型弹框】

Alert 类型弹框，是显示一条弹出提示消息和确认按钮的警告框，用来提示。

- 获取弹窗文本

```

alert = driver.switch_to.alert #获取弹窗对象
print(alert.text)

```

- 点击弹窗确定

```
alert = driver.switch_to.alert #获取弹窗对象
alert.accept()
```

2. 【Confirm 类型弹框】

Confirm 类型弹框，是显示一个含有指定消息和确认和取消按钮的确认框，用来确认。

- 获取弹窗文本

```
confirm = driver.switch_to.alert #获取弹窗对象
print(confirm.text)
```

- 点击弹窗确定

```
confirm = driver.switch_to.alert #获取弹窗对象
confirm.accept()
```

- 点击弹窗取消

```
confirm = driver.switch_to.alert #获取弹窗对象
confirm.dismiss()
```

3. 【Prompt 类型弹框】

Prompt 类型弹框，是显示提示用户进行输入的对话框。用来输入内容。

- 获取弹窗文本

```
prompt = driver.switch_to.alert #获取弹窗对象
time.sleep(2)
print(prompt.text)
```

- 点击弹窗确定

```
prompt = driver.switch_to.alert #获取弹窗对象
time.sleep(2)
prompt.accept()
```

- 点击弹窗取消

```
prompt = driver.switch_to.alert #获取弹窗对象
time.sleep(2)
prompt.dismiss()
```

- 输入弹窗文本


```
prompt = driver.switch_to.alert #获取弹窗对象
time.sleep(2)
prompt.send_keys('web自动化-selenium')
```

5.16 【下拉框操作】模拟操作下拉框

一、学习目标

1. 选择项确认选择
2. 选择项取消选择
3. 获取选择项的值

二、知识点

地址示例：<http://sahitest.com/demo/selectTest.htm>

1. 【确认选择】

- 按索引值选择

```
#1. 按索引选择下拉项
e11 = driver.find_element_by_id("s1Id")

# s = Select(e11) #1. 实例化select对象
# s.select_by_index(2) #2. 按索引选择
```

- 按value属性选择

```
# #2.value属性选择下拉项
# s = Select(e11)
# s.select_by_value("o2")
```

- 按标签文本选择

```
#3. 文本选择下拉项
s = Select(e11)
s.select_by_visible_text("o3")
```

2. 【选择项取消选择】

注意：有一种特殊的select标签，即设置了 `multiple="multiple"` 属性的 `select`，这种 `select` 框是可以多选的，取消操作只适用于添加了 `multiple` 的下拉框(可以多选)，否则会报错

- 按索引值取消选择

```
# #1. 按索引选择，取消
# s = Select(e11)
# s.select_by_index(1)
# time.sleep(2)
# s.deselect_by_index(1)
```

- 按value属性取消选择

```
# #2.按value选择，取消
# s = Select(e11)
# s.select_by_value("o2val") #选择
# time.sleep(2)
# s.deselect_by_value("o2val") #取消选择
```

- 按标签文本取消选择

```
# #3.按文本选择，取消
# s = Select(e11)
# s.select_by_visible_text("o3")
# time.sleep(3)
# s.deselect_by_visible_text("o3")
```

- 全部取消选择

```
#4.全部取消
s = Select(e11)
s.select_by_index(1)
s.select_by_value("o2val")
time.sleep(3)
s.deselect_all()
```

3.【获取选择项的值】

我们还可以获取select标签对象的三个属性（注：不是方法）：

```
import time
from selenium import webdriver
from selenium.webdriver.support.select import Select

driver = webdriver.Chrome(r'C:\Users\Admin\Desktop\chromedriver.exe')

driver.get("http://sahitest.com/demo/selectTest.htm")

e11 = driver.find_element_by_id("s4Id")

s = Select(e11)
a = s.options #获取全部下拉项
print(a)

s.select_by_value("o1val")
s.select_by_index(2)
b = s.all_selected_options
print(b)
c = s.first_selected_option
print(c.text)
```

5.17 【单/复选框】单选框（Radio）复选框（CheckBox）

一、学习目标

- 1.复选框勾选
- 2.单选框勾选
- 3.判断是否勾选

二、知识点

示例网页源码：

```
<html>
<body>
Checkbox:
<input type="checkbox" value="cv1" name="c1">
<input type="checkbox" value="cv2">
<input type="checkbox" value="cv3" name="c1">
<input type="checkbox" value="cv4">
<p>
Radio:
<input type="radio" value="rv1" name="r1">
<input type="radio" value="rv2" name="r1">
</body>
</html>
```

1. 【复选框勾选】

代码示例：

```
#1. 复选框
#方式一
e11 = driver.find_element_by_xpath(".*input[@value='cv1']")
e11.click()
#方式二
e12 = driver.find_element_by_xpath(".*input[@value='cv2']")
e12.send_keys(Keys.SPACE)
```

2. 【单选框勾选】

代码示例：

```
#2. 单选框
#方式一
e13 = driver.find_element_by_xpath(".*input[@value='rv1']")
e14 = driver.find_element_by_xpath(".*input[@value='rv2']")
e13.click()
#方式二
time.sleep(3)
e14.send_keys(Keys.SPACE)
```

3. 【判断是否勾选】

- 语法：

```
element.is_selected()
```

- 参数：

无

- 返回值：

已选择：返回True

未选择：返回False

- 代码示例：

```
# #3.判断是否被勾选
# #复选框
# e11 = driver.find_element_by_xpath("./input[@value='cv1']")
# # e11.click()
# res1 = e11.is_selected()
# print(res1)

#单选框
e13 = driver.find_element_by_xpath("./input[@value='rv1']")
# e13.click()
res2 = e13.is_selected()
print(res2)
```

5.18 【滚动条操作】模拟鼠标滑轮滚动

一、学习目标

- 1.滑动至页面底部
- 2.滑动至顶部
- 3.滑动至具体位置
- 4.滑动至目标元素可见

二、知识点

示例地址：<https://baike.baidu.com/>

1. 【滑动至页面底部】

代码示例：

```
js="window.scrollTo(0,document.body.scrollHeight)"
driver.execute_script(js)
```

2. 【滑动至页面顶部】

代码示例：

```
js="window.scrollTo(0,-document.body.scrollHeight)"
driver.execute_script(js)
```

3. 【滑动至具体位置】

代码示例：

```
js="window.scrollTo(0,500)" # 向下滑动500个像素

js="window.scrollTo(0,-500)" # 向上滚动500个像素

js="window.scrollTo(500,0)" # 向右滑动500个像素

js="window.scrollTo(-500,0)" # 向左滚动500个像素

driver.execute_script(js) # 滑动到具体位置
```

4. 【滑动至目标元素可见】

代码示例：

```
driver.execute_script("arguments[0].scrollIntoView();", element) # 向下滚动至-元素可见

driver.execute_script("arguments[0].scrollIntoView(false);", element) # 向上滚动至-元素可见
```

5.19 【cookies操作】操作当前页面cookies信息（了解）

一、学习目标

1. 获取cookies信息
2. 添加cookies信息
3. 删除cookies信息

二、知识点

示例地址：<https://www.baidu.com/>

1. 【获取cookies信息】

代码示例：

```
#1. 获取cookies信息
res1 = driver.get_cookies()
print(res1)
#单独获取某个cookie
res2 = driver.get_cookie('BD_HOME')
print(res2)
```

2. 【添加cookies信息】

代码示例：

```
#2. 添加cookies信息
cookie = {"name": "aaa", "value": "bbb"}
driver.add_cookie(cookie)
#单独获取某个cookie
res3 = driver.get_cookie('aaa')
print(res3)
```

3. 【删除cookies信息】

代码示例：

```
#3. 删除cookies信息
driver.delete_cookie("aaa")
#单独获取某个cookie
res4 = driver.get_cookie('aaa')
print(res4)
```

5.20 【iframe切换】 切换页面的iframe、handlers

一、学习目标

- 1.iframe切换
- 2.handler切换

二、知识点

1. 【iframe切换】

示例地址：<http://sahitest.com/demo/iframesTest.htm>

- iframe切换

语法：

```
#1. 直接使用id值切换
driver.switch_to.frame(value) #如果iframe标签中有id属性，可以用这个方法直接传id的值
#2. 通过出入元素对象切换
el = driver.find_element_by_xxx(value)
driver.switch_to.frame(el)
#3. 跳回上层的页面
driver.switch_to.parent_frame()
#4. 跳回最外层的页面
driver.switch_to.default_content()
```

代码示例：

```
#切换iframe
e1 = driver.find_element_by_xpath("//*[iframe[@src='index.htm']]")
driver.switch_to.frame(e1) #切换iframe
e11 = driver.find_element_by_xpath("//*[a[text()='Link Test']]")
e11.click()

#切出iframe
# driver.switch_to.parent_frame() #切出到上一层
driver.switch_to.default_content() #切出到最外层
e12 = driver.find_element_by_xpath("//*[input[@value='Click me']]")
e12.click()
```

2. 【handler切换】

示例地址: <https://www.baidu.com/>

- handler切换

语法:

```
#1. 获取当前打开的所有窗口句柄, 返回类型为一个列表
driver.window_handles #注: window_handles是对象属性, 没有括号
#2. 获取当前窗口的句柄
driver.current_window_handle #注: window_handles是对象属性, 没有括号
#3. 切换窗口, handle_id参数代表的是一个窗口句柄
driver.switch_to.window(handle_id)
```

代码示例:

```
#获取当前所有窗体
res1 = driver.window_handles
print(res1)

#当前窗体的id
h1 = driver.current_window_handle
print(h1)

#切换窗体
driver.switch_to.window(res1[-1])

#当前窗体的id
h2 = driver.current_window_handle
print(h2)
```

5.21 【EC模块】 expected_conditions模块介绍

一、学习目标

1. 了解EC判定方法

二、知识点

1. 【判定方法】

- 判定方法

#1.判断当前页面的**title**是否完全等于(==)预期字符串，返回是布尔值
title_is

#2.判断当前页面的**title**是否包含预期字符串，返回布尔值
title_contains

#3.判断某个元素是否被加到了**dom**树里，并不代表该元素一定可见
presence_of_element_located

#4.判断某个元素是否可见。可见代表元素非隐藏，并且元素的宽和高都不等于0
visibility_of_element_located("id","aaa")

#5.跟上面的方法做一样的事情，只是上面的方法要传入**locator**，这个方法直接传定位到的**element**就好了
visibility_of(el)

#6.判断是否至少有1个元素存在于**dom**树中。举个例子，如果页面上有n个元素的**class**都是'**column-md-3**'，那么只要有1个元素存在，这个方法就返回**True**
presence_of_all_elements_located

#7.判断某个元素中的**text**是否 包含 了预期的字符串
text_to_be_present_in_element

#8.判断某个元素中的**value**属性是否包含 了预期的字符串
text_to_be_present_in_element_value

#9.判断该**frame**是否可以**switch**进去，如果可以的话，返回**True**并且**switch**进去，否则返回**False**
frame_to_be_available_and_switch_to_it

#10.判断某个元素中是否不存在于**dom**树或不可见
invisibility_of_element_located

#11.判断某个元素中是否可见并且是**enable**的，这样的话才叫**clickable**
element_to_be_clickable

#12.等某个元素从**dom**树中移除，注意，这个方法也是返回**True**或**False**
staleness_of

#13.判断某个元素是否被选中了，一般用在下拉列表>* **element_selection_state_to_be**:判断某个元素的选中状态是否符合预期
element_to_be_selected

#14.跟上面的方法作用一样，只是上面的方法传入定位到的**element**，而这个方法传入**locator**
element_located_selection_state_to_be

#15.判断页面上是否存在**alert**
alert_is_present

- 代码示例：

判断标题示例地址：<https://www.baidu.com/>

判断元素可见示例地址：<https://sahitest.com/demo/visible.htm>

判断弹窗示例地址：<https://sahitest.com/demo/alertTest.htm>


```
# #1.判断标题完全等于预期文本
# res1 = EC.title_is("百度一下，你就知道")
# print(res1(driver))
#
# #2.判断标题部分等于预期文本
# res2 = EC.title_contains("百度一下")
# print(res2(driver))
#
# #3.判断元素是否可见
# e13 = driver.find_element_by_xpath(".*input[@value='visibility hidden']")
# e13.click()
# time.sleep(1)
# locate = ("id","uv")
# res3 = EC.visibility_of_element_located(locate)
# print(res3(driver))
#
# #4.判断元素是否在dom
# res4 = EC.presence_of_element_located(locate)
# print(res4(driver))
#
#5.判断弹窗是否出现
e15 = driver.find_element_by_name("b1")
e15.click()
res5 = EC.alert_is_present()
print(res5(driver))
```

5.22 【元素等待】三种元素等待方式

一、学习目标

- 1.强制等待
- 2.隐性等待
- 3.显性等待

二、知识点

地址示例：<http://sahitest.com/demo/waitFor.htm>

1.【强制等待】

第一种也是最简单粗暴的一种办法就是强制等待sleep(xx)，不管是元素加载出来，都必须等xx时间。不过不建议总用这种等待方式，太死板，严重影响程序执行速度。

- 语法：

```
time.sleep(3)
```

- 代码示例：

```
#1.强制等待
time.sleep(5)
e11 = driver.find_element_by_id("id2")
res = e11.text
print(res)
```

2.【隐性等待】

隐形等待是设置了一个最长等待时间，如果在规定时间内网页加载完成，则执行下一步，否则一直等到时间截止，然后执行下一步。注意这里有一个弊端，那就是程序会一直等待整个页面加载完成，也就是一般情况下你看到浏览器标签栏那个小圈不再转，才会执行下一步，但有时候页面想要的元素早就在加载完成了，但是因为个别js之类的东西特别慢。

- 语法：

```
driver.implicitly_wait(5)
```

- 代码示例：

```
#2.隐式等待
driver.implicitly_wait(5)
e11 = driver.find_element_by_id("id2")
res = e11.text
print(res)
```

3.【显性等待】

显性等待，WebDriverWait，配合该类的until()和until_not()方法，就能够根据判断条件而进行灵活地等待了。它主要的意思就是：程序每隔xx秒看一眼，如果条件成立，则执行下一步，否则继续等待，直到超过设置的最长时间，然后抛出TimeoutException异常。

- 语法：

```
WebDriverWait(driver,20,0.5).until(EC.presence_of_element_located(locator))
```

- 代码示例：

```
#3.显示等待
loc = ("id","id2")
WebDriverWait(driver,20,0.5).until(EC.presence_of_element_located(loc))
e11 = driver.find_element_by_id("id2")
res = e11.text
print(res)
```

5.23【实战练习】简单实现163邮箱web自动化脚本

一、学习目标

1.实现163邮箱登录web自动化脚本

二、知识点

代码示例：<https://mail.163.com/>

```
#实现163邮箱登录
import time

from selenium import webdriver

#创建浏览器对象
driver = webdriver.Chrome(executable_path=r"C:\Users\Admin\Desktop\chromedriver.exe")

#打开163邮箱
driver.get("https://mail.163.com/")
driver.maximize_window() #最大化

#如何排查定位不到元素
#（1）元素为加载出来：
time.sleep(3)
#（2）为切换iframe，或未退出
# driver.switch_to.frame("x-URS-iframe1647089900611.3906")
#（3）动态元素
e1 = driver.find_element_by_xpath("//*[id='loginDiv']/iframe")
driver.switch_to.frame(e1)

#1.输入用户名
e11 = driver.find_element_by_xpath("//*[id='account-box']/div/input") #定位元素
e11.send_keys("lcyaaa123")
#2.输入密码
e12 = driver.find_element_by_name("password")
e12.send_keys("lcy123456")
#3.点击登录
e13 = driver.find_element_by_id("dologin")
e13.click()
```

第六章 Requests 接口自动化模块介绍

6.1 requests模块介绍和环境安装

一、学习目标

- 1.requests模块介绍
- 2.requests模块安装

二、知识点

1. 【requests模块介绍】

- request库的7个主要方法

方法	说明
requests.request()	构造一个请求，支撑以下各方法的基础方法，其余方法都只是再次封装request方法
requests.get()	获取HTML网页的主要方法，对应HTTP的GET
requests.post()	向HTML网页提交POST请求的方法，对应HTTP的POST
requests.put()	向HTML网页提交PUT请求方法，对应于HTTP的PUT
requests.head()	获取HTML网页头信息，对应于HTTP的HEAD
requests.patch()	向HTML网页提交局部修改请求，对应于HTTP的PATCH
requests.delete()	向HTML页面提交删除请求，对应于HTTP的DELETE

- Response对象属性

属性	说明
r.status_code	HTTP请求的返回状态，200表示连接成功，404表示失败
r.txt	HTTP响应内容的字符串形式，即url对应的页面内容
r.encoding	从HTTP header中猜测的响应内容编码方式
r.apparent_encoding	从内容中分析出的响应内容编码方法（备选编码方式）
r.conent	HTTP响应内容的二进制形式
r.headers	http响应内容的头部内容

2. 【requests模块安装】

- 在线安装：（当前电脑有网）

cmd执行：

```
pip install requests
```

- 离线安装：（当前电脑无网）

去联网的电脑先下载requests模块包：<https://pypi.org/>

解压后cmd执行：

```
python setup.py install
```

6.2 【HTTP协议】http协议简介

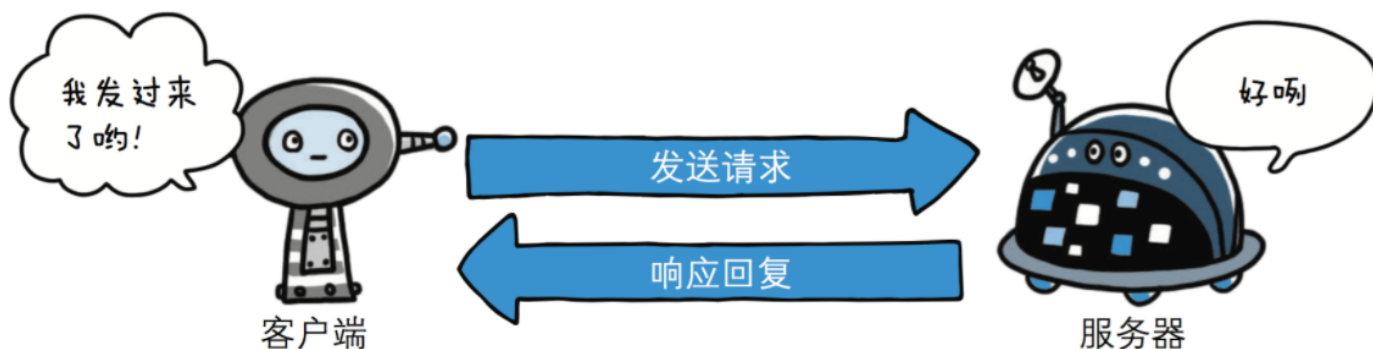
一、学习目标

- 1.HTTP工作原理
- 2.HTTP请求方法
- 3.HTTP状态码

二、知识点

1. 【HTTP工作原理】

HTTP协议定义Web客户端如何从Web服务器请求Web页面，以及服务器如何把Web页面传送给客户端。HTTP协议采用了请求/响应模型。客户端向服务器发送一个请求报文，请求报文包含请求的方法、URL、协议版本、请求头部和请求数据。服务器以一个状态行作为响应，响应的内容包括协议的版本、成功或者错误代码、服务器信息、响应头部和响应数据。



图：请求必定由客户端发出，而服务器端回复响应

- 以下是 HTTP 请求/响应的步骤：

\1. 客户端连接到Web服务器

一个HTTP客户端，通常是浏览器，与Web服务器的HTTP端口（默认为80）建立一个TCP套接字连接。例如，
<http://www.baidu.com>。

\2. 发送HTTP请求

通过TCP套接字，客户端向Web服务器发送一个文本的请求报文，一个请求报文由请求行、请求头部、空行和请求数据4部分组成。

\3. 服务器接受请求并返回HTTP响应

Web服务器解析请求，定位请求资源。服务器将资源复本写到TCP套接字，由客户端读取。一个响应由状态行、响应头部、空行和响应数据4部分组成。

\4. 释放连接TCP连接

若connection 模式为close，则服务器主动关闭TCP连接，客户端被动关闭连接，释放TCP连接；若connection 模式为keepalive，则该连接会保持一段时间，在该时间内可以继续接收请求；

\5. 客户端浏览器解析HTML内容

客户端浏览器首先解析状态行，查看表明请求是否成功的状态代码。然后解析每一个响应头，响应头告知以下为若干字节的HTML文档和文档的字符集。客户端浏览器读取响应数据HTML，根据HTML的语法对其进行格式化，并在浏览器窗口中显示。

2. 【HTTP请求方法】

HTTP/1.1协议中共定义了八种方法（也叫“动作”）来以不同方式操作指定的资源：

- GET

向指定的资源发出“显示”请求。使用GET方法应该只用在读取数据，而不应当被用于产生“副作用”的操作中，例如在Web Application中。其中一个原因是GET可能会被网络蜘蛛等随意访问。

- POST

向指定资源提交数据，请求服务器进行处理（例如提交表单或者上传文件）。数据被包含在请求本文中。这个请求可能会创建新的资源或修改现有资源，或二者皆有。

- HEAD

与GET方法一样，都是向服务器发出指定资源的请求。只不过服务器将不传回资源的本文部分。它的好处在于，使用这个方法可以在不必传输全部内容的情况下，就可以获取其中“关于该资源的信息”（元信息或称元数据）。

- PUT

向指定资源位置上传其最新内容。

- DELETE

请求服务器删除Request-URI所标识的资源。

- TRACE

回显服务器收到的请求，主要用于测试或诊断。

- OPTIONS

这个方法可使服务器传回该资源所支持的所有HTTP请求方法。用'*'来代替资源名称，向Web服务器发送OPTIONS请求，可以测试服务器功能是否正常运作。

- CONNECT

HTTP/1.1协议中预留给能够将连接改为管道方式的代理服务器。通常用于SSL加密服务器的链接（经由非加密的HTTP代理服务器）。

3. 【HTTP状态码】

所有HTTP响应的第一行都是状态行，依次是当前HTTP版本号，3位数字组成的状态代码，以及描述状态的短语，彼此由空格分隔。

状态代码的第一个数字代表当前响应的类型：

- 1xx消息——请求已被服务器接收，继续处理
- 2xx成功——请求已成功被服务器接收、理解、并接受
- 3xx重定向——需要后续操作才能完成这一请求
- 4xx请求错误——请求含有词法错误或者无法被执行
- 5xx服务器错误——服务器在处理某个正确请求时发生错误

6.3 【HTTP协议】请求

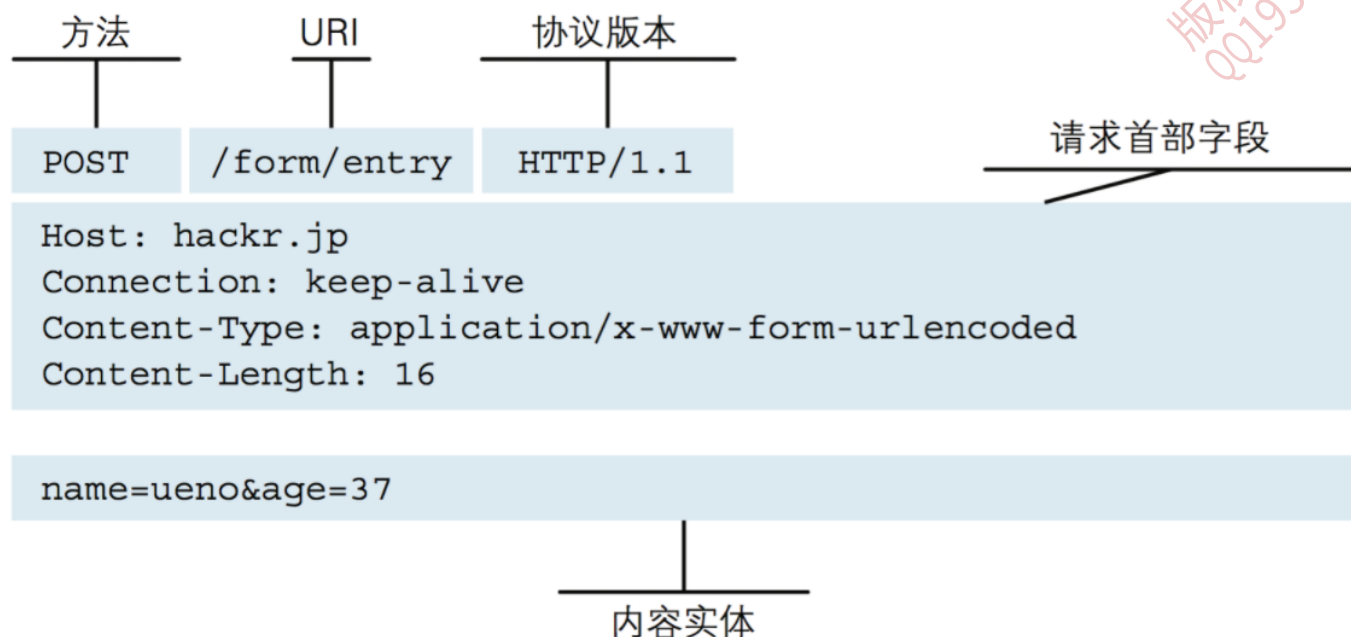
一、学习目标

1.请求报文的构成

2.模拟请求需要准备哪些参数

二、知识点

1. 【请求报文的构成】



图：请求报文的构成

2. 【模拟请求需要准备哪些参数】

我们刚刚已经讲了浏览器发送的接口请求报文的构成，用requests模块模拟浏览器发送请求时，请求数据尽量与浏览器真实的请求一致，所以在模拟请求时，必须有下列数据：

- 请求方法：接口的请求方式，如GET、POST
- 请求地址URL：接口请求目标服务器的url地址，如<http://127.0.0.1:8080/form/entry>
- 请求头：请求头的数据很多没有必要全部传，重点传以下几个即可
 - User-Agent：产生请求的浏览器类型。用以区分该请求是浏览器发的还是脚本发的。
 - Content-Type：代表发送端发送的数据格式。这个在post请求中重点讲解。
 - Cookie：HTTP请求发送时，会把保存在该请求域名下的所有cookie值一起发送给web服务器。登录后的接口一把用cookie验证用户是否有权限访问。
- 请求体：代表发送端发送的具体数据。这个在post请求中重点讲解。

请求体的数据格式是Content-Type规定的，在post请求中不同的数据格式的在requests模块中传参方式是不同的：

- Content-Type为application/x-www-form-urlencoded：requests.post()方法中需要用data参数传参
- Content-Type为application/json：requests.post()方法中需要用json参数传参

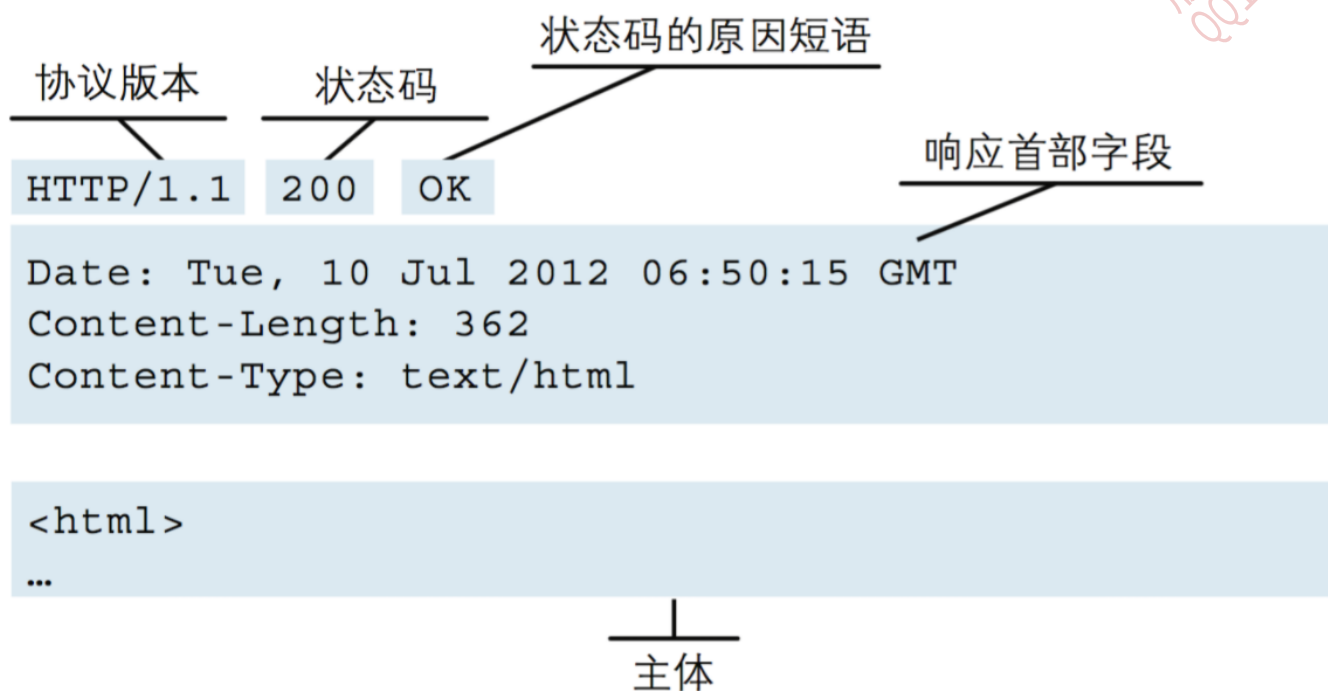
6.4 【HTTP协议】响应

一、学习目标

1. 响应报文的构成
2. 响应数据有哪几种格式

二、知识点

1. 【响应报文的构成】



图：响应报文的构成

2. 【响应数据有哪几种格式】

与请求数据一样，响应头Content-Type中也规定了响应体的数据格式，requests模块对不同的响应数据的获取方式是不同的。

- json类型的响应数据：字典取值

#这种格式的响应数据是{}形式的json类型

Content-Type: application/json; charset=utf-8

#响应体示例

```
{
  "Text": "Fiddler Jam - Your Secure, One-Pass Approach to Finding and Resolving Bugs",
  "Tags": "Troubleshooting",
  "Url": "https://www.telerik.com/blogs/fiddler-jam-your-secure-one-pass-approach-finding-resolving-bugs?utm_source=fiddlerapp&utm_medium=blog&utm_id=product"
}, {
  "Text": "What's the Best way to Build Your web Application? (Infographic)",
  "Tags": "JS",
  "Url": "https://www.telerik.com/blogs/best-way-to-build-web-app-infographic?utm_medium=product&utm_source=fiddlerapp&utm_campaign=kendo-ui-react-blog-dt-build-or-buy-infographic"
}, {
  "Text": "Exploring Fiddler Jam: Get Your Time Back by Solving Issues Faster",
  "Tags": "How To",
  "Url": "https://www.telerik.com/blogs/exploring-fiddler-jam-get-time-back-solving-issues-faster?utm_source=fiddlerapp&utm_medium=blog&utm_id=product"
}
```

- XML类型的响应数据：正则提取

#这种格式的响应数据是<a>形式的xml类型。XML 是一种标记语言，很类似 HTML

Content-Type: text/html; charset=UTF-8

#响应体示例

```
<note>
<to>George</to>
<from>John</from>
<heading>Reminder</heading>
<body>Don't forget the meeting!</body>
</note>
```

- Content-Type: image/jpeg

这种格式的响应数据类型是一些静态资源，什么是静态资源，如图片、音乐、视频等资源数据。

6.5 【Fiddler工具使用】fiddler安装及配置

一、学习目标

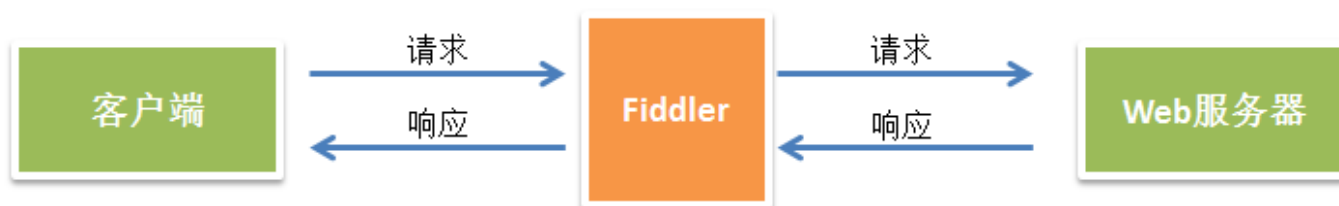
- 1.fiddler工作原理
- 2.安装配置fiddler
- 3.fiddler常用菜单介绍

二、知识点

1. 【fiddler工作原理】

Fiddler是位于客户端和服务端之间的HTTP代理，它能够记录客户端和服务端之间的所有 HTTP(S)请求，可以针对特定的 HTTP(S)请求，分析网络传输的数据，还可以设置断点、修改请求的数据和服务端返回的数据。

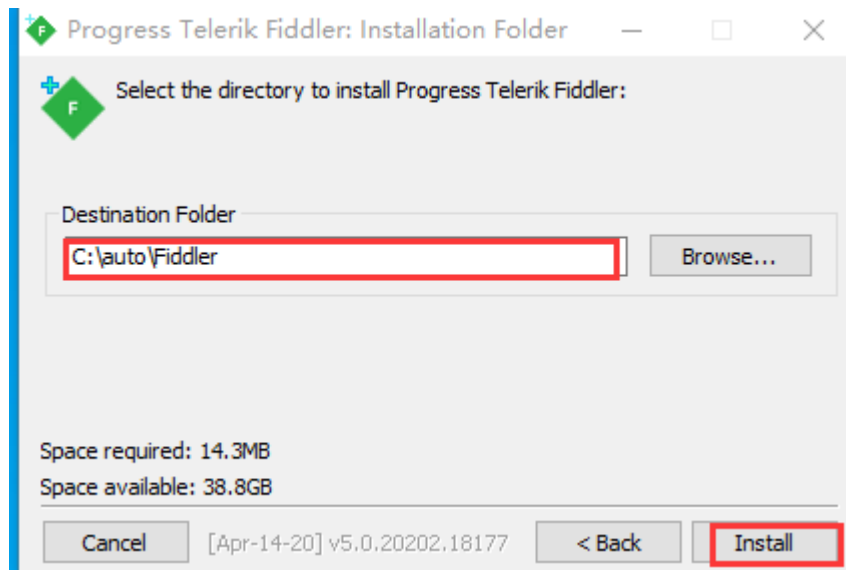
Fiddler在浏览器与服务端之间建立一个代理服务器，Fiddler工作于七层中的应用层，能够捕获通过的HTTP(S)请求。Fiddler启动后会自动将代理服务器设置成本机，默认端口为8888。Fiddler不仅能记录PC上浏览器的网络请求数据，还可以记录同一网络中的其他设备的HTTP(S)请求数据。数据传递流程大致如下：



1. 客户端像WEB服务器发送HTTP(S)请求时，请求会先经过代理Fiddler代理服务器。
2. Fiddler代理服务器截取客户端的请求报文，再转发到WEB服务器，转发之前可以做一些请求报文参数修改的操作。
3. WEB服务器处理完请求以后返回响应报文，Fiddler代理服务器会截取WEB服务器的响应报文。
4. Fiddler处理完响应报文后再返回给客户端。

2. 【安装配置fiddler】

- 安装：
双击运行FiddlerSetup.exe安装程序，然后傻瓜式安装。

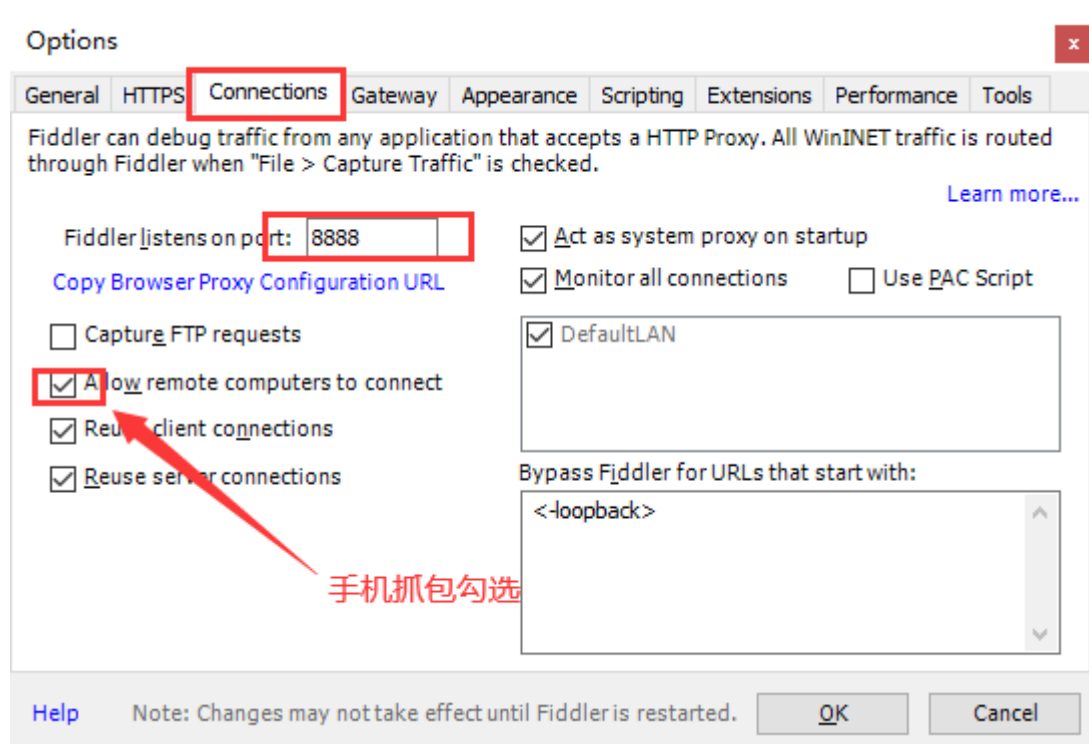


- 配置：

- o Fiddler基本配置

PC端配置：如果只是需要监听本机浏览器HTTP数据包的话不需要做任何额外配置，打开Fiddler即可直接使用。

手机端配置1：如果我们需要监听远程终端设备的网络请求，需要做以下配置。打开Fiddler，在菜单栏中选择 Tools->Options->Connections。输入监听端口（默认是8888），选择Allow remote computers to connect，点击确认然后重启Fiddler。



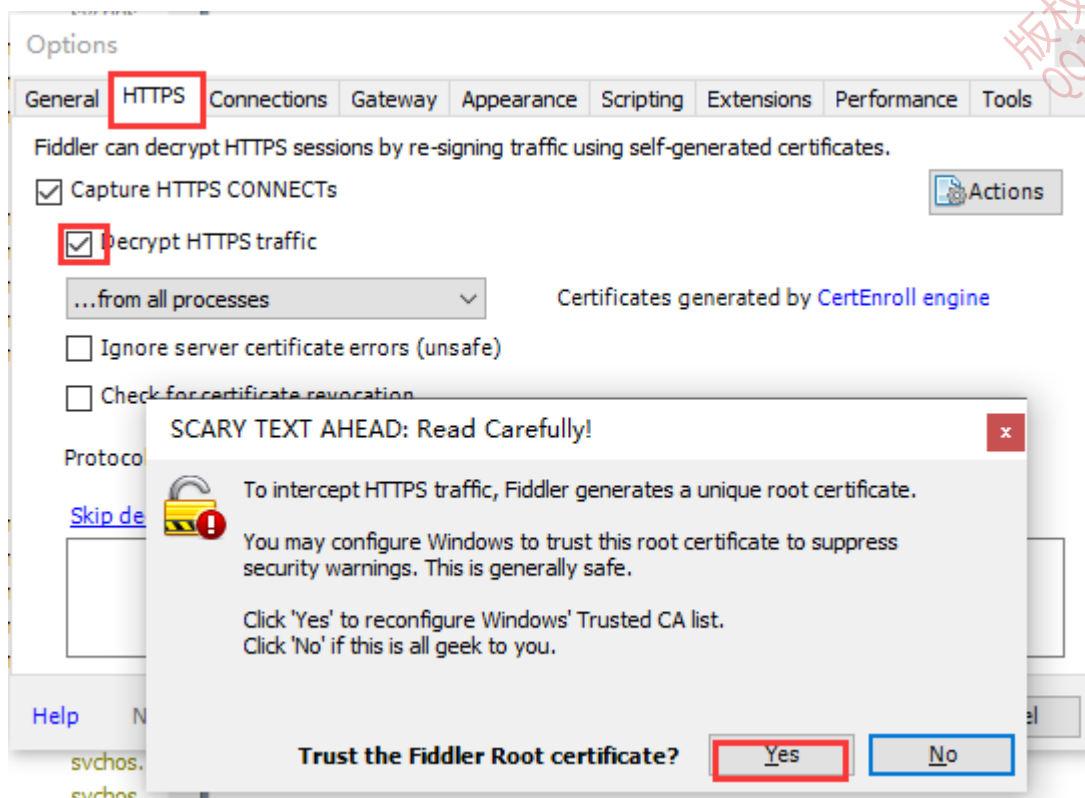
手机端配置2：手机端打开wifi设置界面，进入wifi的高级设置。Fiddler本身就是代理服务器，在wifi高级设置中的代理栏下面选择手动设置，设置输入Fiddler主机的IP地址和监听端口号（前面默认8888那个），点击确认。



o Fiddler抓包HTTPS

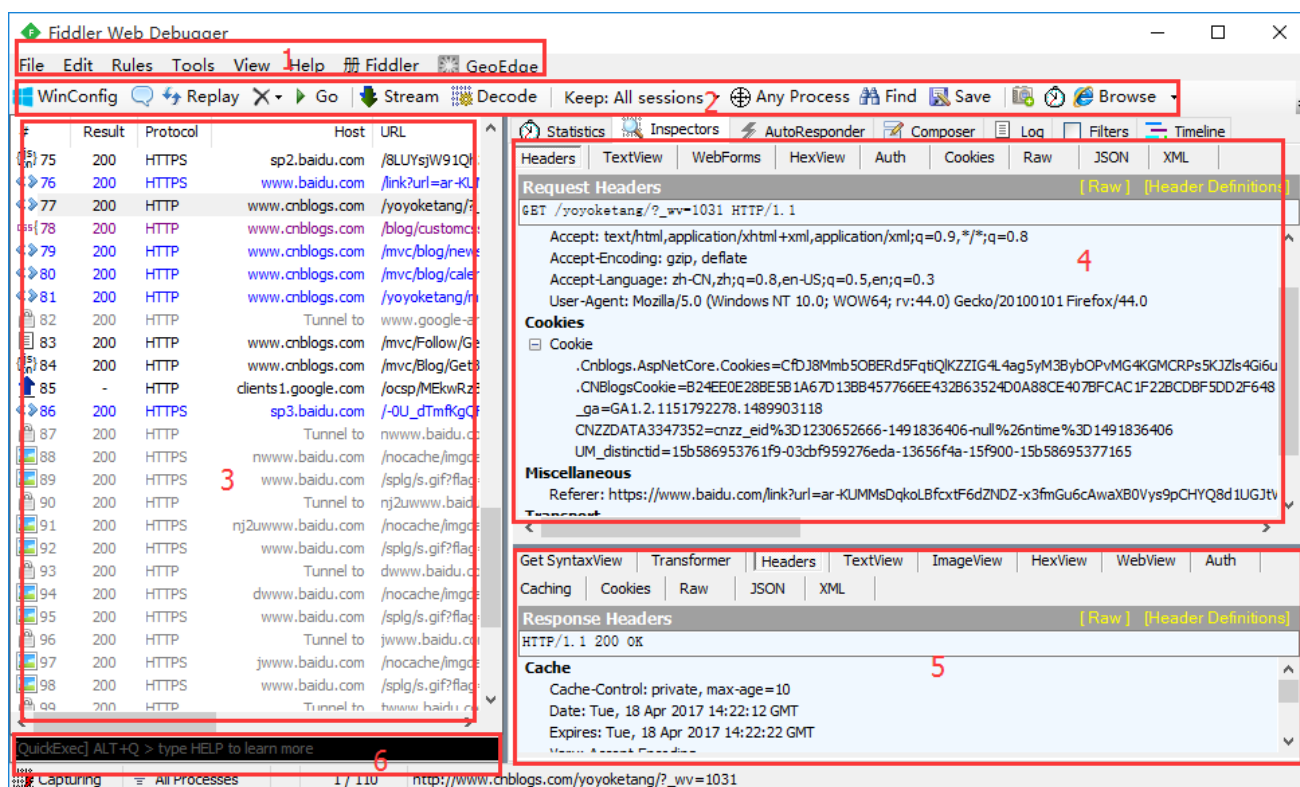
HTTPS数据报文传输的时候涉及到证书及数据加密的问题，所以Fiddler需要抓取HTTPS报文的话还需要做其他配置。

首先还是打开Fiddler配置：Tools->Options->HTTPS：



勾选Capture HTTPS CONNECTS和Decrypt HTTPS traffic选项，如果只是想抓取本机或者远程终端的数据报文，可以在...from all processes这个下拉框中选择。

3. 【fiddler常用菜单介绍】



- 1.第一块区域是设置菜单,这里在安装配置好fiddler后一般不需要动。
- 2.第二块区域是一些快捷菜单，如X按钮可以选择性删除3中的接口请求；Decode选中则接口返回是自动解码。
- 3.第三块左边是抓捕的请求会话列表，每一个请求就是一个会话
- 4.第四块右边上方区域是request请求的详细信息，可以查看headerd、cookies、raw、json等

5.第五块右边下方区域就是response信息，可以查看服务端返回的json数据或xml信息

6.第六块区域左下角黑色的那块小地方，这地方是命令行模式，可以输入简单的指令如：cls，执行清屏的作用等

6.6 【Fiddler工具使用】Inspectors抓取接口信息

一、学习目标

1.fiddler抓包流程

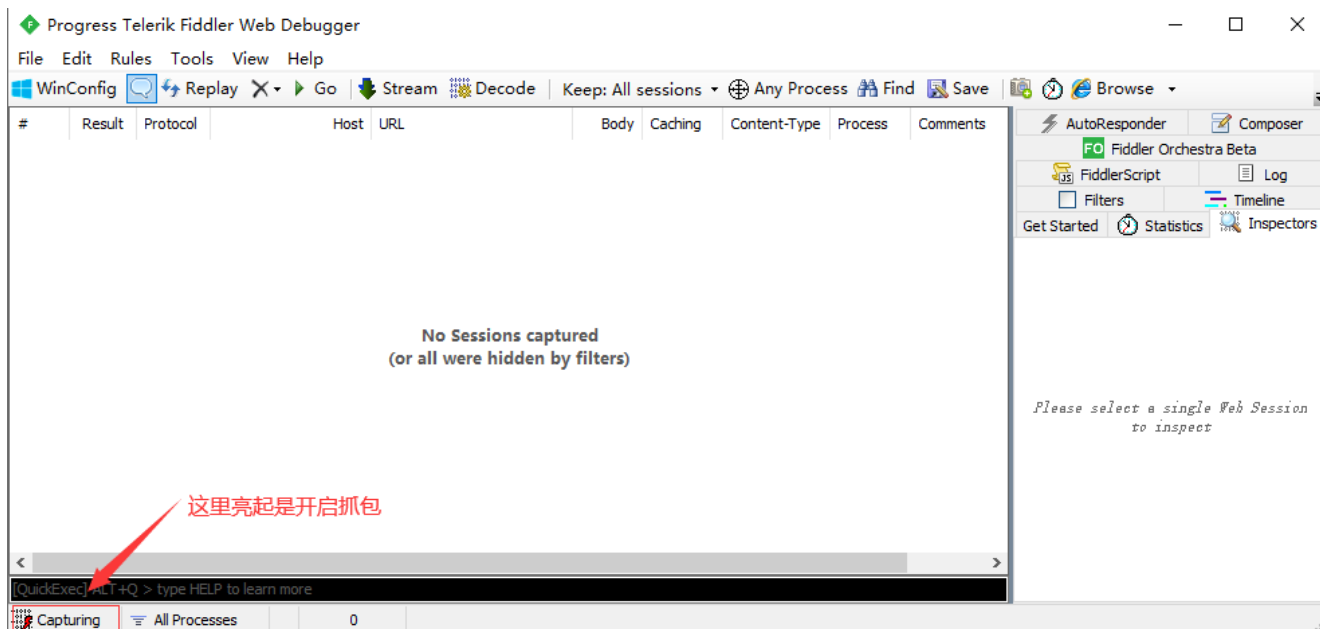
二、知识点

示例地址（禅道）：<http://zentaomax.demo.zentao.net/user-login-Lw==.html>

1. 【fiddler抓包流程】

以禅道登录接口为例：

- 步骤一：打开fiddler,左下角开启抓包

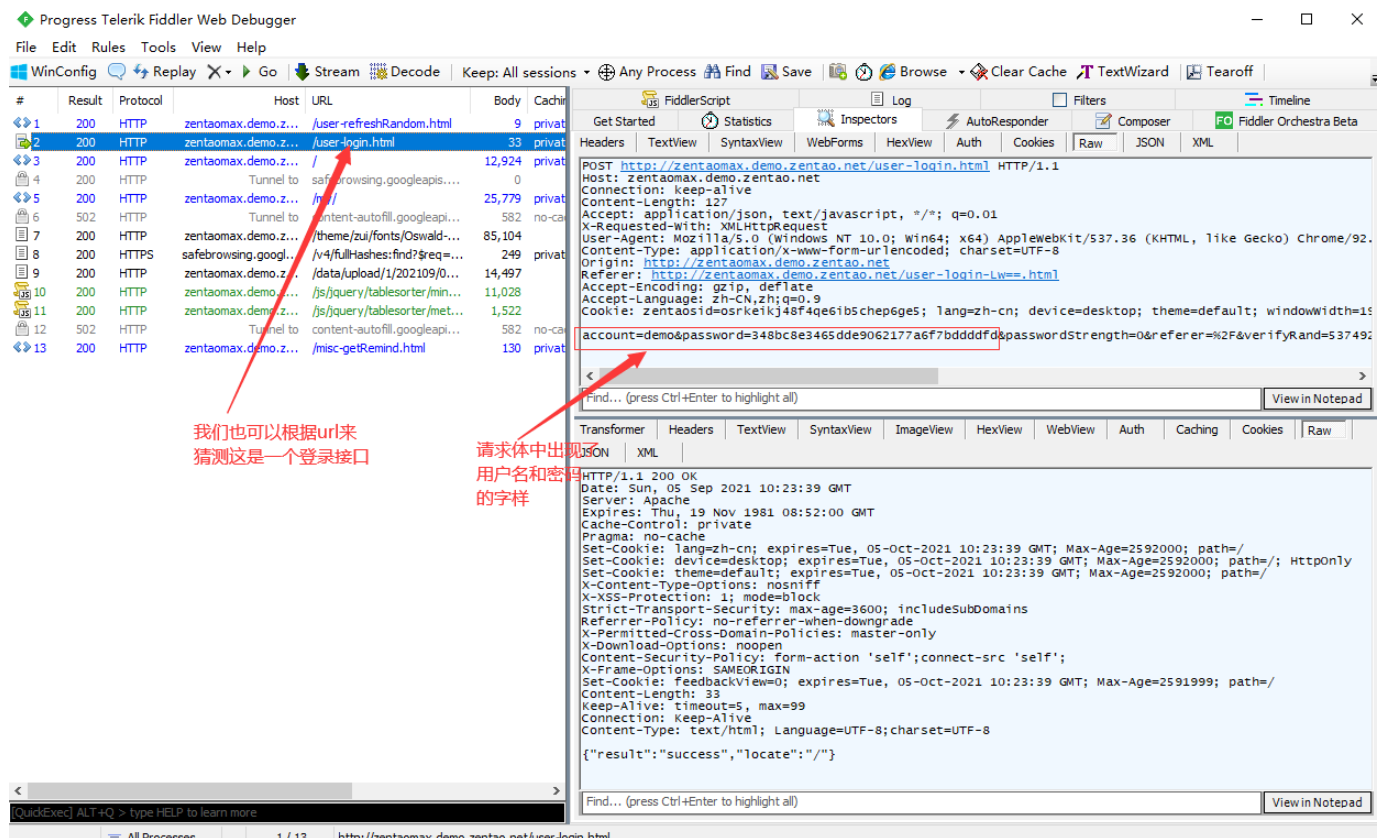


- 步骤二：浏览器手动点击要抓取的功能按钮

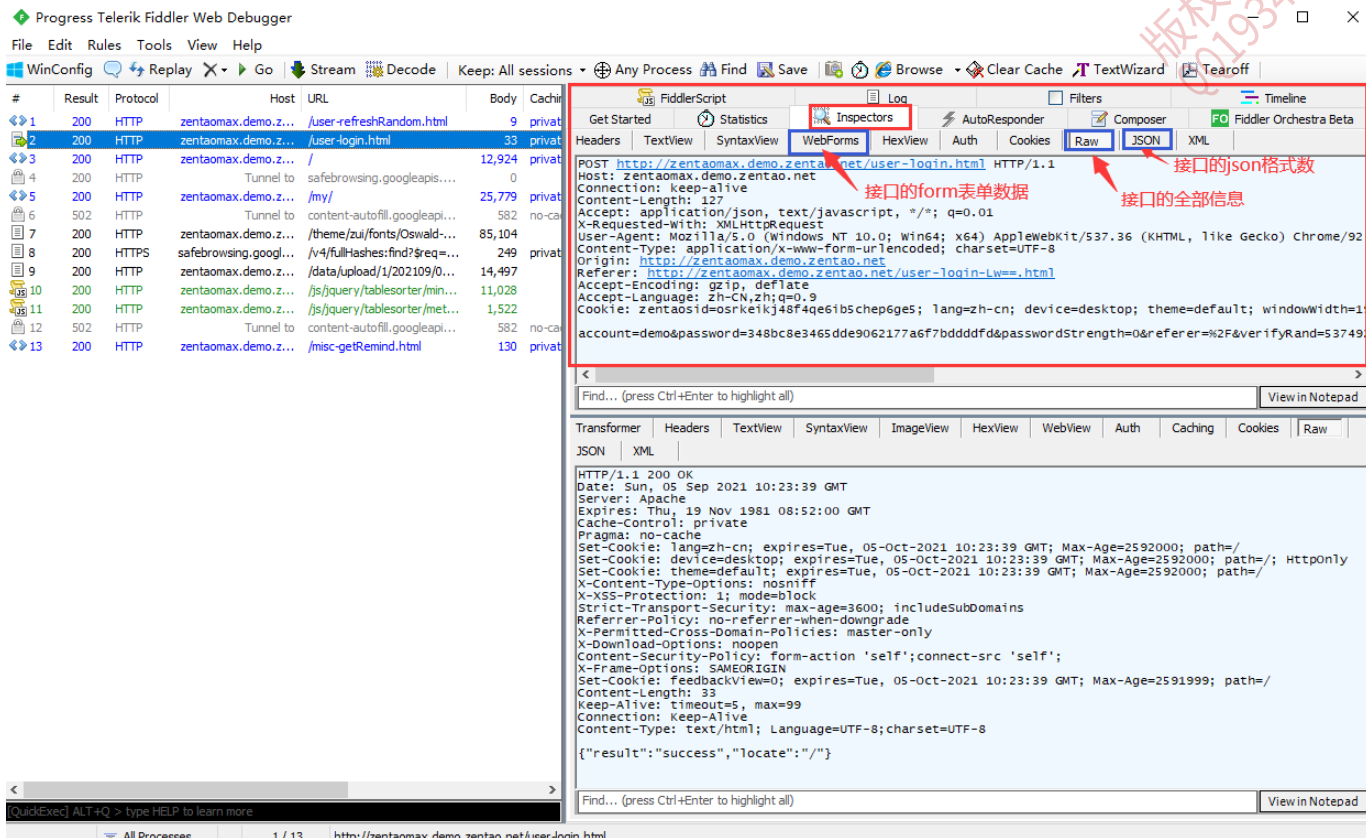


• 步骤三：分析fiddler抓到的接口，找到登录接口

这里有个小技巧：我们在页面点击的时候已经事先知道登录功能需要用户名和密码，那我們可以在fiddler查找用户名demo所在的接口。



注：我们在获取接口的请求信息时，我们只需要关注fiddler右上角的区域。其中raw中是接口的全部信息，包括请求行，请求头，请求体；webforms页签是将form表单的请求数据做详细的展示；JSON页签是对json格式的请求数据做详细的展示。



● 步骤四：提取请求数据

○ 请求方式：

POST

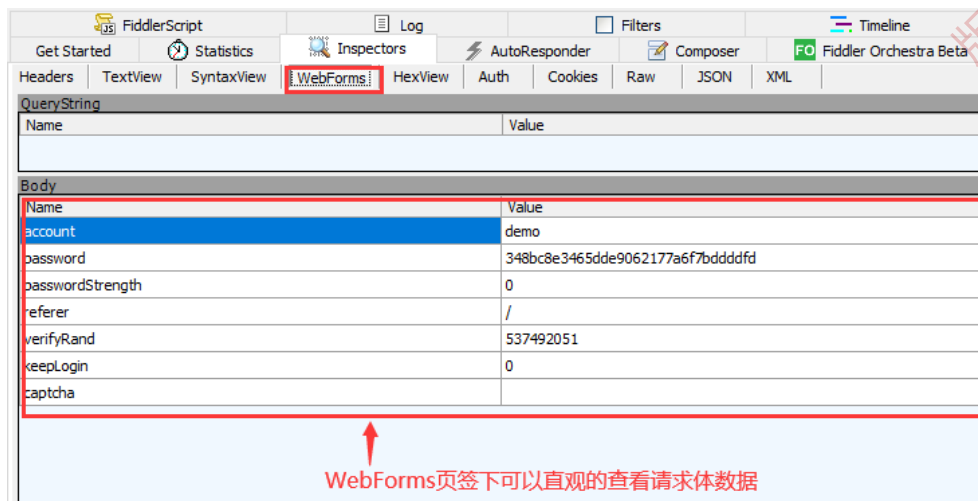
○ 请求URL：

http://zentaomax.demo.zentao.net/user-login.html

○ 请求头（只取两个）：

User-Agent: Mozilla/5.0 (Windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML,like Gecko) Chrome/92.0.4515.131safari/537.36
Content-Type: application/x-www-form-urlencoded; charset=UTF-8

○ 请求体（GET请求没有请求体）



到目前为止，一个登录接口的请求数据就准备好啦，我们后续可以通过调用requests模块中的post方法模拟发送登录请求啦！

6.7【Fiddler工具使用】Filters过滤器（了解）

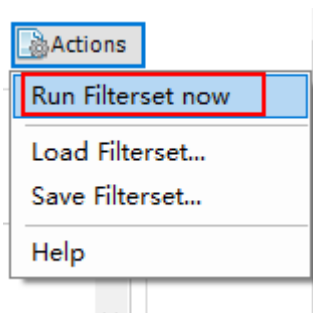
一、学习目标

- 1.按主机过滤
- 2.按进程过滤
- 3.按请求头过滤
- 4.断点
- 5.响应状态码
- 6.响应类型和大小
- 7.响应头

二、知识点

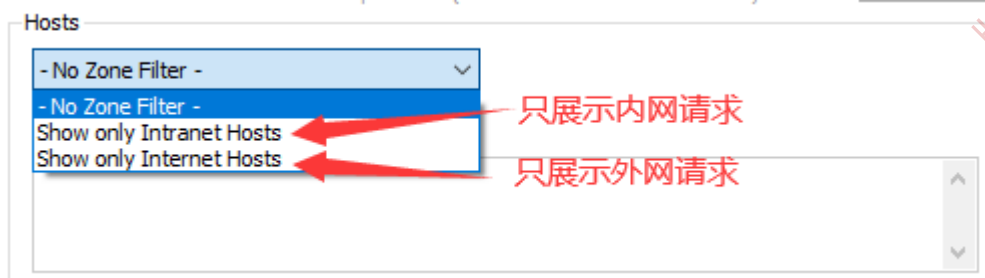
默认情况下过滤是未开启的，需要勾选“Use filters”进行启动。如果有多套监听方式，也可以将其导出，在下次需要的时候进行导入。

配置请求过滤之后：



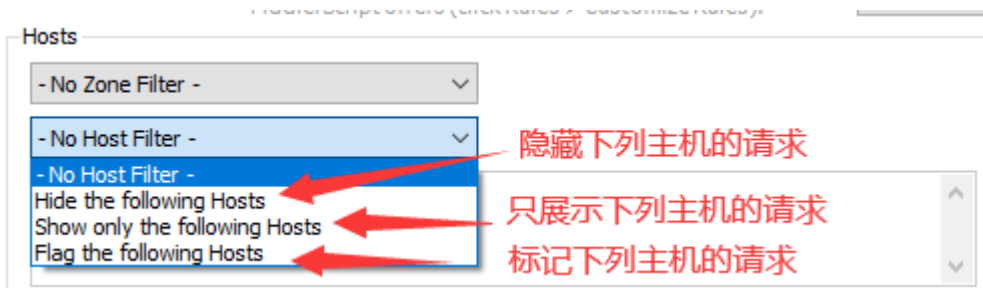
1.【按主机过滤】

- (1) 过滤内网主机/过滤外网主机，一般很少用到，主要使用第二种方式进行过滤。



- No Zone Filter 不分区过滤（内网外网都显示）
- Show only Intranet Hosts 只显示内网主机（可以通过Fiddler的快照功能，在Sessions里边加一个本地的Session，来查看此功能的效果）
- Show only Internet Hosts 只显示互联网主机

(2) 域名过滤，通过配置，只监听/或不监听默认域名。



在选中不同的过滤方式后，配合下面输入框中输入的地址信息，进行过滤。

- Hide the following Hosts：表示在输入框中，输入了哪些域名信息，就不过滤，不进行监听。
- Show only the following Hosts：标识在输入框中输入了哪些域信息，就只监听这些域名，其他的域名将不进行监听。
- Flag the following Hosts：表示在输入框中输入了哪些域名信息，在左侧的session面板中，这些配置的域名在监听到时，会加标识（如：加粗）

2. 【按进程过滤】

进程过滤，通过配置，只监听/不监听哪些进程的请求。



- Show only traffic from：根据进程信息进行过滤，选择后，将只显示由该进程发出的请求；
- Show only Internet Explorer traffic：只关心由IE浏览器发出的请求；
- Hide traffic from Service Host：隐藏来自service host（即由svchost.exe进程发出）的请求

按进程过滤我们用到的比较少，了解即可。

3. 【按请求头过滤】

表示根据请求头信息，进行过滤。

Request Headers

☐ Show only if URL contains

☐ Hide if URL contains

☐ Flag requests with headers

☐ Delete request headers

☐ Set request header

- Show only if URL contains: 可以通过完整url或正则表达式过滤请求地址中包含或不包含的内容
- Hide if URL contains: 与Show only if URL contains相反, 一个是显示, 一个是隐藏;
- Flags requests with headers: 支持指定某个http请求头名称, 如果在web session列表中存在该请求头, 会加粗显示该session;
- Delete requests headers: 与Flags requests with headers类似, 这里是如果包含该请求头, 则删除该请求头;
- Set request header: 将HTTP请求头更新为所输入的值。

4. 【断点设置】

根据断点, 进行配置过滤

Breakpoints

☐ Break request on POST ☐ Break request on GET with query string

☐ Break on XMLHttpRequest

☐ Break response on Content-Type

- Break request on POST: 对POST请求设置断点;
- Break request on GET with query string: 会为所有方法为GET且URL中包含了给定查询条件的请求设置断点;
- Break on XMLHttpRequest: 对所有能够确定是通过XMLHttpRequest对象发送的请求设置断点;
- Break response on Content-Type: 会为所有响应头Content-Type中包含了指定文本的响应设置响应断点。

5. 【响应状态码】

根据响应代码进行过滤

Response Status Code

☐ Hide success (2xx) ☐ Hide non-2xx ☐ Hide Authentication demands (401,407)

☐ Hide redirects (300,301,302,303,307) ☐ Hide Not Modified (304)

- Hide success(2xx): 隐藏响应状态码为2xx的URL请求, 如: 响应状态码为200的URL请求将隐藏掉。
- Hide non-2xx: 隐藏响应状态码不是2xx的URL请求。
- Hide Authentication demands(401,407): 隐藏认证 (响应状态码为: 401 407) 的URL请求。
- Hide redirects(300,301,302,303,307): 隐藏重定向 (响应状态码为: 300, 301, 302, 303, 07) 的URL请求。
- Hide Not Modified(304): 隐藏不是缓存的状态码 (304) 的URL请求。

6. 【响应类型和大小】

根据响应类型和数据大小, 配置过滤

Response Type and Size

☐ Time HeatMap ☐ Block scriptfiles

☐ Hide smaller than ☐ Block image files

☐ Hide larger than ☐ Block SWF files

☐ Block CSS files

- 下拉列表选择要显示的类型

- Hide smaller than：隐藏小于指定大小响应结果的请求。
- Hide larger than：隐藏大于指定大小响应结果的请求。
- Time HeatMap：时间的热图。
- Block script files：阻止返回正常JS文件。如果响应是脚本文件，那么响应 404。
- Block image files：阻止返回正常图片文件。如果响应是图片文件，那么响应 404。
- Block SWF files：阻止返回正常SWF文件。如果响应是SWF 文件，那么响应 404。
- Block CSS files：阻止返回正常CSS文件。如果响应是 CSS 文件，那么响应 404。

7.【响应头】

根据响应头信息，配置过滤

Response Headers

☐ Flag responses that set cookies

☐ Flag responses with headers

☐ Delete response headers

☐ Set response header

- Flag responses that set cookies：会话列表的响应中有cookies信息的加上标识（斜体）。
- Flag responses with headers：会话列表里标记响应中带有特定header的请求（即加粗）。
- Delete response headers：会话列表里删除响应信息中，有指定的Header字段的会话。
- Set response header：在响应中添加header字段。

6.8【Fiddler工具使用】 AutoResponder 自动响应器（了解）

一、学习目标

1. AutoResponder 界面说明

二、知识点

1.【AutoResponder 界面说明】

Fiddler can return previously generated responses instead of using the network.

☒ Enable rules
☐ Accept all CONNECTs
☒ Unmatched requests passthrough
☐ Enable Latency

Add Rule
Import...
Group

Rule Editor

StringToMatch[2]
Test...
Save

Local file to return or *Action to execute
Match only once

If request matches...	then respond with...
☒ http://zentaomax.demo.zentao.net/user-lo...	NEW_RESPONSE_21_25_10
☒ StringToMatch[2]	

(1) 选项：

- **Enable rules**：启用规则。勾选此选项，自动响应器才会激活。
 - **Unmatched requests passthrough**：没有匹配到的请求予以通过。
- 建议开启，没开启时，规则匹配外的请求将失败，报HTTP 404错误。

说明：如果不勾选此选项，抓包的时候，会返回：

```
[Fiddler] The Fiddler AutoResponder is enabled, but this request did not match any of the listed rules. Because the "Unmatched requests passthrough" option on the AutoResponder tab is not enabled, this HTTP/404 response has been generated.
```

意思是：Fiddler的自动响应器激活了，但是请求没匹配到任何列表中的规则。而且因为跳过非匹配请求选项没有激活，所以产生了

```
http/404
```

返回结果。

- **Enable Lagency**：延迟响应时间，单位 ms（毫秒）。
勾选了该选项，则在规则里面就可以设置，立即返回响应，还是隔多少毫秒返回响应。
- **Accept all CONNECTS**：允许所有连接。

推荐：我们在适用Fiddler的时候，一般都会勾选**Enable rules**和**Unmatched requests passthrough**选项。

(2) 按钮：

1. **Add rule**：添加规则，点击此按钮则，会在规则框里插入一条新的规则。
2. **import**：支持导入之前捕获的 .saz 文件。
3. **Group**：对列表中所创建的规则进行分组。
在列表中选一些规则，点击 Group 按钮，就会把这些规则进行单独分组，选中需要合并的组，点击 **Ungroup**，就可以进行解组。

(3) Rule Editor（规则编辑）：

- 第一行：是设置匹配条件。点开下拉，会看到很多Fiddler自带的匹配条件。
即：填写要匹配劫持的地址。

匹配方式：

(1) 纯粹字符串匹配

Fiddler将会按照不区分字母大小写来匹配字符串。

(2) 前缀NOT匹配

跟上面的类似，Fiddler还有一个非字符串匹配的规则：

NOT：该规则会匹配url中不含给定字符的所有请求。

(3) 前缀EXACT匹配（精确匹配）

表达式以EXACT：为前缀，此匹配区分字母大小写。

(4) 前缀REGEX匹配

正则表达式：Fiddler支持以regex:为前缀的正则表达式语法，

使用 .+ 匹配一个或多个字符，

使用 `.` 匹配0个或多个字符，

使用 `^` 匹配字符串开始位置，

使用 `$` 匹配字符串结尾位置。

- 第二行：是设置返回。点开下拉，会看到很多Fiddler自带的返回值类型。

即：就是劫持后真正要访问的地址，或者返回的结果。

可以重定向请求，也可以替换返回结果。

- 包括不同的响应状态码。
- `Find a file....`：找到一个文件作为响应。

6.9【Fiddler工具使用】设置断点（了解）

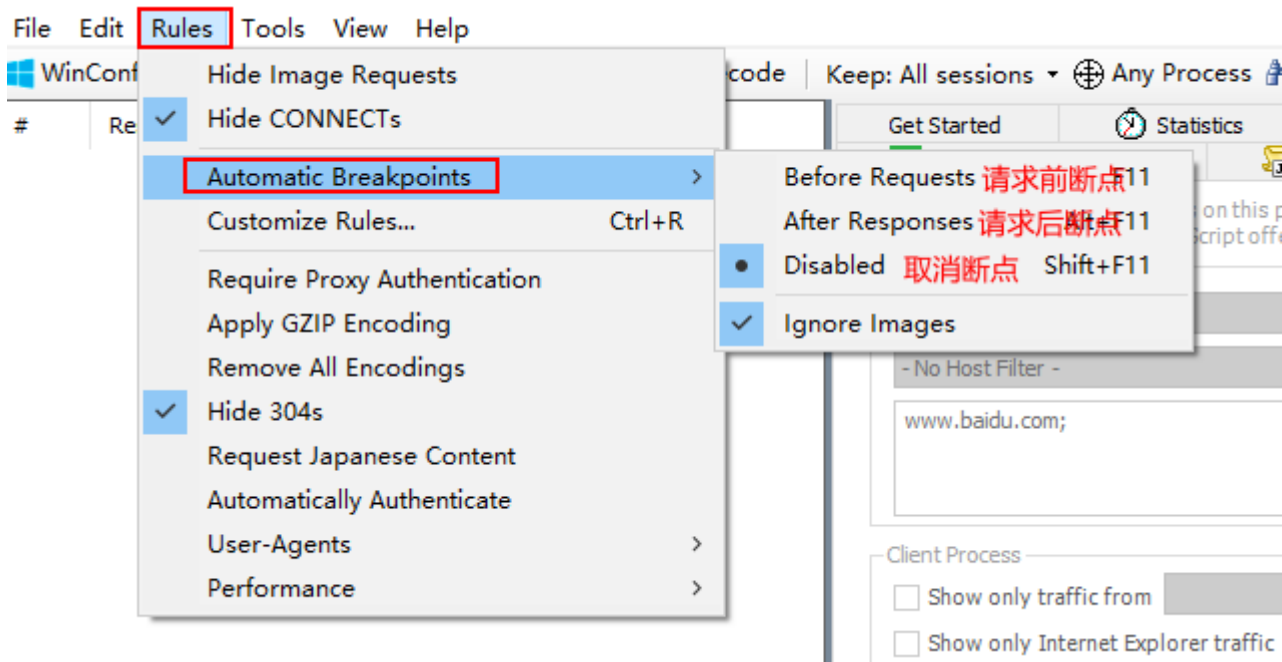
一、学习目标

1.全局断点

2.局部断点

二、知识点

1.【全局断点】



说明：

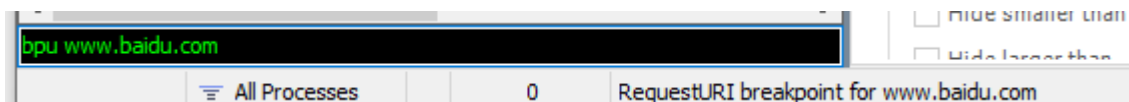
Before request 在请求发送之后 从fiddler到服务器之前

After Responses 在服务器响应之后 从fiddler到客户端之前

注意

在修改之后 点击Run to Completion就能提交了

2.【局部断点】



已经知道了某个接口的请求地址，只需要针对这一条请求打断点调试，在命令行中输入指令就可以了

1.请求前断点 (before response):

bpu 命令行输入: bpu 请求地址

取消断点, 在命令行输入: bpu 回车就可以了

2.响应后断点 (after requests) :

在命令行输入: bpafter 请求地址

取消断点, 在命令行输入: bpafter 回车就可以了

3.bps在特定http状态码时中断。

4.bpv/bpm在特定请求method时中断。

6.10 【接口请求】requests模拟发送get请求

一、学习目标

1.简单的get请求

二、知识点

1.【简单的get请求】

源码:

```
def get(url, params=None, **kwargs):
    r"""Sends a GET request.

    :param url: URL for the new :class:`Request` object.
    :param params: (optional) Dictionary, list of tuples or bytes to send
        in the query string for the :class:`Request`.
    :param **kwargs: Optional arguments that ``request`` takes.
    :return: :class:`Response` <Response> object
    :rtype: requests.Response
    """

    kwargs.setdefault('allow_redirects', True)
    return request('get', url, params=params, **kwargs)
```

- 语法:

```
requests.get(url, params=None, **kwargs)
```

- 参数:

url (必填参数) : 接口的请求地址。

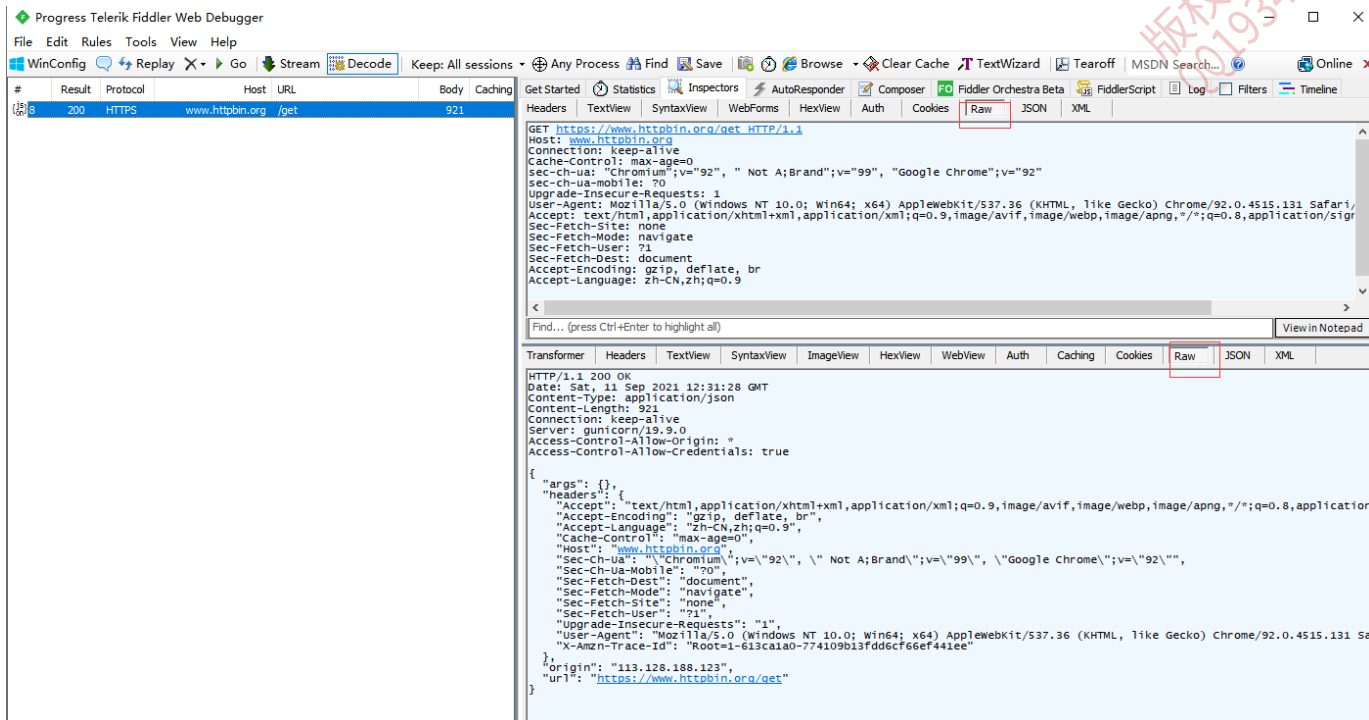
params (选填参数) : 传url后边的查询字符串参数, 如?name=xiaoming&age=18。

**kwargs (选填, 不定长参数) : 代表还可以传其他参数, 如headers,cookies等。

- 返回值:

响应对象

- 代码示例:



```
import requests

res = requests.get('http://www.httpbin.org/get')
print(res.text) #text是响应对象的属性，指响应体的文本内容
```

这里的请求地址是“<http://www.httpbin.org/get>”，/前面是域名或者主机地址，/后面是接口的地址。所以在我们实际的项目中，根据被测项目的不同，主机地址是变化的，不变的是接口的地址，我们一般把这两部分分开，将主机地址或域名设置为配置项，将来写到配置文件当中。

```
import requests
from urllib.parse import urljoin

#urljoin方法可以拼接url
server = 'http://www.httpbin.org' #以后可以把这个参数写到配置文件
api_url = '/get'
url = urljoin(server, api_url)

res = requests.get(url)
print(res.text) #text是响应对象的属性，指响应体的文本内容
```

6.11 【接口请求】requests模拟发送post请求

一、学习目标

1.简单的post请求

二、知识点

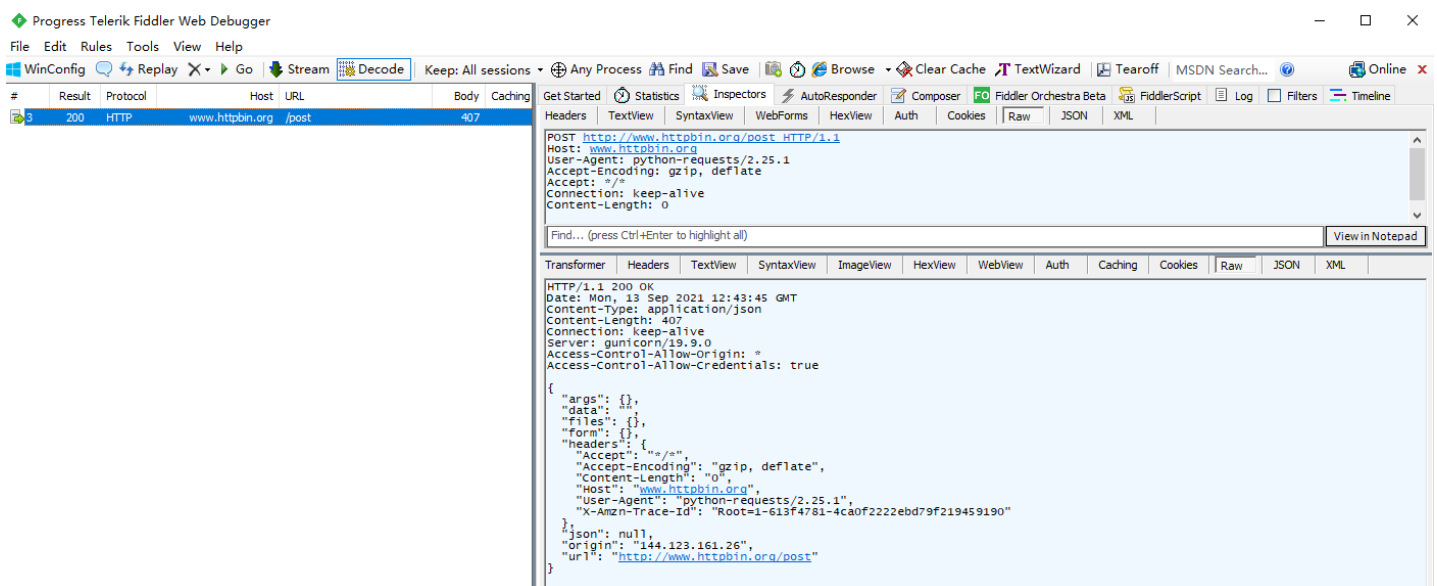
1.【简单的post请求】

源码：

```
def post(url, data=None, json=None, **kwargs):
    """Sends a POST request.

    :param url: URL for the new :class:`Request` object.
    :param data: (optional) Dictionary, list of tuples, bytes, or file-like
        object to send in the body of the :class:`Request`.
    :param json: (optional) json data to send in the body of the :class:`Request`.
    :param \**kwargs: Optional arguments that ``request`` takes.
    :return: :class:`Response` object
    :rtype: requests.Response
    """

    return request('post', url, data=data, json=json, **kwargs)
```



• 语法:

```
requests.post(url, data=None, json=None, params=None, **kwargs)
```

• 参数:

url (必填参数) : 接口的请求地址。

data (选填参数) :接口的请求体, 传入的参数可以是字典格式 (推荐), 也可以是字符串。

json (选填参数) : 接口的请求体, 传入的参数可以是字典格式 (推荐), 也可以是字符串。

**kwargs (选填, 不定长参数) : 代表还可以传其他参数, 如headers,cookies等。

• 返回值:

响应对象

• 示例:

```
import requests

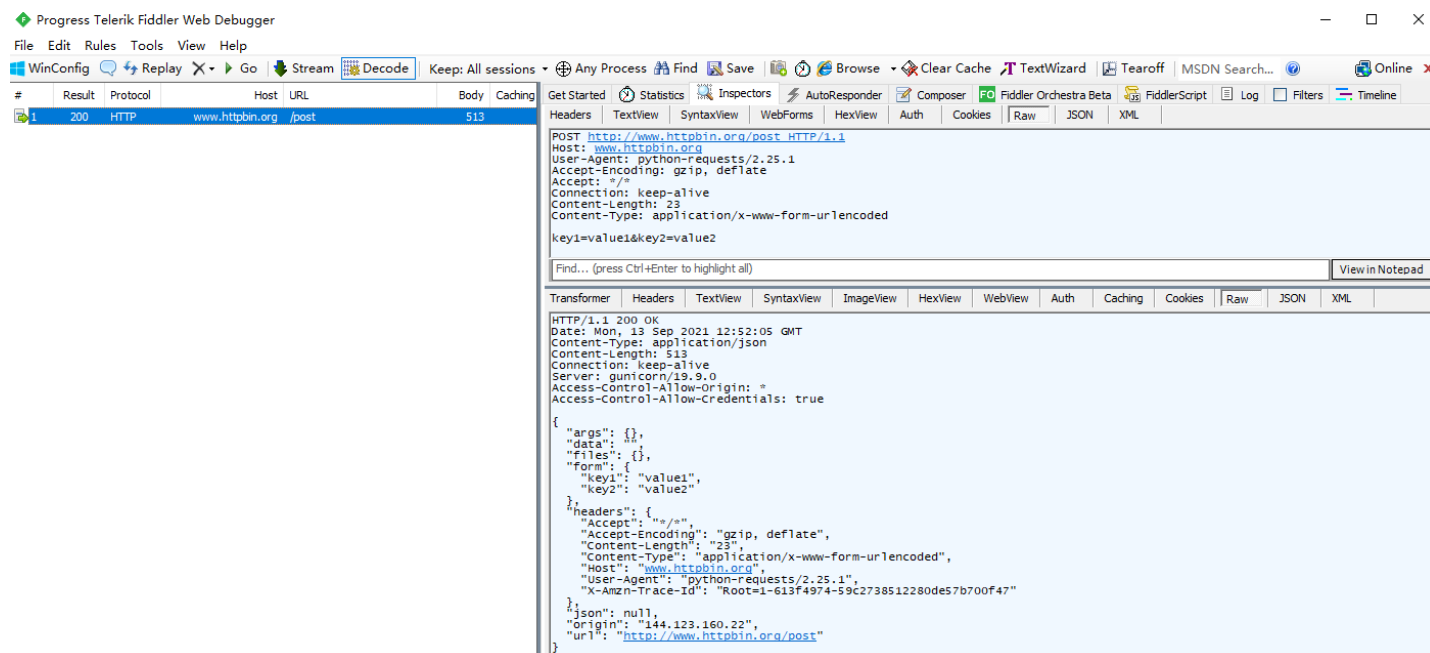
res = requests.post('http://www.httpbin.org/post')
print(res.text) #text是响应对象的属性, 指响应体的文本内容
```


6.12 【接口请求】POST请求体的data格式传参

一、学习目标

1. data 格式传参

二、知识点



1. 【application/x-www-form-urlencoded格式传参】

当看到接口请求头的Content-Type: application/x-www-form-urlencoded时，需要用到requests.post()方法中的data参数进行传参，传参的格式可以是字符串，也可以是字典。

#Content-Type: application/x-www-form-urlencoded

```
import requests
url = 'http://www.httpbin.org/post'
```

#data的字典格式

```
d = {
    "key1": "value1",
    "key2": "value2"
}
res = requests.post(url, data=d)
print(res.json())
```

#data的字符串格式

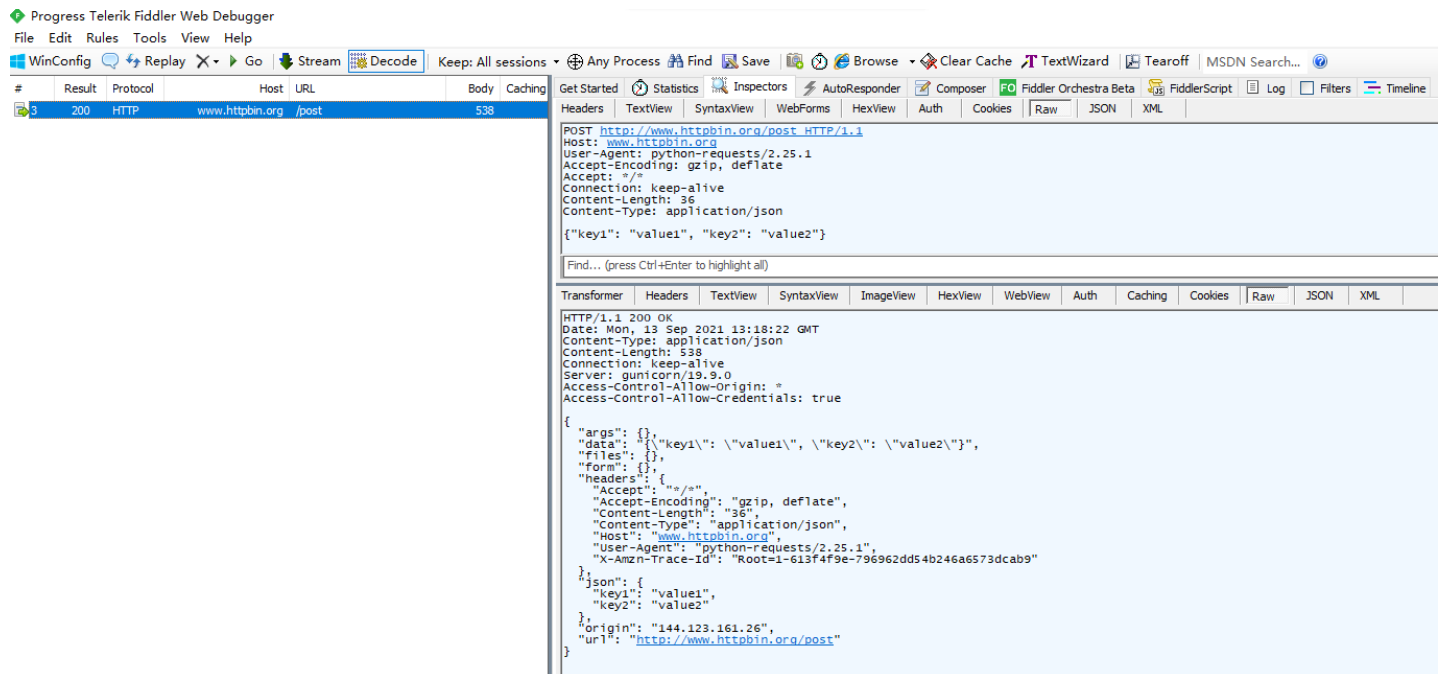
```
s = 'key1=value1&key2=value2'
res = requests.post(url, data=s)
print(res.json())
```

6.13 【接口请求】POST请求体的json格式传参

一、学习目标

1. json 格式传参

二、知识点



1. 【application/json格式传参】

这里我们可以看到的Content-Type:application/json，**application/json** 这个 Content-Type 作为响应头大家肯定不陌生。实际上，现在越来越多的人把它作为请求头，用来告诉服务端消息主体是序列化后的JSON 字符串。由于JSON 规范的流行，服务端语言也都有处理JSON 的函数，使用JSON 不会遇上什么麻烦。

```
#Content-Type: application/json

import requests
url = 'http://www.httpbin.org/post'
d = {
    "key1": "value",
    "key2": "value2"
}
res = requests.post(url, json=d)
print(res.json())
```

其实，Content-Type:application/json这种格式的接口，也可以通过data参数进行传参，但是需要将data的字典数据类型转成json数据类型才能传参。

```
#json用data传参
url = 'http://www.httpbin.org/post'
d = {
    "key1":"value",
    "key2":"value2"
}
#d转换为json数据
data_json = json.dumps(d)

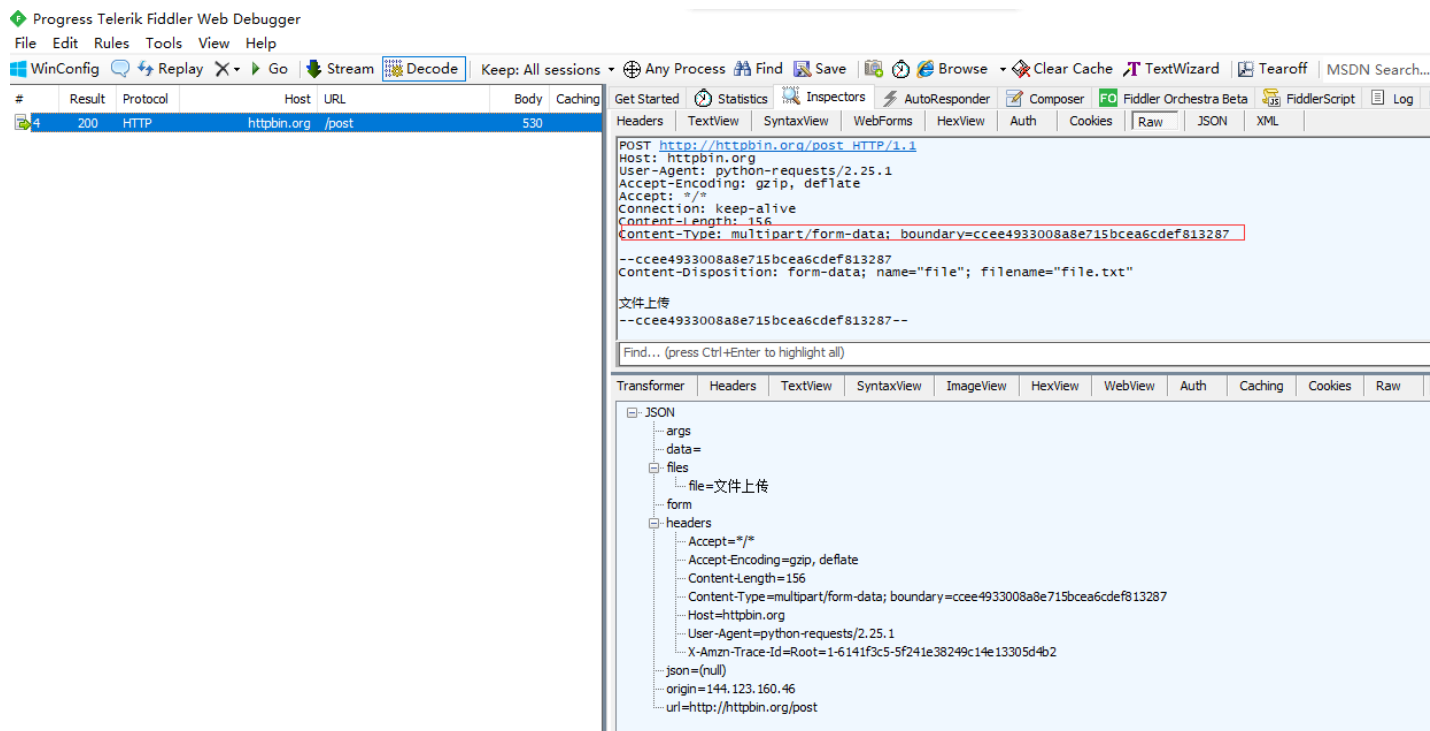
res = requests.post(url,data=data_json)
print(res.json())
```

6.14 【接口请求】POST请求体的files格式传参(文件上传)

一、学习目标

1. files 格式传参

二、知识点



The screenshot shows the Fiddler Web Debugger interface. The top pane displays the raw HTTP request details for a POST to `http://httpbin.org/post`. The request headers include `Host: httpbin.org`, `User-Agent: python-requests/2.25.1`, `Accept-Encoding: gzip, deflate`, `Connection: keep-alive`, `Content-Length: 156`, and `Content-Type: multipart/form-data; boundary=ccee4933008a8e715bcea6cdef813287`. The body of the request is a multipart form with a single part named `file` with the filename `file.txt`. The bottom pane shows the JSON response from the server, which includes the `files` section: `{ "files": { "file": "文件上传" } }`.

1. 【multipart/form-data格式传参】

图中的这种数据传输格式，就是典型的文件上传接口的格式类型，通过抓包结果可以看出，这个接口只有一个参数（通过name来判断）file参数。

```
#Content-Type: multipart/form-data
```

```
import requests
url = 'http://httpbin.org/post'
d = {
    "file": open('./file.txt', 'rb')
}
res = requests.post(url, files=d)
print(res.json())
```

6.15 【接口请求】requests模拟QueryString查询字符串传参

一、学习目标

1. params 查询字符串传参

二、知识点

The screenshot shows the Fiddler Web Debugger interface. The top menu bar includes File, Edit, Rules, Tools, View, and Help. Below the menu is a toolbar with various icons for actions like WinConfig, Replay, Go, Stream, Decode, and others. The main window is divided into several panes. The top pane shows a list of captured HTTP sessions. The middle pane displays the details of the selected session, including the request and response headers, body, and caching information. The bottom pane shows the JSON representation of the request and response data.

The selected session is a POST request to `http://httpbin.org/post?name=xiaoming&age=18`. The request headers are:

```
Host: httpbin.org
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Length: 19
Content-Type: application/x-www-form-urlencoded
```

The request body is:

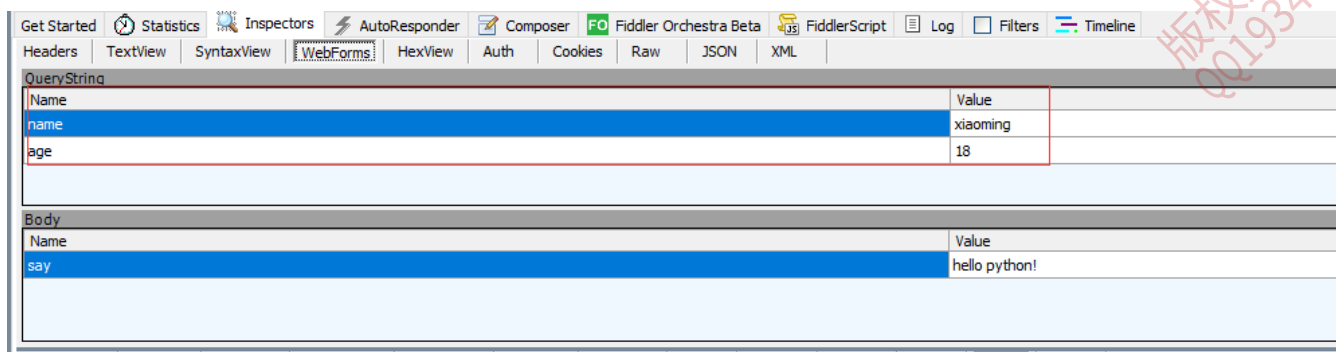
```
say=hello+python%21
```

The JSON representation of the request is shown in the bottom pane:

```
{
  "args": {
    "age": "18",
    "name": "xiaoming"
  },
  "data": "",
  "files": {},
  "form": {
    "say": "hello python!"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Content-Length": "19",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.25.1",
    "X-Amzn-Trace-Id": "Root=1-6141fd96-3a035d4759b451c72c8d2850"
  },
  "json": null,
  "origin": "113.128.188.74",
  "url": "http://httpbin.org/post?name=xiaoming&age=18"
}
```

1. 【QueryString查询字符串传参】

像这种<http://httpbin.org/post?name=xiaoming&age=18>，在url中存在?等一系列参数，这些参数就是查询字符串，也可以在fiddler的WebForms中直观查看



代码示例：

#querystring查询字符串参数

```
import requests
url = 'http://httpbin.org/post'
d = {
    "name": "xiaoming",
    "age": "18"
}

d1 = {
    "say": "hello python!"
}
res = requests.post(url, params=d, data=d1)
print(res.json())
```

同理，get请求也是这种传参方式：

#get请求传递查询字符串

```
url = 'http://httpbin.org/get'

d = {
    "name": "xiaoming",
    "age": "18"
}
res = requests.get(url, params=d)
print(res.json())
```

6.16 【接口请求】requests模拟headers传参

一、学习目标

1.headers传参

二、知识点

1. 【headers传参】

User-Agent，浏览器的用户代理字符串。告诉HTTP服务器，客户端使用的[操作系统](#)和浏览器的名称和版本。

Content-Type，来表示具体请求中的媒体类型信息，确切的说客户端告知服务端，自己即将发送的请求消息携带的数据结构类型，好让服务端接收后以合适的方式处理。

#post中headers的参数传递

```
url = 'http://httpbin.org/post?name=xiaoming&age=18'
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/48.0.2564.48 Safari/537.36",
    "Content-Type": "application/x-www-form-urlencoded"
}
res = requests.post(url, headers=headers)
print(res.json())
```

同理，get请求也是这种传参方式：

#get中headers的参数传递

```
url = 'http://httpbin.org/get?name=xiaoming&age=18'
headers = {
    "User-Agent": "Mozilla/6.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/48.0.2564.48 Safari/537.36"
}
res = requests.get(url, headers=headers)
print(res.json())
```

6.17 【接口请求】requests模拟cookies传参

一、学习目标

1.cookies 传参

三、知识点

The screenshot shows the Fiddler Web Debugger interface. The top pane displays the request details for a GET request to `http://httpbin.org/cookies?name=xiaoming&age=18`. The request headers are visible, including `User-Agent`, `Accept-Encoding`, `Connection`, `Content-Type`, and `Cookie: sessionId=123456`. The bottom pane shows the response body, which is a JSON object: `{ "cookies": { "sessionId": "123456" } }`.

1. 【cookies传参】

#cookies参数传递

```
import requests
url = 'http://httpbin.org/cookies'

query_data = {
```

```

    "name": "xiaoming",
    "age": "18"
}

header_data = {
    "Content-Type": "application/x-www-form-urlencoded",
    "User-Agent": "Mozilla/5.0 (Windows NT 6.3; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/48.0.2564.48 Safari/537.36"
}

cookies_data = {
    "sessionId": "123456"
}
res = requests.get(url, params=query_data, headers=header_data, cookies=cookies_data)
print(res.json())

```

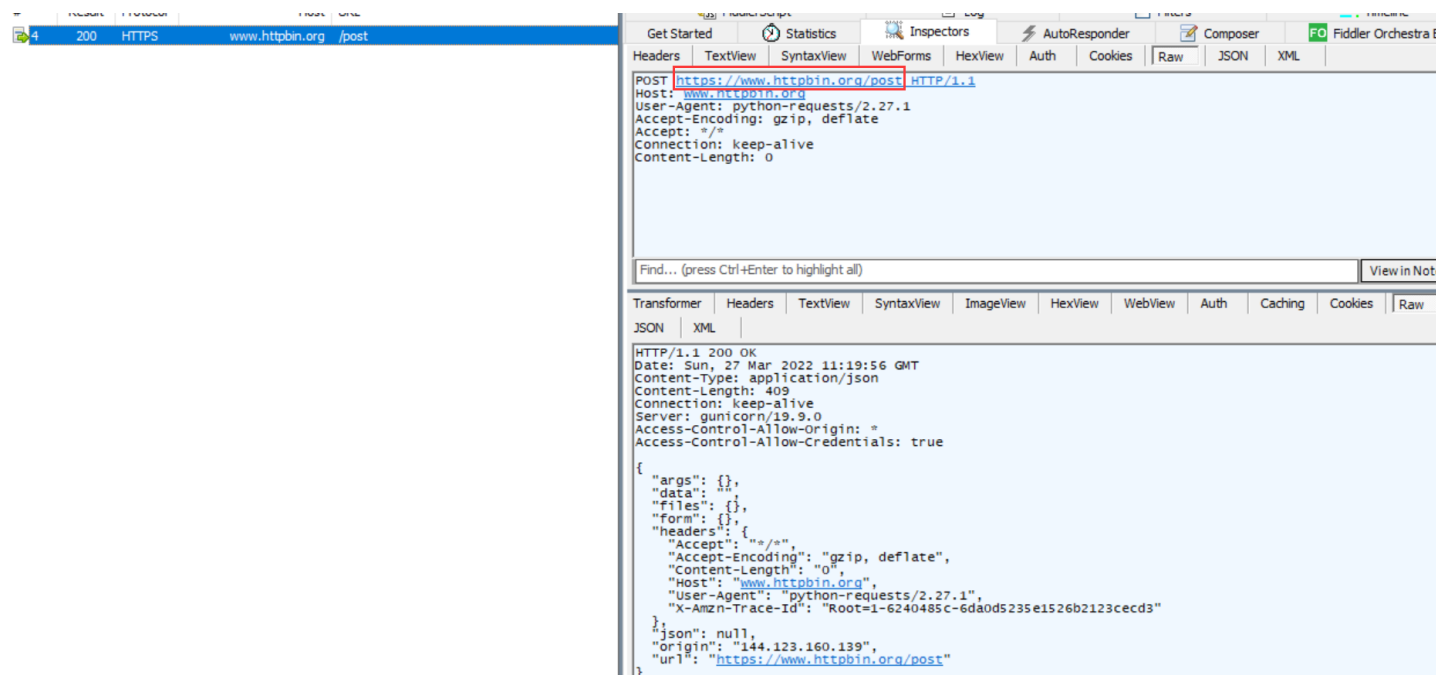
同理，post请求传cookies参数也是一样的

6.18 【接口请求】requests模拟发送https请求

一、学习目标

1. 【发送https请求】

二、知识点



1. 【发送https请求】

注：当请求https接口时，会报如下错误：

```
ValueError: check_hostname needs a SSL context with either CERT_OPTIONAL or CERT_REQUIRED
```

这是由于urllib3版本太高导致的，所以在模拟https请求前，需要降低urllib3：

```
pip install urllib3==1.23 -i http://pypi.douban.com/simple --trusted-host pypi.douban.com
```

fiddler作为中间人获取了网站的公钥和客户端的密钥，然后伪装成客户端和服务端通信（使用网站公钥和自己的密钥），再伪装成服务端把收到的信息发送给客户端（使用自己的公钥和客户端的密钥）。fiddler的公钥不是ca认证的，所以requests就无法通过内置的ca公钥解密，验证就会失败。所以要是不验证，要不指定和特定的证书进行对比验证，就能解决问题。

代码示例：

```
import requests
url = 'https://www.httpbin.org/post'
res = requests.post(url, verify=False) #关闭ssl认证
print(res.json())
```

执行代码，我们发现，将验证设置忽略后，可以跳过SSL验证，但存在一个警告信息InsecureRequestWarning：

```
InsecureRequestWarning: Unverified HTTPS request is being made to host '127.0.0.1'. Adding
certificate verification is strongly advised. See:
https://urllib3.readthedocs.io/en/latest/advanced-usage.html#ssl-warnings
InsecureRequestWarning,
```

【最终版】消除警告：

post:

```
#https请求

import requests
import urllib3
from urllib3.exceptions import InsecureRequestWarning

urllib3.disable_warnings(InsecureRequestWarning)
url = 'https://www.httpbin.org/post'
res = requests.post(url, verify=False) #关闭ssl认证
print(res.json())
```

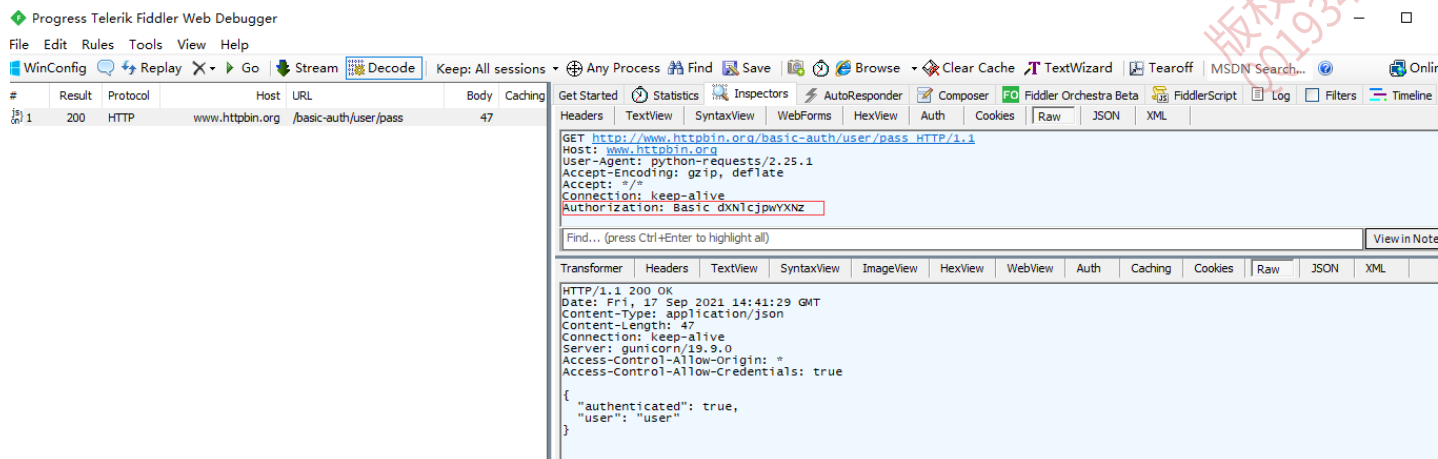
6.19 【接口请求】requests模拟发送认证接口

一、学习目标

- 1.HTTP Basic Auth基本身份认证
- 2.Digest Authentication摘要式身份认证
- 3.token认证

二、知识点

1. 【HTTP Basic Auth基本身份认证】



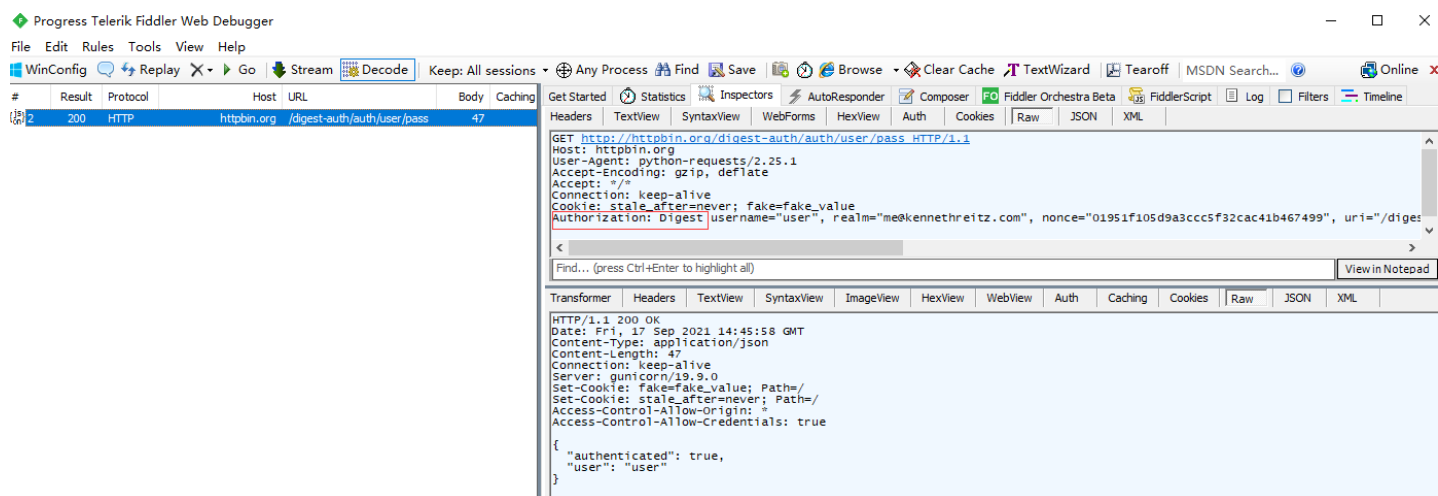
代码示例:

```
import requests
from requests.auth import HTTPBasicAuth

url = 'http://www.httpbin.org/basic-auth/user/pass'

res = requests.get(url,auth=HTTPBasicAuth('user','pass'))
print(res.json())
```

2. 【Digest Authentication摘要式身份认证】



代码示例:

```
#2. 摘要式身份认证
from requests.auth import HTTPDigestAuth
url = 'http://httpbin.org/digest-auth/auth/user/pass'
res = requests.get(url,auth=HTTPDigestAuth('user','pass'))
print(res.json())
```

3. 【token认证】

Token是服务端生成的一串字符串，以作客户端进行请求的一个令牌，当第一次登录后，服务器生成一个Token便将此Token返回给客户端，以后客户端只需带上这个Token前来请求数据即可，无需再次带上用户名和密码。

注：token值一般通过请求头中的Authorization参数传递。在登录成功后会返回token值，将token值传入登录后的接口的请求头即可成功请求！如 `"Authorization": "dsfsgfh1232465"`

代码示例：

```
#3.token认证
#事先模拟登录
url = 'http://请求地址'
header = {
    "Authorization": "dsfdsfdsfdsfd"
}
res = requests.post(url,headers=header)
print(res.text)
```

6.20 【接口响应】解析text/html响应

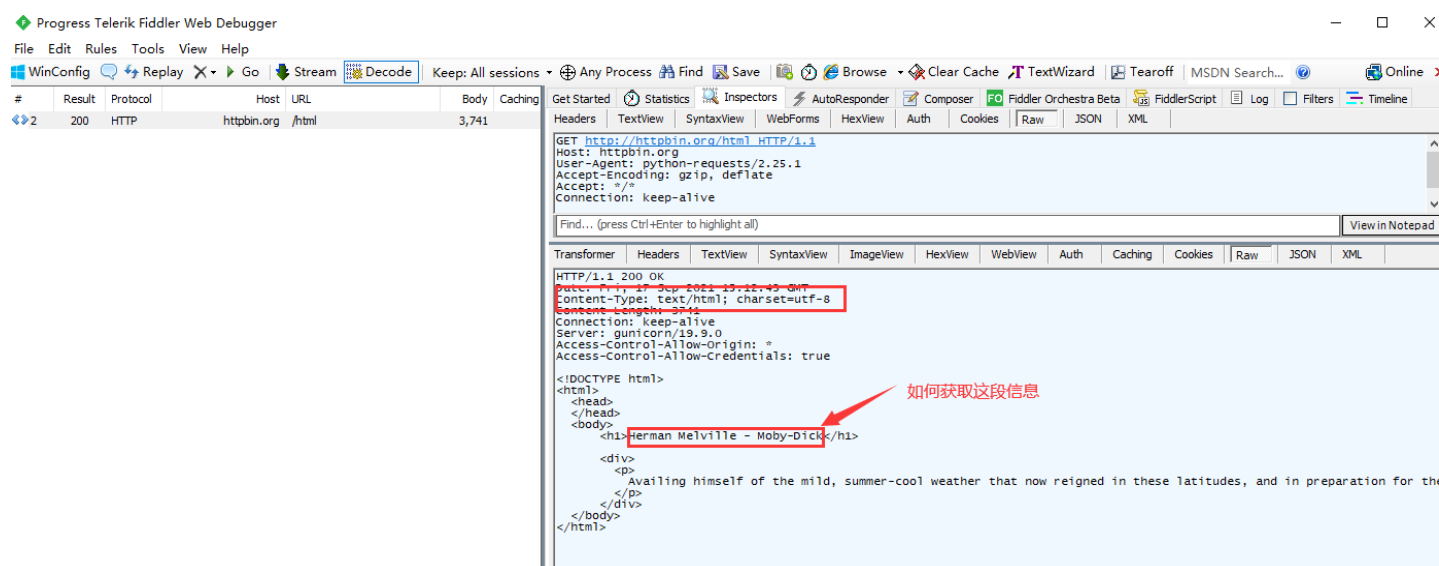
一、学习目标

1.解析提取text/html响应体

二、知识点

1.【解析text/html响应体】

为什么要解析响应体？我们做接口自动化是需要断言的，即断言接口返回信息是否符合预期结果。当看到响应体格式为Content-Type: text/html时，则可以通过res.text获取响应体的全部内容，然后用正则表达式提取想要的信息。



代码示例：

```
#接口响应：text/html解析
import re
import requests
url = 'http://httpbin.org/html'
res = requests.get(url)
print(res.status_code) #接口的状态码

#响应体内容
content = res.text #获取响应体内容
print(content)

#解析响应体内容
```

```
res2 = re.findall('<h1>(.*?)</h1>', content)[0]
print(res2)
```

说明：re.findall()：正则表达式提取文本中匹配的所有数据，返回列表，所有后面加[0]，表示取列表中第一个值。

(.*?)：表示匹配任意字符，固定写法。将你需要截取的文本位置用它来代替。

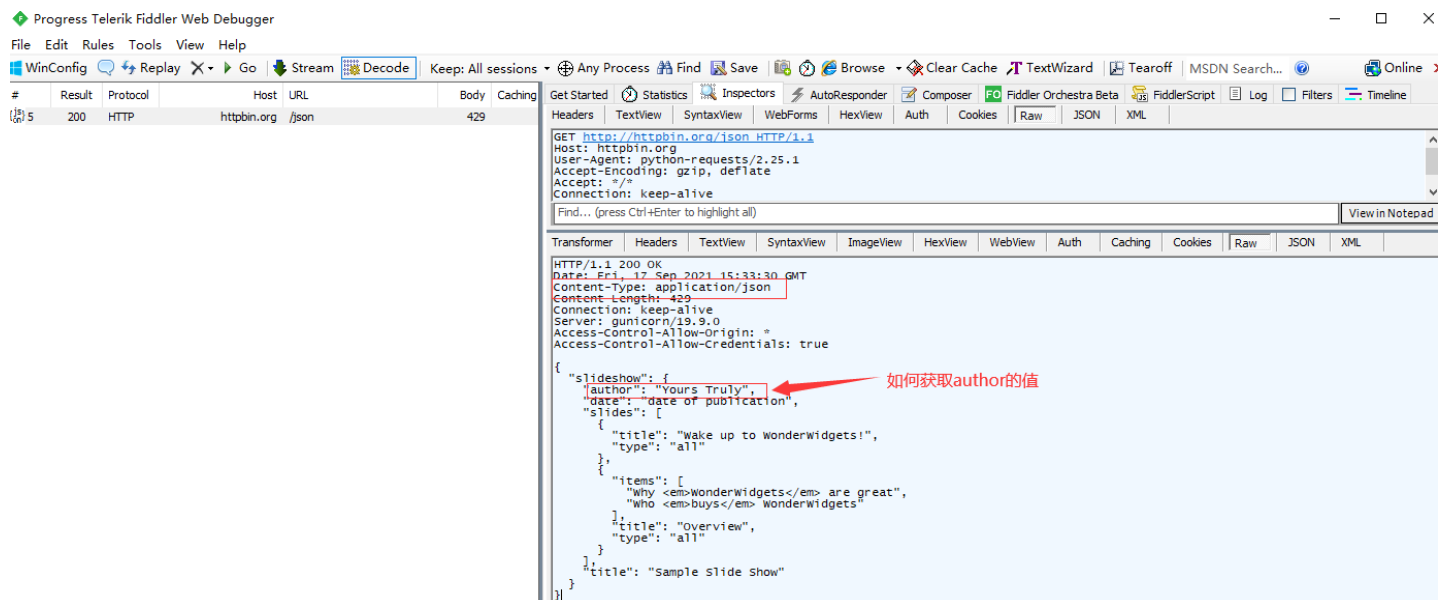
6.21 【接口响应】解析application/json响应

一、学习目标

1.解析提取application/json响应体

二、知识点

1.【解析提取application/json响应体】



顾名思义，这种格式的响应体是json格式，我们可以通过res.json()方法先获取响应体的全部内容。在python中json格式和字典数据类型类似，都是{}形式，由于python中只有字典数据类型而没有json，所以python会自动把json数据认作是字典数据，我们可以根据字典取值来提取需要的特定值。

#接口响应：json解析

```
import requests
url = 'http://httpbin.org/json'
res = requests.get(url)
print(res.status_code)    #获取状态码
```

#获取响应体的内容

```
content = res.json()
print(content, type(content))
```

#解析接口响应

```
author = content['slideshow']['author']
print(author)
```

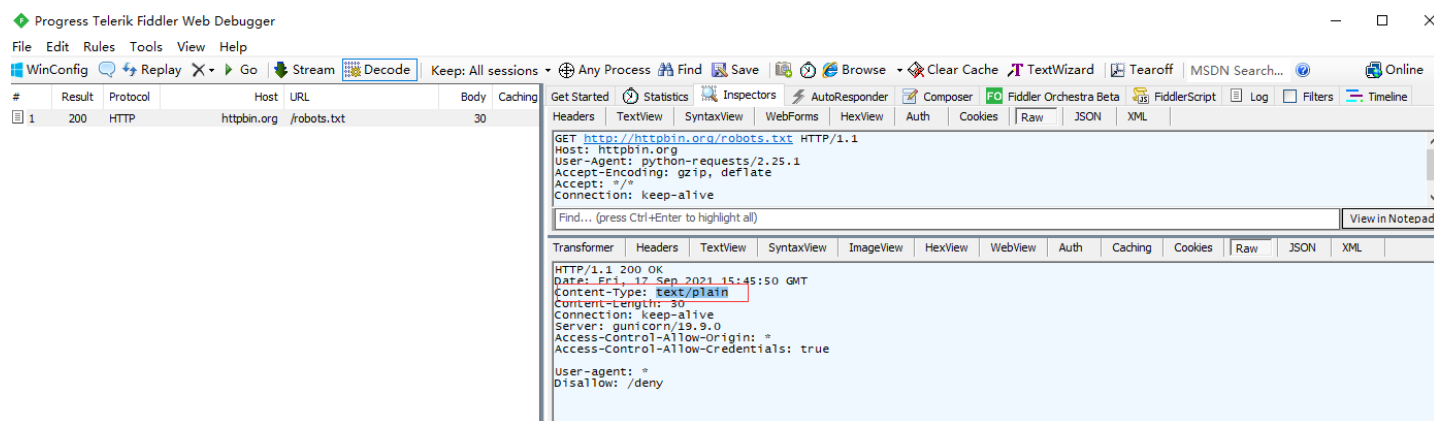
6.22 【接口响应】解析静态文件响应（文件下载）

一、学习目标

1.文件下载

二、知识点

1.【获取静态文件】



静态资源文件，是通过`res.content`来获取文件的二进制内容，将二进制内容写入文本即可达到下载文件的效果。

代码示例：

#接口响应：静态文件解析

```
import requests
url = 'http://httpbin.org/robots.txt'
res = requests.get(url)
print(res.status_code) #获取状态码

#获取响应体内容
c = res.content
print(c,type(c))

#写入文件
with open('robots.txt','wb') as f:
    f.write(c)
```

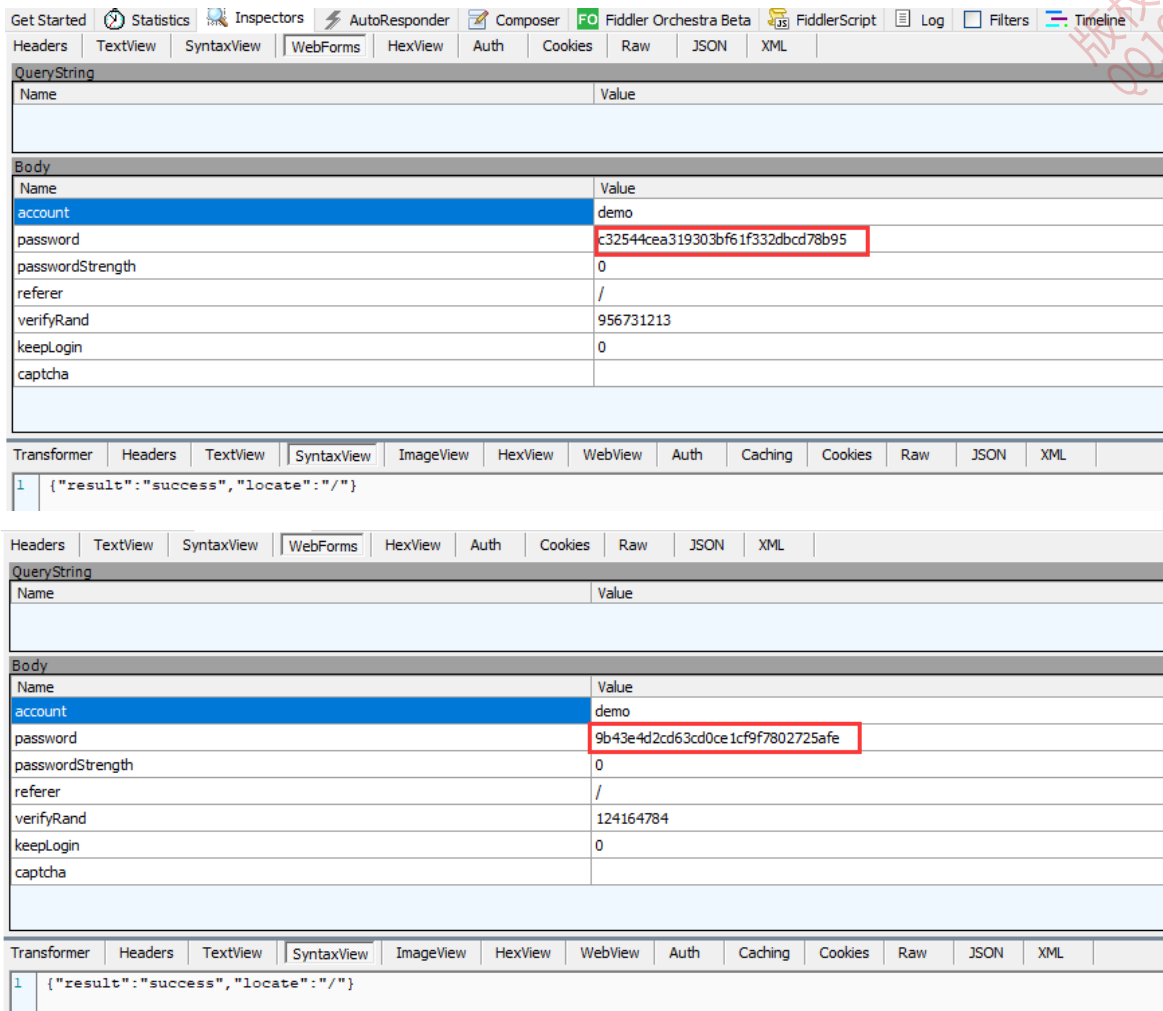
6.23 【多接口请求】前端页面中js解密

一、学习目标

1.js破解禅道登录

二、知识点

地址：<http://zentaomax.demo.zentao.net>



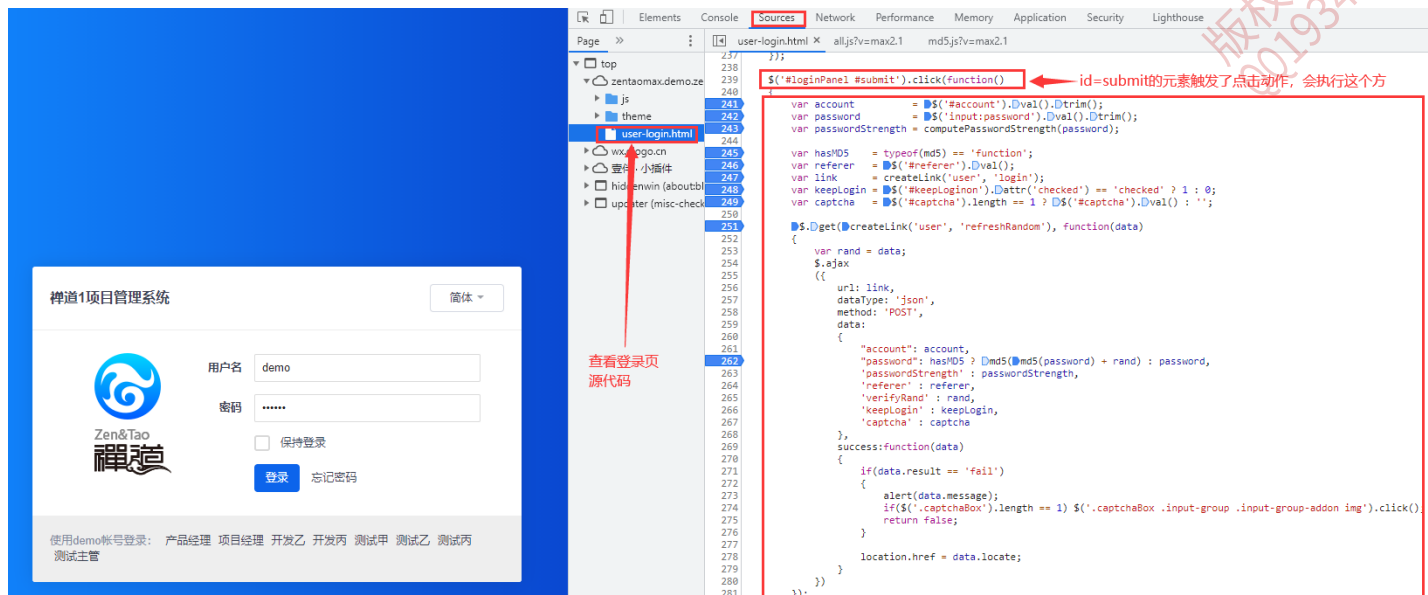
这是禅道两次登录抓的包，可以看到，两次登录的密码都被加密了，而且每次加密的数值还是变化的。这就造成当前抓取的密码不能用到下一次请求中。

1. 【js破解禅道登录】

既然要获取密码的加密值，我们需要搞清楚前端页面的密码加密逻辑，这需要有一点js前端基础，否则会很难理解。

- 步骤一：找到点击登录时，触发了哪段js代码





说明：在js语法中#代表id属性，刚刚我们已经知道了登录按钮的id为submit,查看登录页源代码的js中，发现点击动作出发了如图所示的js函数。

- 步骤二：找到js代码对密码的加密逻辑

```
$('#loginPanel #submit').click(function()
{
    var account      = $('#account').Dval().Dtrim();
    var password      = $('#input:password').Dval().Dtrim();
    var passwordStrength = computePasswordStrength(password);

    var hasMD5        = typeof(md5) == 'function';
    var referer        = $('#referer').Dval();
    var link           = createLink('user', 'login');
    var keepLogin      = $('#keepLoginon').Dattr('checked') == 'checked' ? 1 : 0;
    var captcha        = $('#captcha').length == 1 ? D$('#captcha').Dval() : '';

    $.Dget(DcreateLink('user', 'refreshRandom'), function(data)
    {
        var rand = data;
        $.ajax
        ({
            url: link,
            dataType: 'json',
            method: 'POST',
            data:
            {
                "account": account,
                "password": hasMD5 ? Dmd5(Dmd5(password) + rand) : password,
                "passwordStrength": passwordStrength,
                "referer": referer,
                "verifyRand": rand,
                "keepLogin": keepLogin,
                "captcha": captcha
            },
            success: function(data)
            {
                if(data.result == 'fail')
                {
                    alert(data.message);
                    if($('#captchaBox').length == 1) $('#captchaBox .input-group .input-group-addon img').click();
                    return false;
                }
                location.href = data.locate;
            }
        })
    });
});
```

说明：这一段js代码就表明了密码的加密逻辑，hasMD5 ? md5(md5(password) + rand) : password，这是一段匿名函数，他需要传入两个值rand,password。

并将这两个值进行md5(md5(password) + rand)运算。所以，我们需要：找到rand,password值和md5方法是怎么来的；

- 步骤三：获取rand,password值和md5方法。

通过谷歌浏览器的断点，来获取值。

```

239 $('#loginPanel #submit').click(function()
240 {
241     var account      = $('#account').Dval().Dtrim();    account = "demo"
242     var password     = $('#input:password').Dval().Dtrim(); password = "123456"
243     var passwordStrength = computePasswordStrength(password); passwordStrength = 0
244
245     var hasMD5       = typeof(md5) == 'function';    hasMD5 = true
246     var referer      = $('#referer').Dval();    referer = "/"
247     var link         = createLink('user', 'login'); link = "/user-login.html"
248     var keepLogin    = $('#keepLoginon').Dattr('checked') == 'checked' ? 1 : 0;
249     var captcha      = $('#captcha').length == 1 ? $('#captcha').Dval() : '';
250
251     $.Dget(createLink('user', 'refreshRandom'), function(data)
252     {
253         var rand = data;
254         $.ajax
255         ({
256             url: link,
257             dataType: 'json',
258             method: 'POST',
259             data:
260             {
261                 "account": account,
262                 "password": hasMD5 ? md5(md5(password) + rand) : password,
263                 "passwordStrength" : passwordStrength,
264                 "referer" : referer,
265                 "verifyRand" : rand,
266                 "keepLogin" : keepLogin,
267                 "captcha" : captcha
268             },
269             success:function(data)
270             {
271                 if(data.result == 'fail')
272                 {
273                     alert(data.message);
274                     if($('#captchaBox').length == 1) $('#captchaBox .input-group .input-group-addon img').click();
275                     return false;
276                 }
277                 location.href = data.locate;
278             }
279         });
280     });
281

```

断点

rand：是通过这个方法获取的get(createLink('user', 'refreshRandom'), get请求：<http://zentaomax.demo.zentao.net/user-refreshRandom.html>

password：123456

md5方法：引用的js文件：<http://zentaomax.demo.zentao.net/js/md5.js?v=max2.1>

• 步骤四：编码

若要获取随机加密的密码，我们需要用python实现一遍js的加密逻辑，我们上面已经理清楚了密码是如何加密的，我们可以按照js的逻辑来用python实现。

先封装一个python执行js方法的类，用于调用js文件中的md5方法：

需要先安装模块：`pip install PyExecJS`

```

#encrypt.py
import execjs
import os

rootPath = os.path.dirname(__file__)
jsPath = os.path.join(rootPath, "md5.js")

class ExecJS(object):

    _instance = False

    def _get_js(self, name):
        js_str = ''
        with open(name, 'r', encoding="utf-8") as f:
            line = f.readline()
            while line:

```

```

        js_str = js_str + line
        line = f.readline()
    return js_str

def get_encrypt_pwd(self, function,*args):
    ctx = execjs.compile(self._get_js(jsPath))
    return ctx.call(function, *args)

if __name__ == "__main__":
    e = ExecJs()
    print(e.get_encrypt_pwd('md5', '123ABCdef*', "zx3nhqeupprmh2f"))

```

构造请求参数，发送请求：

```

import requests
#密码的加密逻辑：md5(md5(password) + rand)
#1.md5方法：md5.js
#2.password:123456
#3.rand:http://zentaomax.demo.zentao.net/user-refreshRandom.html

#获取cookie
url_cookie = 'http://zentaomax.demo.zentao.net/user-login.html'
header_cookie = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "Referer": "http://zentaomax.demo.zentao.net/my/"
}
res_cookie = requests.get(url_cookie,headers=header_cookie)
d = requests.utils.dict_from_cookiejar(res_cookie.cookies)
print(d)

#密码的加密
#1.rand获取
url_rand = 'http://zentaomax.demo.zentao.net/user-refreshRandom.html'
header_rand = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "X-Requested-With": "XMLHttpRequest",
    "Referer": "http://zentaomax.demo.zentao.net/user-login.html"
}
res_rand = requests.get(url_rand,headers=header_rand,cookies=d)
print(res_rand.text)

#2.python实现调用js文件中的方法md5
password = '123456'
from encrypt import ExecJs
e = ExecJs()
step1 = e.get_encrypt_pwd('md5',password)
step2 = e.get_encrypt_pwd('md5',step1 + res_rand.text)
print(step2)

#禅道登录

```



```
url = 'http://zentaomax.demo.zentao.net/user-login.html'
data = {
    "account": "demo",
    "password": step2,
    "passwordstrength": "0",
    "referer": "/",
    "verifyRand": res_rand.text,
    "keepLogin": "0",
    "captcha": ""
}
header = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
    "Host": "zentaomax.demo.zentao.net",
    "Connection": "keep-alive",
    "Content-Length": "127",
    "Accept": "application/json, text/javascript, */*; q=0.01",
    "X-Requested-With": "XMLHttpRequest",
    "Origin": "http://zentaomax.demo.zentao.net",
    "Referer": "http://zentaomax.demo.zentao.net/user-login.html",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "zh-CN,zh;q=0.9"
}
res = requests.post(url,data=data,headers=header,cookies=d)
print(res.text)
```

6.24 【多接口请求】requests进行session会话保持

一、学习目标

1.session会话保持

二、知识点

1. 【session会话保持】

session会话保持是将登录成功的cookies传递给登录后的接口，来告知后台当前接口是有请求权限的，那么，当我们不登录，直接模拟请求接口会怎样？

- 禅道查看个人档案接口：

```
# 个人档案接口
import requests
url = 'http://zentaomax.demo.zentao.net/my-profile.html'
query_str = {
    "onlybody": "yes"
}
header = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.75 Safari/537.36"
}
res = requests.get(url=url,headers=header,params=query_str)
print(res.text)
```

运行代码的同时，用fiddler抓取模拟的接口请求，看看接口返回有没有我们需要的信息：



西部魔影

姓名	西部魔影	性别	男
用户名	demo	邮箱	
部门	/	职位	
入职日期		权限	guest 设计开发人员 1 123 智慧 关务项目组 智慧关务项目组 智 慧关务项目组 AAAA 777777 hhhh 77777777777777 77777888
手机		微信	
电话		QQ	
邮编		通讯地址	
源代码帐号	admin	Skype	
访问次数	16048	WhatsApp	
最后登录	2021-09-23 21:04:00	Slack	
最后IP	171.83.9.16	钉钉	

修改密码 编辑档案 更换头像

Progress Telerik Fiddler Web Debugger

File Edit Rules Tools View Help

WinConfig WinConfig Replay X Go Stream Decode Keep: All sessions + Any Process Find Save Browse Clear Cache TextWizard Fiddler Orchestra Beta FiddlerScript Log Filters Timeline

#	Result	Protocol	Host	URL	Body	Caching
1	200	HTTP	zentaomax.demo.z...	/user-refreshRandom.html	9	private
2	200	HTTP	zentaomax.demo.z...	/user-login.html	33	private
3	200	HTTP	zentaomax.demo.z...	/	51,041	private
4	200	HTTP	zentaomax.demo.z...	/my/	214,862	private
11	200	HTTP	zentaomax.demo.z...	/misc-getRemind.html	133	private
12	200	HTTP	zentaomax.demo.z...	/my-profile.html?onlybody=...	29,075	private
14	200	HTTP	zentaomax.demo.z...	/my-profile.html?onlybody=...	28,543	private

Request Headers

GET /my-profile.html?onlybody=yes HTTP/1.1

Client

Accept: */*
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/93.0.4577.63 Safari/537.36

Miscellaneous

Referer: http://zentaomax.demo.zentao.net/my/

Transport

Connection: keep-alive
Host: zentaomax.demo.zentao.net

Transformer Headers Textview Syntaxview Imageview Hexview Webview Auth Caching Cookies Raw JSON XML

HTTP/1.1 200 OK
Date: Thu, 23 Sep 2021 12:59:47 GMT
Server: Apache
Set-Cookie: zentaosid=d3vqe9ue8u5oj61g13f5s4js7; path=/; HttpOnly
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: private
Pragma: no-cache
Set-Cookie: lang=zh-cn; expires=Sat, 23-Oct-2021 12:59:47 GMT; Max-Age=2592000; path=/
Set-Cookie: device=desktop; expires=Sat, 23-Oct-2021 12:59:47 GMT; Max-Age=2592000; path=/
Set-Cookie: theme=default; expires=Sat, 23-Oct-2021 12:59:47 GMT; Max-Age=2592000; path=/
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=3600; includeSubDomains
Referrer-Policy: no-referrer-when-downgrade
X-Permitted-Cross-Domain-Policies: master-only
X-Download-Options: noopen
Content-Security-Policy: form-action 'self'; connect-src 'self';
X-Frame-Options: SAMEORIGIN
Vary: Accept-Encoding
Content-Length: 28543
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; Language=UTF-8; charset=UTF-8

<!DOCTYPE html>
<html lang="zh-cn">
<head>
<meta charset="utf-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="renderer" content="webkit">
<title>地盘-我的档案 - 禅道</title>
<script>window.config={"webRoot":"/","debug":0,"appName":"","cookieLife":30,"requestType":"PATH_INFO","requestFix":"","window.lang":"submitting":"","u7a0d\u0019...","save":"","u4fdd\u00b8","expand":"","usc55\u00f0\u0016\u00e8","timeout":"","u8fde...</script>
<script>config.onlybody = "yes";</script><link rel="stylesheet" href="/theme/default/zh-cn.default.css?v=max2.1" type="text/css">
<script src="/js/all.js?v=max2.1"></script>
<style>.dropdown-menu.with-search {padding: 0; min-width: 150px; overflow: hidden; max-height: 302px;}
<.dropdown-menu > .menu-search .input-group {width: 100%;}
<.dropdown-menu > .menu-search .input-group-addon {position: absolute; right: 10px; top: 0; z-index: 10; background: none;>

查询名字，发现没有返回成功

西部魔影

View in Notepad

Alt+Q > type HELP to learn more

All Processes 1 / 7 http://zentaomax.demo.zentao.net/my-profile.html?onlybody=yes

上图可以发现，虽然接口调用成功了（状态码200），但是业务没有处理成功，说明我们的接口模拟失败了，那么到底是什么原因呢，答案就是当前的查看个人档案接口是没有登录成功的授权的，所以，我们需要先模拟登录，保持登录会话，然后再请求查看个人档案接口。

- 完整代码：

```

import requests
#密码的加密逻辑: md5(md5(password) + rand)
#1.md5方法: md5.js
#2.password:123456
#3.rand:http://zentaomax.demo.zentao.net/user-refreshRandom.html

s = requests.session() #创建会话保存器

#获取cookie
url_cookie = 'http://zentaomax.demo.zentao.net/user-login.html'
header_cookie = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "Referer": "http://zentaomax.demo.zentao.net/my/"
}
res_cookie = s.get(url_cookie, headers=header_cookie)
d = requests.utils.dict_from_cookiejar(res_cookie.cookies)
print(d)

#密码的加密
#1.rand获取
url_rand = 'http://zentaomax.demo.zentao.net/user-refreshRandom.html'
header_rand = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "X-Requested-With": "XMLHttpRequest",
    "Referer": "http://zentaomax.demo.zentao.net/user-login.html"
}
res_rand = s.get(url_rand, headers=header_rand)
print(res_rand.text)

#2.python实现调用js文件中的方法md5
password = '123456'
from encrypt import ExecJs
e = ExecJs()
step1 = e.get_encrypt_pwd('md5', password)
step2 = e.get_encrypt_pwd('md5', step1 + res_rand.text)
print(step2)

#禅道登录
url = 'http://zentaomax.demo.zentao.net/user-login.html'
data = {
    "account": "demo",
    "password": step2,
    "passwordstrength": "0",
    "referer": "/",
    "verifyRand": res_rand.text,
    "keepLogin": "0",
    "captcha": ""
}

header = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
    "Host": "zentaomax.demo.zentao.net",

```

```

"Connection": "keep-alive",
"Content-Length": "127",
"Accept": "application/json, text/javascript, */*; q=0.01",
"X-Requested-With": "XMLHttpRequest",
"Origin": "http://zentaomax.demo.zentao.net",
"Referer": "http://zentaomax.demo.zentao.net/user-login.html",
"Accept-Encoding": "gzip, deflate",
"Accept-Language": "zh-CN,zh;q=0.9"
}
res = s.post(url,data=data,headers=header)
print(res.text)

url = 'http://zentaomax.demo.zentao.net/my-profile.html'
query_str = {
    "onlybody": "yes"
}
header = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/100.0.4896.75 Safari/537.36"
}
res = s.get(url=url,headers=header,params=query_str)
print(res.text)

```

说明：在第一个请求开始前，先创建会话保持器，下面的每个请求都用这个会话保持器来发送，requests模块提供的这个会话保持器的作用就是将每次请求的最新cookies自动的加到下一个接口的cookies参数中，由于session信息一般是保存在cookies中的，所以达到保持登录的效果。

The screenshot shows the Fiddler Web Debugger interface. The left pane displays a list of requests, with the fourth request (HTTP 200) selected. The right pane shows the details of this request, including the Request Headers, Client information, Cookies, and the raw HTML response. A red arrow points to the 'Cookies' section, which lists several cookies including 'device=desktop', 'feedbackView=0', 'lang=zh-cn', and 'theme=default'. Below the cookies, the raw HTML content is displayed, showing a form for user profile management. The form includes fields for 'user-name' and 'user-role', both containing the value '西部魔影' (Xibu Moying).

加了会话保持，接口就能正常请求啦！

6.25 【多接口请求】requests进行多接口中参数传递

一、学习目标

- 1.函数返回值方式传递参数
- 2.单例实现全局参数存储

二、知识点

如何跨py文件使用s参数？

1. 【函数返回值方式传递参数】

login.py:

```
import requests
#密码的加密逻辑: md5(md5(password) + rand)
#1.md5方法: md5.js
#2.password:123456
#3.rand:http://zentaomax.demo.zentao.net/user-refreshRandom.html

def login():
    s = requests.session() #创建会话保存器

    #获取cookie
    url_cookie = 'http://zentaomax.demo.zentao.net/user-login.html'
    header_cookie = {
        "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like
        Gecko) Chrome/100.0.4896.60 Safari/537.36",
        "Referer": "http://zentaomax.demo.zentao.net/my/"
    }
    res_cookie = s.get(url_cookie,headers=header_cookie)
    d = requests.utils.dict_from_cookiejar(res_cookie.cookies)
    print(d)

    #密码的加密
    #1.rand获取
    url_rand = 'http://zentaomax.demo.zentao.net/user-refreshRandom.html'
    header_rand = {
        "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like
        Gecko) Chrome/100.0.4896.60 Safari/537.36",
        "X-Requested-With": "XMLHttpRequest",
        "Referer": "http://zentaomax.demo.zentao.net/user-login.html"
    }
    res_rand = s.get(url_rand,headers=header_rand)
    print(res_rand.text)

    #2.python实现调用js文件中的方法md5
    password = '123456'
    from encrypt import ExecJs
    e = ExecJs()
    step1 = e.get_encrypt_pwd('md5',password)
    step2 = e.get_encrypt_pwd('md5',step1 + res_rand.text)
    print(step2)
```

```

#禅道登录
url = 'http://zentaomax.demo.zentao.net/user-login.html'
data = {
    "account": "demo",
    "password": step2,
    "passwordStrength": "0",
    "referer": "/",
    "verifyRand": res_rand.text,
    "keepLogin": "0",
    "captcha": ""
}
header = {
    "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
    "Host": "zentaomax.demo.zentao.net",
    "Connection": "keep-alive",
    "Content-Length": "127",
    "Accept": "application/json, text/javascript, */*; q=0.01",
    "X-Requested-With": "XMLHttpRequest",
    "Origin": "http://zentaomax.demo.zentao.net",
    "Referer": "http://zentaomax.demo.zentao.net/user-login.html",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "zh-CN,zh;q=0.9"
}
res = s.post(url,data=data,headers=header)
print(res.text)
return s

```

myinfo.py:

```

# 个人档案接口
import requests
from login import login

def myinfo(s):
    url = 'http://zentaomax.demo.zentao.net/my-profile.html'
    query_str = {
        "onlybody": "yes"
    }
    header = {
        "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/100.0.4896.75 Safari/537.36"
    }
    res = s.get(url=url,headers=header,params=query_str)
    return res.text

#1. 登录
s = login()
#2. 个人档案
res = myinfo(s)
print(res)

```

2. 【单例实现全局参数存储】

GlobalManager全局变量管理器封装：

```

class GlobalManager(object):
    """模块间共享变量"""
    _globaldict = {}
    _instance = False

    def get_value(self, name):
        try:
            return self._globaldict[name]
        except KeyError as e:
            print("获取的变量名称: {}不存在!!!!!!!!!!!!!!!!".format(name))
            return None

    def set_value(self, name, value):
        self._globaldict[name] = value

    def __new__(cls, *args, **kwargs): #单例，固定写法
        if cls._instance == False:
            cls._instance = super().__new__(cls, *args, **kwargs)
        return cls._instance

if __name__ == "__main__":
    globalm = GlobalManager()

```

举个栗子：

```

import requests
from globalmanager import GlobalManager
#密码的加密逻辑: md5(md5(password) + rand)
#1.md5方法: md5.js
#2.password:123456
#3.rand:http://zentaomax.demo.zentao.net/user-refreshRandom.html

gm = GlobalManager()

def login():
    s = requests.session() #创建会话保存器
    gm.set_value("huihua",s)
    #获取cookie
    url_cookie = 'http://zentaomax.demo.zentao.net/user-login.html'
    header_cookie = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
        Gecko) Chrome/100.0.4896.60 Safari/537.36",
        "Referer": "http://zentaomax.demo.zentao.net/my/"
    }
    res_cookie = gm.get_value('huihua').get(url_cookie,headers=header_cookie)
    d = requests.utils.dict_from_cookiejar(res_cookie.cookies)
    print(d)
    #密码的加密

```

```

#1.rand获取
url_rand = 'http://zentaomax.demo.zentao.net/user-refreshRandom.html'
header_rand = {
    "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "X-Requested-With": "XMLHttpRequest",
    "Referer": "http://zentaomax.demo.zentao.net/user-login.html"
}
res_rand = gm.get_value('huihua').get(url_rand,headers=header_rand)
print(res_rand.text)

#2.python实现调用js文件中的方法md5
password = '123456'
from encrypt import ExecJs
e = ExecJs()
step1 = e.get_encrypt_pwd('md5',password)
step2 = e.get_encrypt_pwd('md5',step1 + res_rand.text)
print(step2)

#禅道登录
url = 'http://zentaomax.demo.zentao.net/user-login.html'
data = {
    "account": "demo",
    "password": step2,
    "passwordStrength": "0",
    "referer": "/",
    "verifyRand": res_rand.text,
    "keepLogin": "0",
    "captcha": ""
}
header = {
    "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/100.0.4896.60 Safari/537.36",
    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
    "Host": "zentaomax.demo.zentao.net",
    "Connection": "keep-alive",
    "Content-Length": "127",
    "Accept": "application/json, text/javascript, */*; q=0.01",
    "X-Requested-With": "XMLHttpRequest",
    "Origin": "http://zentaomax.demo.zentao.net",
    "Referer": "http://zentaomax.demo.zentao.net/user-login.html",
    "Accept-Encoding": "gzip, deflate",
    "Accept-Language": "zh-CN,zh;q=0.9"
}
res = gm.get_value('huihua').post(url,data=data,headers=header)
print(res.text)

```

```

# 个人档案接口
import requests
from login import login
from globalmanager import GlobalManager

gm = GlobalManager()

```



```
def myinfo():
    url = 'http://zentaomax.demo.zentao.net/my-profile.html'
    query_str = {
        "onlybody": "yes"
    }
    header = {
        "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like
        Gecko) Chrome/100.0.4896.75 Safari/537.36"
    }
    res = gm.get_value('huihua').get(url=url,headers=header,params=query_str)
    return res.text

#1. 登录
login()
#2. 个人档案
res = myinfo()
print(res)
```

6.26 【实战练习】接口自动化实现图片壁纸爬虫

一、学习目标

1. 爬取图片壁纸

三、知识点

【爬虫案例】

```
import requests
import os
from lxml import etree

if not os.path.exists('./picture'):
    os.mkdir('./picture')

url = 'https://pic.netbian.com/4kmeinv/'
header = {
    "User-Agent": "Mozilla/5.0 (windows NT 10.0; win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/100.0.4896.75 Safari/537.36"
}
res = requests.get(url,headers=header,verify=False)
res.encoding = 'gbk'
page_info = res.text

#解析响应体（页面标签结构）
tree = etree.HTML(page_info)
li_list = tree.xpath("//ul[@class='clearfix']/li")
print(li_list)
for li in li_list:
    path = 'https://pic.netbian.com/' + li.xpath("./a/img/@src")[0]
    name = li.xpath("./a/b/text()")[0] + '.jpg'
    res = requests.get(path,headers=header,verify=False).content
    with open('./picture/'+name,'wb') as f:
```

```
f.write(res)
print("图片下载完成:", name)
```

第七章 Pytest单元测试框架模块介绍

7.1 Pytest模块介绍和环境安装

一、学习目标

1.pytest模块介绍

2.pytest模块安装

二、知识点

1.【pytest模块介绍】

- 非常容易上手，入门简单，文档丰富，文档中有很多实例可以参考
- 能够支持简单的单元测试和复杂的功能测试
- 支持参数化
- 执行测试过程中可以将某些测试跳过，或者对某些预期失败的case标记成失败
- 支持重复执行失败的case
- 支持运行由 unittest编写的测试case
- 具有很多第三方插件，并且可以自定义扩展
- 方便和持续集成工具集成

2.【pytest模块安装】

- 在线安装：（当前电脑有网）

cmd执行：

```
pip install pytest==5.4.1
```

- 离线安装：（当前电脑无网）

去联网的电脑先下载pytest模块包：<https://pypi.org/>

解压后cmd执行：

```
python setup.py install
```

7.2 Pytest用例设计

一、学习目标

1.pytest用例编写规则

2.实例演示

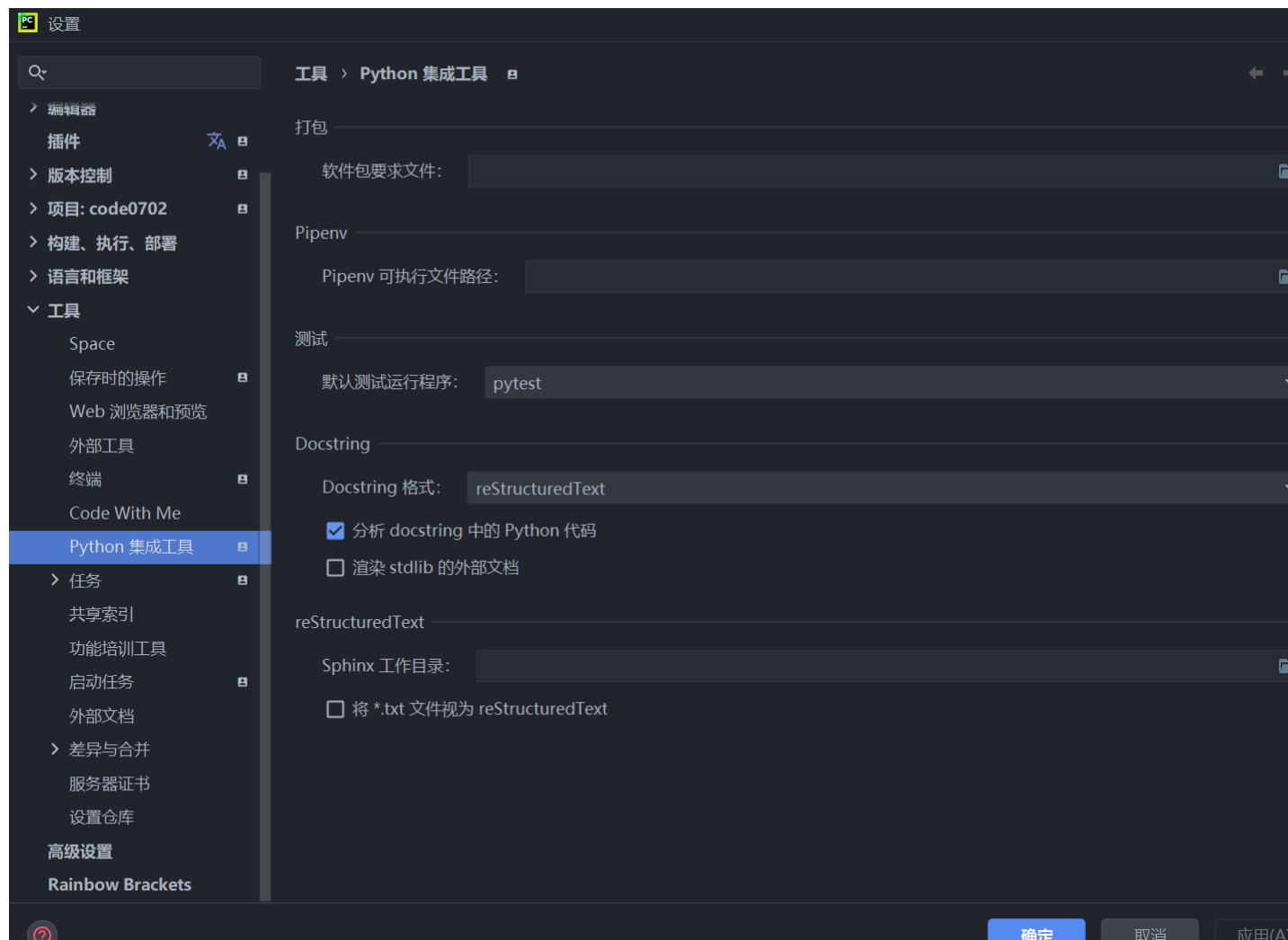
二、知识点

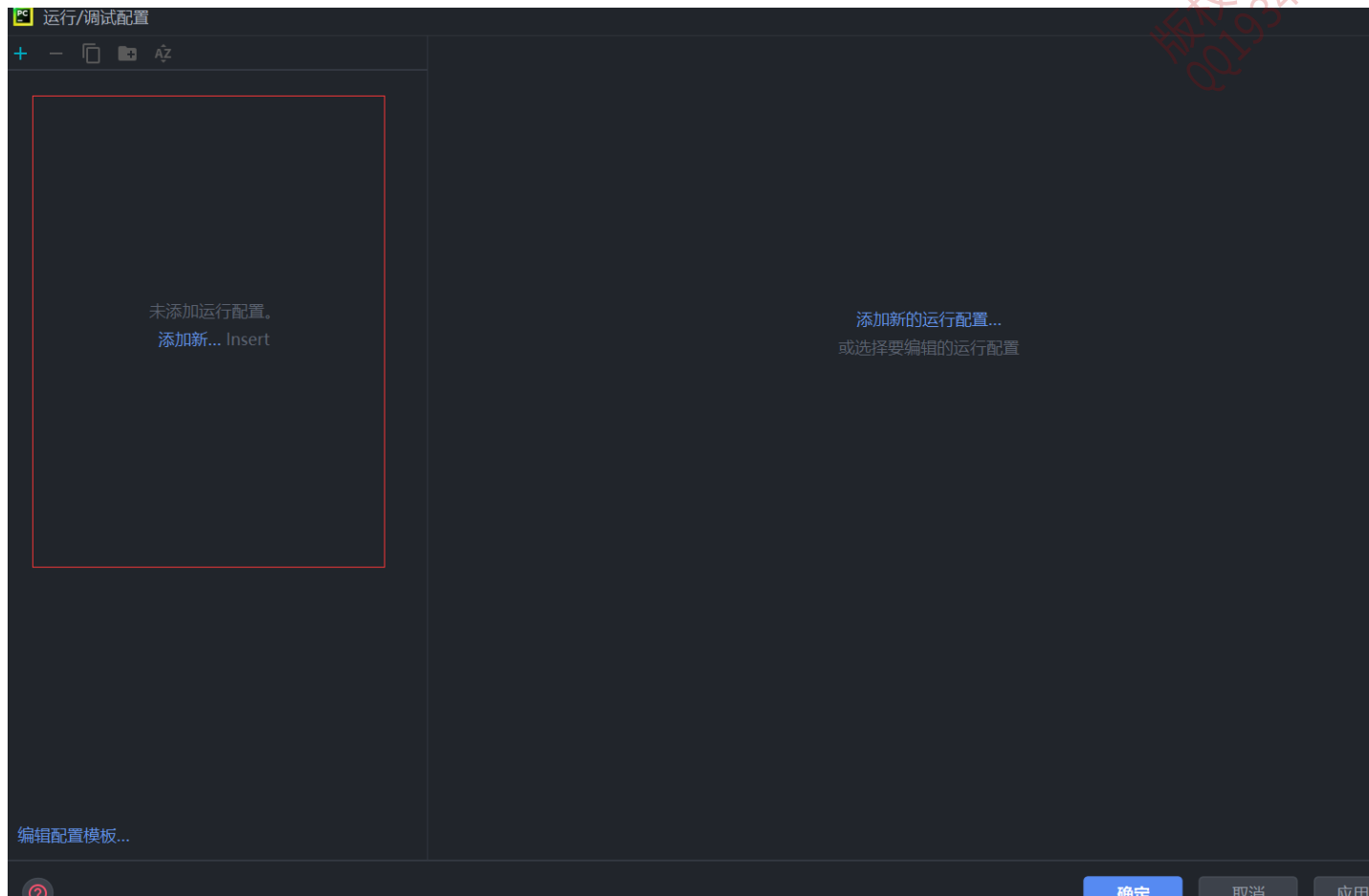
1.【pytest用例编写规则】

- 文件名以 `test_.py` 开头或 `_test.py` 结尾
- 以 `test_` 开头的函数
- 以 `Test` 开头的类
- 以 `test_` 开头的方法
- 所有的包 `pakege` 必须要有 `__init__.py` 文件

2. 【实例演示】

修改配置：





- 不在类中的测试用例函数：

- 新建用例：

```
def test_case01():  
    '''验证1+1=2'''  
    num = 1+1  
    assert num == 2, "断言失败：1+1不等于2"  
    print("断言成功：1+1等于2")
```

说明：`test_demo.py` 作为编写测试用例的文件；`test_case01()` 是测试用例函数。点击运行后测试用例被自动执行了，即使我们没有写main函数主动调用，pytest会自动的找到哪些函数属于测试用例并自动运行。

- 在类中的测试用例函数（推荐）：

- 新建用例：

```
#2. 在类中的自动化用例  
  
class TestCase():  
  
    def test_case02(self):  
        '''1+1=2'''  
        num = 1 + 1  
        assert num == 2, "断言失败：1+1不等于2"  
        print("断言成功：1+1等于2")
```

说明：`TestCase` 作为测试用例的类，一个类下可以写多条测试用例，我们可以把测试同一个功能的测试用例放到一个测试类下，方便管理。

7.3 Pytest前置和后置运行级别

一、学习目标

- 1.类内外执行：`setup/teardown`
- 2.类外执行：`setup_module/teardown_module`，`setup_function/teardown_function`
- 3.类内执行：`setup_class/teardown_class`，`setup_method/teardown_method`

二、知识点

1.【类内外执行：`setup/teardown`】（推荐）

此前置和后置是不区分测试用例是在类中还是不在类中，两种都能用。

我们用`print()`打印来验证前置和后置的执行顺序。

- 语法说明：

setup/teardown: 无论是类内还是类外，每个方法都会执行1次。

setup: 方法运行前执行

teardown: 方法运行完才执行

- 代码示例：

#1.不在类中的前置后置

```
def setup():
    '''用例前置'''
    print("这是不在类中的用例前置！")

def teardown():
    '''用例的后置'''
    print("这是不在类中的用例后置！")

def test_case01():
    '''1+1=2'''
    num = 1 + 1
    assert num == 2, "断言失败：1+1不等于2"
    print("断言成功：1+1等于2")

def test_case02():
    '''1+2=3'''
    num = 1 + 2
    assert num == 3, "断言失败：1+2不等于3"
    print("断言成功：1+2等于3")

class TestCase():
```

```
def setup(self):
    '''用例前置'''
    print("这是不在类中的用例前置！")

def teardown(self):
    '''用例的后置'''
    print("这是不在类中的用例后置！")

def test_case01(self):
    '''1+1=2'''
    num = 1 + 1
    assert num == 2, "断言失败：1+1不等于2"
    print("断言成功：1+1等于2")

def test_case02(self):
    '''1+2=3'''
    num = 1 + 2
    assert num == 3, "断言失败：1+2不等于3"
    print("断言成功：1+2等于3")
```

说明：从运行结果可以看出，setup在每一个测试用例执行前执行，teardown在每一个测试用例执行后执行。setup和teardown即可以在类中应用，也可以不在类中应用。这也是在项目中常用的前置和后置。

2. 【类外执行：setup_module/teardown_module， setup_function/teardown_function】

此前置和后置只能作用于不在类中的测试用例函数。

- 语法说明：

setup_module/teardown_module:

只有前置后置方法在class外部时，才会被执行。整个.py文件只执行1次。

setup_module：在.py文件执行前执行。

teardown_module：在.py文件执行后执行。

setup_function/teardown_function:

只有类外部的的方法才会执行。每个方法执行一次。

setup_function：类外部的的方法前执行。

teardown_function：类外部的的方法后执行。

- 代码示例：

#只能在类的外部使用的前置后置函数

```
def setup_module():
    print("setup_module执行开始！")

def teardown_module():
    print("teardown_module执行开始！")

def setup_function():
    print("setup_function执行开始！")

def teardown_function():
    print("teardown_function执行开始！")
```

```
def test_case01():
    '''1+1=2'''
    num = 1 + 1
    assert num == 2, "断言失败：1+1不等于2"
    print("断言成功：1+1等于2")

def test_case02():
    '''1+2=3'''
    num = 1 + 2
    assert num == 3, "断言失败：1+2不等于3"
    print("断言成功：1+2等于3")
```

说明：setup_module/teardown_module是模块级的前置和后置，所以在当前py模块只执行了一次。
setup_function/teardown_function是函数级的前置和后置，所以在每个用例函数分别执行了前置和后置。

3. 【类内执行：setup_class/teardown_class，setup_method/teardown_method】

此前置和后置只能作用于在类中的测试用例函数。

- 语法说明：

setup_class/teardown_class:

只有方法在class内部时，才可以执行。class内只运行1次。

setup_class: 在class类运行前执行。

teardown_class: 在class类运行后执行。

setup_method/teardown_method:

只有方法在class内部时，才可以执行。class内运行多次。

setup_method: 每个method前执行一次。

teardown_method: 每个method后执行一次。

- 代码示例：

#在类的内部执行的前置后置

```
class TestCase():

    def setup_class(self):
        print("setup_class执行开始！")

    def teardown_class(self):
        print("teardown_class执行开始！")

    def setup_method(self):
        print("setup_method执行开始！")

    def teardown_method(self):
        print("teardown_method执行开始！")

    def test_case01(self):
        '''1+1=2'''
        num = 1 + 1
        assert num == 2, "断言失败：1+1不等于2"
```

```
print("断言成功: 1+1等于2")

def test_case02(self):
    '''1+2=3'''
    num = 1 + 2
    assert num == 3, "断言失败: 1+2不等于3"
    print("断言成功: 1+2等于3")
```

说明：setup_class/teardown_class是类级的前置和后置，所以在当前类下只执行了一次。
setup_method/teardown_method是方法级的前置和后置，所以在每个用例函数分别执行了前置和后置。

7.4 Pytest 自定义前置后置：fixture简单应用

一、学习目标

1. `conftest.py` 文件介绍
2. `fixture` 的简单应用
3. `fixture` 前置传参

二、知识点

1. `conftest.py` 文件介绍

`fixture`实际应用需要结合 `conftest.py` 文件来使用，我们将前置函数写在 `conftest.py` 中，可以跨文件调用前置。

`conftest.py` 是`fixture`函数的一个集合，可以理解为公共的提取出来放在一个文件里，然后供其它模块调用。不同于普通被调用的模块，`conftest.py` 使用时不需要导入，`Pytest` 会自动查找

- `conftest.py` 特点：
 - (一) 文件名必须是 `conftest.py`，不得改变。
 - (二) 不需要 `import` 导入 `conftest.py`，`pytest` 用例会自动识别该文件。
 - (三) 放到项目的根目录下就可以全局目录调用，放在某个文件夹下则只能在文件夹内的测试用例调用。
- `conftest.py` 的使用场景
 - (一) 接口自动化公用的`session`或`token`
 - (二) `web`自动化时`driver`实例化操作

2. 【`fixture`的简单应用】

- 语法:

```
@pytest.fixture(scope = "function",params=None,autouse=False,ids=None,name=None)
```

- 参数说明:

`fixture`里面有个`scope`参数可以控制`fixture`的作用范围，`scope`：有四个级别参数`"function"`（默认），`"class"`，`"module"`，`"session"`

params: 一个可选的参数列表，它将导致多个参数调用fixture功能和所有测试使用它。

autouse: 如果True，则为所有测试激活fixture func可以看到它。如果为False则显示需要参考来激活fixture

ids: 每个字符串id的列表，每个字符串对应于params这样他们就是测试ID的一部分。如果没有提供ID它们将从params自动生成

name: fixture的名称。这默认为装饰函数的名称

- 代码示例:

```
import pytest

@pytest.fixture(scope='function')
def init_function():
    print("这是方法前置！")
    yield
    print("这是方法后置！")
```

```
import pytest

class TestCase():

    def test_case01(self):
        '''1+1=2'''
        num = 1 + 1
        assert num == 2, "断言失败: 1+1不等于2"
        print("断言成功: 1+1等于2")

    @pytest.mark.usefixtures('init_function')
    def test_case02(self):
        '''1+2=3'''
        num = 1 + 2
        assert num == 3, "断言失败: 1+2不等于3"
        print("断言成功: 1+2等于3")
```

说明: conftest.py文件定义前置函数，在测试用例方法调用，调用方式有两种，一种是直接往用例方法中传如前置函数的函数名；一种是通过usefixtures调用，这种方式可以传多个前置，也可以叠加调用前置（遵循就近原则执行前置）。

3. 【fixture前置传参】

当前置操作中产生的返回值需要被用例使用，需要用到前置传参。如在web自动化中可以将driver浏览器对象写入前置并传入测试用例中使用。我们在定义前置函数时可以用yield关键字返回参数，供测试用例方法使用。

conftest.py文件:

```
@pytest.fixture(scope='function')
def init_function03():
    print("这是方法前置03！")
    num = 1+1
    yield num
    print("这是方法后置03！")
```

测试用例文件:

```
class TestCase():

    @pytest.mark.usefixtures('init_function03')
    def test_case01(self, init_function03):
        '''1+1=2'''
        assert init_function03 == 2, "断言失败: 1+1不等于2"
        print("断言成功: 1+1等于2")
```

7.5 Pytest自定义前置后置：fixture参数详解（了解）

一、学习目标

- 1.scope参数实例讲解
- 2.params参数实例讲解
- 3.autouse参数实例讲解
- 4.ids参数实例讲解
- 5.name参数实例讲解

二、知识点

1.scope参数实例讲解

- 用于控制Fixture的作用范围
- 作用类似于Pytest的setup/teardown
- 默认取值为function（函数级别），控制范围的排序为：session > module > class > function

取值	范围	说明
function	函数级	每一个函数或方法都会调用
class	类级别	每个测试类只运行一次
module	模块级	每一个.py文件调用一次
session	会话级	每次会话只需要运行一次，会话内所有方法及类，模块都共享这个方法

代码示例：

```
#conftest.py前置定义文件
import pytest

@pytest.fixture(scope="session")
def init_session():
    print("会话前置")
    yield
    print("会话后置")

@pytest.fixture(scope="module")
def init_module():
    print("模块前置")
    yield
```

```

print("模块后置")

@pytest.fixture(scope="class")
def init_class():
    print("类的前置")
    yield
    print("类的后置")

@pytest.fixture(scope="function")
def init_function():
    print("用例前置")
    yield
    print("用例后置")

```

```

import pytest

@pytest.mark.usefixtures('init_function')
class TestCase():

    def test_case01(self):
        '''1+1=2'''
        num = 1 + 1
        assert num == 2, "断言失败：1+1不等于2"
        print("断言成功：1+1等于2")

    @pytest.mark.usefixtures('init_session', 'init_module', 'init_class')
    def test_case02(self):
        '''1+2=3'''
        num = 1 + 2
        assert num == 3, "断言失败：1+2不等于3"
        print("断言成功：1+2等于3")

```

2.params参数实例讲解

- Fixture的可选形参列表，支持列表传入
- 默认None，每个param的值
- fixture都会去调用执行一次，类似for循环
- 可与参数ids一起使用，作为每个参数的标识，详见ids
- 被Fixture装饰的函数要调用是采用：request.param(固定写法)

代码示例：

```

@pytest.fixture(params=["a", "b", {"name": "user1"}, {"name": "user2"}])
def demo(request):
    res = request.param
    yield res

```

```

@pytest.mark.usefixtures("demo")
def test_case03(self, demo):
    print("获取的params的参数：{}".format(demo))

```

3.ids参数实例讲解

- 用例标识ID
- 与params配合使用，一对一关系

代码示例：

```
@pytest.fixture(params=["a", "b", {"name": "user1"}, {"name": "user2"}], ids=
["one", "two", "three", "four"])
def demo(request):
    res = request.param
    yield res
```

```
@pytest.mark.usefixtures("demo")
def test_case03(self, demo):
    print("获取的params的参数: {}".format(demo))
```

4.autouse参数实例讲解

- 默认False
- 若为True，每个测试函数都会自动调用该fixture,无需传入fixture函数名
- 由此我们可以总结出调用fixture的三种方式：
 - 1.函数或类里面方法直接传fixture的函数参数名称
 - 2.使用装饰器@pytest.mark.usefixtures()修饰
 - 3.autouse=True自动调用，无需传任何参数，作用范围跟着scope走（谨慎使用）

代码示例：

```
@pytest.fixture(scope="session", autouse=True)
def init_session():
    print("回话前置")
    yield
    print("回话后置")
```

```
class TestCase():

    def test_case01(self):
        '''1+1=2'''
        num = 1 + 1
        assert num == 2, "断言失败：1+1不等于2"
        print("断言成功：1+1等于2")
```

5.name参数实例讲解

- fixture的重命名
- 通常来说使用 fixture 的测试函数会将 fixture 的函数名作为参数传递，但是 pytest 也允许将fixture重命名
- 如果使用了name,那只能将name传如，函数名不再生效

代码示例：

```
@pytest.fixture(scope="function", name="fun")
def init_function():
    print("用例前置")
    yield
    print("用例后置")
```

```
def test_case04(fun):
    '''1+1=2'''
    num = 1 + 1
    assert num == 2, "断言失败：1+1不等于2"
    print("断言成功：1+1等于2")
```

7.6 Pytest参数化：parametrize简单应用

一、学习目标

- 1.parametrize的简单应用
- 2.parametrize传多组数据

二、知识点

1. 【parametrize的简单应用】

- 语法：

```
@pytest.mark.parametrize(argnames, argvalues, indirect=False, ids=None, scope=None))
```

- 参数说明：

argnames：必传，参数名，以逗号分隔的字符串，表示一个或多个参数名称(key)，或参数字符串的列表/元组

argvalues：必传，参数值，若argnames有一个刚单值列表传入，若argnames有多个，以套用元组的列表展示，无组内与参数名一一对应

indirect：为true时，那argnames一定是一个fixture函数名称，argvalues值将传入对应的fixture内，相当于@pytest.fixture(params=)的用法，默认False

ids：标记子用例执行名称，与argvalues数量一致，未指定自动生成，默认None

scope：如果指定，则表示参数的范围。范围用于按参数实例对测试进行分组。它还将覆盖任何fixture函数定义的范围，允许使用测试上下文或配置设置动态范围

- 代码示例：

```
import pytest

class TestCase():

    test_data = [{"username": "user1", "password": "psw1"}]

    @pytest.mark.parametrize('casedata', test_data)
    def test_login(self, casedata):
        '''登录'''
        print("请输入用户名: {}".format(casedata['username']))
        print("请输入密码: {}".format(casedata['password']))
        assert 1 == 1
```

2. 【pytest参数化传多组数据】

代码示例：

```
import pytest

class TestCase():

    test_data = [{"username": "user1", "password": "psw1"}, {"username": "user2", "password": "psw2"},
                  {"username": "user3", "password": "psw3"}]

    @pytest.mark.parametrize('casedata', test_data)
    def test_login(self, casedata):
        '''登录'''
        print("请输入用户名: {}".format(casedata['username']))
        print("请输入密码: {}".format(casedata['password']))
        assert 1 == 1
```

7.7 Pytest参数化：pytest参数详解（了解）

一、学习目标

1. argnames、argvalues参数实例讲解
2. indirect参数实例讲解
3. ids参数实例讲解
4. scope参数实例讲解
5. 与其它mark一起使用

二、知识点

1. 【argnames、argvalues参数实例讲解】

- 单参数单值

代码示例：

#1.单参数单值

```
class TestCase():

    @pytest.mark.parametrize('arg',[1])
    def test_case01(self,arg):
        print("传入的值是：{}".format(arg))
```

- 单参数多值

- 1.单参数多值，argvalues可以传入多样的python数据类型：列表，嵌套了元组的列表，字典，字符串
- 2.传入多个值时，测试用例会被执行多次，每次取一个值去运行

代码示例：

#2.单参数多值

```
@pytest.mark.parametrize('arg',[1,'abc',{'a':1,'b':2}),(4,5)])
def test_case01(self,arg):
    print("传入的值是：{}".format(arg))
```

- 多参数多值

argnames参数的个数，必须要和argvalues的每一组的个数一致，否则报错。

代码示例：

#3.多参数多值

```
@pytest.mark.parametrize('arg1,arg2',[(3+5,8),(5-2,3)])
def test_case01(self,arg1,arg2):
    print("原值：{}，预期：{}".format(arg1,arg2))
    assert arg1 == arg2
```

- parametrize叠加使用：

代码示例：

#4.叠加使用

```
@pytest.mark.parametrize('arg2',["a","b","c"])
@pytest.mark.parametrize('arg1',["1","2","3"])
def test_case04(self,arg1,arg2):
    all = arg1 + arg2
    print("叠加结果：{}".format(all))
```

2. 【indirect参数实例讲解】

说明：

- (一) indirect一般与Pytest的request、fixture组合使用。

(二) 当indirect为True时，argnames则要传入fixture函数名称，不再是一个普通参数，而是要被调用的fixture函数，argvalues则是要给这个函数传的值。

(三) 作法其实与@pytest.fixture(params)一样，但使用了@pytest.mark.parametrize相当于参数化了fixture，而不是只有固定的一套数据传入使用。

- 单fixture单值 (通过列表)

代码示例：

```
@pytest.fixture()
def indirect_demo(request):
    user = request.param
    print("传入的用户名: {}".format(user))
    yield user
```

```
#1. 单fixture单值
user = ['张三', '李四']
@pytest.mark.parametrize('indirect_demo', user, indirect=True)
def test_one_param(indirect_demo):
    print("测试方法的用户名: {}".format(indirect_demo))
```

- 单fixture多值(通过字典)

代码示例：

```
@pytest.fixture()
def indirect_demo2(request):
    userinfo = request.param
    print("传入的用户名: {}".format(userinfo['user']))
    print("传入的密码: {}".format(userinfo['psw']))
    yield userinfo
```

```
#2. 单fixture多值
userinfo = [{"user": "张三", "psw": "123"}, {"user": "李四", "psw": "456"}]

@pytest.mark.parametrize('indirect_demo2', userinfo, indirect=True)
def test_one_param(indirect_demo2):
    print("测试方法的用户名: {}".format(indirect_demo2['user']))
    print("测试方法的密码: {}".format(indirect_demo2['psw']))
```

- 传多fixture多值 (通过嵌套元组的列表)

代码示例：


```
@pytest.fixture()
def indirect_demo3(request):
    user = request.param
    print("传入的用户名: {}".format(user))
```

```
@pytest.fixture()
def indirect_demo4(request):
    psw = request.param
    print("传入的密码: {}".format(psw))
```

#3. 多fixture多值

```
usermsg= [("张三", "123"), ("李四", "456")]
```

```
@pytest.mark.parametrize('indirect_demo3, indirect_demo4', usermsg, indirect=True)
def test_one_param(indirect_demo3, indirect_demo4):
    print("这是测试用例！")
    print(indirect_demo3, indirect_demo4)
```

• 叠加fixture (单值列表)

代码示例:

```
@pytest.fixture()
def indirect_demo3(request):
    user = request.param
    print("传入的用户名: {}".format(user))
```

```
@pytest.fixture()
def indirect_demo4(request):
    psw = request.param
    print("传入的密码: {}".format(psw))
```

#4. 叠加使用

```
user = ['张三', '李四']
```

```
psw = ["123", "456"]
```

```
@pytest.mark.parametrize('indirect_demo4', psw, indirect=True)
@pytest.mark.parametrize('indirect_demo3', user, indirect=True)
def test_case04(indirect_demo3, indirect_demo4):
    print("这是测试用例！")
```

3. 【ids参数实例讲解】

作用：标记子用例执行名称，增加可读性，中文需要转码

代码示例:

```
import pytest

userinfo = [('张三', '123'), ('李四', '456')]

@pytest.mark.parametrize('arg', userinfo, ids=["one", "two"])
def test_case01(arg):
    print("用户数据: {}".format(arg))
```

4. 【scope参数实例讲解】

1. scope的作用范围取值与fixture scope一致，当indirect=True才会被使用。
2. scope的作用范围会覆盖fixture的scope范围，如果同一个被调用的fixture有多个parametrize定义了scope,取第一条的范围。

代码示例：

```
@pytest.fixture(scope='function')
def indirect_demo5(request):
    userinfp = request.param
    print("传入的元祖: {}".format(userinfp))
```

#5.scope参数的使用

```
usermsg= [("张三", "123"), ("李四", "456")]

class TestCase():

    @pytest.mark.parametrize('indirect_demo5', usermsg, indirect=True, scope='class')
    def test_case01(self, indirect_demo5):
        print("这是第一个测试方法！")

    @pytest.mark.parametrize('indirect_demo5', usermsg, indirect=True, scope='class')
    def test_case02(self, indirect_demo5):
        print("这是第二个测试方法！")
```

5. 【与其它mark一起使用】

代码示例：

```
#测试用例文件test_xxx.py
import pytest
a = '跳过'
@pytest.mark.parametrize("x,y",
                        [
                            ('test1', 123456),
                            pytest.param("test1", 123456, marks=pytest.mark.xfail),
                            pytest.param("test1", 123456, marks=pytest.mark.xfail),
                            pytest.param("test2", 123456, marks=pytest.mark.skip("跳过这条用例")),
                            pytest.param("test3", 123456, marks=pytest.mark.skipif(a == '跳
过', reason="有条件的跳过"))
                        ]
)
```

```
def test_one(x,y):  
    assert x== 'test1'  
    assert y == 123456
```

7.8 Pytest跳过某个测试用例：skip和skipif

一、学习目标

1. @pytest.mark.skip 跳过执行测试函数
2. @pytest.mark.skipif 若满足条件，则跳过测试函数
3. 自定义 @pytest.mark.skip() 标签
4. 通过 pytest.skip 方法跳过测试函数
5. 跳过测试类
6. 跳过模块
7. pytest.importorskip 模块导入失败则跳过

二、知识点

1. 【skip跳过执行测试函数】

- 语法：

```
@pytest.mark.skip(reason=None) #reason非必填，自定义跳过的原因
```

- 代码示例：

```
import pytest  
  
@pytest.mark.skip(reason="no reason")  
def test_01():  
    print("---用例1执行---")  
  
class TestCase():  
  
    @pytest.mark.skip(reason="no reason")  
    def test_02(self):  
        print("---用例2执行---")  
  
    def test_03(self):  
        print("---用例3执行---")
```

2. 【skipif若满足条件，则跳过测试函数】

- 语法：

```
@pytest.mark.skipif(condition, reason=None) #condition为一个表达式，表达式成立则跳过
#注：如果多个标签一起使用，满足其中一个跳过条件则会跳过该测试函数。
```

- 代码示例：

```
import pytest

@pytest.mark.skipif(condition= 2>1,reason="no reason")
def test_01():
    print("---用例1执行---")

class TestCase():

    @pytest.mark.skipif(condition= 2>1,reason="no reason")
    def test_02(self):
        print("---用例2执行---")

    def test_03(self):
        print("---用例3执行---")
```

3. 【自定义@pytest.mark.skip()标签】

- 语法：

```
myskip = pytest.mark.skip() 或 myskip = pytest.mark.skipif(condition=...)

#装饰时用该变量代替标签即可：@myskip
```

- 代码示例：

```
import pytest

myskip = pytest.mark.skipif(condition=2>1,reason='no reason')

@myskip
def test_01():
    print("---用例1执行---")

class TestCase():

    @myskip
    def test_02(self):
        print("---用例2执行---")

    def test_03(self):
```

```
print("----用例3执行----")
```

4. 【通过pytest.skip()方法跳过测试函数】

- 语法：

```
pytest.skip(msg="no reason") #写在用例函数内，msg非必填
```

- 代码示例：

```
import pytest

def test_01():
    pytest.skip(msg='no reason')
    print("----用例1执行----")

class TestCase():

    def test_02(self):
        pytest.skip()
        print("----用例2执行----")

    def test_03(self):
        print("----用例3执行----")
```

5. 【跳过测试类】

- 语法：

```
@pytest.mark.skip()和@pytest.mark.skipif()
#跳过测试类其实和跳过测试方法一样，用他们装饰测试类就好啦。
```

- 代码示例：

```
import pytest

def test_01():
    print("----用例1执行----")

@pytest.mark.skipif(condition=2>1,reason="class skip")
class TestCase():

    def test_02(self):
```

```
print("----用例2执行----")

def test_03(self):
    print("----用例3执行----")
```

6. 【跳过模块】

- 语法：

```
pytestmark = pytest.mark.skip(condition, reason=None)
#注：pytestmark不能改名
```

- 代码示例：

```
import pytest

pytestmark = pytest.mark.skipif(condition=2>1, reason='no reason')

def test_01():
    print("----用例1执行----")

class TestCase():

    def test_02(self):
        print("----用例2执行----")

    def test_03(self):
        print("----用例3执行----")
```

7. 【pytest.importorskip 模块导入失败则跳过】

- 语法：

```
#当引入某个模块失败时，我们同样可以跳过后续部分的执行：
docutils = pytest.importorskip("docutils")
#我们也可以为其指定一个最低满足要求的版本，判断的依据是检查引入模块的__version__属性：
docutils = pytest.importorskip("docutils", minversion="0.3")
```

- 代码示例：

```
import pytest

docutils = pytest.importorskip("docutils", minversion='0.3')

def test_01():
    print("----用例1执行----")
```

```
class TestCase():  
  
    def test_02(self):  
        print("----用例2执行----")  
  
    def test_03(self):  
        print("----用例3执行----")
```

7.9 Pytest标记预计失败的用例：xfail

一、学习目标

1. `@pytest.mark.xfail()` 使用例执行失败
2. `pytest.xfail()` 用例执行过程中使用例执行失败
3. 运行时忽略xfail标记

二、知识点

注：用例执行失败时（`XFAIL`：符合预期的失败）、用例执行成功时（`XPASS`：不符合预期的成功）

1. 【`@pytest.mark.xfail()`】

- 语法：

```
@pytest.mark.xfail(condition=None, reason=None, raises=None, run=True, strict=False)
```

- 参数：

- `reason` 关键字参数，默认值为 `None`

可以指定一个字符串，表明标记用例的原因；

- `strict` 关键字参数，默认值为 `False`

当 `strict=False` 时，如果用例执行失败，结果标记为 `XFAIL`，表示符合预期的失败；如果用例执行成功，结果标记为 `XPASS`，表示不符合预期的成功；

当 `strict=True` 时，如果用例执行成功，结果将标记为 `FAILED`，而不再是 `XPASS` 了；

我们也可以在 `pytest.ini` 文件中配置：

```
[pytest]  
xfail_strict=true
```

- `raises` 关键字参数，默认值为 `None`

可以指定为一个异常类或者多个异常类的元组，表明我们期望用例上报指定的异常；

如果用例的失败不是因为所期望的异常导致的，`pytest` 将会把测试结果标记为 `FAILED`；

- `run` 关键字参数，默认值为 `True`：

当 `run=False` 时，`pytest` 不会再执行测试用例，直接将结果标记为 `XFAIL`；

- 代码示例：

```
import pytest

@pytest.mark.xfail(condition= 2>1,reason='123',strict=True)
def test_01():
    print("---用例1执行啦---")

class TestCase():

    @pytest.mark.xfail(condition=2>1,reason=456)
    def test_02(self):
        print("---用例2执行啦---")
        assert False

    def test_03(self):
        print("---用例3执行啦---")
```

2. 【pytest.xfail()】

- 语法：

```
#用例函数内使用
pytest.xfail(reason="")
```

- 代码示例：

```
import pytest

def test_01():
    pytest.xfail(reason='123')
    print("---用例1执行啦---")

class TestCase():

    def test_02(self):
        pytest.xfail(reason='456')
        print("---用例2执行啦---")
        assert False

    def test_03(self):
        print("---用例3执行啦---")
```

3. 【运行时忽略xfail标记】

我们可以通过命令行选项 `pytest --runxfail` 来去掉 `xfail` 标记，使这些用例变成正常执行用例，仿佛没有被标记过一样：

同样，`pytest.xfail()` 方法也将会失效；

7.10 Pytest设置用例标签：mark

一、学习目标

1. @pytest.mark. 自定义标签

2. 组合运行用例

3. 注册、管理 mark 标记

二、知识点

1. 【@pytest.mark. 自定义标签】

(一) 可以标记测试方法、测试类，标记名可以自定义，最好起有意义的名字；

(二) 同一测试类/方法可同时拥有多个标记；

- 语法：

```
@pytest.mark.标签名
```

- 代码示例：

```
import pytest

@pytest.mark.login
class TestLogin():

    @pytest.mark.smoke
    @pytest.mark.sucess
    def test_login_sucess(self):
        '''登录成功'''
        print("----用例1: 登录成功")

    @pytest.mark.failed
    def test_login_failed(self):
        '''登录失败'''
        print("----用例2: 登录失败")

@pytest.mark.logout
class TestLogout():

    @pytest.mark.smoke
    @pytest.mark.sucess
    def test_logout_sucess(self):
        '''退出成功'''
        print("----用例3: 退出成功")

    @pytest.mark.failed
    def test_logout_failed(self):
        '''退出失败'''
        print("----用例4: 退出失败")
```

- 运行：

运行语法：`pytest -m "自定义标签名"` 或 `pytest.main(['-m 自定义标签名'])` 可以在终端运行或py文件的主函数运行。

```
pytest -m "smoke" test_demo.py
```

2. 【注册、管理 mark 标记】

由上面的运行结果可以看出，用例只运行了标签为"smoke"的用例，但是运行会出现告警信息。那么如何解决这些告警呢？首先要明白，使用mark标记时，是需要注册的，如果不注册就会出现告警。

- 方式一：在conftest.py文件当中，通过hook注册：

```
def pytest_configure(config):
    config.addinvalue_line('markers', 'smoke: 标记为运行冒烟用例')
    config.addinvalue_line('markers', 'sucess: 标记为运行成功用例')
    config.addinvalue_line('markers', 'failed: 标记为运行失败用例')
    config.addinvalue_line('markers', 'login: 标记为运行登录用例')
    config.addinvalue_line('markers', 'logout: 标记为运行退出用例')
```

- 方式二：创建pytest.ini文件，在文件中注册：

```
[pytest]
markers =
    smoke: smoke case
    sucess: sucess case
    failed: failed case
    login: login case
    logout: logout case
```

“addopts = --strict” 参数来严格规范 mark 标记的使用，当使用未注册的 mark 标记时，pytest会直接报错。

pytest.ini文件文件的更多用法，我们后面专门拿出一节课来介绍。

3. 【组合运行用例】

其实当我们运行的时候，也可以指定多个标签。

1. 使用 -m 参数运行标记的测试用例；
2. -m 参数支持 and、or、not 等表达式；

终端运行：

```
pytest -m "login and not failed" .\test_demo01.py
pytest -v -m "login and not sucess" .\test_demo01.py
```

7.11 Pytest常用插件：pytest-ordering调整用例执行顺序

一、学习目标

1. pytest-ordering 安装

2. pytest-ordering 应用

二、知识点

1. 【pytest-ordering 安装】

既然pytest-ordering是pytest的一个插件，插件就需要安装，插件其实就是一个python的三方模块。

```
pip install pytest-ordering
```

2. 【pytest-ordering 应用】

- 语法：

```
@pytest.mark.run(order) #order序号
```

- 代码示例：

```
import pytest

@pytest.mark.run(order=2)
def test_01():
    print("---用例1执行啦---")

class TestCase():

    @pytest.mark.run(order=1)
    def test_02(self):
        print("---用例2执行啦---")

    @pytest.mark.run(order=3)
    def test_03(self):
        print("---用例3执行啦---")
```

7.12 Pytest常用插件: pytest-rerunfailures失败用例重跑

一、学习目标

1. pytest-rerunfailures 安装

2. pytest-rerunfailures 应用

二、知识点

1. 【pytest-rerunfailures 安装】

插件安装：

```
pip install pytest-rerunfailures
```

2. 【pytest-rerunfailures 应用】

使用方式有两种：

- 命令行参数
 - 语法：

```
pytest --reruns 3 --reruns-delay 1 test_demo.py
#3是重试次数，1是每次间隔1秒
pytest --reruns 3 --only-rerun AssertionError --only-rerun ValueError test_demo.py
#也可以指定异常类型重跑，其中多个--only-rerun之间是或的关系
```

- 代码示例：

```
class TestCase():

    def test_01(self):
        print("---用例1开始啦---")
        assert False
```

- 装饰器方式
 - 语法：

```
@pytest.mark.flaky(reruns=3, reruns_delay=1)#3是重试次数，1是每次间隔1秒
```

- 代码示例：

```
import pytest

class TestCase():

    @pytest.mark.flaky(reruns=3, reruns_delay=1)
    def test_01(self):
        print("---用例1开始啦---")
        assert False
```

- 注意：

不可以与类，模块还有包级别的fixture装饰器一起使用：@pytest.fixture()

该插件与pytest-xdist的--looponfail 参数运行不兼容

最后总结，这个插件虽然还用，但是坑还是不少，建议主要使用失败重试次数和重试间隔的功能即可。

7.13 Pytest常用插件：pytest-repeat重复运行用例

一、学习目标

1. pytest-repeat 安装
2. pytest-repeat 应用

二、知识点

1. pytest-repeat 安装

插件安装：

```
pip install pytest-repeat
```

2. pytest-repeat 应用

使用方式有两种：

- 命令行参数
 - 语法：

```
pytest --count=3 --repeat-scope=class test_demo.py
#3是重复运行的次数
#-repeat-scope的参数有 choices=('function', 'class', 'module', 'session') 默认为function；控制用例重复执行时机
```

- 代码示例：

```
class TestCase():

    def test_01(self):
        print("---用例1执行啦")

    def test_02(self):
        print("---用例2执行啦")
```

- 装饰器方式
 - 语法：

```
@pytest.mark.repeat(num) #num运行次数
```

- 代码示例：

```
import pytest

class TestCase():

    def test_01(self):
        print("---用例1执行啦")

    @pytest.mark.repeat(3)
    def test_02(self):
        print("---用例2执行啦")
```

7.14 Pytest常用插件：pytest-assume多重断言

一、学习目标

1. pytest-assume 安装
2. pytest-assume 应用

二、知识点

1. pytest-assume 安装

插件安装：

```
pip install pytest-assume
```

2. pytest-assume 应用

- 语法：

```
pytest.assume(布尔型/表达式)
```

- 原生的assert和pytest-assume断言对比

代码示例：

```
import pytest

class TestCase():

    def test_01(self):
        print("---用例1执行啦---")
        assert False, "用例1第一次断言失败"
        assert False, "用例1第二次断言失败"

    def test_02(self):
        print("---用例2执行啦---")
        pytest.assume(1>2, "用例2第一次断言失败")
        pytest.assume(False, "用例2第二次断言失败")
        print("123")
```

7.15 Pytest常用插件：pytest-xdist分布式执行用例

一、学习目标

1. 分布式执行用例的设计原则
2. pytest-xdist 插件安装
3. pytest-xdist 插件应用

二、知识点

1. 分布式执行用例的设计原则

- 用例之间是独立的，用例之间没有依赖关系，用例可以完全独立运行【独立运行】
- 用例执行没有顺序，随机顺序都能正常执行【随机执行】
- 每个用例都能重复运行，运行结果不会影响其他用例【不影响其他用例】

2. pytest-xdist 插件安装

插件安装：

```
pip install pytest-xdist
```

3. pytest-xdist 插件应用

原理：xdist会产生一个或多个workers，workers都通过master来控制；每个worker负责执行完整的测试用例集，然后按照master的要求运行测试，而master机不执行测试任务

测试用例示例代码：

```
from selenium import webdriver

class TestCase():

    def setup(self):
        '''初始化driver'''
        self.driver = webdriver.Chrome(executable_path=r'C:\Users\Admin\Desktop\chromedriver.exe')

    def teardown(self):
        self.driver.quit()

    def test_case01(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('Python')
        self.driver.find_element_by_id('su').click()

    def test_case02(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('Java')
        self.driver.find_element_by_id('su').click()

    def test_case03(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('go')
        self.driver.find_element_by_id('su').click()

    def test_case04(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('php')
        self.driver.find_element_by_id('su').click()

    def test_case05(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('c++')
        self.driver.find_element_by_id('su').click()
```

- 使用分布式的执行：

- `pytest -n num` (代表使用num个CPU)
- `pytest -n auto`
 - `n auto`：可以自动检测到系统的CPU核数；从测试结果来看，检测到的是逻辑处理器的数量
 - 使用`auto`等于利用了所有CPU来跑用例，此时CPU占用率会特别高

说明：建议最多使用1/2的CPU个数来进行执行，消耗资源太多，导致电脑太卡。

- 自定义执行模式

将按照同一个作用域方法来分组，然后将每个测试组发给可以执行的worker，确保同一个组的测试用例在同一个进程中执行：

```
--dist=loadscope #每个worker按类执行
```

示例：`pytest -v -n 3 --dist=loadscope test_demo.py`

按照同一个文件名来分组，然后将每个测试组发给可以执行的worker，确保同一个组的测试用例在同一个进程中执行：

```
--dist=loadfile #每个worker按文件执行
```

示例：`pytest -v -n 3 --dist=loadfile test_xdist.py test_xdist_02.py test_xdist_03.py`

是将每个用例，分别发给所有的执行器worker，相当于开了几个执行器worker，同一个用例就执行几遍：

```
--dist=each
```

示例：`pytest -v -n 3 --dist=each test_xdist.py`

将待运行的用例随机发给可用的执行器worker，用例执行顺序随机的，目前默认采用这种方式：

```
--dist=load 和 --dist=no
```

示例：`pytest -v -n 3 --dist=load test_xdist.py`

- 如何让scope=session的fixture在test session中仅仅执行一次

`pytest-xdist`是让每个worker进程执行属于自己的测试用例集下的所有测试用例，这意味着在不同进程中，不同的测试用例可能会调用同一个scope范围级别较高（例如session）的fixture，该fixture则会被执行多次，这不符合scope=session的预期。

虽然`pytest-xdist`没有内置的支持来确保会话范围的夹具仅执行一次，但是可以通过使用锁定文件进行进程间通信来实现。

文件锁在os中很常见，如果多个程序同时访问或修改同一个文件，很容易因为文件数据不同步而出现问题。给文件加一个锁，同一时间只能有一个程序修改此文件，或者程序都只能读取文件就解决同步问题，保证了线程安全。

```
import json
import os
import uuid
```



```
import pytest
from filelock import FileLock

@pytest.fixture(scope='session')
def login(tmp_path_factory, worker_id):
    if worker_id == 'master':
        # 【伪代码：获取登录id】
        uuid_value = str(uuid.uuid1())
        print("fixture:请求登录接口，获取token", uuid_value)
        os.environ['token'] = uuid_value
        return uuid_value
    root_tmp_dir = tmp_path_factory.getbasetemp().parent
    fn = root_tmp_dir / "data.json"
    with FileLock(str(fn) + '.lock'):
        if fn.is_file():
            token = json.loads(fn.read_text())
            print("读取文件的token:{}".format(token))
        else:
            # 【伪代码：获取登录id】
            token = str(uuid.uuid1())
            print("fixture:请求登录接口，获取token", token)
            fn.write_text(json.dumps(token))
            print("首次运行，token:{}".format(token))
            os.environ['token'] = token
    return token
```

1. 示例只需要执行一次login（因为它是只需要执行一次来定义配置选项，等等）
2. 当第一次请求这个fixture时，则会利用FileLock仅产生一次fixture数据
3. 当其他进程再次请求这个fixture时，则会从文件中读取数据

7.16 Pytest常用插件：pytest-html生成简洁的测试报告

一、学习目标

- 1.pytest-html插件安装
- 2.pytest-html插件应用
- 3.其他格式的测试报告

二、知识点

1. 【pytest-html插件安装】

插件安装：

```
pip install pytest-html==2.1.1
```

2. 【pytest-html插件应用】

测试用例示例代码：

```
from selenium import webdriver
```

```
class TestCase():

    def setup(self):
        '''初始化driver'''
        self.driver = webdriver.Chrome(executable_path=r'C:\Users\Admin\Desktop\chromedriver.exe')

    def teardown(self):
        self.driver.quit()

    def test_case01(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('Python')
        self.driver.find_element_by_id('su').click()

    def test_case02(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('Java')
        self.driver.find_element_by_id('su').click()

    def test_case03(self):
        self.driver.get('https://www.baidu.com/')
        self.driver.find_element_by_id('kw').send_keys('go')
        self.driver.find_element_by_id('su').click()
```

- 创建一个独立的报告

命令行执行测试用例，增加参数：

```
--html=report.html --self-contained-html
```

示例：pytest --html=/report/auto_reports.html --self-contained-html test_html_demo.py

说明：--html=report.html，指定报告的生成路径；--self-contained-html，可以是报告独立，复制给他人。

auto_reports.html

1.报告标题

Report generated on 25-Oct-2021 at 09:49:54 by pytest-html v3.1.1

Environment

2.项目环境

JAVA_HOME	C:\Program Files\Java\jdk1.8.0_181
Packages	["pluggy": "1.0.0", "py": "1.10.0", "pytest": "6.2.5"]
Platform	Windows-10-0.19041-SP0
Plugins	["forked": "1.3.0", "html": "3.1.1", "metadata": "1.11.0", "xdist": "2.4.0"]
Python	3.6.8

Summary

3.报告摘要信息

3 tests ran in 11.33 seconds.
(Un)check the boxes to filter the results.

☒ 3 passed, ☐ 0 skipped, ☐ 0 failed, ☐ 0 errors, ☐ 0 expected failures, ☐ 0 unexpected passes

Results

4.报告结果汇总

Show all details / Hide all details

Result	Test	Duration	Links
Passed (show details)	test_html_demo.py::TestCase::test_case_01	3.78	
Passed (show details)	test_html_demo.py::TestCase::test_case_02	3.73	
Passed (show details)	test_html_demo.py::TestCase::test_case_03	3.74	

- 自定义报告展示

- 修改报告标题

默认情况下，报告标题将是报告的文件名，您可以使用 `pytest_html_report_title` 钩子对其进行编辑：

```
def pytest_html_report_title(report):
    report.title = "My very own title!"
```

My very own title!

Report generated on 25-Oct-2021 at 09:55:13 by pytest-html v3.1.1

Environment

JAVA_HOME	C:\Program Files\Java\jdk1.8.0_181
Packages	["pluggy": "1.0.0", "py": "1.10.0", "pytest": "6.2.5"]
Platform	Windows-10-10.0.19041-SP0
Plugins	["forked": "1.3.0", "html": "3.1.1", "metadata": "1.11.0", "xdist": "2.4.0"]
Python	3.6.8

Summary

3 tests ran in 11.62 seconds.

(Un)check the boxes to filter the results:

☒ 3 passed, ☐ 0 skipped, ☐ 0 failed, ☐ 0 errors, ☐ 0 expected failures, ☐ 0 unexpected passes

Results

Show all details / Hide all details

Result	Test	Duration	Links
Passed (show details)	test_html_demo.py::TestCase::test_case_01	3.93	
Passed (show details)	test_html_demo.py::TestCase::test_case_02	3.84	
Passed (show details)	test_html_demo.py::TestCase::test_case_03	3.74	

o 修改环境信息

环境部分由提供 `pytest-metadata` 的插件，并且可以通过访问 `pytest_configure` 和 `pytest_sessionfinish` 钩子函数修改环境信息。

要在运行测试前修改 `Environment` 部分，请使用：`pytest_configure`

```
def pytest_configure(config):
    config._metadata["其他环境"] = "无网环境"
```

要在运行测试后修改 `Environment` 部分，请使用：`pytest_sessionfinish`

```
import pytest

@pytest.hookimpl(tryfirst=True)
def pytest_sessionfinish(session, exitstatus):
    session.config._metadata["其他环境"] = "联网环境"
```

My very own title!

Report generated on 25-Oct-2021 at 10:06:25 by pytest-html v3.1.1

Environment

JAVA_HOME	C:\Program Files\Java\jdk1.8.0_181
Packages	["pluggy": "1.0.0", "py": "1.10.0", "pytest": "6.2.5"]
Platform	Windows-10-10.0.19041-SP0
Plugins	["forked": "1.3.0", "html": "3.1.1", "metadata": "1.11.0", "xdist": "2.4.0"]
Python	3.6.8
其他环境	联网环境

Summary

3 tests ran in 11.39 seconds.

(Un)check the boxes to filter the results:

☒ 3 passed, ☐ 0 skipped, ☐ 0 failed, ☐ 0 errors, ☐ 0 expected failures, ☐ 0 unexpected passes

Results

Show all details / Hide all details

Result	Test	Duration	Links
Passed (show details)	test_html_demo.py::TestCase::test_case_01	3.80	
Passed (show details)	test_html_demo.py::TestCase::test_case_02	3.74	
Passed (show details)	test_html_demo.py::TestCase::test_case_03	3.76	

o 修改摘要信息

可以使用钩子编辑摘要部分 `pytest_html_results_summary`：

```
from py.xml import html

def pytest_html_results_summary(prefix, summary, postfix):
    prefix.extend([html.p("自动化测试工程师：工具人")])
```

My very own title!

Report generated on 25-Oct-2021 at 10:46:43 by pytest-html v3.1.1

Environment

JAVA_HOME	C:\Program Files\Java\jdk1.8.0_181
Packages	["pluggy": "1.0.0", "py": "1.10.0", "pytest": "6.2.5"]
Platform	Windows-10-10.0.19041-SP0
Plugins	["borked": "1.3.0", "html": "3.1.1", "metadata": "1.11.0", "xdist": "2.4.0"]
Python	3.6.8
其他环境	无环境

Summary

自动化测试工程师 工具人

3 tests ran in 11.71 seconds.

(Un)check the boxes to filter the results:

☒ 3 passed, ☐ 0 skipped, ☐ 0 failed, ☐ 0 errors, ☐ 0 expected failures, ☐ 0 unexpected passes

Results

Show all details / Hide all details

Result	Test	Duration	Links
Passed (show details)	test_html_demo.py::TestCase::test_case_01	4.00	
Passed (show details)	test_html_demo.py::TestCase::test_case_02	3.82	
Passed (show details)	test_html_demo.py::TestCase::test_case_03	3.80	

修改报告结果

可以通过为标题和行实施自定义挂钩来修改报告的列。下面的示例 `confstest.py` 添加了一个带有测试函数 注释 的 `description` 列，添加了一个可排序的时间列，并删除了 `links` 列：

```
from datetime import datetime
from py.xml import html
import pytest

def pytest_html_results_table_header(cells):
    cells.insert(1, html.th("Description")) #1代表列的索引，如0, 1, 2
    cells.insert(2, html.th("Time"))
    cells.pop()

def pytest_html_results_table_row(report, cells):
    if hasattr(report, 'description'):
        cells.insert(1, html.td(report.description))
        cells.insert(2, html.td(datetime.utcnow()))
        cells.pop()
    else:
        print("插入测试报告报错: {}".format(report.longreprtext))

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call): #item是所有的测试用例，call是测试结果
    outcome = yield
    report = outcome.get_result() #获取测试报告对象，为下面自定义测试报告做铺垫
    report.description = str(item.function.__doc__) #修改描述信息，获取测试用例的__doc__，也就是
    测试函数的注释
```

My very own title!

Report generated on 25-Oct-2021 at 11:05:23 by pytest-html v3.1.1

Environment

JAVA_HOME	C:\Program Files\Java\jdk1.8.0_181
Packages	["pluggy": "1.0.0", "py": "1.10.0", "pytest": "6.2.5"]
Platform	Windows-10-10.0.19041-SP0
Plugins	["borked": "1.3.0", "html": "3.1.1", "metadata": "1.11.0", "xdist": "2.4.0"]
Python	3.6.8
其他环境	无环境

Summary

自动化测试工程师 工具人

3 tests ran in 11.53 seconds.

(Un)check the boxes to filter the results:

☒ 3 passed, ☐ 0 skipped, ☐ 0 failed, ☐ 0 errors, ☐ 0 expected failures, ☐ 0 unexpected passes

Results

Show all details / Hide all details

Result	Time	Test	Description	Duration
Passed (show details)	2021-10-25 03:05:23.577629	test_html_demo.py::TestCase::test_case_01	测试用例一	3.82
Passed (show details)	2021-10-25 03:05:23.577629	test_html_demo.py::TestCase::test_case_02	测试用例二	3.75
Passed (show details)	2021-10-25 03:05:23.577629	test_html_demo.py::TestCase::test_case_03	测试用例三	3.87

• 额外内容

可以通过在报告对象上创建“额外”列表来向 HTML 报告添加详细信息。以下是可以添加的额外内容类型：

类型	例子
原始 HTML	<code>extra.html('<div>Additional HTML</div>')</code>
JSON	<code>extra.json({'name': 'pytest'})</code>
纯文本	<code>extra.text('Add some simple Text')</code>
网址	<code>extra.url('http://www.example.com/')</code>
图片	<code>extra.image(image, mime_type='image/gif', extension='gif')</code>
图片	<code>extra.image('/path/to/file.png')</code>
图片	<code>extra.image('http://some_image.png')</code>

注意：从文件添加图像时，路径可以是绝对路径，也可以是相对路径。

还有几种图像格式的方便类型：

图像格式	例子
PNG	<code>extra.png(image)</code>
JPEG格式	<code>extra.jpg(image)</code>
SVG	<code>extra.svg(image)</code>

这里介绍一下如何从测试报告中自定义插入图片，在自动化测试中可以展示失败的截图，方便定位问题：

```
from selenium import webdriver
driver =
webdriver.Chrome(executable_path=r"C:\Users\19344\Desktop\xdist_demo\chromedriver.exe")

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item):
    """
    当测试失败的时候，自动截图，展示到html报告中
    :param item:
    """
    pytest_html = item.config.pluginmanager.getplugin('html')
    outcome = yield
    report = outcome.get_result()
    # 获取调用结果的测试报告，返回一个report对象，report对象的属性包括when（setup, call, teardown三个阶段）、nodeid(测试用例的名字)、outcome(用例的执行结果, passed, failed)
    extra = getattr(report, 'extra', [])
    # 以上是固定写法，目的是为了获取report对象，下面对report做自定义修改
    if report.when == 'call' or report.when == "setup": #先判断用例执行到哪一阶段
        xfail = hasattr(report, 'wasxfail')
        if (report.skipped and xfail) or (report.failed and not xfail): #判断用例如果是失败的
            file_name = report.nodeid.replace("::", "_")+".png"
```

```

screen_img = _capture_screenshot()
if file_name:
    html = '<div></div>' % screen_img
    extra.append(pytest_html.extras.html(html))
report.extra = extra

def _capture_screenshot():
    """
    截图保存为base64，展示到html中
    :return:
    """
    return driver.get_screenshot_as_base64()
    
```

说明：测试用例的测试步骤是执行完常规钩子函数返回的report报告有个属性叫report.when，先执行when='setup' 返回setup 的执行结果；然后执行when='call' 返回call 的执行结果；最后执行when='teardown'返回teardown 的执行结果。

My very own title!

Report generated on 25-Oct-2021 at 13:45:42 by pytest-html v3.1.1

Environment

JAVA_HOME	C:\Program Files\Java\jdk1.8.0_101
Packages	["pluggy": "1.0.0", "py": "1.10.0", "pytest": "6.2.5"]
Platform	Windows-10-10.0.19041-SP0
Plugins	["forked": "1.3.0", "html": "3.1.1", "metadata": "1.11.0", "xdist": "2.4.0"]
Python	3.8.8
其他环境	无列环境

Summary

自动化测试工程师 工具人

1 tests ran in 0.86 seconds

(Un)check the boxes to filter the results.

☐ 0 passed, ☐ 0 skipped, ☒ 1 failed, ☐ 0 errors, ☐ 0 expected failures, ☐ 0 unexpected passes

Results

Show all details / Hide all details

Result	Time	Test	Description	Duration
Failed (hide details)		test_screenshot.py::TestCase::test_case_01		0.76
<pre> self = <test_screenshot.TestCase object at 0x0000029C96D9CFD0> def test_case_01(self): """测试用例""" driver.get(url) driver.find_element_by_name('wd').send_keys('Python') print("输入Python成功") driver.find_element_by_id('su').click() print("点击百度一下成功") > assert False E AssertionError test_screenshot.py:16: AssertionError -----Captured stdout call----- 输入Python成功 点击百度一下成功 </pre>				

3. 【其他格式的测试报告】

- 生成JunitXML 格式的测试报告

在CMD中切换到该测试用例路径下，执行指令：

```

#其中report\auto_report.xml为生成log的路径
pytest -v test_demo.py --junitxml=report/auto_report.xml
    
```

- 生成result log格式的测试报告（新版本的pytest可能没有了）

在CMD中切换到该测试用例路径下，执行指令：

```

pytest -v test_demo.py --result-log=report/log.txt
    
```

- 生成在线测试报告

```
pytest -v test_demo.py --pastebin=all
```

7.17 Pytest常用插件：allure-pytest生成精美的测试报告(一)

一、学习目标

- 1.allure-pytest 插件安装
- 2.allure-pytest 插件应用

二、知识点

1.allure-pytest 插件安装

注：allure 是基于 Java 的一个程序，需要 Java1.8 的环境，没有安装需要去安装一下。

- 安装allure-pytest插件

```
pip install allure-pytest
```

- 安装allure命令工具

Windows 安装 allure-commandline：

- 下载 allure-commandline

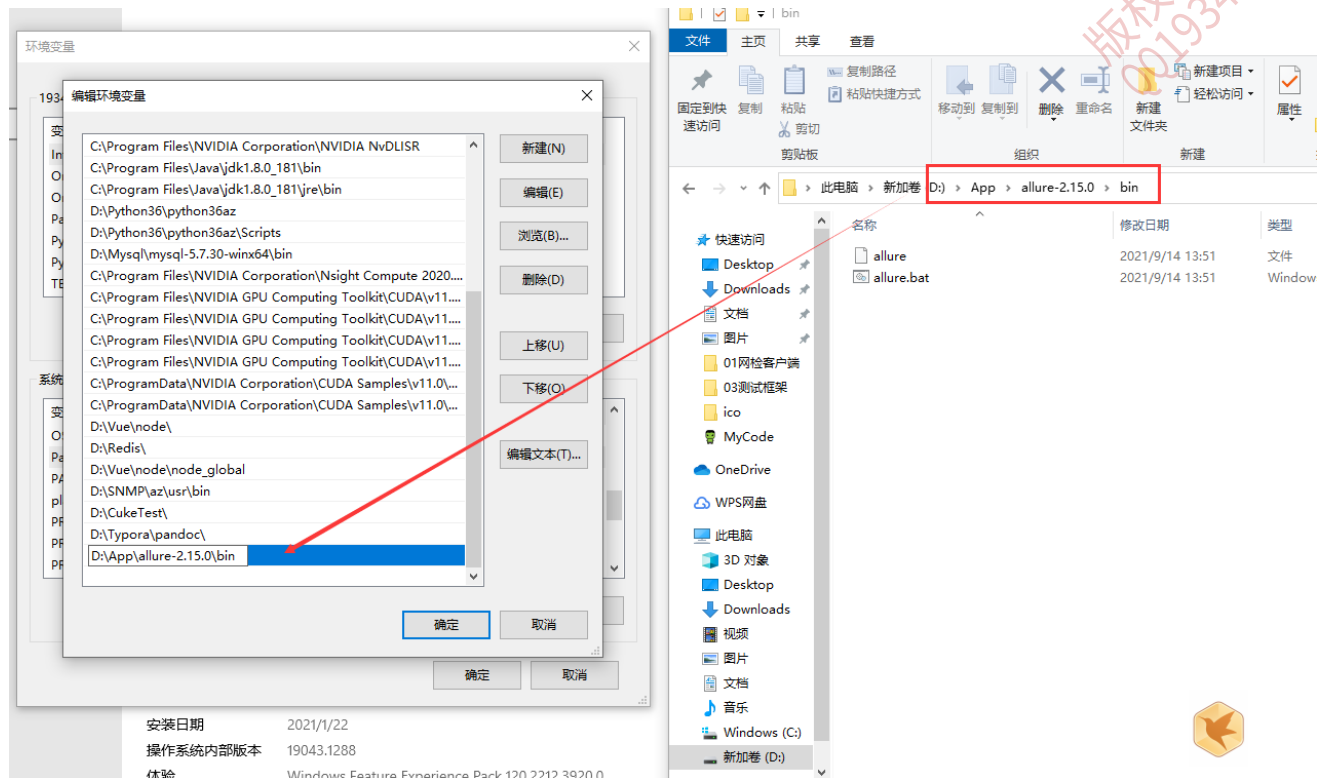
<https://repo.maven.apache.org/maven2/io/qameta/allure/allure-commandline/> #下载地址，下最新版就可以

← → ↻ <https://repo.maven.apache.org/maven2/io/qameta/allure/allure-commandline/>

应用 百度一下, 你就知道 书签栏 自动化测试 Socket airTest 数据可视化 Django Pyqt5

allure-commandline-2.15.0-javadoc.jar	2021-09-14 13:52	79180
allure-commandline-2.15.0-javadoc.jar.asc	2021-09-14 13:52	821
allure-commandline-2.15.0-javadoc.jar.asc.md5	2021-09-14 13:52	32
allure-commandline-2.15.0-javadoc.jar.asc.sha...	2021-09-14 13:52	40
allure-commandline-2.15.0-javadoc.jar.asc.sha...	2021-09-14 13:52	64
allure-commandline-2.15.0-javadoc.jar.asc.sha...	2021-09-14 13:52	128
allure-commandline-2.15.0-javadoc.jar.md5	2021-09-14 13:52	32
allure-commandline-2.15.0-javadoc.jar.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0-javadoc.jar.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0-javadoc.jar.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0-sources.jar	2021-09-14 13:52	17895
allure-commandline-2.15.0-sources.jar.asc	2021-09-14 13:52	821
allure-commandline-2.15.0-sources.jar.asc.md5	2021-09-14 13:52	32
allure-commandline-2.15.0-sources.jar.asc.sha...	2021-09-14 13:52	40
allure-commandline-2.15.0-sources.jar.asc.sha...	2021-09-14 13:52	64
allure-commandline-2.15.0-sources.jar.asc.sha...	2021-09-14 13:52	128
allure-commandline-2.15.0-sources.jar.md5	2021-09-14 13:52	32
allure-commandline-2.15.0-sources.jar.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0-sources.jar.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0-sources.jar.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.jar	2021-09-14 13:52	21616
allure-commandline-2.15.0.jar.asc	2021-09-14 13:52	821
allure-commandline-2.15.0.jar.asc.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.jar.asc.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.jar.asc.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.jar.asc.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.jar.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.jar.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.jar.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.jar.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.pom	2021-09-14 13:52	7230
allure-commandline-2.15.0.pom.asc	2021-09-14 13:52	821
allure-commandline-2.15.0.pom.asc.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.pom.asc.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.pom.asc.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.pom.asc.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.pom.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.pom.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.pom.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.pom.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.tgz	2021-09-14 13:52	19629420
allure-commandline-2.15.0.tgz.asc	2021-09-14 13:52	821
allure-commandline-2.15.0.tgz.asc.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.tgz.asc.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.tgz.asc.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.tgz.asc.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.tgz.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.tgz.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.tgz.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.tgz.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.zip	2021-09-14 13:52	19644665
allure-commandline-2.15.0.zip.asc	2021-09-14 13:52	821
allure-commandline-2.15.0.zip.asc.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.zip.asc.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.zip.asc.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.zip.asc.sha512	2021-09-14 13:52	128
allure-commandline-2.15.0.zip.md5	2021-09-14 13:52	32
allure-commandline-2.15.0.zip.sha1	2021-09-14 13:52	40
allure-commandline-2.15.0.zip.sha256	2021-09-14 13:52	64
allure-commandline-2.15.0.zip.sha512	2021-09-14 13:52	128

- 解压后，将bin目录添加到环境变量



- o 验证安装成功

cmd输入：

```
allure --version
```

2.allure-pytest插件应用

示例代码：

```
import pytest

def test_success():
    """this test succeeds"""
    assert True

def test_failure():
    """this test fails"""
    assert False

def test_skip():
    """this test is skipped"""
    pytest.skip('for a reason!')

def test_broken():
    raise Exception('oops')
```

- 生成Allure测试报告
 - o 第一步，生成测试报告数据

在pytest执行测试的时候，指定--alluredir选项及结果数据保存的目录：

```
pytest test_allure_demo1.py --alluredir ./result/
```

○ 第二步，生成测试报告页面

■ 生成离线测试报告

#通过下面的命令将./result/目录下的测试数据生成测试报告页面
allure generate ./result/ -o ./report/ --clean

-clean选项目的是先清空测试报告目录，再生成新的测试报告。

■ 生成在线测试报告：

```
allure serve result #result是报告源文件目录
```

注：生成在线报告必须以管理员权限运行命令。

● 修改报告的LOGO

1.修改配置文件，进入'E:\allure-2.13.7\config' 目录，打开'allure.yml' 文件，在最下面增加一行 '- custom-logo-plugin '，特别要注意层级关系。

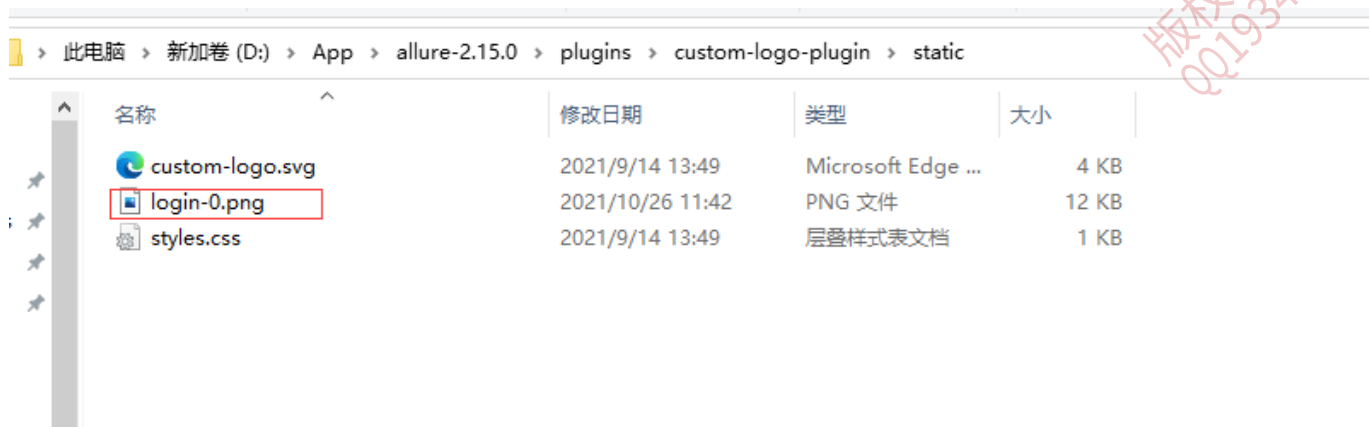
> 此电脑 > 新加卷 (D:) > App > allure-2.15.0 > config

名称	修改日期	类型	大小
allure.yml	2021/10/26 13:07	YML 文件	1 KB
allure-cucumber.yml	2021/9/14 13:47	YML 文件	0 KB
allure-junit.yml	2021/9/14 13:47	YML 文件	1 KB

```
plugins:
  - junit-xml-plugin
  - xunit-xml-plugin
  - trx-plugin
  - behaviors-plugin
  - packages-plugin
  - screen-diff-plugin
  - xctest-plugin
  - jira-plugin
  - xray-plugin
  - custom-logo-plugin
```

2.上传logo

进入按照allure路径下D:\App\allure-2.15.0\plugins\custom-logo-plugin\static



3.修改'styles.css' 文件 来配置logo, 不会的可以参考我的设置。

```
.side-nav__brand {
background: url('login-0.png') no-repeat left center !important;
padding-left: 180px !important;
margin-left: 10px;
}
```

7.18 Pytest常用插件：allure-pytest生成精美的测试报告(二)

一、学习目标

- 1.allure 装饰器介绍
- 2.allure 装饰器应用

二、知识点

1. 【allure装饰器介绍】

使用方法	参数值	参数说明
@allure.epic()	epic描述	敏捷里面的概念，定义史诗，往下是feature
@allure.feature()	模块名称	功能点的描述，往下是story
@allure.story()	用户故事	用户故事，往下是title
@allure.title(用例的标题)	用例的标题	重命名html报告名称
@ allure.testcase()	测试用例的链接地址	对应功能测试用例系统里面的case
@allure.issue()	缺陷	对应缺陷管理系统里面的链接
@allure.description()	用例描述	测试用例的描述
@allure.step()	操作步骤	测试用例的步骤
@allure.severity()	用例等级	blocker, critical, normal, minor, trivial
@allure.link()	链接	定义一个链接，在测试报告展现
@allure.attachment()	附件	报告添加附件

2. 【allure装饰器应用】

测试用例代码：

生成报告：

```
pytest test_demo1.py --alluredir ./result/      #第一步，运行用例生成报告原始数据
allure generate ./result/ -o ./report/ --clean  #第二步，生成报告
```

以上是不做修改的报告的用例展示。下面我们通过上面示例代码的基础上，增加自定义内容。

- step——测试用例的步骤

用来给报告上的测试用例添加步骤描述。

- 语法：

```
@allure.step("step:步骤")      #方式一，写在函数的上方
with allure.step("step:步骤")  #方式二，写在函数的内部
```

- 参数：

传入步骤描述信息，字符串类型

- 代码示例：

```
import allure

@allure.step('输入用户名')
def input_username(username):
    print("输入用户名: {}".format(username))

@allure.step('输入输入密码')
def input_password(password):
    print("输入密码: {}".format(password))

class TestCase():

    def test_login_case01(self):
        '''正确的用户名，正确密码登录'''
        input_username('user01')
        input_password('psw01')
        print("点击登录")

    def test_login_case02(self):
        '''错误的用户名，正确密码登录'''
        with allure.step("输入用户名"):
            print("输入用户名: user02")
        with allure.step("输入密码"):
            print("输入密码: psw02")
        print("点击登录")
```

```
def test_login_case03(self):
    '''正确的用户名，错误的密码登录'''
    print("输入用户名: user03")
    print("输入密码: psw03")
    print("点击登录")
```

- feature——测试用例特性（主要功能模块）

修改Behaviors菜单的功能描述，用来给功能模块增加描述信息，如登录模块，用户管理模块等。

- 语法：

```
@allure.feature("功能模块")
```

- 参数：

字符串

- 代码示例：

```
@allure.feature("登录功能")
class TestCase():
```

- story——feature功能模块下的分支功能

修改Behaviors菜单的功能描述，功能模块下的子分支。

- 语法：

```
@allure.story("子功能描述")
```

- 参数：

字符串

- 代码示例：

```
@allure.story("正确的用户名，正确密码登录")
def test_login_case01(self):
    '''正确的用户名，正确密码登录'''
    input_username('user01')
    input_password('psw01')
    print("点击登录")

@allure.story("错误的用户名，正确密码登录")
def test_login_case02(self):
    '''错误的用户名，正确密码登录'''
    with allure.step("输入用户名"):
        print("输入用户名: user02")
    with allure.step("输入密码"):
        print("输入密码: psw02")
    print("点击登录")

@allure.story("正确的用户名，错误的密码登录")
def test_login_case03(self):
    '''正确的用户名，错误的密码登录'''
    print("输入用户名: user03")
```

```
print("输入密码: psw03")
print("点击登录")
```

- link/issue/testcase——链接

- 语法:

```
@allure.link(url,name=None)
@allure.issue(url,name=None)
@allure.testcase(url,name=None)
```

- 参数:

字符串, 链接地址, 链接名

- 代码示例:

```
@allure.link('https://www.baidu.com',name='link')
@allure.issue("https://www.baidu.com",name='缺陷链接')
@allure.testcase('https://www.baidu.com',name="用例链接")
@allure.story("正确的用户名, 正确密码登录")
def test_login_case01(self):
    '''正确的用户名, 正确密码登录'''
    input_username('user01')
    input_password('psw01')
    print("点击登录")
```

- description——用例描述

- 语法:

```
@allure.description("用例描述") 提供描述字符串的装饰器
@allure.description_html("用例描述") 提供一些HTML在测试用例的描述部分
```

- 参数:

字符串

- 代码示例:

```
@allure.description('这是用例二的描述信息')
@allure.story("错误的用户名, 正确密码登录")
def test_login_case02(self):
    '''错误的用户名, 正确密码登录'''
    with allure.step("输入用户名"):
        print("输入用户名: user02")
    with allure.step("输入密码"):
        print("输入密码: psw02")
    print("点击登录")
```

- severity——测试用例的严重级别

Allure中对严重级别的定义:

blocker级别: 中断缺陷 (客户端程序无响应, 无法执行下一步操作)

critical级别: 临界缺陷 (功能点缺失)

normal级别：普通缺陷（数值计算错误）

minor级别：次要缺陷（界面错误与UI需求不符）

trivial级别：轻微缺陷（必填项无提示，或者提示不规范）

如果不指定用例级别则默认为normal

◦ 语法：

```
@allure.severity(allure.severity_level.CRITICAL) 或者 @allure.severity('critical')
```

◦ 参数：

字符串

◦ 代码示例：

```
@allure.severity('trivial')
@allure.story("正确的用户名，错误的密码登录")
def test_login_case03(self):
    '''正确的用户名，错误的密码登录'''
    print("输入用户名: user03")
    print("输入密码: psw03")
    print("点击登录")
```

• attach——用于向测试报告中输入一些附加的信息，通常是一些测试数据信息

◦ 语法：

```
allure.attach(body, name, attachment_type, extension)
allure.attach.file(source, name, attachment_type, extension)
```

◦ 参数：

body：要写入文件的原始内容

name：包含文件名的字符串

attachment_type -：其中一个allure.attachment_type值

extension：提供的将用作创建文件的扩展名

source：文件路径

```
#allure.attachment_type的所有值:
TEXT = ("text/plain", "txt")
CSV = ("text/csv", "csv")
TSV = ("text/tab-separated-values", "tsv")
URI_LIST = ("text/uri-list", "uri")

HTML = ("text/html", "html")
XML = ("application/xml", "xml")
JSON = ("application/json", "json")
YAML = ("application/yaml", "yaml")
PCAP = ("application/vnd.tcpdump.pcap", "pcap")

PNG = ("image/png", "png")
JPG = ("image/jpeg", "jpg")
SVG = ("image/svg+xml", "svg")
```

```
GIF = ("image/gif", "gif")
BMP = ("image/bmp", "bmp")
TIFF = ("image/tiff", "tiff")

MP4 = ("video/mp4", "mp4")
OGG = ("video/ogg", "ogg")
WEBM = ("video/webm", "webm")

PDF = ("application/pdf", "pdf")
```

- o 测试用例中添加文本、图片、html

代码示例：

```
@allure.severity('trivial')
@allure.story("正确的用户名，错误的密码登录")
def test_login_case03(self):
    '''正确的用户名，错误的密码登录'''
    print("输入用户名: user03")
    print("输入密码: psw03")
    print("点击登录")
    #增加附加文本
    allure.attach(body='这是一段文本',name='文本展示',attachment_type=allure.attachment_type.TEXT)
    #增加附加图片
    allure.attach.file(source="./pic.png",name='图片展示',attachment_type=allure.attachment_type.PNG)
    #增加附加HTML
    allur
```

7.19 Pytest执行用例的多种方式

一、学习目标

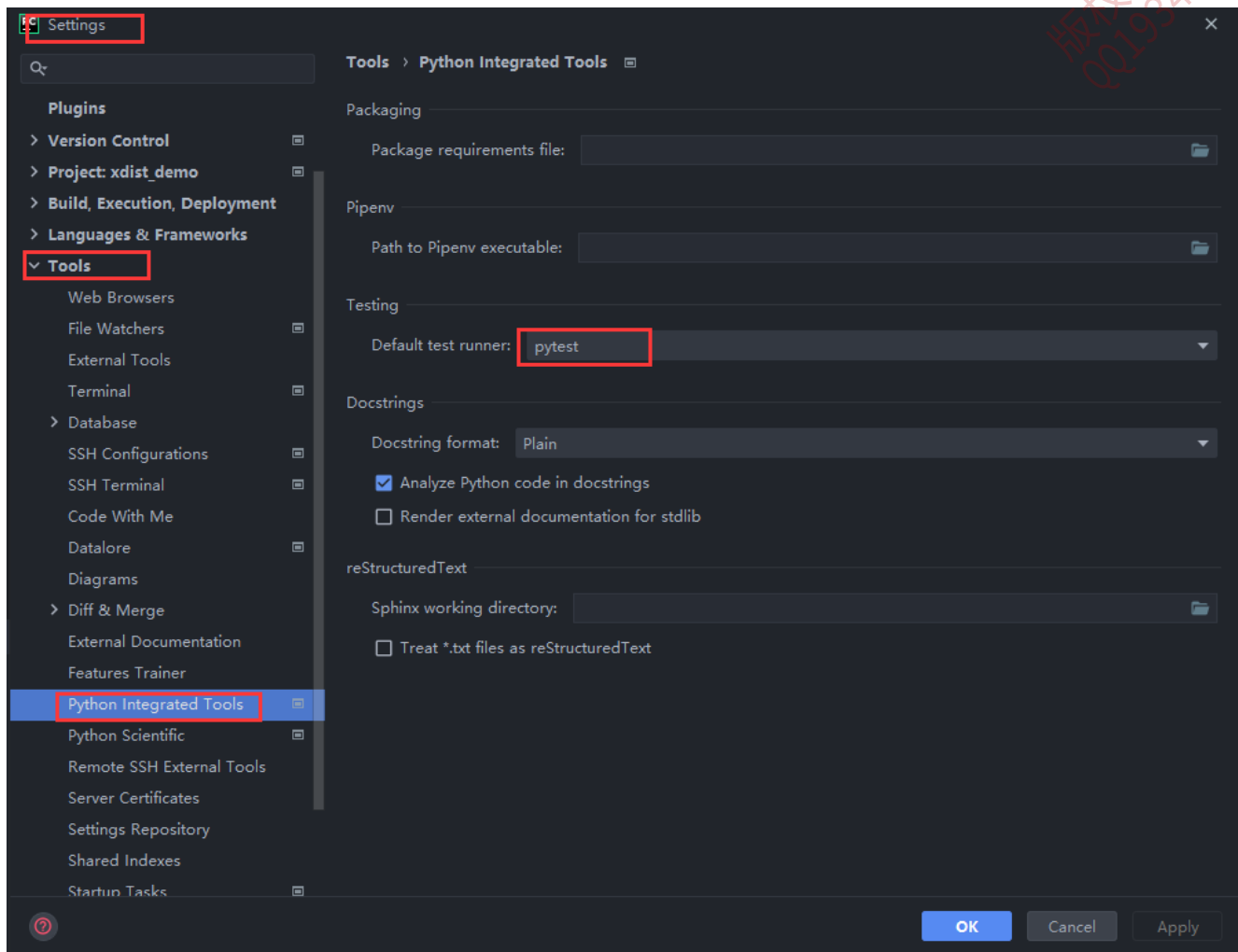
- 1.通过IDE运行
- 2.通过命令行运行
- 3.通过main函数运行
- 4.通过pytest.ini运行

二、知识点

1.【通过IDE运行】

- 步骤一：Pycharm运行方式配置

在使用IDE如Pycharm运行用例之前，需要修改一个配置，来指定pytest模式来执行项目：



- 步骤二：点击箭头运行

修改完配置后，测试用例前面会有绿色的箭头，点击箭头我们可以指定用例运行测试用例，也可以直接右键整个py文件运行全部用例：



2. 【通过命令行运行】

- 语法：

```
#命令行中Terminal中输入
pytest 【运行参数】 【指定用例】
#如pytest -s -v test_demo.py
```

- 运行参数

pytest -s test_demo.py	#显示代码中的print或日志打印
pytest -v test_demo.py	#显示用例详情
pytest -q test_demo.py	#简化显示，与-v相反
pytest --collect-only test_demo.py	#检查哪些测试用例会被运行
pytest -k "not 02" test_demo.py	#选择用例运行，可以用not, or
pytest -x test_demo.py	#遇到失败用例自动停止运行
pytest --maxfail=2 test_demo.py	#达到最大失败数自动停止运行
pytest --tb=short test_demo.py	#失败信息的显示方式，三个固定值short
: 仅输出assert 的一行以及系统判定的内容；line : 只使用一行输出显示所有的错误信息；no : 屏蔽所有的回溯信息	
pytest -m "markname" test_demo.py	#指定标签名运行，配合前面讲的mark标
签装饰器	
pytest -n 3 test_demo.py	#分布式运行，指定3个进程，配合前面讲
的xdist插件	
pytest --reruns 3 --reruns-delay 1 test_demo.py	#失败重复运行3次，每次1s间隔，配合前
面讲的pytest-rerunfailures插件	
pytest --html=report.html --self-contained-html test_demo.py	#输出测试报告，配合前面讲的pytest-
html插件	
pytest --alluredir ./result/ test_demo.py	#输出allure报告数据到result目录，
配合前面讲的所有ure-pytest插件	

注：以上运行参数可以互相组合使用。

- 指定用例

pytest -s testsuit/	#指定文件夹下所有用例运行
pytest -s testsuit/test_demo.py	#指定测试文件所有用例运行
pytest -s testsuit/test_demo.py::TestCase	#指定测试类的所有用例运行，文件内
用::连接	
pytest -s testsuit/test_demo.py::TestCase::test_case_01	#指定单个测试用的所有用例运行

3. 【通过main函数运行】

除了命令行运行，我们可以使用函数来调用执行测试用例。

- 语法：

```
pytest.main(args=None, plugins=None) #传参数列表，如['-s', 'test_demo.py']
```

- 参数：

`args` 为运行测试的一些参数，和命令行运行时的参数相同，在列表 `List` 里以字符串 `str` 的形式，多参数以“,” 隔开，也可以传入测试case的路径。

`plugins` 为插件参数，也是列表形式，是运行 `pytest` 时运行下载的插件的参数。

- 代码示例

```
pytest.main('-v', 'test_demo.py')
```

4. 【通过pytest.ini运行】

最后一种也是最常用的一种方式，就是在 `pytest.ini` 文件中配置 `pytest` 的运行参数。

`pytest.ini` 是优先级最高的一种运行方式，可以改变`pytest`的默认运行方式。

可以使用 `pytest -h` 查看 `pytest.ini` 文件的参数。

下节课做详细介绍。

7.20 Pytest.ini文件介绍

一、学习目标

1. `pytest.ini` 文件要注意

2. `pytest.ini` 运行用例配置

二、知识点

1. 【`pytest.ini` 文件要注意】

1. 存放位置：一般放在项目的根目录（即当前项目的顶级文件夹下）。
2. 编码格式：必须是ANSI编码格式，可以使用notepad++修改编码格式。
3. `pytest.ini` 文件中不能使用任何中文符号（也可以用，最好少用）。
4. 配置了 `pytest.ini` 文件后，不管是主函数模式运行用例，还是命令行模式运行用例，都会去读取 `pytest.ini` 配置文件中的内容。
5. CMD命令中执行 `pytest -h` 命令，可以查看 `pytest.ini` 的设置选项。

2. 【`pytest.ini` 运行用例配置】

- 常用参数：
 - `addopts` 命令行的参数，用空格分隔
 - `testpaths` 测试用例的路径
 - `markers` 标记参数，赋值方式为 `key:value`
 - `python_files` 模块的命名规则 `xx.py`
 - `python_classes` 类名的命名规则 `Xxx`
 - `python_functions` 方法的命名规则 `**`
 - `required_plugins` 插件的使用
 - `xfail_strict = true` 禁用`xpass`
- 代码示例：

```
[pytest]
```

```
addopts = --html=auto_reports.html --self-contained-html
testpaths = ./
python_files = test_*.py
python_classes = Test*
python_functions = test*
markers =
    smoke:冒烟测试用例
    login:登录测试用例
```

第八章 PO设计模式、数据驱动

8.1 【数据驱动】数据驱动介绍

一、学习目标

- 1.什么是数据驱动
- 2.数据驱动的好处
- 3.如何实现数据驱动

二、知识点

1. 【什么是数据驱动】

相同的测试脚本使用不同的测试数据来执行，测试数据和测试行为进行了完全的分离，这样的测试脚本设计模式叫做数据驱动。换句话说就是输入N组数据，框架会自动构造出N个测试用例，在测试结果中把每一个测试用例运行的结果单独展示出来。

所以，数据传递和数据驱动是两个概念，数据传递只是单纯的将变量A的数据传递给变量B。

2. 【数据驱动的好处】

数据格式的选择		
	优点	缺点
Excel	生成数据方便	二进制文件不利于版本管理
CSV	可使用Excel编辑	文本格式方便版本管理
YAML	格式完备，可读性好	格式简单
XML	格式完备	冗长复杂
JSON	格式完备，可读性一般	不能编写注释，格式死板

- 1、可以大大的减少代码量；
- 2、提高了代码的可读性和可维护性；
- 3、一组数据对应一条case，用例自动加载生成；

3. 【如何实现数据驱动】

前面的章节中讲到，pytest的数据驱动是通过@pytest.mark.parametrize()装饰器来实现的，所以，实现一个完整的数据驱动，可以分为以下步骤：

步骤一：将测试数据存入yaml、excel等文件

步骤二：从数据文件中读数据，将数据处理成@pytest.mark.parametrize()需要的传入格式，也就是形如[{"user1":"psw1","user2":"psw2"}]列表嵌套字典的格式。其中，每一个字典就是每一组数据，N个字典，则测试用例自动执行N次。

步骤三：使用装饰器@pytest.mark.parametrize()，传入步骤二中处理后的数据。

8.2【数据驱动】excel数据读取

一、学习目标

1.xlrd 模块安装

2.xlrd 模块讲解

3.xlrd 模块应用

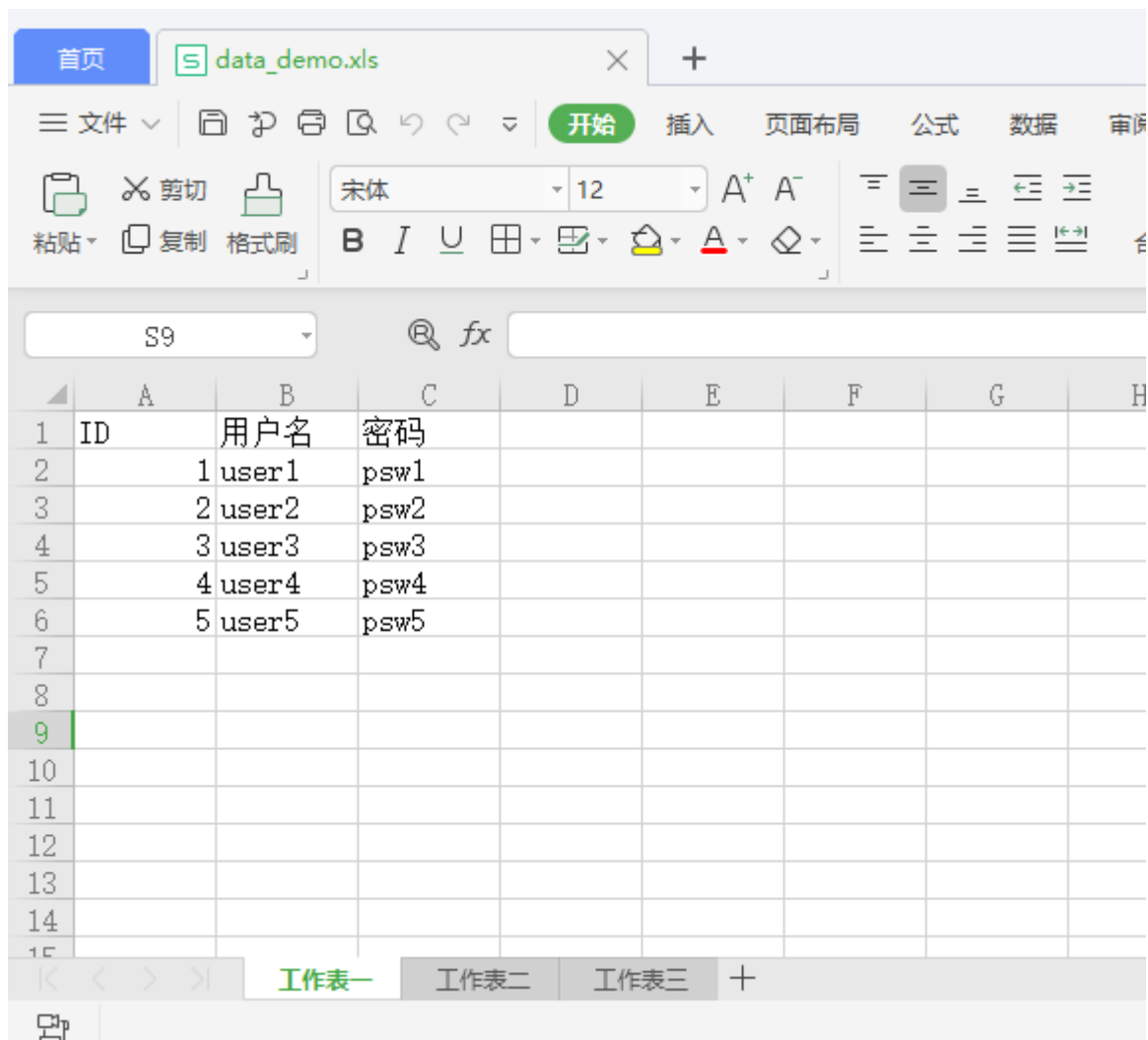
二、知识点

1.【xlrd 模块安装】

xlrd库是专门用来读取excel的三方库，建议下载1.2.0之前的版本，新的版本只支持xls不支持xlsx格式的表格文件：

```
pip install xlrd==1.2.0
```

2.【xlrd 模块讲解】



- 导入模块

```
import xlrd
```

- 打开Excel文件

```
data = xlrd.open_workbook("data_demo.xls")#文件名以及路径。返回一个Book对象
```

- 获取book中一个工作表（Sheet页）

```
table = data.sheets()[0]           #通过索引顺序获取，返回Sheet对象
table = data.sheet_by_index(0)     #通过索引顺序获取，返回Sheet对象
table = data.sheet_by_name("工作表一") #通过名称获取，返回Sheet对象，一般用这个
```

我们还可以获取工作表信息：

```
names = data.sheet_names()         #返回book中所有工作表的名字，如：['工作表一'，'工作表二'，'工作表三']
state = data.sheet_loaded("工作表一" or 1) #检查某个sheet是否导入完毕，返回布尔型，如：True
```

- 行（row）的操作

```
nrows = table.nrows                #获取该sheet中的有效行数
row = table.row(rowx)               #返回由该行中所有的单元格对象组成的列表，列表中是一个个Cell对象（数据类型：值）。0是第一行
row = table.row_slice(rowx)         #返回由该行中所有的单元格对象组成的列表，与上一个一样
len = table.row_len(rowx)           #返回该行的有效单元格长度

#获取第三行数据，第start列到第end列(start~end列)[不含第end列]。返回由该行中所有单元格的数据类型组成的列表，如：array('B', [2, 1, 1])
#注：0 empty, 1 string, 2 number, 3 date, 4 boolean, 5 error
types = table.row_types(rowx, start_colx=0, end_colx=None)

#获取第四行数据，第start列到第end列(start~end列)[不含第end列]。返回由该行中所有单元格的数据组成的列表
values = table.row_values(rowx, start_colx=0, end_colx=None)
```

参数rowx代表行的索引，第一行是0。

- 列(colnum)的操作

```
ncols = table.ncols                #获取列表的有效列数
col = table.col(colx, start_rowx=0, end_rowx=None) #返回由该列中所有的单元格对象组成的列表
col = table.col_slice(colx, start_rowx=0, end_rowx=None) #返回由该列中所有的单元格对象组成的列表
types = table.col_types(colx, start_rowx=0, end_rowx=None) #返回由该列中所有单元格的数据类型组成的列表
values = table.col_values(colx, start_rowx=0, end_rowx=None) #返回由该列中所有单元格的数据组成的列表
```

参数colx代表列的索引，第一列是0。

- 单元格的操作

```
cell = table.cell(rowx,colx)          #返回单元格对象
type = table.cell_type(rowx,colx)     #返回单元格中的数据类型
value = table.cell_value(rowx,colx)   #返回单元格中的数据
```

参数rowx,colx, 分别是行的索引和列的索引, 第一行是0。

3. 【xlrd 模块应用】

读取表格数据代码实现:

```
# [{"ID":1,"用户名":"user01","密码":"psw01"}, {"ID":2,"用户名":"user02","密码":"psw02"}]

import xlrd

def dict_data():
    data = xlrd.open_workbook('data_demo.xls')
    table = data.sheet_by_name('Sheet1')
    header = table.row_values(0)
    print(header)
    rowNum = table.nrows
    colNum = table.ncols
    if rowNum <= 1:
        print("没有数据")
    else:
        r = []
        j = 1
        for i in range(rowNum - 1):
            s = {}
            values = table.row_values(j)
            print(values)
            j = j + 1
            for x in range(colNum):
                s[header[x]] = values[x]
            r.append(s)
        return r

res = dict_data()
print(res)
```

8.3 【数据驱动】excel数据写入

一、学习目标

1. xlwt 模块安装
2. xlwt 模块讲解
3. xlwt 模块应用

二、知识点

1. 【xlwt 模块安装】

```
pip install xlwt
```

2. 【xlwt 模块讲解】

- 导入模块

```
import xlwt
```

- 创建工作簿

```
workbook = xlwt.Workbook(encoding='ascii', style_compression=0)#encoding='utf-8', 设置字符编码就可以在excel中输出中文了。默认是ascii。style_compression 表示是否压缩，不常用
```

workbook对象的属性和方法：

style_compression 表示是否压缩，不常用。

workbook 还有一些属性：

owner 设置文档所有者。

country_code: 国家码

wnd_protect: 窗口保护

obj_protect: 对象保护

Protect: 保护

backup_on_save: 保存时备份

Hpos: 横坐标

Vpos: 纵坐标

width: 宽度

Height: 高度

active_sheet: 活动sheet

tab_width: tab宽度

wnd_visible: 窗口是否可见

wnd_mini: 窗口最小化

hscroll_visible: 横向滚动条是否可见。

vscroll_visible: 纵向滚动条是否可见。

tabs_visible: tab是否可见。

dates_1904: 是否使用1904日期系统

use_cell_values: 单元格的值

default_style: 默认样式

colour_RGB: 颜色

方法有: add_sheet, add_style, add_font, add_str, del_str, str_index, add_rt, rt_index, get_sheet, raise_bad_sheetname, convert_sheetindex, setup_xcall, add_sheet_reference.

- 创建工作表

```
oneworkSheet = workbook.add_sheet("sheet1")
```

- 写入数据(行, 列, 数据)

```
oneworkSheet.write(0, 0, "hello")
```


- 保存数据,入参为文件路径

```
workbook.save("./myExcel.xls")
```

3. 【xlwt 模块应用】

我们前面讲到读取的数据格式为：

```
list_data = [{ 'ID': 1.0, '用户名': 'user1', '密码': 'psw1'}, { 'ID': 2.0, '用户名': 'user2', '密码': 'psw2'}, { 'ID': 3.0, '用户名': 'user3', '密码': 'psw3'}, { 'ID': 4.0, '用户名': 'user4', '密码': 'psw4'}, { 'ID': 5.0, '用户名': 'user5', '密码': 'psw5'}]
```

写入表格数据代码实现：

```
import xlwt

list_data = [{ 'ID': 1.0, '用户名': 'user1', '密码': 'psw1'}, { 'ID': 2.0, '用户名': 'user2', '密码': 'psw2'}, { 'ID': 3.0, '用户名': 'user3', '密码': 'psw3'}, { 'ID': 4.0, '用户名': 'user4', '密码': 'psw4'}, { 'ID': 5.0, '用户名': 'user5', '密码': 'psw5'}]

def excel_write(list_data, excel_path):
    if not isinstance(list_data, list):
        raise TypeError('数据必须是列表')
    # 1.创建book对象
    workbook = xlwt.Workbook(encoding='utf-8')
    # 2.创建sheet页
    worksheet = workbook.add_sheet("工作表一")
    # 3.写入
    #写入标题
    keys_data = list(list_data[0].keys())
    print(keys_data)
    num = len(keys_data)
    for c in range(num):
        worksheet.write(0, c, keys_data[c])
    #写入内容
    rows_num = len(list_data)
    for r in range(rows_num):
        print(r)
        values_data = list(list_data[r].values())
        print(values_data)
        cows_num = len(values_data)
        for c in range(cows_num):
            worksheet.write(r+1, c, values_data[c])
    # 4.保存数据
    workbook.save(excel_path)

excel_write(list_data, 'new_demo.xls')
```

8.4【数据驱动】excel数据驱动实践

一、学习目标

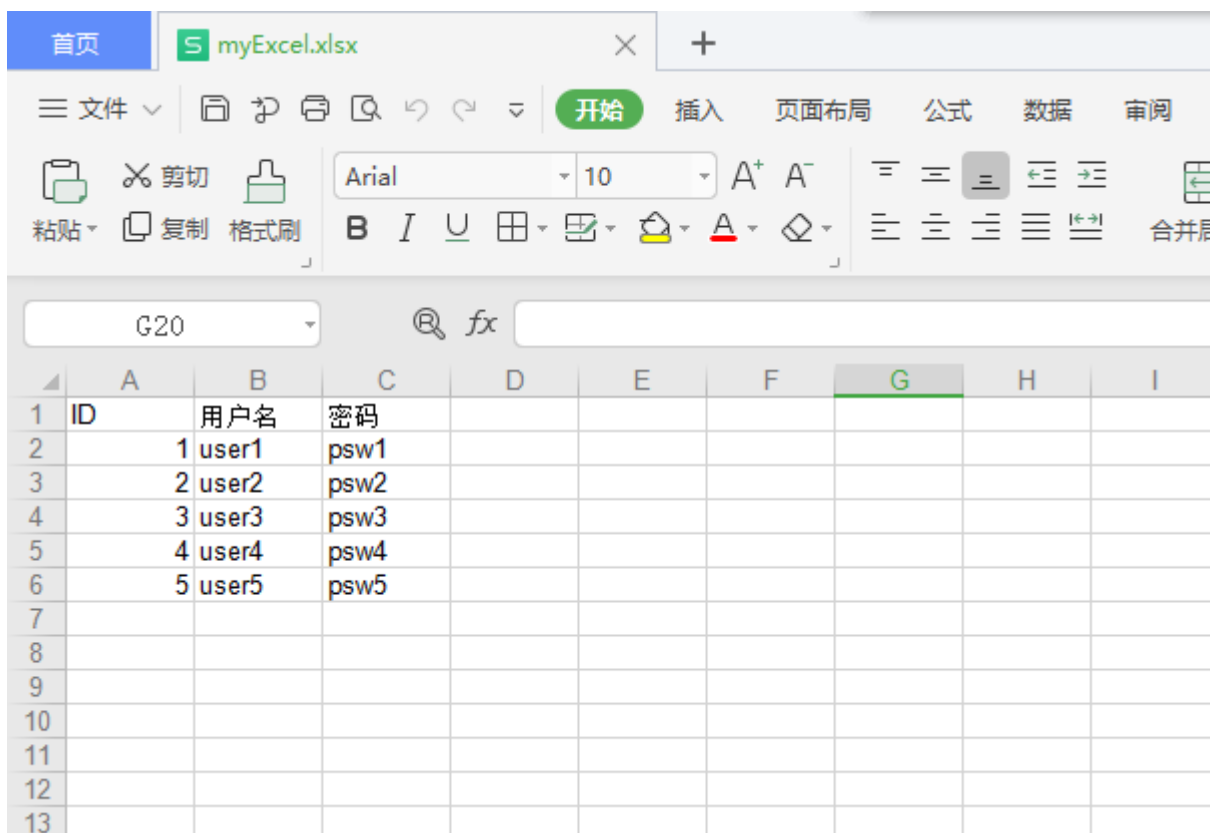
1.excel数据驱动实践

二、知识点

1.【excel数据驱动实践】

- 数据驱动实现步骤：
 - 从数据文件中读取数据，数据格式必须是列表中套多个字典的形式，如[{}]
 - 将读取到的数据通过parametrize装饰器传入测试用例函数中
- 代码实现：

Excel数据文件：



The screenshot shows an Excel spreadsheet with a table containing login data. The table has three columns: ID, 用户名 (Username), and 密码 (Password). The data rows are as follows:

ID	用户名	密码
1	user1	psw1
2	user2	psw2
3	user3	psw3
4	user4	psw4
5	user5	psw5

这里的登录功能用例用到了一个表格，我们针对不同的功能可以使用多个表格来分别存储不同功能的测试数据，也可以使用多个sheet页来划分不同功能的测试数据。

代码实现：

```
import pytest
from read_excel import dict_data

#1. 读取数据
case_data = dict_data('new_demo.xls', '工作表一')
print(case_data)

class TestCase():

    @pytest.mark.parametrize('data', case_data)
    def test_login_case(self, data):
```

```
'''登录测试'''
print("输入用户名: {}".format(data['用户名']))
print("输入密码: {}".format(data['密码']))
print("点击登录")
assert 1 == 1
```

8.5 【数据驱动】yaml 格式介绍

一、学习目标

1. yaml 的语法规则
2. yaml 的数据格式-对象
3. yaml 的数据格式-数组
4. yaml 的数据格式-纯量
5. yaml 数据引用
6. 同一个 yaml 文件表示多个文档

二、知识点

1. 【yaml的语法规则】

- 基本语法规则（与python很相似）：
 - 大小写敏感
 - 使用缩进表示层级关系
 - 缩进时不容许使用Tab键，只容许使用空格。
 - 缩进的空格数目不重要，只要相同层级的元素左侧对齐便可
 - # 表示注释，从这个字符一直到行尾，都会被解析器忽略。
- 支持的数据结构：
 - 对象：键值对的集合，又称为映射（mapping） / 哈希（hashes） / 字典（dictionary）
 - 数组：一组按次序排列的值，又称为序列（sequence） / 列表（list）
 - 纯量（scalars）：单个的、不可再分的值

2. 【yaml的数据格式-对象】

对象的一组键值对，使用冒号结构表示，可以用来表示python的字典数据类型。

- 单层字典表示
 - Python写法：

```
{"animal": "dog"}
```

- Yaml写法：

```
animal: dog    #用英文:表示，注意:后面必须加一个空格。
```

- 多层字典表示
 - Python写法：

```
{"animal": {"name": "xiaohuang", "age": "3"}}
```

- Yaml写法:

```
#写法一
animal:
  name: xiaohuang
  age: 3

#写法二
animal: {"name": "xiaohuang", "age": "3"} #也支持{}表示字典
```

3. 【yaml的数据格式-数组】

一组连词线开头的行，构成一个数组。

- 单层列表表示

- Python写法:

```
["dog", "cat", "mouse"]
```

- Yaml写法:

```
- dog
- cat
- mouse
```

- 多层列表表示

- Python写法:

```
[["dog", "cat", "mouse"]]
```

- Yaml写法:

```
#方式一
-
  - Cat
  - Dog
  - mouse

#方式二
[ [ 'Cat', 'Dog', 'Goldfish' ] ] #也支持[]表示列表
```

- Yaml列表和字典嵌套（数据驱动格式）

```
[{"username": "user1", "password", "psw1"}, {"username": "user2", "password", "psw2"}]
- username: user1
  password: psw1
- username: user2
  password: psw2
- username: user3
  password: psw3
- username: user4
  password: psw4
- username: user5
  password: psw5
```

- 表示python元组
 - Python写法:

```
("hahaha", "lalala")
```

- Yaml写法:

```
!!python/tuple ["hahaha", "lalala"]
```

4. 【yaml的数据格式-纯量】

纯量是最基本的、不可再分的值。如下数据类型都属于 JavaScript 的纯量。

字符串、布尔值、整数、浮点数、Null、时间、日期

- Python写法:

```
{
    'number': 15.01,
    'string': 'hi',
    'bool': True,
    'nothing': None,
    'date': datetime.date(2018, 1, 1),
    'time': datetime.datetime(2018, 1, 1, 12, 12, 12)
}
```

- Yaml写法:

```
number: 15.01
string: hi
bool: true
nothing: ~
date: 2018-01-01
time: 2018-01-01 12:12:12
```

4. 【yaml数据引用】

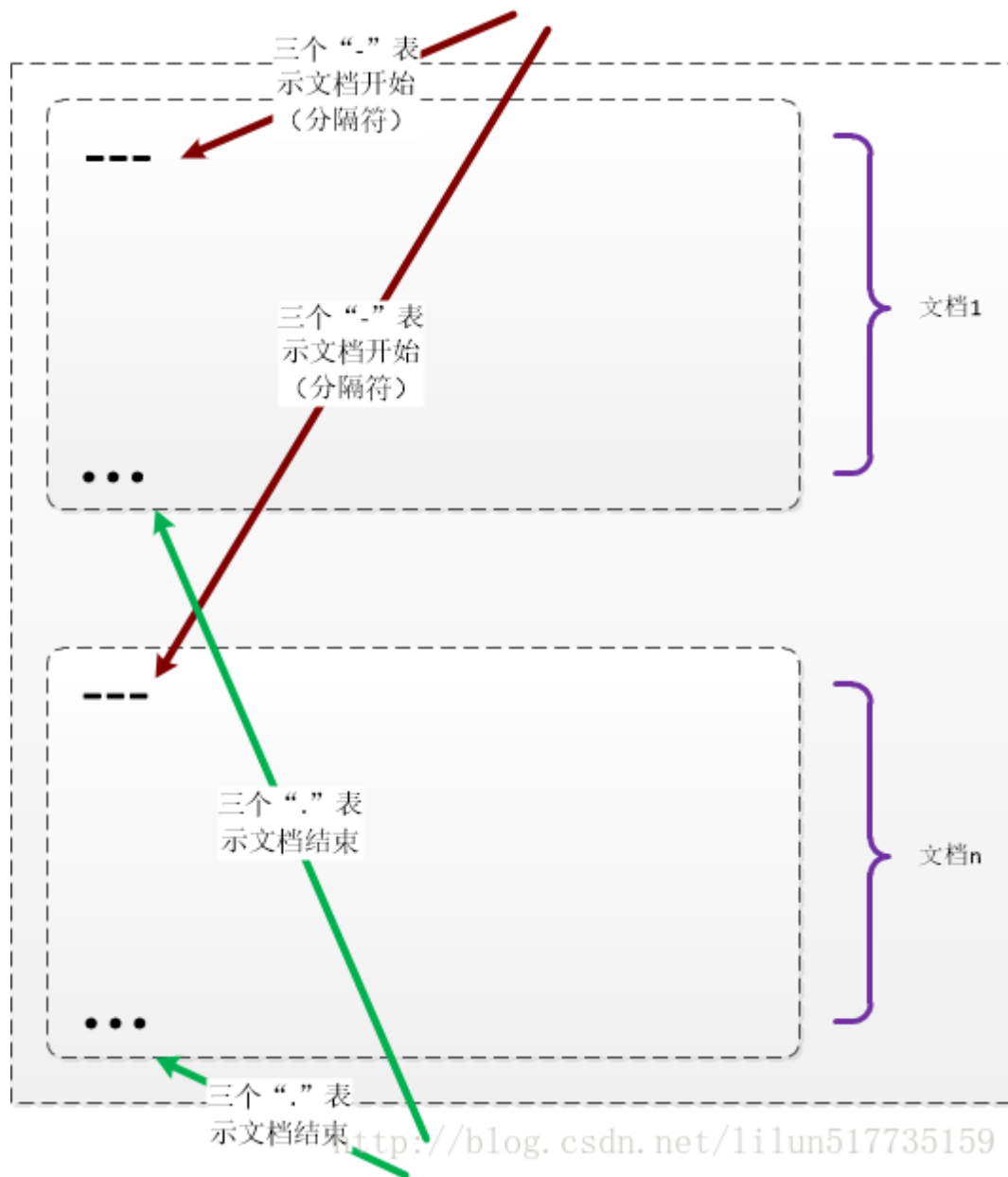
& 用来创建锚点 (defaults) , << 表示合并到当前数据, * 用来引用锚点。

- Yaml写法:

```
defaults: &defaults
  adapter: postgres
  host:    localhost

development:
  database: myapp_development
  <<: *defaults
```

5. 【同一个yaml文件表示多个文档】



yaml写法:

```
---
name: qiyu
age: 20岁
...
---
name: qingqing
age: 19岁
...
```

8.6 【数据驱动】yaml 数据读取

一、学习目标

1. pyyaml 模块安装
2. 读取yaml文件应用

二、知识点

1. 【pyyaml 模块安装】

```
pip install pyyaml
```

2. 【读取yaml文件应用】

准备一个yaml 文件数据：

```
- username: user1
  password: psw1
- username: user2
  password: psw2
- username: user3
  password: psw3
- username: user4
  password: psw4
- username: user5
  password: psw5
```

- 单个yaml文档读取：
 - 语法：

```
yaml.load(yaml_str, Loader=None)
```

```
yaml.safe_load #等效于yaml.load(yaml_str, Loader=yaml.SafeLoader)
```

```
yaml.full_load #等效于yaml.load(yaml_str, Loader=yaml.FullLoader)
```

```
yaml.unsafe_load #等效于yaml.load(yaml_str, Loader=yaml.UnsafeLoader)
```

- 参数说明：

解析 `yaml_str`，返回一个对象

`Loader` 有四种模式：

`BaseLoader`：仅加载最基本的 YAML；

`SafeLoader`：安全地加载 YAML 语言的子集。建议加载不可信的输入；

`FullLoader`：加载完整的 YAML 语言。避免执行任意代码。

`UnsafeLoader`：（也要求 `Loader` 向后兼容）不受信任的数据输入可以轻易利用的原始 `Loader` 代码。

◦ 代码实现：

```
import os
import yaml

def read_yaml(yamlpath):
    '''读取yaml数据'''
    if not os.path.isfile(yamlpath):
        raise FileNotFoundError("文件路径不存在，请检查路径是否正确：{}".format(yamlpath))
    #open读取yaml,获取文本内容
    with open(yamlpath, 'r', encoding='utf-8') as f:
        cfg = f.read()
        content = yaml.load(cfg, Loader=yaml.FullLoader)
    return content

if __name__ == '__main__':
    content = read_yaml(r'C:\Users\Admin\Desktop\stuCode\code0806\data.yaml')
    print(content)
```

• 多个yaml文档读取：（了解）

如果string或文件包含几块yaml文档，可以使用 `yaml.load_all` 来解析全部的文档。

◦ 语法：

```
yaml.load_all(ymlfile, Loader=None) #返回对象
```

◦ 参数说明：

同上

◦ 代码实现：

```
def read_yaml_group(yamlpath):
    '''读取yaml数据'''
    if not os.path.isfile(yamlpath):
        raise FileNotFoundError("文件路径不存在，请检查路径是否正确：{}".format(yamlpath))
    #open读取yaml,获取文本内容
    with open(yamlpath, 'r', encoding='utf-8') as f:
        cfg = f.read()
        content = yaml.load_all(cfg, Loader=yaml.FullLoader)
    return content
```

• yaml文件中的变量传递

如果我们需要动态替换 `yaml` 文件中的值，`string` 模块提供了实现这些规则的 `Template` 类可以替换我们想要替换掉的值。

- o yaml 示例：

```
- method: post
  params:
    username: $user
    password: $psw
  url: http://127.0.0.1:8888/login
```

- o 代码示例：

```
import yaml
from string import Template
import os

def yaml_template(yamlpath, change_data):
    '''yaml变量的动态替换'''
    if not os.path.isfile(yamlpath):
        raise FileNotFoundError("文件路径不存在，请检查路径是否正确：{}".format(yamlpath))
    if not isinstance(change_data, dict):
        raise Exception("修改内容必须是字典：{}".format(change_data))
    #读取文本内容
    with open(yamlpath, 'r', encoding='utf-8') as f:
        cfg = f.read()
        content = Template(cfg).substitute(**change_data)
    return content

if __name__ == '__main__':
    change_data = {
        "user": "user01",
        "psw": "psw01"
    }
    res =
    yaml_template(r'C:\Users\Admin\Desktop\stuCode\code0806\post_data.yaml', change_data)
    print(res)
```

8.7 【数据驱动】yaml 数据写入

一、学习目标

1.yaml数据写入应用

二、知识点

1. 【yaml数据写入应用】

写入的yaml数据：

```
list_data = [{'username': 'user1', 'password': 'psw1'}, {'username': 'user2', 'password': 'psw2'},
{'username': 'user3', 'password': 'psw3'}, {'username': 'user4', 'password': 'psw4'}, {'username':
'user5', 'password': 'psw5'}]
```

- 单个 yaml 文档写入：

o 语法：

```
yaml.dump(data, stream=None, allow_unicode=True)
```

o 参数说明：

data: 要写入的数据

stream: 打开的文件

allow_unicode: True参数防止写入的中文乱码。

o 代码实现：

```
#yaml文件的写入
import os

import yaml

list_data = [{'username': 'user1', 'password': 'psw1'}, {'username': 'user2', 'password': 'psw2'}, {'username': 'user3', 'password': 'psw3'}, {'username': 'user4', 'password': 'psw4'}, {'username': 'user5', 'password': 'psw5'}]

def write_yaml(yamlpath, data):
    '''yaml写入封装'''
    # if not os.path.isfile(yamlpath):
    #     raise FileNotFoundError("文件路径不存在, 请检查: {}".format(yamlpath))
    if not isinstance(data, list):
        raise TypeError("数据类型错误, 请输入列表数据: {}".format(data))
    with open(yamlpath, 'w', encoding='utf-8') as f:
        yaml.dump(data=data, stream=f, allow_unicode=True)

if __name__ == '__main__':
    write_yaml('data_demo.yaml', list_data)
```

• 多个yaml文档写入：（了解）

o 语法：

```
yaml.dump_all(data, stream=None, allow_unicode=True)
```

o 参数说明：

data: 要写入的数据

stream: 打开的文件

allow_unicode: True参数防止写入的中文乱码。

o 代码实现：

```
def write_yaml_group(yamlpath,data):
    '''yaml写入封装'''
    # if not os.path.isfile(yamlpath):
    #     raise FileNotFoundError("文件路径不存在，请检查：{}".format(yamlpath))
    if not isinstance(data,list):
        raise TypeError("数据类型错误，请输入列表数据：{}".format(data))
    with open(yamlpath,'w',encoding='utf-8') as f:
        yaml.dump_all(documents=data,stream=f,allow_unicode=True)
```

8.8 【数据驱动】yaml 数据驱动实践

一、学习目标

1.yaml数据驱动实践

二、知识点

1. 【yaml 数据驱动实践】

- 数据驱动实现步骤

- (1) 从数据文件中读取数据，数据格式必须是列表中套多个字典的形式，如[{}]
- (2) 将读取到的数据通过parametrize装饰器传入测试用例函数中

- 代码实现

Yaml数据文件：

```
main.py × data_demo.yaml × test_yaml_demo.py ×
1  - 用户名: user1
2    密码: psw1
3  - 用户名: user2
4    密码: psw2
5  - 用户名: user3
6    密码: psw3
7  - 用户名: user4
8    密码: psw4
9  - 用户名: user5
10   密码: psw5
```

代码实现：

```
from read_yaml import read_yaml
import pytest
case_data = read_yaml(r'C:\Users\Admin\Desktop\stuCode\code0808\data_demo.yaml')
print(case_data)

class TestCase():

    @pytest.mark.parametrize('data', case_data)
    def test_login_case(self, data):
```

```
'''登录用例'''  
print("输入用户名: {}".format(data['用户名']))  
print("输入密码: {}".format(data['密码']))  
print("点击登录")  
assert 1 == 1
```

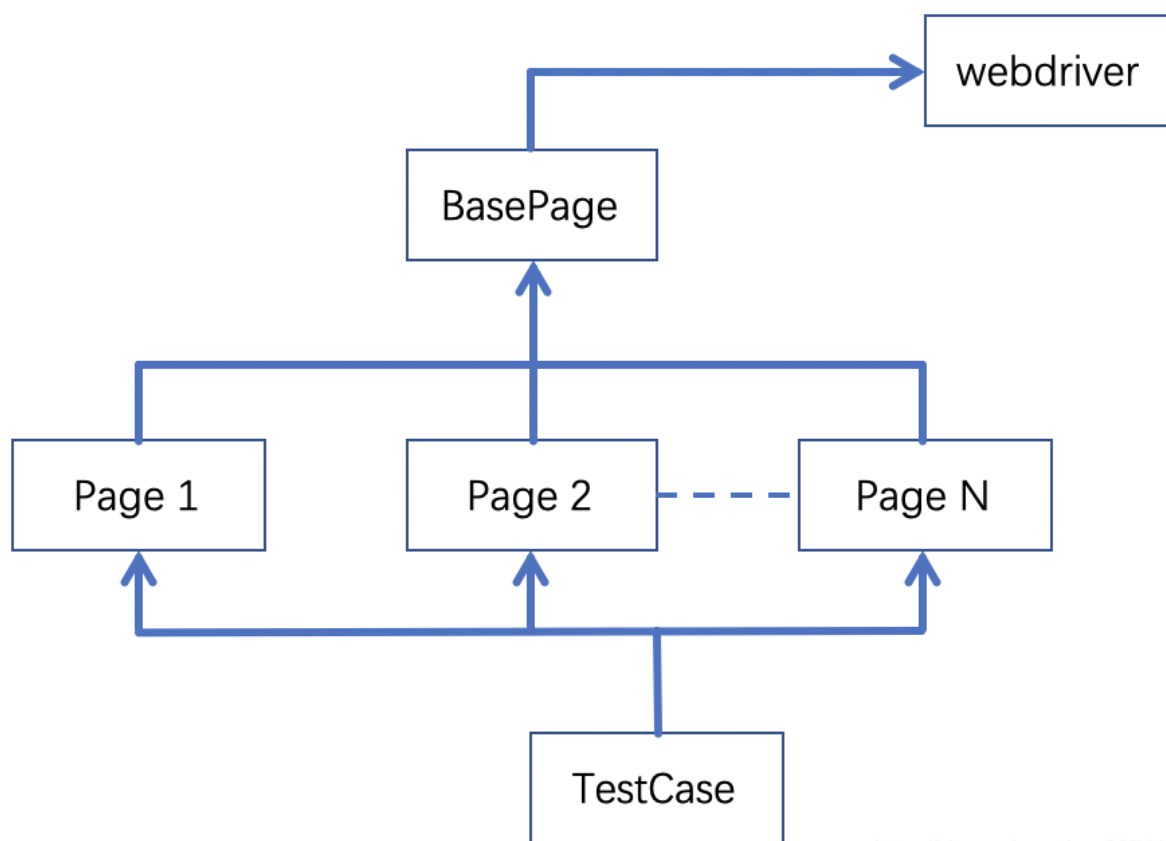
8.9 【PO设计模式】PO结构设计讲解

一、学习目标

1. PO 是什么
2. PO 的优势
3. PO 模式的核心要素

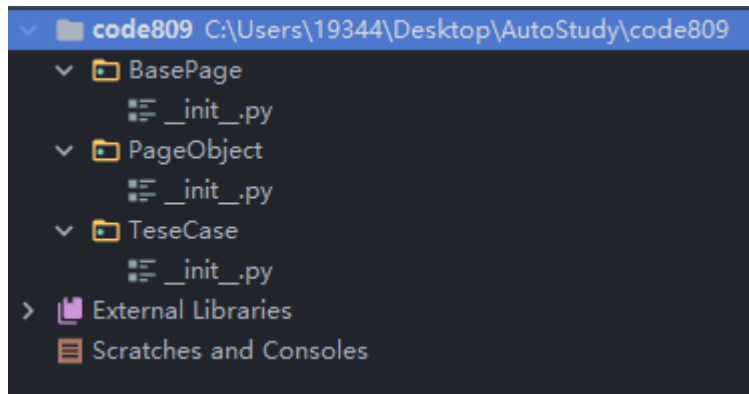
二、知识点

1. 【PO是什么】



https://blog.csdn.net/qc_38741986

基于这种模式，我们设计的目录结构：



- 1、页面对象模型（PO）是一种设计模式，也可以理解为代码编写规范，用来管理维护一组web元素的对象库。
- 2、在PO下，应用程序的每一个页面都有一个对应的page class。
- 3、每一个page class维护着该web页的元素集和操作这些元素的方法。
- 4、page class中的方法命名最好根据对应的业务场景进行。
- 5、实际上，PO模式是一种分层思想，将完整的自动化脚本的编写流程分离。

2. 【PO的优势】

- 1、PO提供了一种业务流程与页面元素操作分离的模式，这使得测试代码变得更加清晰。
- 2、页面对象与用例分离，使得我们更好的复用对象。
- 3、可复用的页面方法代码会变得更加优化
- 4、更加有效的命名方式使得我们更加清晰的知道方法所操作的UI元素。例如我们要回到首页，方法命名为：gotoHomePage()，通过方法名即可清晰的知道具体的功能实现。

2. 【PO模式的核心要素】

本质上PO设计模式是一种分层思想，将脚本的实现过程分为：基础层，逻辑层，业务层。后面的课程中会对这三层作详细的讲解。

不止是web自动化测试，我们可以把PO设计模式的思想应用于整个测试框架的设计过程中。

8.10 【PO设计模式】基础层

一、学习目标

- 1.基础层方法封装

二、知识点

1. 【基础层方法封装】

webbase.py 代码示例：

```
#WEB自动化的基础层封装
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException

class WebBase():
```

```

def __init__(self,driver):
    self.driver = driver

def get_url(self,url):
    '''打开浏览器地址'''
    self.driver.get(url)
    self.driver.maximize_window()

def findElement(self,locator):
    '''定位元素方法'''
    try:
        if isinstance(locator,list):
            locator = tuple(locator)
        if not isinstance(locator,tuple):
            print("locator参数类型错误，必须是元祖或列表！")
        print("正在定位元素信息：定位方式->%s, value值->%s" % (locator[0], locator[1]))
        ele = WebDriverWait(self.driver,10,0.5).until(EC.presence_of_element_located(locator))
        print("定位元素{}成功".format(locator))
        return ele
    except TimeoutException as e:
        print('未定位到元素',locator)
        raise e

def sendkeys(self,locator,text):
    '''输入文本'''
    try:
        ele = self.findElement(locator)
        ele.send_keys(text)
        print('send keys {} 成功'.format(text))
    except Exception as e:
        print('send keys {} 失败'.format(text))
        raise e

def click(self,locator):
    try:
        ele = self.findElement(locator)
        ele.click()
        print('click {} 成功'.format(locator))
    except Exception as e:
        print('click {} 失败'.format(locator))
        raise e

```

8.11 【PO设计模式】逻辑层

一、学习目标

1.逻辑层方法简单封装

二、知识点

1.【逻辑层方法简单封装】

homePage.py 代码示例：

```

from BasePage.webbase import webBase

class HomePage(webBase):
    '''百度首页'''

    loc_input = ["id", "kw"]
    loc_button = ["id", "su"]
    loc_news = ["xpath", ".*//a[text()='新闻']"]
    loc_map = ["xpath", ".*//a[text()='地图']"]

    def __init__(self, driver):
        super().__init__(driver)
        self.get_url('https://www.baidu.com/')

    def search_info(self, text):
        '''搜索功能'''
        #1. 输入文本
        self.sendkeys(self.loc_input, text)
        #2. 点击百度一下
        self.click(self.loc_button)

    def goto_news(self):
        '''点击新闻'''
        self.click(self.loc_news)

    def goto_map(self):
        '''点击地图'''
        self.click(self.loc_map)

if __name__ == '__main__':
    from selenium import webdriver
    driver =
    webdriver.Chrome(executable_path=r'C:\Users\Admin\Desktop\stuCode\code0809\chromedriver.exe')
    home = HomePage(driver)
    home.goto_map()

```

8.12 【PO设计模式】业务层

一、学习目标

1. 业务层方法简单封装

二、知识点

1. 【业务层方法简单封装】

test_baidu_case.py 代码示例：

```

#业务层
from PageObject.homePage import HomePage
from selenium import webdriver

```

```
driver =  
webdriver.Chrome(executable_path=r'C:\Users\Admin\Desktop\stuCode\code0809\chromedriver.exe')  
  
class TestCase():  
  
    def test_baidu_case01(self):  
        '''测试用例一：查询功能'''  
        home = HomePage(driver)  
        home.search_info('python')  
  
    def test_baidu_case02(self):  
        '''测试用例二：跳转新闻页'''  
        home = HomePage(driver)  
        home.goto_news()  
  
    def test_baidu_case03(self):  
        '''测试用例三：跳转到地图页'''  
        home = HomePage(driver)  
        home.goto_map()
```

第九章 自动化测试常用模块

9.1 【SYS模块】系统操作讲解与应用

一、学习目标

1. sys 模块常用方法介绍
2. sys 模块重点方法详解

二、知识点

1. 【sys模块常用方法介绍】

函数	说明
<code>dir(sys)</code>	<code>dir()</code> 方法查看模块中可用的方法。注意：如果是在编辑器，一定要注意要事先声明代码的编码方式，否则中文会乱码。
<code>sys.argv</code>	实现从程序外部向程序传递参数
<code>sys.exit([arg])</code>	程序中间的退出， <code>arg=0</code> 为正常退出
<code>sys.getdefaultencoding()</code>	获取系统当前编码，一般默认为 <code>ascii</code>
<code>sys.setdefaultencoding()</code>	设置系统默认编码，执行 <code>dir(sys)</code> 时不会看到这个方法，在解释器中执行不通过，可以先执行 <code>reload(sys)</code> ，再执行 <code>setdefaultencoding('utf8')</code> ，将系统编码设置为 <code>utf8</code>
<code>sys.getfilesystemencoding()</code>	获取文件系统编码方式， <code>windows</code> 下返回 <code>'mbcs'</code> ， <code>mac</code> 下返回 <code>'utf-8'</code>
<code>sys.path</code>	获取指定模块搜索路径的字符串集合，可以将写好的模块放在得到的某个路径下，就可以在程序中import时正确找到
<code>sys.platform</code>	获取当前系统平台。
<code>sys.stdin/sys.stdout/sys.stderr</code>	<code>stdin</code> ， <code>stdout</code> ，以及 <code>stderr</code> 变量包含与标准I/O流对应的流对象。如果需要更好地控制输出，而 <code>print</code> 不能满足要求，它们就是你所需要的。你也可以替换它们，重定向输出和输入到其它设备(device)，或者以非标准的方式处理它们。
<code>sys.modules</code>	是一个全局字典，该字典是python启动后就加载在内存中。每当程序员导入新的模块， <code>sys.modules</code> 将自动记录该模块。当第二次再导入该模块时，python会直接到字典中查找，从而加快程序运行的速度。它拥有字典所拥有的一切方法。

2. 【sys模块重点方法详解】

- `sys.argv`

传递给Python脚本的命令行参数列表，第一个参数是程序本身的路径，实现了从程序外部向程序内传递参数。

代码示例：

```
import sys

for i in sys.argv:
    print('命令行中传进来的参数：', i)
```

- `sys.path`

import模块的目录名位置列表，列表中的第一项为当前位置。

模块要处于Python搜索路径中的目录里才能被导入，`sys.path` 是个列表，所以在末尾添加目录是很容易的，用 `sys.path.append` 就行了。当这个append执行完之后，新目录即时起效，以后的每次import操作都可能会检查这个目录。

```
PROJECT_ROOT = os.path.dirname(os.path.abspath(__file__)) #根目录
sys.path.append(PROJECT_ROOT) #讲根目录加到python的搜索路径当中，这样就不会由于找不到路径而报导入错误
```

- `sys.exit([arg])`

程序中间的退出, `arg=0` 为正常退出。在一般情况下, 执行到主程序末尾, 解释器自动退出, 但是如果需要中途退出程序, 可以调用 `sys.exit` 函数 (0是正常退出, 其他为异常)。

代码示例:

```
import sys

print('111')
print('222')
sys.exit('错误')
print('333')
```

在如上例子中, 首先打印出hello, 在执行 `sys.exit('错误')` 时, 将'错误'作为参数传递到函数中, 然后将'错误'打印出来, 程序正常退出, 不会执行后面的语句, 也就不会打印出world。

- `sys.platform`

获取当前系统平台, 返回操作系统的名称。

```
print(sys.platform)
```

- `sys.modules`

返回系统导入的模块字段。

```
print(sys.modules['os'])
```

- `sys.getdefaultencoding()` / `sys.getfilesystemencoding()`

`sys.getdefaultencoding()` : 获取解释器默认编码。

`sys.getfilesystemencoding()` : 获取内存数据存到文件里的默认编码。

```
print(sys.getdefaultencoding())
```

9.2 【OS模块】常用方法讲解与应用

一、学习目标

1. `os` 模块常用方法介绍
2. `os` 模块重点方法详解

二、知识点

1. 【`os` 模块常用方法介绍】

常用方法: `os` 模块的常见函数列表(`import os`)

函数	说明
<code>os.environ</code>	一个dictionary 包含环境变量的映射关系
<code>os.name</code>	显示当前使用的平台
<code>os.sep</code>	显示当前平台下路径分隔符
<code>os.linesep</code>	给出当前平台使用的行终止符
<code>os.remove('filename')</code>	删除一个文件
<code>os.rename("oldname","newname")</code>	重命名文件
<code>os.getcwd()</code>	显示当前python脚本工作路径
<code>os.chdir(dir)</code>	改变当前目录，注意windows下用到转义
<code>os.listdir('dirname')</code>	返回指定目录下的所有文件和目录名
<code>os.makedirs('dirname/dirname')</code>	可生成多层递规目录
<code>os.rmdir('dirname')</code>	删除单级目录
<code>os.getlogin()</code>	得到用户登录名称
<code>os.getenv('key')</code>	得到环境变量配置
<code>os.putenv('key')</code>	设置环境变量
<code>os.system()</code>	运行shell命令，注意：这里是打开一个新的shell，运行命令，当命令结束后，关闭shell。

2. 【os 模块重点方法详解】

- `os.sep`：返回系统路径的分隔符

代码示例：

```
#1.os.sep
print(os.sep)
```

- `os.name`：返回当前平台操作系统

代码示例：

```
#2.os.name
print(os.name)
```

当返回的值为 `posix` 时，则为 `linux` 系统；返回值为 `nt` 时，则为 `windows` 系统。

- `os.environ`：系统的环境变量,输出为一个字典形式，系统环境信息全部存储在该字典中

`os.getenv()` #从环境变量取值

代码示例：

```
#3.os.environ
print(os.environ)
print(os.getenv('PATH'))
```

- **os.uname()：返回操作系统的详细信息,也可以得到具体分项的信息**

注：os.uname()只能用于Linux。os.uname()是不支持windows系统的。

代码示例：

```
import os #导入模块
uname = os.uname() #使用变量接收
print(uname)      # 输出系统所有详细信息
print(uname.sysname) # 输出系统操作系统类型
print(uname.nodename) # 输出主机名称
print(uname.release) # 输出内核版本
print(uname.version) # 输出系统版本
print(uname.machine) #输出硬件架构
```

- **对文件以及目录的操作**

- (1) 文件操作：创建文件以及删除文件Linux

```
os.mknod(文件名称) # 当该文件存在的时候出现报错
os.remove(文件名称) # 当该文将不存在的时候出现报错
```

```
import os
os.mknod('./dir01')
os.remove('./dir01')
```

- (2) 目录操作：创建目录以及删除目录

```
os.listdir(目录名称) # 列出目录中的所有文件以及目录，返回一个列表
os.mkdir(目录名称) # 创建一个目录，不能递归创建，目录存在时再创建报错。
os.makedirs('目录名称') # 递归创建目录，目录存在时再创建报错
os.rmdir(目录名称) # 删除一个空目录，不能递归删除，目录不存在时以及存在文件时候报错。
os.removedirs('目录名称') # 递归删除空目录，目录不存在或者目录中存在文件时报错
```

```
#4.目录操作
os.mkdir('./dir01') #单个文件夹创建
os.makedirs('./dir02/dir03') #创建多个文件夹
os.rmdir('./dir01') #删除单个文件夹
os.removedirs('./dir02/dir03')
```

- (3) 对文件以及目录进行重命名

```
os.rename('oldname', 'newname') # 当新名字已经被使用的时候，重命名失败
```

```
#5.文件重命名
os.rename('./dir01', './new01')
os.rename('test.txt', 'new.txt')
```

9.3 【OS模块】路径操作讲解与应用

一、学习目标

1. os 模块 path 常用方法介绍
2. os 模块 path 重点方法详解

二、知识点

1. 【os 模块 path 常用方法介绍】

常用方法：

函数	说明
<code>os.path.abspath()</code>	获取绝对路径 <code>os.path.abspath("1.txt") == os.path.join(os.getcwd(), "1.txt")</code>
<code>os.path.split()</code>	用于分开一个目录名称中的目录部分和文件名称部分。
<code>os.path.splitext(path)</code>	分割路径，返回路径名和文件扩展名的元组
<code>os.pardir</code>	表示当前平台下上一级目录的字符 ..
<code>os.path.join(path, name)</code>	连接目录和文件名。
<code>os.path.basename(path)</code>	返回文件名
<code>os.path.dirname(path)</code>	返回文件路径
<code>os.path.getctime("/root/1.txt")</code>	返回1.txt的ctime(创建时间)时间戳
<code>os.path.exists(os.getcwd())</code>	判断文件是否存在
<code>os.path.isfile(os.getcwd())</code>	判断是否是文件名，1是0否
<code>os.path.isdir('c:\Python\temp')</code>	判断是否是目录，1是0否
<code>os.path.islink('/home/111.sql')</code>	是否是符号连接，windows下不可用
<code>os.path.samefile(os.getcwd(), '/home')</code>	看看两个文件名是不是指的是同一个文件
<code>os.walk()</code>	能够把给定的目录下的所有目录和文件遍历出来。
<code>os.path.walk('/home/huaying', test_fun, "a.c")</code>	遍历/home/huaying下所有子目录包括本目录,对于每个目录都会调用函数test_fun。

2. 【os模块path重点方法详解】

- 文件路径获取

```
#os路径处理
import os

#1. 获取上一级目录
```

```
print(os.path.pardir)

#2. 获取绝对路径
print(os.path.abspath(__file__))

#3. 获取文件上一层路径
print(os.getcwd())
print(os.path.dirname(os.path.abspath(__file__)))

#4. 文件夹与文件名分割
print(os.path.split(os.path.abspath(__file__)))

#5. 拼接目录与文件
print(os.path.join(r'C:\Users\Admin\Desktop\stuCode\code0901', 'main3.py'))
```

- `os.walk(top, topdown=True, onerror=None)`

- 1、参数`top`表示需要遍历的顶级目录的路径。
 - 2、参数`topdown`的默认值是“True”表示首先返回顶级目录下的文件，然后再遍历子目录中的文件。当`topdown`的值为“False”时，表示先遍历子目录中的文件，然后再返回顶级目录下的文件。
 - 3、参数`onerror`默认值为“None”，表示忽略文件遍历时的错误。如果不为空，则提供一个自定义函数提示错误信息后继续遍历或抛出异常中止遍历。
- 返回值：函数返回一个元组，含有三个元素。这三个元素分别是：每次遍历的路径名、路径下子目录列表、目录下文件列表。

- `os.path.walk(top, func, arg)`

- 1、参数`top`表示需要遍历的目录路径。
 - 2、参数`func`表示回调函数，即对遍历路径进行处理的函数。所谓回调函数，是作为某个函数的参数使用，当某个时间触发时，程序将调用定义好的回调函数处理某个任务。注意：`walk`的回调函数必须提供三个参数：第1个参数为`os.path.walk`的参数`arg`，第2个参数表示目录`dirname`，第3个参数表示文件列表`names`。注意：`os.path.walk`的回调函数中的文件列表不和`os.walk()`那样将子目录和文件分开，而是混为了一谈，需要在回调函数中判断是文件还是子目录。
 - 3、参数`arg`是传递给回调函数的元组，为回调函数提供处理参数，`arg`可以为空。回调函数的第1个参数就是用来接收这个传入的元组的。
- 区别：`os.path.walk()`与`os.walk()`产生的文件名列表并不相同。`os.walk()`产生目录树下的目录路径和文件路径，而`os.path.walk()`只产生文件路径（是子目录与文件的混合列表）。

9.4 【logging模块】日志模块的基本使用

一、学习目标

- 1.简单的日志打印应用
- 2.logging相关概念介绍

二、知识点

1.【简单的日志打印应用】

代码示例：

```
import logging

logger = logging.getLogger(__name__) #__name__是当前模块名
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s') #基本设置

logger.info("start print log")
logger.debug("Do something")
logger.warning("Something maybe fail.")
logger.info("Finish")
```

logging中可以选择很多消息级别，如debug、info、warning、error以及critical。通过赋予logger或者handler不同的级别，开发者就可以只输出错误信息到特定的记录文件，或者在调试时只记录调试信息。

2. 【logging相关概念介绍介绍】

• logging.basicConfig函数各参数

filename: 指定日志文件名；
 filemode: 和file函数意义相同，指定日志文件的打开模式，'w'或者'a'；
 format: 指定输出的格式和内容，format可以输出很多有用的信息
 %(levelname)s: 打印日志级别的数值
 %(levelname)s: 打印日志级别的名称
 %(pathname)s: 打印当前执行程序的路径，其实就是sys.argv[0]
 %(filename)s: 打印当前执行程序名
 %(funcName)s: 打印日志的当前函数
 %(lineno)d: 打印日志的当前行号
 %(asctime)s: 打印日志的时间
 %(thread)d: 打印线程ID
 %(threadName)s: 打印线程名称
 %(process)d: 打印进程ID
 %(message)s: 打印日志信息
 datefmt: 指定时间格式，同time.strftime()；
 level: 设置日志级别，默认为logging.WARNING；
 stream: 指定将日志的输出流，可以指定输出到sys.stderr，sys.stdout或者文件，默认输出到sys.stderr，当stream和filename同时指定时，stream被忽略；

• python中配置logging有三种方式

第一种：基础配置

```
logging.basicConfig(filename="config.log",filemode="w",format="%(asctime)s-%(name)s-%(levelname)s-%(message)s",level=logging.INFO)
```

第二种：使用配置文件的方式配置logging,使用fileConfig(filename,defaults=None,disable_existing_loggers=True)函数来读取配置文件。

第三种：使用一个字典方式来写配置信息，然后使用dictConfig(dict,defaults=None,disable_existing_loggers=True)函数来瓦按成logging的配置。

9.5 【logging模块】将日志写入文件

一、学习目标

- 1.将日志写入到文件
- 2.将日志同时输出到屏幕和日志文件

二、知识点

1. 【将日志写入到文件】

设置logging，创建一个 `FileHandler`，并对输出消息的格式进行设置，将其添加到logger，然后将日志写入到指定的文件中。

```
import logging

#1. 创建日志对象
logger = logging.getLogger(__name__)
logger.setLevel(level=logging.INFO)          #设置总级别

#2. 创建文件处理器
handler = logging.FileHandler("log.txt")
handler.setLevel(logging.INFO)              #设置文件输出级别
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
handler.setFormatter(formatter)            #设置日志打印格式

#3. 将文件处理器添加到日志对象
logger.addHandler(handler)

#4. 日志打印
logger.info("Start print log")
logger.debug("Do something")
logger.warning("Something maybe fail.")
logger.info("Finish")
```

2. 【将日志同时输出到屏幕和日志文件】

logger中添加 `StreamHandler`，可以将日志输出到屏幕上。

```
import logging

logger = logging.getLogger(__name__)

#文件输出设置
logger.setLevel(level=logging.INFO)

handler = logging.FileHandler("log.txt")
handler.setLevel(logging.INFO)
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
handler.setFormatter(formatter)

#控制台输出设置
console = logging.StreamHandler()
console.setLevel(logging.INFO)
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
console.setFormatter(formatter)

#加入到handler
```



```

logger.addHandler(handler)
logger.addHandler(console)

logger.info("Start print log")
logger.debug("Do something")
logger.warning("Something maybe fail.")
logger.info("Finish")

```

可以发现，logging有一个日志处理的主对象，其他处理方式都是通过addHandler添加进去，logging中包含的handler主要有如下几种：

handler名称	位置	作用
StreamHandler	logging.StreamHandler	日志输出到流，可以是sys.stderr，sys.stdout或者文件
FileHandler	logging.FileHandler	日志输出到文件
BaseRotatingHandler	logging.handlers.BaseRotatingHandler	基本的日志回滚方式
RotatingHandler	logging.handlers.RotatingHandler	日志回滚方式，支持日志文件最大数量和日志文件回滚
TimeRotatingHandler	logging.handlers.TimeRotatingHandler	日志回滚方式，在一定时间区域内回滚日志文件
SocketHandler	logging.handlers.SocketHandler	远程输出日志到TCP/IP sockets
DatagramHandler	logging.handlers.DatagramHandler	远程输出日志到UDP sockets
SMTPHandler	logging.handlers.SMTPHandler	远程输出日志到邮件地址
SysLogHandler	logging.handlers.SysLogHandler	日志输出到syslog
NTEventLogHandler	logging.handlers.NTEventLogHandler	远程输出日志到Windows NT/2000/XP的事件日志
MemoryHandler	logging.handlers.MemoryHandler	日志输出到内存中的指定buffer
HTTPHandler	logging.handlers.HTTPHandler	通过"GET"或者"POST"远程输出到HTTP服务器

9.6 【logging模块】日志打印设置

一、学习目标

- 1.设置消息的等级
- 2.捕获traceback

二、知识点

1.【设置消息的等级】

可以设置不同的日志等级，用于控制日志的输出级别。

日志等级	使用范围
FATAL	致命错误
CRITICAL	特别糟糕的事情，如内存耗尽、磁盘空间为空，一般很少使用
ERROR	发生错误时，如IO操作失败或者连接问题
WARNING	发生很重要的事件，但是并不是错误时，如用户登录密码错误
INFO	处理请求或者状态变化等日常事务
DEBUG	调试过程中使用DEBUG等级，如算法中每个循环的中间状态

```
import logging

logger = logging.getLogger(__name__)
logger.setLevel(level=logging.WARNING)          #设置总级别WARNING, 则最终输出WARNING及更高级别

handler = logging.FileHandler("log.txt")
handler.setLevel(logging.WARNING)              #设置文件输出级别
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
handler.setFormatter(formatter)

logger.addHandler(handler)

logger.info("Start print log")
logger.debug("Do something")
logger.warning("Something maybe fail.")
logger.info("Finish")
```

2. 【捕获traceback】

Python中的traceback模块被用于跟踪异常返回信息，可以在logging中记录下traceback。

```
import logging

logger = logging.getLogger(__name__)
logger.setLevel(level=logging.INFO)
handler = logging.FileHandler("log.txt")
handler.setLevel(logging.INFO)
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
handler.setFormatter(formatter)

console = logging.StreamHandler()
console.setLevel(logging.INFO)

logger.addHandler(handler)
logger.addHandler(console)

logger.info("Start print log")
logger.debug("Do something")
logger.warning("Something maybe fail.")
try:
    print(1/0)
```

```
except Exception:
    logger.error("分母不能为0", exc_info=True)

logger.info("Finish")
```

也可以使用：`logger.exception("分母不能为0")`

9.7 【logging模块】文件配置logging模块

一、学习目标

1. `ini` 文件存储日志配置
2. `Yaml` 文件存储日志配置

二、知识点

1. 【ini文件配置】

- 配置文件.ini

```
[log]
formatter = %(asctime)s - %(name)s - %(levelname)s - %(message)s
level = WARNING
stream_handler_level = DEBUG
file_handler_level = DEBUG
```

- 代码实现

```
import logging
from configparser import RawConfigParser

def read_config_ini(configPath):
    config = RawConfigParser()
    config.read(configPath, encoding='utf-8')
    return config

config = read_config_ini('配置文件.ini')['log']

logger = logging.getLogger(__name__)
#文件输出设置
logger.setLevel(level=config['level'])
handler = logging.FileHandler("log.txt")
handler.setLevel(config['file_handler_level'])
formatter = logging.Formatter(config['formatter'])
handler.setFormatter(formatter)

#控制台输出设置
console = logging.StreamHandler()
console.setLevel(config['stream_handler_level'])
console.setFormatter(formatter)
```

```
#加入到handler
logger.addHandler(handler)
logger.addHandler(console)

#示例代码
logger.info("Start print log")
logger.debug("Do something")
logger.warning("Something maybe fail.")
logger.info("Finish")
```

2. 【Yaml文件配置】

- 配置文件.yaml

```
log:
  formatter: '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
  level: WARNING
  stream_handler_level: DEBUG
  file_handler_level: DEBUG
```

- 代码实现

```
import logging
import os
import yaml

def read_yaml(yamlPath):
    '''读取yaml文件内容realPath: 文件的真实绝对路径'''
    if not os.path.isfile(yamlPath):
        raise FileNotFoundError("文件路径不存在, 请检查路径是否正确: %s" % yamlPath)
    with open(yamlPath, 'r', encoding='utf-8') as f:
        cfg = f.read()
        content = yaml.load(cfg, Loader=yaml.FullLoader)
        return content

config = read_yaml('配置文件.yaml')['log']

logger = logging.getLogger(__name__)
#文件输出设置
logger.setLevel(level=config['level'])
handler = logging.FileHandler("log.txt")
handler.setLevel(config['file_handler_level'])
formatter = logging.Formatter(config['formatter'])
handler.setFormatter(formatter)

#控制台输出设置
console = logging.StreamHandler()
console.setLevel(config['stream_handler_level'])
console.setFormatter(formatter)
```

```
#加入到handler
logger.addHandler(handler)
logger.addHandler(console)

#示例代码
logger.info("Start print log")
logger.debug("Do something")
logger.warning("Something maybe fail.")
logger.info("Finish")
```

9.8 【SMTP邮件发送】发送普通邮件

一、学习目标

- 1.邮箱的发送账号开启SMTP服务
- 2.发送普通邮件

二、知识点

1.【邮箱的发送账号开启SMTP服务】

- 步骤一：打开邮箱账号，设置-账户-POP3/IMAP/SMTP/Exchange/CardDAV/CalDAV服务

邮箱首页 | 设置 - 换肤

POP3/IMAP/SMTP/Exchange/CardDAV/CalDAV服务

开启服务：

POP3/SMTP服务 (如何使用 Foxmail 等软件收发邮件？)	已开启 关闭	开启
IMAP/SMTP服务 (什么是 IMAP，它又是如何设置？)	已开启 关闭	
Exchange服务 (什么是Exchange，它又是如何设置？)	已关闭 开启	
CardDAV/CalDAV服务 (什么是CardDAV/CalDAV，它又是如何设置？)	已关闭 开启	
(POP3/IMAP/SMTP/CardDAV/CalDAV服务均支持SSL连接。如何设置？)		

收取选项： 的邮件

☐ 收取“我的文件夹”

☐ 收取“QQ邮件订阅”

☒ SMTP发信后保存到服务器 勾选

(以上收取选项对POP3/IMAP/SMTP/Exchange均生效。了解更多)

☐ 收取垃圾邮件隔离提醒

(该收取选项只对POP3生效。我使用了IMAP/Exchange协议，怎么办？)

同步选项： ☐ 禁止收信软件删信 (为什么会有收信软件删信？)

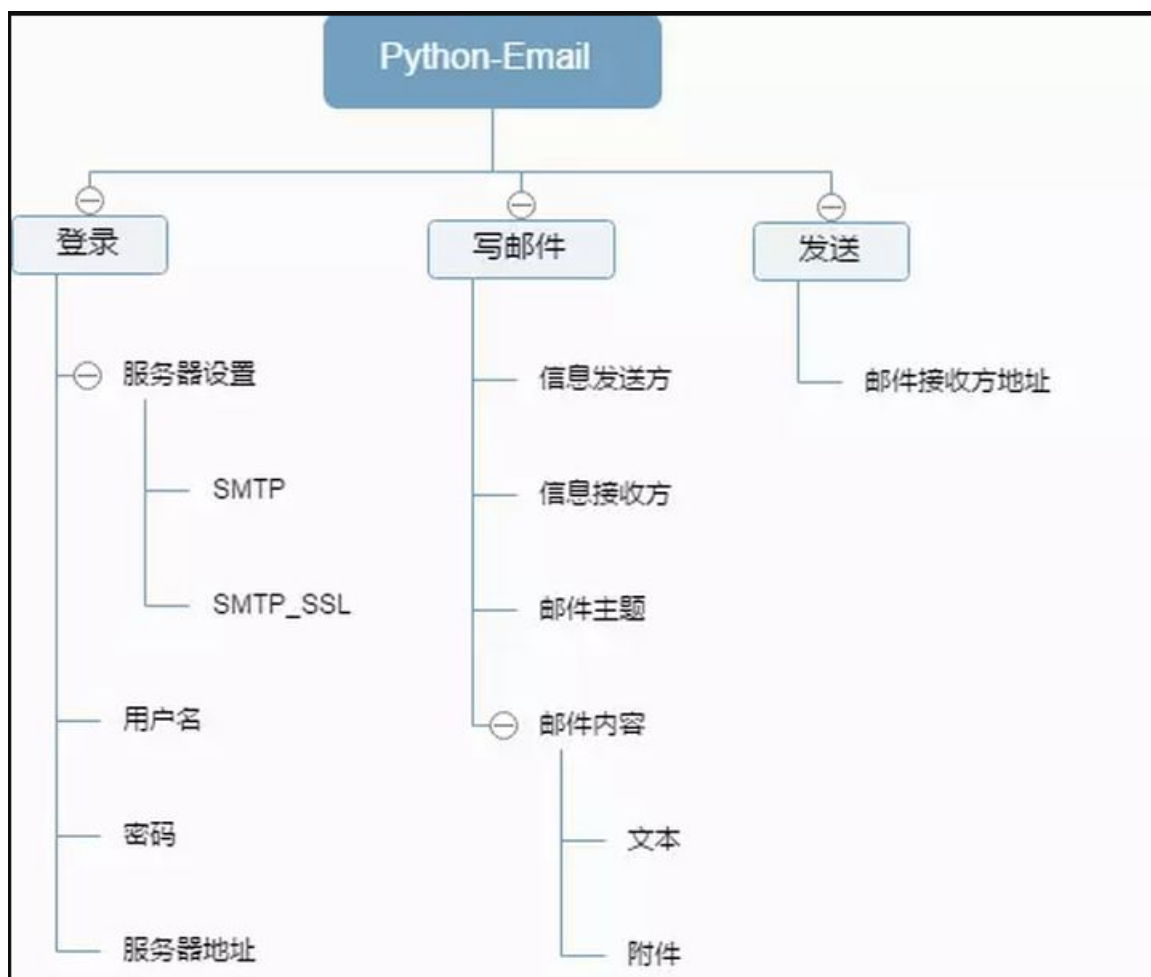
(该收取选项只对POP3生效。)

- 步骤二：获取授权码 (可以是多个，选一个就行)



2. 【发送普通邮件】

- 发送邮件流程



smtpplib模块 发送邮件的步骤：

- ①、创建一个SMTP对象
- ②、连接SMTP服务器，端口一般是465（QQ邮箱是25）
- ③、登录自己的邮箱账号

④、调用发送消息函数，参数：发件人、收件人、消息内容

⑤、关闭连接

- 代码实现

```
#需要使用smtpplib库，来进行邮箱的连接
import smtpplib
#处理邮件内容 MIMEText为文本内容
from email.mime.text import MIMEText

#步骤一：登录
#第一参数是smtp服务器，第二参数是端口
smtp = smtpplib.SMTP("smtp.qq.com", port=25)
#使用用户名和授权码登录
smtp.login("193XXXXXX", "pswxxxxxxx")

#步骤二：写邮件
content = '测开星辰教你学习自动化！'
#三个参数：第一个为文本内容，第二个 plain 设置文本格式，第三个 utf-8 设置编码
msg = MIMEText(content, 'plain', "utf-8")
#定义邮件主题
msg['Subject'] = "邮件主题"
#发件人
msg['From'] = "193XXXXXX"
#收件人
msg["To"] = "xxxxxxxxxx"

#步骤三：发送
#三个参数：发件人、收件人、as_string()是将msg(MIMEText或MIMEMultipart对象)变为str
smtp.sendmail("193XXXXXX", "pswxxxxxxx", msg.as_string())

smtp.quit()
```

9.9 【SMTP邮件发送】发送HTML、附件、图片内容

一、学习目标

1.邮件发送HTML内容

2.邮件发送附件

3.邮件发送图片

二、知识点

1.【邮件发送HTML内容】

```
#需要使用smtpplib库，来进行邮箱的连接
import smtpplib
#处理邮件内容 MIMEText为文本内容
from email.mime.text import MIMEText

#步骤一：登录
#第一参数是smtp服务器，第二参数是端口
```

```

smtp = smtplib.SMTP("smtp.qq.com", port=25)
#使用用户名和授权码登录
smtp.login("193XXXXXX", "pswxxxxxxx")

#步骤二：写邮件
content = """
<p>xx系统测试结果说明：</p>
<p><a href="http://www.baidu.com">本次测试通过，点击即可查看详细信息</a></p>
"""
#三个参数：第一个为文本内容，第二个 plain 设置文本格式，第三个 utf-8 设置编码
msg = MIMEText(content, 'plain', "utf-8")
#定义邮件主题
msg['Subject'] = "邮件主题"
#发件人
msg['From'] = "193XXXXXX"
#收件人
msg["To"] = "xxxxxxxxxx"

#步骤三：发送
#三个参数：发件人、收件人、as_string()是将msg(MIMEText或MIMEMultipart对象)变为str
smtp.sendmail("193XXXXXX", "pswxxxxxxx", msg.as_string())

smtp.quit()

```

2. 【邮件发送附件(非图片)】

```

#需要使用smtplib库，来进行邮箱的连接
import smtplib
#处理邮件内容 MIMEText为文本内容
from email.mime.text import MIMEText
#MIMEMultipart创建带附件的邮箱实例
from email.mime.multipart import MIMEMultipart

#步骤一：登录
#第一参数是smtp服务器，第二参数是端口
smtp = smtplib.SMTP("smtp.qq.com", port=25)
#使用用户名和授权码登录
smtp.login("193XXXXXX", "pswxxxxxxx")

#步骤二：写邮件
content = '测开星辰教你学习自动化！'
#创建带附件的邮件实例
msg = MIMEMultipart()
#定义邮件主题
msg['Subject'] = "邮件主题"
#发件人
msg['From'] = "193XXXXXX"
#收件人
msg["To"] = "xxxxxxxxxx"
#三个参数：第一个为文本内容，如果是文本内容则写，第三个 utf-8 设置编码
msg.attach(MIMEText(content, 'plain', "utf-8"))

#构造附件，传送当前目录下的 test.txt 文件
att = MIMEText(open('test.txt', 'rb').read(), 'base64', 'utf-8')
att["Content-Type"] = 'application/octet-stream'

```



```
# 这里的filename可以任意写，写什么名字，邮件中显示什么名字
att["Content-Disposition"] = 'attachment; filename="test.txt"'
msg.attach(att)

#步骤三：发送
#三个参数：发件人、收件人、as_string()是将msg(MIMEText或MIMEMultipart对象)变为str
smtp.sendmail("193XXXXXX", "xxxxxxxxxx", msg.as_string())

smtp.quit()
```

3.【邮件发送图片】

```
#需要使用smtplib库，来进行邮箱的连接
import smtplib
#处理邮件内容 MIMEText为文本内容
from email.mime.text import MIMEText
#MIMEMultipart创建带附件的邮箱实例
from email.mime.multipart import MIMEMultipart

#步骤一：登录
#第一参数是smtp服务器，第二参数是端口
smtp = smtplib.SMTP("smtp.qq.com", port=25)
#使用用户名和授权码登录
smtp.login("193XXXXXX", "pswxxxxxxxx")

#步骤二：写邮件
content = '测开星辰教你学习自动化！'
#创建带附件的邮件实例
msg = MIMEMultipart()
#定义邮件主题
msg['Subject'] = "邮件主题"
#发件人
msg['From'] = "193XXXXXX"
#收件人
msg["To"] = "xxxxxxxxxx"
#三个参数：第一个为文本内容，如果是文本内容则写，第三个 utf-8 设置编码
msg.attach(MIMEText(content, 'plain', "utf-8"))

#构造附件，传送当前目录下的 test.png 文件
att = MIMEText(open('test.png', 'rb').read(), 'base64', 'utf-8')
att["Content-Type"] = 'application/octet-stream'
# 这里的filename可以任意写，写什么名字，邮件中显示什么名字
att["Content-Disposition"] = 'attachment; filename="1.jpg"'
msg.attach(att)

#步骤三：发送
#三个参数：发件人、收件人、as_string()是将msg(MIMEText或MIMEMultipart对象)变为str
smtp.sendmail("193XXXXXX", "xxxxxxxxxx", msg.as_string())

smtp.quit()
```

9.10 【pymysql 模块】mysql 数据库安装

一、学习目标

1. Windows 下 mysql 数据库安装

二、知识点

1. 【Windows 下 mysql 数据库安装】

- 步骤一：首先下载 mysql，并解压。

此电脑 > 本地磁盘 (D:) > Mysql

名称	修改日期	类型	大小
mysql-5.7.30-winx64	2021/12/27 20:45	文件夹	
mysql-5.7.30-winx64.zip	2020/11/6 15:13	360压缩 ZIP 文件	376,653 KB

- 步骤二：添加环境变量

(1) 新建编辑系统变量：变量名：MYSQL_HOME 变量值：D:\mysql-5.7.31-winx64

编辑系统变量

变量名(N):

MYSQL_HOME

变量值(V):

D:\Mysql\mysql-5.7.30-winx64

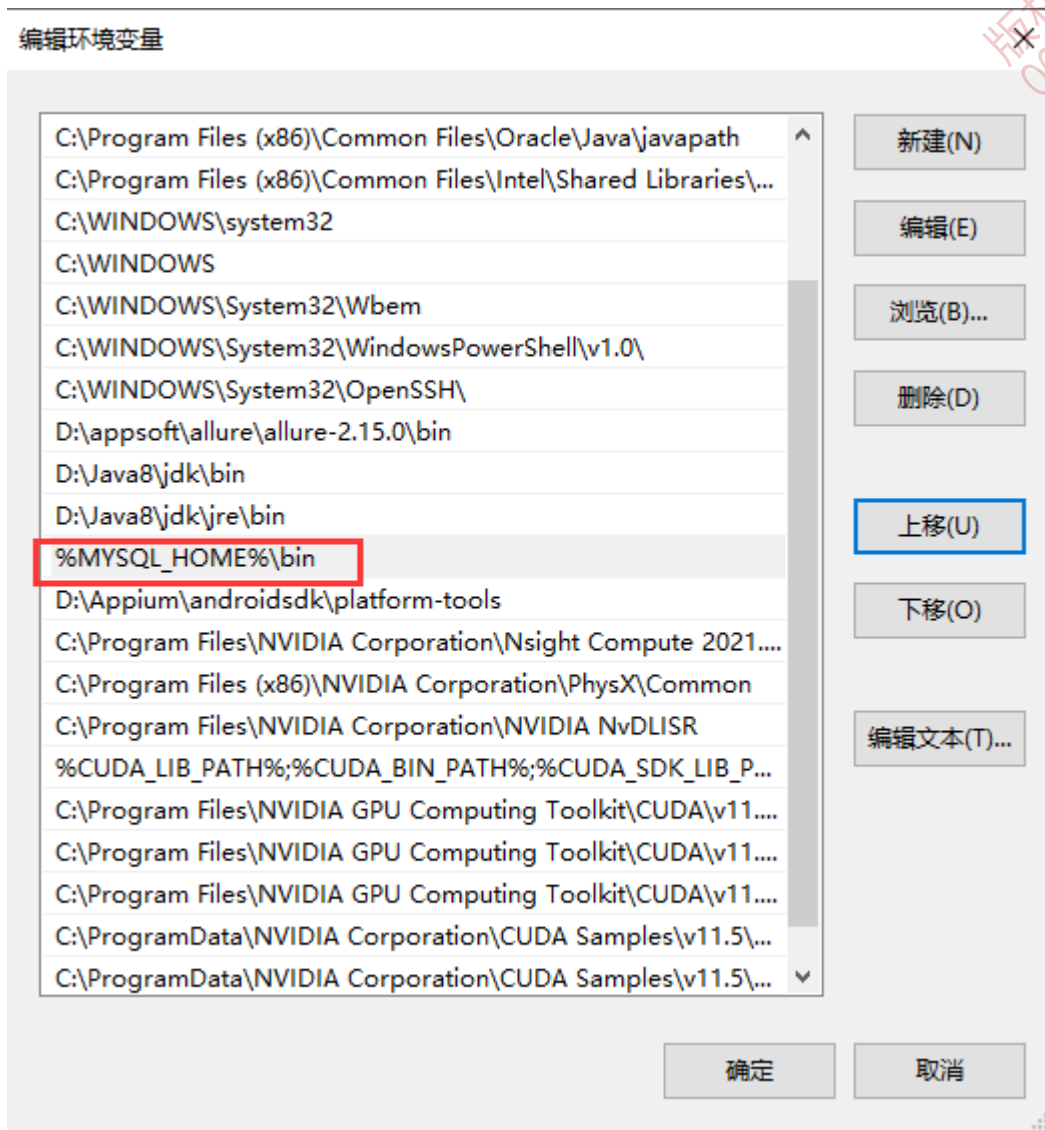
浏览目录(D)...

浏览文件(F)...

确定

取消

(2) MYSQL 的 bin 目录 %MYSQL_HOME%\bin;添加到系统path中



- 步骤三：添加配置文件

(1) 在安装目录下新建配置文件，my.ini

```
[mysql]

port=3306

basedir=D:\Mysql\mysql-5.7.30-winx64

datadir=D:\Mysql\mysql-5.7.30-winx64\data

max_connections=200

character-set-server=utf8

default-storage-engine=INNODB

sql_mode=NO_ENGINE_SUBSTITUTION,STRICT_TRANS_TABLES

[mysql]

default-character-set=utf8
```

basedir与datadir要改成自己本电脑的mysql的安装路径，我的是D:\Mysql\mysql-5.7.30-winx64。

- 步骤四：安装及配置MySQL

(1) 以**管理员**身份打开cmd，运行：

```
mysqld --install
```

(2) 初始化数据库的命令如下：

```
mysqld --initialize-insecure --user=mysql
```

(3) 启动MySQL服务的命令如下：

```
net start mysql
```

(4) 登录并修改root用户密码的命令如下：

```
mysql -uroot -p  
Enter password: (空密码，直接回车进入mysql)  
#修改密码为123  
set password for root@localhost = password('123');
```

(5) 重新进行MySQL界面并且修改root远程访问权限的命令如下：

```
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' IDENTIFIED BY '123';  
  
FLUSH PRIVILEGES;
```

(6) 验证数据库是否安装成功：

```
show databases;
```

9.11 【pymysql 模块】Mysql 数据库操作流程详解

一、学习目标

1. pymysql 模块安装
2. pymysql模块详解

二、知识点

1. 【pymysql模块安装】

- 模块安装：

```
pip install pymysql
```

2. 【pymysql模块详解】

- 步骤一：数据库连接

o 语法：

```
connection = pymysql.connect(host='127.0.0.1',
                             user='root',
                             password='123',
                             database='test',
                             port=3306,
                             charset='utf8mb4',
                             cursorclass=pymysql.cursors.DictCursor)
```

o 参数解释：

这里除了 host、user、password 等还有很多参数可以选择 详见：

参数	解释
host	数据库服务器地址
user	用户名
password	登录密码
database	操作的数据库
port	数据库端口
charset	数据库编码
cursorclass	设置默认的游标类型
bind_address	当客户端有多个网络接口时，指定连接到主机的接口。参数可以是主机名或IP地址。
unix_socket	unix 套接字地址，区别于 host 连接
read_timeout	读取数据超时时间，单位秒，默认无限制
write_timeout	写入数据超时时间，单位秒，默认无限制
connect_timeout	连接超时时间，默认 10，最小 1，最大 31536000
sql_mode	指定默认的 SQL_MODE
init_command	当连接建立完成之后执行的初始化 SQL 语句
autocommit	是否自动提交，默认不自动提交，参数值为 None 表示以服务器为准
max_allowed_packet	发送给服务器的最大数据量，默认为 16MB
defer_connect	是否惰性连接，默认为立即连接
db	参数 database 的别名,为了兼容python2.7
passwd	参数 password 的别名,为了兼容python2.7

o 返回：

返回connection连接对象，该连接对象支持以下方法：

connection对象支持的方法

cursor()	使用该连接创建并返回游标
commit()	提交当前事务
rollback()	回滚当前事务
close()	关闭连接

• 步骤二：创建游标

连接完数据库，接着就该获取游标，之后才能进行执行、提交等操作。

◦ 语法

```
cursor = connection.cursor(cursor=None)
#如cursor = connection.cursor(cursor=pymysql.cursors.DictCursor)
```

◦ 参数

cursor: Cursor--默认，元组类型；DictCursor--字典类型；SSCursor--无缓冲元组类型；SSDictCursor-- 无缓冲字典类型

◦ 返回

返回cursor游标对象，该对象支持以下方法：

```
execute(self, query, args):执行单条sql语句,接收的参数为sql语句本身和使用的参数列表,返回值为受影响的行数
executemany(self,query, args):执行单条sql语句,但是重复执行参数列表里的参数,返回值为受影响的行数
nextset(self):移动到下一个结果集
cursor用来接收返回值的方法:
fetchall(self):接收全部的返回结果行.
fetchmany(self, size=None):接收size条返回结果行.如果size的值大于返回的结果行的数量,则会返回cursor.arraysize条数据.
fetchone(self):返回一条结果行.
scroll(self, value, mode='relative'):移动指针到某一行.如果mode='relative',则表示从当前所在行移动value条,如果mode='absolute',则表示从结果集的第一行移动value条.
callproc(self, procname, args):用来执行存储过程,接收的参数为存储过程名和参数列表,返回值为受影响的行数
rowcount(): 返回数据条数或影响行数
close(): 关闭游标对象
```

• 步骤三：执行sql语句

创建完游标就可以执行sql语句啦，需要用到execute()方法。

◦ 语法：

```
cursor.execute(query, args=None)
cursor.executemany(query, args=None) #可以批量执行
```

◦ 参数：

query: 执行的sql语句，字符串。需要动态变化值用%s占位

args: 参数列表，替换%s的参数（可以不用args参数）

• 步骤四：获取返回结果

我们使用了fetchall()这个方法，将会是查询返回的全部结果。

```

cursor.fetchall() #取出全部的数据，可以返回一个结果集
#其他方法：
fetchmany(size) #取出一定数量的数据
fetchone() #取出一条数据

```

• 步骤五：事务处理

```

connection.commit() #提交修改
connection.rollback() #事务回滚
connection.begin() #事务开始（这个基本不用）

```

• 步骤六：关闭数据库

当数据库操作执行完后一定要关闭游标和连接，否则会阻塞执行。

```

cursor.close() #关闭游标
connection.close() #关闭数据库连接

```

9.12 【pymysql 模块】Mysql 数据库增删改查实现

一、学习目标

1. Mysql 数据查询
2. Mysql 数据新增
3. Mysql 数据更新
4. Mysql 数据删除

二、知识点

1. 【Mysql数据查询】

user表中先提前手动新增一条数据。

```

import pymysql

#步骤一：数据库连接

connection = pymysql.connect(
    host= '127.0.0.1',
    user= 'root',
    password= '123',
    database= 'test',
    port= 3306,
    charset= 'utf8mb4',
    cursorclass= pymysql.cursors.DictCursor
)

#步骤二：创建游标
# cursor = connection.cursor()

try:

```

```

with connection.cursor() as cursor:
    #步骤三：执行sql语句
    sql = 'select * from user where userid = %(id)s'
    count = cursor.execute(sql,args={'id':1})
    print(count)
    #步骤四：获取返回结果
    res = cursor.fetchall()[0]
    print(res)
    #步骤五：事务处理
except Exception as e:
    connection.rollback()
finally:
    # 步骤六：关闭数据库
    connection.close()

```

2. 【Mysql数据新增】

```

import pymysql

#步骤一：数据库连接

connection = pymysql.connect(
    host= '127.0.0.1',
    user= 'root',
    password= '123',
    database= 'test',
    port= 3306,
    charset= 'utf8mb4',
    cursorclass= pymysql.cursors.DictCursor
)

#步骤二：创建游标
# cursor = connection.cursor()
try:
    with connection.cursor() as cursor:
        #步骤三：执行sql语句
        sql = 'insert into user (name,userid) values (%s,%s)'
        count = cursor.execute(sql,args=('李四',2))
        print(count)
        #步骤五：事务处理
        connection.commit()
except Exception as e:
    connection.rollback()
finally:
    # 步骤六：关闭数据库
    connection.close()

```

3. 【Mysql数据更新】

```

import pymysql

#步骤一：数据库连接

connection = pymysql.connect(
    host= '127.0.0.1',

```



```

user= 'root',
password= '123',
database= 'test',
port= 3306,
charset= 'utf8mb4',
cursorclass= pymysql.cursors.DictCursor
)
#步骤二：创建游标
# cursor = connection.cursor()
try:
    with connection.cursor() as cursor:
        #步骤三：执行sql语句
        sql = 'update user set name = %s where userid = %s'
        count = cursor.execute(sql,args=('张三丰',1))
        print(count)
        #步骤五：事务处理
        connection.commit()
except Exception as e:
    connection.rollback()
finally:
    # 步骤六：关闭数据库
    connection.close()

```

4. 【Mysql数据删除】

```

import pymysql

#步骤一：数据库连接

connection = pymysql.connect(
    host= '127.0.0.1',
    user= 'root',
    password= '123',
    database= 'test',
    port= 3306,
    charset= 'utf8mb4',
    cursorclass= pymysql.cursors.DictCursor
)
#步骤二：创建游标
# cursor = connection.cursor()
try:
    with connection.cursor() as cursor:
        #步骤三：执行sql语句
        sql = 'delete from user where userid = %s'
        count = cursor.execute(sql,args=(1))
        print(count)
        #步骤五：事务处理
        connection.commit()
except Exception as e:
    connection.rollback()
finally:
    # 步骤六：关闭数据库
    connection.close()

```

9.13 【JDBC 模块】JDBC 方式数据库操作流程详解

一、学习目标

1. 安装 JayDeBeApi 模块
2. JDBC 方式操作数据库详解

二、知识点

1. 【安装JayDeBeApi模块】

- 模块安装

```
pip install JayDeBeApi
```

2. 【JDBC方式操作数据库详解】

- 步骤一：配置参数

创建连接，首先需要有以下参数，缺一不可：

```
jdbcString: Driver.class所在位置（如com.oscar.Driver）

driverPath: jar包所在位置（如C:/ShenTong/jdbc/oscarJDBC.jar）

urlString: url连接串（如jdbc:<数据库类型>://<host>:<post>/<数据库名>）

userName: 数据库用户名

password: 密码
```

我已经提前准备了各种数据库的配置信息：

```
sqlite:
    类名: org.sqlite.JDBC
    url模板: jdbc:sqlite:{dbPATH}
    添加文件: sqlite-jdbc-3.7.2.jar

mysql:
    类名: com.mysql.cj.jdbc.Driver
    url模板: jdbc:mysql://{IP}:{PORT}/{DATABASE}?serverTimezone=Asia/Shanghai&useSSL=false
    添加文件: mysql-connector-java-8.0.17.jar

oracle:
    类名: oracle.jdbc.OracleDriver
    url模板: jdbc:oracle:thin:@{IP}:{PORT}:{DATABASE}
    添加文件: ojdbc6-11.2.0.3.jar

sqlserver:
    类名: com.microsoft.sqlserver.jdbc.SQLServerDriver
    url模板: jdbc:sqlserver://{IP}:{PORT};databaseName={DATABASE}
    添加文件: sqljdbc4-4.0.jar

informix:
```

类名：com.informix.jdbc.IfxDriver

url模板： jdbc:informix-sqli://{IP}:{PORT}/{DATABASE}:informixserver=ol_informix1170

添加文件： ifxjdbc.jar

sybase:

类名：com.sybase.jdbc3.jdbc.SybDriver

url模板： jdbc:sybase:Tds://{IP}:{PORT}/{DATABASE}

添加文件： lib/jconn3-1.0.0-SNAPSHOT.jar

达梦:

类名：dm.jdbc.driver.DmDriver

url模板： jdbc:dm://{IP}:{PORT}/{DATABASE}

添加文件： Dm7JdbcDriver16.jar

人大金仓:

类名：com.kingbase.Driver

url模板： jdbc:kingbase://{IP}:{PORT}/{DATABASE}

添加文件： kingbasejdbc4.jar

神通:

类名：com.oscar.Driver

url模板： jdbc:oscar://{IP}:{PORT}/{DATABASE}

添加文件： oscarJDBC.jar

南大通用:

类名：com.gbase.jdbc.Driver

url模板： jdbc:gbase://{IP}:{PORT}/{DATABASE}

添加文件： gbase-connector-java-8.3.81.53-build55.2.1-bin.jar

瀚高:

类名：com.highgo.jdbc.Driver

url模板： jdbc:highgo://{IP}:{PORT}/{DATABASE}

添加文件： hgdb-4.0-jdbc41.jar

优炫:

类名：com.uxsino.uxdb.Driver

url模板： jdbc:uxdb://{IP}:{PORT}/{DATABASE}

添加文件： uxdbjdbc-4.2.jar

db2:

类名： com.ibm.db2.jcc.DB2Driver)

url模板： jdbc:db2://{IP}:{PORT}/{DATABASE}

添加文件： db2jcc4.jar

postgresql:

类名：org.postgresql.Driver

url模板： jdbc:postgresql://{IP}:{PORT}/{DATABASE}

添加文件： 没下载

access:

类名：net.ucanaccess.jdbc.UcanaccessDriver

url模板： jdbc:ucanaccess://D:/ideaproject/Dds/Dds/jsonconf/dbtest.mdb

添加文件： 没下载

注意：jar驱动文件的版本必须与数据库的版本一致，否则会报错。

• 步骤二：创建连接

◦ 语法：

```
connection = jaydebeapi.connect(jclassname, url, driver_args=None, jars=None, libs=None)
```

◦ 参数：

jclassname: JDBC驱动程序的Java类名。

url: JDBC驱动程序所需的数据库url。

driver_args: 要传递给的字典或参数序列，一般是用户名和密码。

jars: JDBC驱动程序的jar文件名或文件名序列。

libs: JDBC驱动程序将Dll/so文件名或Dll/sos序列（一般不填）

◦ 返回：

返回connection连接对象。

◦ 示例：

```
driver = 'com.mysql.cj.jdbc.Driver'
url = 'jdbc:mysql://127.0.0.1:3306/auto?serverTimezone=Asia/Shanghai&useSSL=false'
user = 'root'
passwd = '123'
jar_file = 'mysql-connector-java-8.0.17.jar'

connection = jaydebeapi.connect(driver, url, [user, passwd], jar_file)
```

• 步骤三：创建游标

```
cursor = connection.cursor()
```

• 步骤四：执行sql语句

```
sqlStr = 'SELECT VERSION()'
cursor.execute(sqlStr)
```

• 步骤五：获取结果

```
res = cursor.fetchall()    #查询所有
# res = cursor.fetchone() #查询一条
```

• 步骤六：事务处理

注：在这之前需要改一下数据的配置，关闭自动提交 `connection.jconn.setAutoCommit(False)`

```
connection.commit() #提交修改
connection.rollback() #事务回滚
```

• 步骤七：关闭连接

```
cursor.close()          #关闭游标
connection.close()       #关闭连接
```

9.14 【JDBC 模块】JDBC 方式数据库增删改查实现

一、学习目标

1. Mysql 数据查询
2. Mysql 数据新增
3. Mysql 数据更新
4. Mysql 数据删除

二、知识点

1. 【Mysql数据查询】

```
import jaydebeapi

'''
mysql:
    类名: com.mysql.cj.jdbc.Driver
    url模板: jdbc:mysql://{IP}:{PORT}/{DATABASE}?serverTimezone=Asia/Shanghai&useSSL=false
    添加文件: mysql-connector-java-8.0.17.jar
'''

***步骤一: 配置参数**
driver = 'com.mysql.cj.jdbc.Driver'
url = 'jdbc:mysql://127.0.0.1:3306/test?serverTimezone=Asia/Shanghai&useSSL=false'
jar_file = 'mysql-connector-java-8.0.17.jar'
user = 'root'
password = '123'

***步骤二: 创建连接**
connection = jaydebeapi.connect(driver,url,[user,password],jar_file)
connection.jconn.setAutoCommit(False)

***步骤三: 创建游标**
try:
    with connection.cursor() as cursor:
        ***步骤四: 执行sql语句**
        sql = 'select * from user where userid = 1'
        cursor.execute(sql)
        ***步骤五: 获取结果**
        res = cursor.fetchall()
        for i in res:
            print(i)
except Exception as e:
    # **步骤六: 事务处理**
    connection.rollback()
finally:
    # **步骤七: 关闭连接**
    connection.close()
```

2. 【Mysql数据新增】

```

import jaydebeapi

'''
mysql:
    类名: com.mysql.cj.jdbc.Driver
    url模板: jdbc:mysql://{IP}:{PORT}/{DATABASE}?serverTimezone=Asia/Shanghai&useSSL=false
    添加文件: mysql-connector-java-8.0.17.jar
'''

***步骤一: 配置参数**
driver = 'com.mysql.cj.jdbc.Driver'
url = 'jdbc:mysql://127.0.0.1:3306/test?serverTimezone=Asia/Shanghai&useSSL=false'
jar_file = 'mysql-connector-java-8.0.17.jar'
user = 'root'
password = '123'

***步骤二: 创建连接**
connection = jaydebeapi.connect(driver,url,[user,password],jar_file)
connection.jconn.setAutoCommit(False)

***步骤三: 创建游标**
try:
    with connection.cursor() as cursor:
        ***步骤四: 执行sql语句**
        sql = 'insert into user (name,userid) values ("王五",3)'
        cursor.execute(sql)
        connection.commit()
except Exception as e:
    # **步骤六: 事务处理**
    connection.rollback()
finally:
    # **步骤七: 关闭连接**
    connection.close()

```

3. 【Mysql数据更新】

```

import jaydebeapi

'''
mysql:
    类名: com.mysql.cj.jdbc.Driver
    url模板: jdbc:mysql://{IP}:{PORT}/{DATABASE}?serverTimezone=Asia/Shanghai&useSSL=false
    添加文件: mysql-connector-java-8.0.17.jar
'''

***步骤一: 配置参数**
driver = 'com.mysql.cj.jdbc.Driver'
url = 'jdbc:mysql://127.0.0.1:3306/test?serverTimezone=Asia/Shanghai&useSSL=false'
jar_file = 'mysql-connector-java-8.0.17.jar'
user = 'root'
password = '123'

***步骤二: 创建连接**
connection = jaydebeapi.connect(driver,url,[user,password],jar_file)
connection.jconn.setAutoCommit(False)

***步骤三: 创建游标**
try:
    with connection.cursor() as cursor:
        ***步骤四: 执行sql语句**

```

```

        sql = 'update user set name = "张三丰" where userid = 1'
        cursor.execute(sql)
        connection.commit()
except Exception as e:
    # **步骤六：事务处理**
    connection.rollback()
finally:
    # **步骤七：关闭连接**
    connection.close()

```

4. 【Mysql数据删除】

```

import jaydebeapi

'''
mysql:
    类名: com.mysql.cj.jdbc.Driver
    url模板: jdbc:mysql://{IP}:{PORT}/{DATABASE}?serverTimezone=Asia/Shanghai&useSSL=false
    添加文件: mysql-connector-java-8.0.17.jar
'''

***步骤一：配置参数**
driver = 'com.mysql.cj.jdbc.Driver'
url = 'jdbc:mysql://127.0.0.1:3306/test?serverTimezone=Asia/Shanghai&useSSL=false'
jar_file = 'mysql-connector-java-8.0.17.jar'
user = 'root'
password = '123'

***步骤二：创建连接**
connection = jaydebeapi.connect(driver,url,[user,password],jar_file)
connection.jconn.setAutoCommit(False)

***步骤三：创建游标**
try:
    with connection.cursor() as cursor:
        ***步骤四：执行sql语句**
        sql = 'delete from user where userid = 3'
        cursor.execute(sql)
        connection.commit()
except Exception as e:
    # **步骤六：事务处理**
    connection.rollback()
finally:
    # **步骤七：关闭连接**
    connection.close()

```

9.15 【系统行为模拟】Python执行cmd或shell命令

一、学习目标

1. subprocess.Popen 类介绍
2. subprocess.Popen 类执行shell实现

二、知识点

1. 【subprocess.Popen 介绍】

• 语法:

```
subprocess.Popen( args,
    bufsize=0,
    executable=None,
    stdin=None,
    stdout=None,
    stderr=None,
    preexec_fn=None,
    close_fds=False,
    shell=False,
    cwd=None,
    env=None,
    universal_newlines=False,
    startupinfo=None,
    creationflags=0)
```

• 参数:

- `args`: 要执行的shell命令，可以是字符串，也可以是命令各个参数组成的序列。当该参数的值是一个字符串时，该命令的解释过程是与平台相关的，因此通常建议将 `args` 参数作为一个序列传递。
- `bufsize`: 指定缓存策略，0表示不缓冲，1表示行缓冲，其他大于1的数字表示缓冲区大小，负数 表示使用系统默认缓冲策略。
- `stdin, stdout, stderr`: 分别表示程序标准输入、输出、错误句柄。
- `preexec_fn`: 用于指定一个将在子进程运行之前被调用的可执行对象，只在Unix平台下有效。
- `close_fds`: 如果该参数的值为True，则除了0,1和2之外的所有文件描述符都将会在子进程执行之前被关闭。
- `shell`: 该参数用于标识是否使用shell作为要执行的程序，如果shell值为True，则建议将 `args` 参数作为一个字符串传递而不要作为一个序列传递。
- `cwd`: 如果该参数值不是None，则该函数将会在执行这个子进程之前改变当前工作目录。
- `env`: 用于指定子进程的环境变量，如果 `env=None`，那么子进程的环境变量将从父进程中继承。如果 `env!=None`，它的值必须是一个映射对象。
- `universal_newlines`: 如果该参数值为True，则该文件对象的 `stdin`，`stdout` 和 `stderr` 将会作为文本流被打开，否则他们将会被作为二进制流被打开。
- `startupinfo` 和 `creationflags`: 这两个参数只在Windows下有效，它们将被传递给底层的 `CreateProcess()` 函数，用于设置子进程的一些属性，如主窗口的外观，进程优先级等。

2. 【subprocess.Popen 类执行shell实现】

```
import subprocess
import os
import io

def popen_cmd(cmd, buffering=-1):
    if not isinstance(cmd, str):
        raise TypeError("cmd类型必须是字符串!")
    if buffering == 0 or buffering is None:
        raise ValueError("buffering不支持0和None")
    proc = subprocess.Popen(cmd, bufsize=buffering, shell=True, stdout=subprocess.PIPE)
    print("CMD命令执行成功: {}".format(cmd))
    return os._wrap_close(io.TextIOWrapper(proc.stdout), proc)
```



```
if __name__ == '__main__':  
    res = popen_cmd('dir').read()  
    print(res)
```

9.16 【系统行为模拟】文件的新增、删除、复制、剪切

一、学习目标

- 1.文件新增
- 2.文件复制
- 3.文件剪切
- 4.文件删除

二、知识点

1.【文件新增】

windows的新建空白文件命令 (cmd) : `type nul > test.doc`

windows的新建内容文件命令 (cmd) : `echo 内容 > test.doc`

这样我们可以使用python调用cmd,来实现新增文件:

```
def file_add(path, filename, opt=1, content=None):  
    '''文件新增'''  
    cmd1 = path[:2]  
    if opt == 1:  
        popen_cmd('%s&cd %s&type nul > %s'%(cmd1, path, filename))  
        print("空文件新增完成, 文件: %s"%filename)  
    elif opt == 2:  
        popen_cmd('%s&cd %s&echo %s > %s'%(cmd1, path, content, filename))  
        print("有内容文件新增完成, 文件: %s"%filename)
```

2.【文件复制】

```
def file_copy(filepath, copypath):  
    '''文件复制'''  
    try:  
        copypath = shutil.copy(filepath, copypath)  
        print("文件复制成功: {}".format(copypath))  
        return copypath  
    except Exception as e:  
        print("文件复制失败! ")  
        raise e
```

3.【文件剪切】

```
def file_cut(filepath, cutpath):  
    '''文件剪切'''  
    try:  
        cutpath = shutil.move(filepath, cutpath)  
        print("文件剪切成功: {}".format(cutpath))  
        return cutpath  
    except Exception as e:  
        print("文件剪切失败!")  
        raise e
```

4. 【文件删除】

删除文件目录下的所有文件：

```
def file_all_dele(path):  
    '''删除所有文件'''  
    try:  
        for filename in os.listdir(path):  
            os.unlink(os.path.join(path, filename))  
        print("所有的文件删除成功: {}".format(path))  
    except Exception as e:  
        print("文件删除失败!")  
        raise e
```

9.17 【SSH远程连接】Python远程执行shell命令

一、学习目标

1. paramiko 模块安装
2. Ubuntu18.04 下安装配置SSH服务
3. paramiko 实现远程服务器的连接

二、知识点

1. 【paramiko 模块安装】

```
pip install paramiko
```

2. 【Ubuntu18.04 下安装配置SSH服务】

仅仅安装完 paramiko 模块还是不够的，如果要访问远程的服务器，首先得保证远程服务器有ssh并且运行远程连接，我们可以在远程机器上做如下配置：

- 步骤一：打开终端键入如下命令

```
apt-get update  
  
apt-get install openssh-server
```

执行完第二个命令可能会报错，如下：

```

lcym@lcym-virtual-machine:~$ sudo apt-get install openssh-server
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
有一些软件包无法被安装。如果您用的是 unstable 发行版，这也许是
因为系统无法达到您要求的状态造成的。该版本中可能会有一些您需要的软件
包尚未被创建或是它们已被从新到(Incoming)目录移出。
下列信息可能会对解决问题有所帮助：

下列软件包有未满足的依赖关系：
openssh-server : 依赖: openssh-client (= 1:7.6p1-4)
                  依赖: openssh-sftp-server 但是它将不会被安装
                  推荐: ssh-import-id 但是它将不会被安装
E: 无法修正错误，因为您要求某些软件包保持现状，就是它们破坏了软件包间的依赖关系。
lcym@lcym-virtual-machine:~$ /etc/init.d/ssh start

```

遇到这种错误时，我们需要先安装对应版本的openssh-client，提示信息是需要1:7.6p1-4版本：

```
sudo apt-get install openssh-client=1:7.6p1-4
```

- 步骤二：启动SSH服务

```

/etc/init.d/ssh start      #启动SSH服务

/etc/init.d/ssh restart   #重启SSH服务
/etc/init.d/ssh stop      #关闭SSH服务

```

- 步骤三：检查是否启动成功

```
ps -e | grep sshd
```

- 步骤四：允许root用户登陆

1.首先，打开“/etc/ssh/sshd_config”：

```

sudo chmod 777 /etc/ssh/sshd_config    #先放开文件编辑权限
gedit /etc/ssh/sshd_config              #编辑

```

2.查看是否有“PermitRootLogin yes”，没有添加即可，完成后保存退出

```

$OpenBSD: sshd_config,v 1.101 2017/03/14 07:19:07 djm Exp $

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/bin:/bin:/usr/sbin:/sbin

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

```

3. 【paramiko实现远程服务器的连接】

- 创建SSH对象

```
ssh = paramiko.SSHClient()
```

- 连接服务器

```

# 允许连接不在know_hosts文件中的主机
ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
# 连接服务器
ssh.connect(hostname='192.168.133.128', port=22, username='lcy', password='123')

```

- 执行命令

```
stdin, stdout, stderr = ssh.exec_command('ifconfig -a')
```

- 获取命令结果

```

result = stdout.read()
print(result.decode('utf-8'))

```

- 关闭连接

```
ssh.close()
```

9.18 【SSH远程连接】远程服务器文件上传、文件下载

一、学习目标

- 1.远程服务器文件上传
- 2.远程服务器文件下载

二、知识点

1.【远程服务器文件上传】

- 实例化一个transport对象

```
transport = paramiko.Transport(('192.168.133.128',22))
```

- 建立连接

```
transport.connect(username='lcy', password='123')
```

- 上传文件

```
sftp = paramiko.SFTPClient.from_transport(transport)
sftp.put('./test.txt', '/home/lcy/桌面/test/test.txt') #本地地址、远端地址
```

- 关闭连接

```
transport.close()
```

2.【远程服务器文件下载】

- 实例化一个transport对象

```
transport = paramiko.Transport(('192.168.133.128',22))
```

- 建立连接

```
transport.connect(username='lcy', password='123')
```

- 下载文件

```
sftp = paramiko.SFTPClient.from_transport(transport)
sftp.get('/home/lcy/桌面/test/test.txt', './test1.txt',) #远端地址、本地地址
```

- 关闭连接

```
transport.close()
```

第十章 自动化测试框架从零封装及搭建

10.1 【框架封装】为什么要封装自动化测试框架？

一、学习目标

- 1.什么是自动化测试框架
- 2.为什么要封装自动化测试框架
- 3.自动化测试框架包含哪些东西

二、知识点

1.【什么是自动化测试框架】

- 也许你可能会一点Python脚本，会一点selenium，会一点点这个编程，但是这只是框架的一部分，大家可能还很模糊。有些朋友可能说这个搭建测试框架到底是什么？
- 在软件工程中，框架（Framework）是整个或部分系统的可重用设计，表现为一组抽象构件及构件实例间交互的方法；另一种定义认为，框架是可被应用开发者定制的应用骨架，前者是从应用方面而后者是从目的方面给出的定义。
- 说通俗点就是，自动化测试框架是一个通用的，适合所有项目的，一套完整测试脚本编写流程的骨架。你不需要自己写代码来实现日志的管理、报告的输出、邮件的发送、数据驱动等一系列功能，这些框架都帮你自动做了，你所需要做的就是调用框架提供的基类函数，组装成你所需要的自动化脚本即可。

2.【为什么要封装自动化测试框架】

一套好的测试框架，可以让团队其他同事不需要有很强的代码基础，就能编写自动化测试用例,维护测试用例，执行自动化用例，利于团队协作，起到提质增效的作用。

为什么要自己搭建自动化测试框架，不直接使用现有自动化测试工具？

现有工具不支持自动化测试case的版本控制，即无法支持团队协作

对于复杂加密和签名的接口，工具不支持

涉及到很多协议的情况不支持（postman只支持http，jmeter好一点，还支持https）

不容易排错

生成的自动化测试报告不美观

接口串联场景不太支持

数据库连接校验不支持

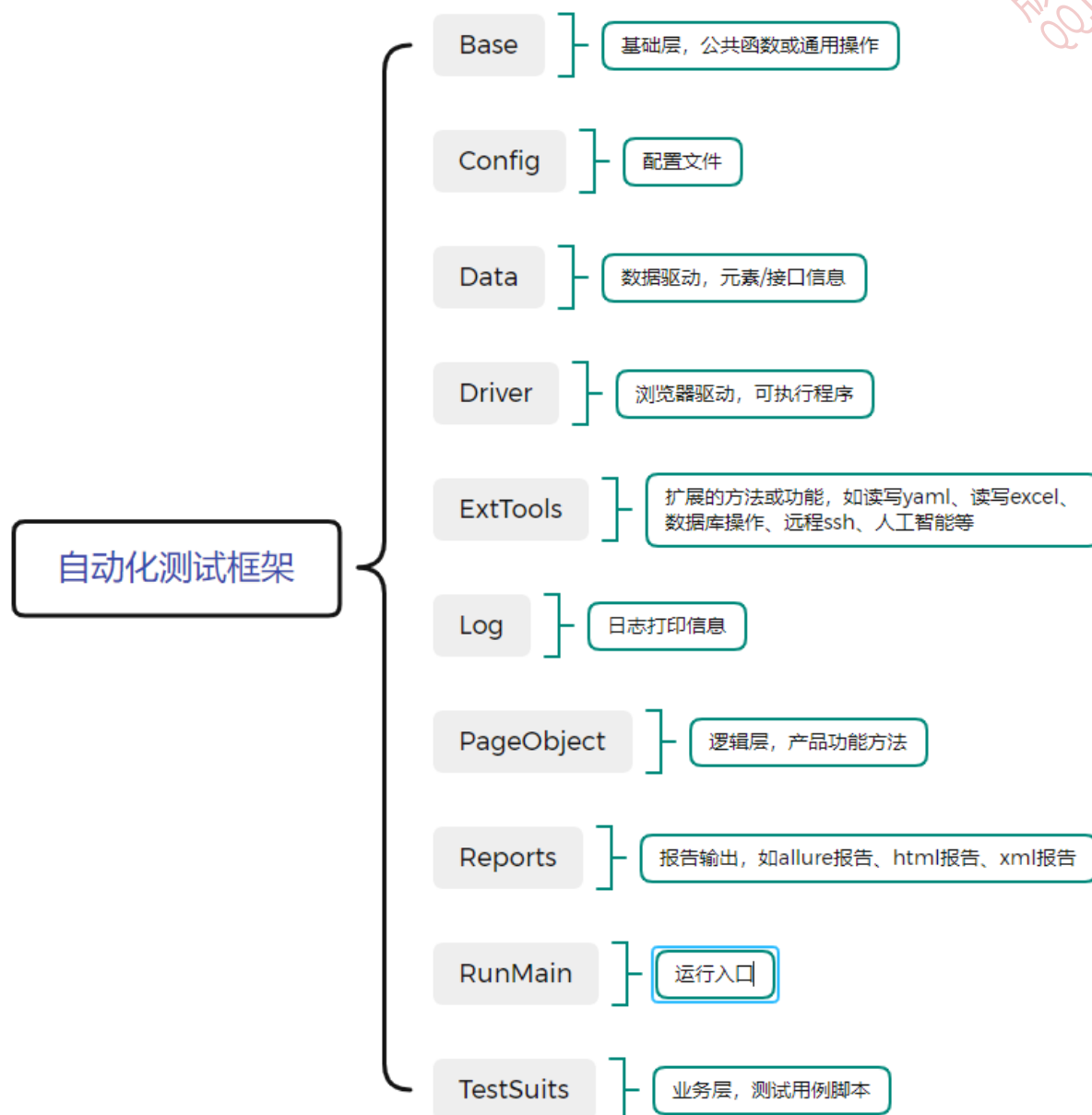
日志监控不支持

web化自动化+接口自动化的组合场景不支持

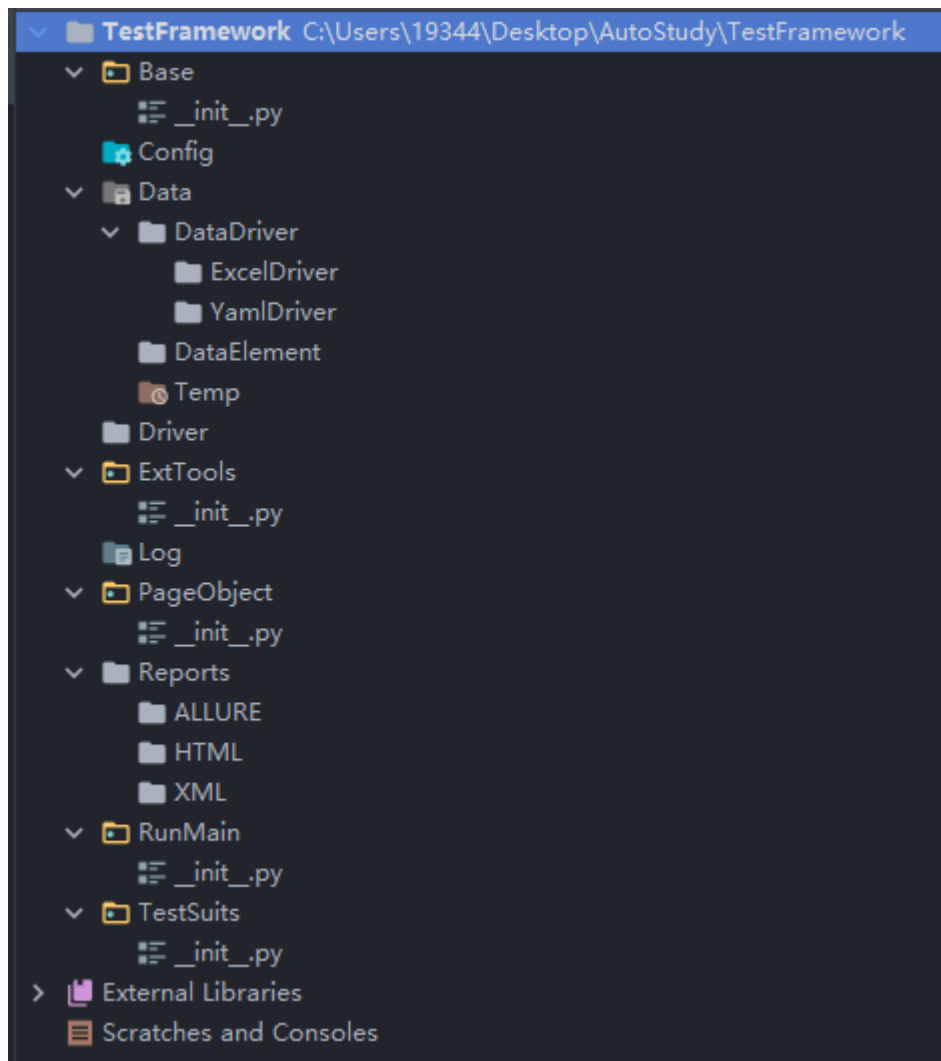
3.【自动化测试框架包含哪些东西】

- 首先，框架的整体设计思路遵循pageobject设计模式思想，所以框架要有基础层、逻辑层、业务层。基础层用来存放基础方法，如web自动化的点击，输入操作，接口自动化的发送get或post方法；逻辑层，存放功能逻辑方法，调用基础层的方法来实现如登录方法，新增方法；业务层，存放业务逻辑的方法，调用逻辑层的方法组合实现我们的测试用例。
- 其次，在po设计模式的基础上需要进行代码和测试数据的分离，目的是为了便于维护，只需要修改数据而不用动代码即可达到测试不同用例的目的。数据其实又分为两大块：一是数据驱动，也就是测试用了需要的数据；二是元素信息，也就是逻辑层需要的数据，如元素的xpath数据。
- 然后，自动化用例运行完需要自动产出结果，所以测试报告输出也是必须要有的。本节课提供了三种报告：allure报告、html报告、xml报告
- 前面的三项是自动化测试框架的基本功能，我们为了使框架的使用更加的人性化，我们还可以增加如下的功能：
配置文件全局配置、数据库操作、日志打印、邮件发送、统一的运行入口等功能。

通过上面的总结，我们可以得到这样一个目录结构：



新建自动化测试框架项目目录：



10.2 【框架封装】文件路径的统一管理封装

一、学习目标

1. 路径管理类封装

二、知识点

1. 【路径管理类封装】

```
import os
# C:\Users\Admin\Desktop\stuCode\TestFramework_po
class BasePath():

    PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    CONFIG_DIR = os.path.join(PROJECT_ROOT, 'Config')
    DATA_DIR = os.path.join(PROJECT_ROOT, "Data")
    DATA_DRIVER_DIR = os.path.join(DATA_DIR, "DataDriver")
    DATA_ELEMENT_DIR = os.path.join(DATA_DIR, "DataElement")
    DATA_TEMP_DIR = os.path.join(DATA_DIR, "Temp")
    DRIVER_DIR = os.path.join(PROJECT_ROOT, "Driver")
    LOG_DIR = os.path.join(PROJECT_ROOT, "Log")
```



```
ALLURE_DIR = os.path.join(PROJECT_ROOT, 'Reports', 'ALLURE')
HTML_DIR = os.path.join(PROJECT_ROOT, 'Reports', 'HTML')
XML_DIR = os.path.join(PROJECT_ROOT, 'Reports', 'XML')
```

10.3 【框架封装】数据配置的统一管理封装

一、学习目标

- 1.提取框架运行中各项配置
- 2.封装配置信息读取方法

二、知识点

1. 【提取框架运行中各项配置】

[客户端自动化配置]

```
#设置鼠标移动速度：0为立即执行
duration = 0.2
#每次点击间隔时间：0为立即执行
interval = 0.25
#隐式等待时间
minSearchTime = 5
#设置图片识别信任度
confidence = 0.97
#是否灰度匹配
grayscale = True
```

[WEB自动化配置]

```
#选择浏览器类型ie、firefox、chrome、chromeheadless
browser = ie
```

[日志打印配置]

```
formatter = %(asctime)s - %(name)s - %(levelname)s - %(message)s
level = INFO
stream_handler_level = DEBUG
file_handler_level = DEBUG
```

[邮件发送配置]

```
#地址
host = smtp.qq.com
#端口
port = 25
#发送方email
send_email = xxxxxx
#接收方email群发是列表形式 ["xxx",xxx,...]
receiver = ['xxxxxx']
#授权码
pwd = pirlfkxxxxxxxx
#发送方的名字，一般指测试人员
sender = 测试中心：工具人
#主题 格式字符串
subject = 自动化测试报告
```

```
[数据库连接配置]
dbtype = mysql
host = 127.0.0.1
user = root
passwd = 1111
port = 3306
database = lportal
```

```
[项目运行设置]
```

```
#自动化测试类型：HTTP、WEB、CLIENT
AUTO_TYPE = CLIENT
#测试报告类型：ALLURE、HTML、XML
REPORT_TYPE = ALLURE
#数据驱动类型：YamlDriver、ExcelDriver
DATA_DRIVER_TYPE = YamlDriver
#项目名称：空为全部项目,TestSuits下的文件名
TEST_PROJECT = project01_auto_test
#项目测试地址
TEST_URL = http://127.0.0.1
#是否发送邮件,yes,no
IS_EMAIL = no
```

2. 【封装配置信息读取方法】

新建py文件：TestFramework\Base\utils.py

```
from configparser import RawConfigParser

def read_config_ini(configPath):
    '''读取配置文件.ini'''
    config = RawConfigParser()
    config.read(configPath,encoding='utf-8')
    return config
```

10.4 【框架封装】日志管理封装

一、学习目标

1.日志参数配置

2.日志基类封装

二、知识点

1. 【日志参数配置】

```
[日志打印配置]
formatter = %(asctime)s - %(name)s - %(levelname)s - %(message)s
level = INFO
stream_handler_level = DEBUG
file_handler_level = DEBUG
```

2. 【日志基类封装】

新建py文件：TestFramework_po\Base\baseLogger.py

```
import logging
import time
import os
from Base.basePath import BasePath as BP
from Base.utils import read_config_ini

config = read_config_ini(BP.CONFIG_FILE)['日志打印配置']
rq = time.strftime('%Y%m%d_%H',time.localtime()) + '.log'

class Logger(object):

    def __init__(self,name):
        self.name = name
        self.logger = logging.getLogger(self.name)
        self.logger.setLevel(config['level'])
        self.streamHandler = logging.StreamHandler()
        self.fileHandler = logging.FileHandler(os.path.join(BP.LOG_DIR,rq),'a',encoding='utf-8')
        self.formatter = logging.Formatter(config['formatter'])
        self.fileHandler.setFormatter(self.formatter)
        self.streamHandler.setFormatter(self.formatter)
        self.streamHandler.setLevel(config['stream_handler_level'])
        self.fileHandler.setLevel(config['file_handler_level'])
        self.logger.addHandler(self.streamHandler)
        self.logger.addHandler(self.fileHandler)

    def getLogger(self):
        return self.logger
```

10.5 【框架封装】全局变量管理器封装

一、学习目标

1.全局变量管理器封装

二、知识点

1.【全局变量管理器封装】

新建py文件：TestFramework\Base\baseContainer.py

```
class GlobaManager(object):

    _globaldict = {}
    _instance = False

    def set_value(self,name,value):
        '''存变量'''
        self._globaldict[name] = value

    def get_value(self,name):
        '''取变量'''
```

```

    try:
        return self._globaldict[name]
    except KeyError as e:
        print("获取的变量不存在: {}".format(name))

def __new__(cls, *args, **kwargs):
    if cls._instance == False:
        cls._instance = super().__new__(cls, *args, **kwargs)
    return cls._instance

```

10.6【框架封装】数据文件自动读取封装

一、学习目标

1. yaml 文件读写
2. excel 文件读写
3. 元素或接口数据自动读取
4. 测试用例数据自动读取（数据驱动）

二、知识点

1.【yaml文件读写】

新建py文件：TestFramework_po\Base\baseYaml.py

```

#!/usr/bin/python
# -*- coding:utf-8 -*-

import yaml
import os

def read_yaml(yamlPath):
    '''读取yaml文件内容
    realPath: 文件的真实绝对路径'''
    if not os.path.isfile(yamlPath):
        raise FileNotFoundError("文件路径不存在，请检查路径是否正确: %s" % yamlPath)
    # open方法打开直接读出来
    with open(yamlPath, 'r', encoding='utf-8') as f:
        cfg = f.read()
    content = yaml.load(cfg, Loader=yaml.FullLoader)
    # 用load方法转字典
    return content

def write_yaml(yamlPath, data):
    '''写入yaml文件单组数据'''
    with open(yamlPath, 'w', encoding='utf-8') as f:
        yaml.dump(data=data, stream=f, allow_unicode=True)

if __name__ == '__main__':

```

```
yamlPath = r''
data = read_yaml(yamlPath)
print(data)
```

2. 【excel文件读写】

新建py文件：TestFramework_po\Base\baseExcel.py

```
import xlrd
import xlwt

class ExcelRead():

    def __init__(self, excelPath, sheetName="Sheet1"):
        self.data = xlrd.open_workbook(excelPath)
        self.table = self.data.sheet_by_name(sheetName)
        #获取第一行为key值
        self.keys = self.table.row_values(0)
        #获取总行数
        self.rowNum = self.table.nrows
        #获取总列数
        self.colNum = self.table.ncols

    def dict_date(self):
        if self.rowNum <=1:
            print("总行数小于1")
        else:
            r = []
            j = 1
            for i in range(self.rowNum-1):
                s = {}
                #从第二行取对应的values值
                values = self.table.row_values(j)
                for x in range(self.colNum):
                    s[self.keys[x]] = values[x]
                r.append(s)
                j+=1
            return r

    def get_rowinfo(self, row):
        '''获取exl中行信息,row--行数(int)'''
        if row <= 1:
            rowdata = None
        else:
            testdates = self.dict_date()
            rowdata = testdates[row-2]
        return rowdata

    def get_colinfo(self, name):
        '''获取exl中列信息'''
        name_data = []
        testdates = self.dict_date()
        for data in testdates:
            name_data.append(data[name])
```

```

        return name_data

def get_cellinfo(self, row, name):
    '''获取exl中某一单元格信息, row--行数(int); name--列名(char)'''
    if row <= 1:
        rowdata = None
    else:
        testdates = self.dict_date()
        rowdata = testdates[row-2][name]
    return rowdata

class Excelwrite():

    def __init__(self, sheetName="Sheet1"):
        self.workbook = xlwt.Workbook(encoding='utf-8')
        # 获取工作表对象worksheet
        self.worksheet = self.workbook.add_sheet(sheetName)

    def set_header(self, list_data):
        if not isinstance(list_data, list):
            raise TypeError("数据必须是列表")
        keys_data = list(list_data[0].keys())
        num = len(keys_data)
        for c in range(num):
            self.worksheet.write(0, c, label=keys_data[c])

    def excl_write(self, list_data, excel_path):
        '''表格写入全部数据'''
        if not isinstance(list_data, list):
            raise TypeError("数据必须是列表")
        # 获取每行列表
        self.set_header(list_data)
        rows_num = len(list_data)
        for r in range(rows_num):
            values_data = list(list_data[r].values())
            cows_num = len(values_data)
            for c in range(cows_num):
                self.worksheet.write(r + 1, c, values_data[c])
        # 保存数据到硬盘
        self.workbook.save(excel_path)

if __name__ == "__main__":
    path = r'C:\Users\Administrator\Desktop\zyjw_auto\data\download\涉密专用服务器信息_1590883213892.xls'
    data = ExcelRead(path, '涉密专用服务器信息')
    col = data.get_colinfo('服务器名称')
    print(col)

```

3. 【元素或接口数据自动读取】

新建py文件：TestFramework_po\Base\baseData.py

```
import os
```

```

from string import Template
import yaml
from Base.baseContainer import GlobalManager
from Base.baseLogger import Logger
from Base.basePath import BasePath as BP
from Base.utils import read_config_ini
from Base.baseYaml import read_yaml
from Base.baseExcel import ExcelRead

logger = Logger('baseData.py').getLogger()

def init_file_path(pic_path):
    '''遍历文件夹下所有yaml文件路径'''
    path = {}    #{"web元素信息-登录": "C:\\Users\\Admin\\Desktop\\stuCode\\TestFramework_po\\Data\\DataElement\\project01_auto_test\\web元素信息-登录.yaml"}
    path_lists = [path_list for path_list in os.walk(pic_path)]
    print(path_lists)
    for file_tup in path_lists:
        for file_path in file_tup:
            if isinstance(file_path, str):
                value = file_path
            elif isinstance(file_path, list):
                for file_name in file_path:
                    path[file_name.split('.')[0]] = os.path.join(value, file_name)
    return path

def is_file_exist(file_path, yamlName):
    """检查json中读取的图片名称获取全路径后是否存在"""
    abs_path = file_path.get(yamlName)
    if not abs_path:
        raise FileNotFoundError('el:{0} 不存在检查文件名或检查配置文件TEST_PROJECT! '.format(yamlName))
    return abs_path

class DataBase():
    '''逻辑层数据读取'''

    def __init__(self, yamlName=None):
        self.gm = GlobalManager()
        self.yamlName = yamlName
        self.config = read_config_ini(BP.CONFIG_FILE)
        self.run_config = self.config['项目运行设置']
        self.api_path =
        init_file_path(os.path.join(BP.DATA_ELEMENT_DIR, self.run_config['TEST_PROJECT']))
        if not self.run_config['AUTO_TYPE'] == 'CLIENT':
            self.abs_path = is_file_exist(self.api_path, self.yamlName)

    def get_element_data(self, change_data=None):
        if change_data:
            with open(self.abs_path, 'r', encoding='utf-8') as f:
                cfg = f.read()
                content = Template(cfg).safe_substitute(**change_data)
                return yaml.load(content, Loader=yaml.FullLoader)

```

```

        else:
            return read_yaml(self.abs_path)

if __name__ == '__main__':
    ele = DataBase('接口元素信息-登录')
    change_data = {
        "username": "用户",
        "password": "密码"
    }
    res = ele.get_element_data(change_data)
    print(res)

```

4. 【测试用例数据自动读取（数据驱动）】

```

class DataDriver():
    '''读取数据'''

    def __init__(self):
        self.gm = GlobalManager()
        self.config = read_config_ini(BP.CONFIG_FILE)

    def get_case_data(self, yamlName):
        '''获取数据驱动'''
        data_type = self.config['项目运行设置']['DATA_DRIVER_TYPE']
        abs_path = init_file_path(os.path.join(BP.DATA_DRIVER_DIR, data_type, self.config['项目运行设置']['TEST_PROJECT']))
        data_path = is_file_exist(abs_path, yamlName)
        if data_type == 'YamlDriver':
            return read_yaml(data_path)
        elif data_type == 'ExcelDriver':
            return ExcelRead(data_path).dict_date()

```

10.7 【框架封装】邮件发送封装

一、学习目标

1. 邮件发送基类封装

二、知识点

1. 邮件发送基类封装

TestFramework_po\Base\utils.py 先编写打包函数，将报告打包成zip压缩包：

```

import os
import zipfile

def make_zip(localPath, pname):
    '''打包zip'''
    zipf = zipfile.ZipFile(pname, 'w', zipfile.ZIP_DEFLATED)
    pre_len = len(os.path.dirname(localPath))
    for parent, dirnames, filenames in os.walk(localPath):
        for filename in filenames:

```



```

        pathfile = os.path.join(parent, filename)
        arcname = pathfile[pre_len:].strip(os.path.sep)
        zipf.write(pathfile, arcname)
    zipf.close()
    return pname

```

新建py文件：TestFramework_po\Base\baseSendEmail.py

```

from datetime import datetime
import os
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from Base.basePath import BasePath as BP
from Base.utils import read_config_ini, make_zip

class HandleEmail():

    def __init__(self):
        config = read_config_ini(BP.CONFIG_FILE)
        email_config = config['邮件发送配置']
        self.host = email_config['host']
        self.port = int(email_config['port'])
        self.sender = email_config['sender']
        self.send_email = email_config['send_email']
        self.receiver = eval(email_config['receiver'])
        self.pwd = email_config['pwd']
        self.subject = email_config['subject']

    # 添加文本
    def add_text(self, text):
        return MIMEText(text, "plain", "utf-8")

    # 添加html文本
    def add_html_text(self, html):
        return MIMEText(html, "html", "utf-8")

    # 添加附件, 图片, txt, pdf, zip
    def add_accessory(self, filepath):
        res = MIMEText(open(filepath, "rb").read(), "base64", "utf-8")
        res.add_header('Content-Disposition', 'attachment', filename=os.path.basename(filepath))
        return res

    # 添加主题 发件人, 收件人
    def add_subject_attach(self, attach_info: tuple, send_date=None):
        msg = MIMEMultipart('mixed')
        msg['Subject'] = self.subject
        # msg['From'] = '{0} <{1}>'.format(datas["send_email"][send_email][0], send_email)
        msg['From'] = '{0} <{1}>'.format(self.sender, self.send_email)
        msg['To'] = ";".join(self.receiver)
        if send_date:
            msg['Date'] = send_date
        else:
            msg['Date'] = datetime.strftime(datetime.now(), "%Y-%m-%d %H:%M:%S")

```

```

        if isinstance(attach_info, tuple):
            for i in attach_info:
                msg.attach(i)
        return msg

#发送邮件
def send_email_oper(self, msg):
    try:
        smtp = smtplib.SMTP(self.host, port=self.port)
        smtp.login(self.send_email, self.pwd)
        smtp.sendmail(self.send_email, self.receiver, msg.as_string())
        print("{0}给{1}发送邮件成功,发送时间:{2}".format(self.send_email, self.receiver,
            datetime.strftime(datetime.now(), "%Y-%m-%d
%H:%M:%S"))))
    except Exception as e:
        smtp = smtplib.SMTP_SSL(self.host, port=self.port)
        smtp.login(self.send_email, self.pwd)
        smtp.sendmail(self.send_email, self.receiver, msg.as_string())
        print("{0}给{1}发送邮件成功,发送时间:{2}".format(self.send_email, self.receiver,
            datetime.strftime(datetime.now(), "%Y-%m-%d
%H:%M:%S"))))
    finally:
        smtp.quit()

def send_public_email(self, send_date=None, text='', html='', filetype='HTML'):
    ''' 邮件发送公共方法'''
    attach_info = []
    text_plain = self.add_text(text=text)
    attach_info.append(text_plain)
    if html:
        text_html = self.add_html_text(html=html)
        attach_info.append(text_html)
    elif filetype == 'ALLURE':
        allure_zip = make_zip(BP.ALLURE_REPORT, os.path.join(BP.ALLURE_REPORT, 'allure.zip'))
        file_attach = self.add_accessory(filepath=allure_zip)
        attach_info.append(file_attach)
    elif filetype == 'HTML':
        file_attach =
self.add_accessory(filepath=os.path.join(BP.HTML_DIR, 'auto_reports.html'))
        attach_info.append(file_attach)
    elif filetype == 'XML':
        file_attach = self.add_accessory(filepath=os.path.join(BP.XML_DIR, 'auto_reports.xml'))
        attach_info.append(file_attach)
    # 添加主题和附件信息到msg
    attach_info = tuple(attach_info)
    msg = self.add_subject_attach(attach_info=attach_info, send_date=send_date)
    # 发送邮件
    self.send_email_oper(msg)

if __name__ == '__main__':
    text = '本邮件由系统自动发出, 无需回复! \n各位同事, 大家好, 以下为本次测试报告!'
    HandleEmail().send_public_email(send_date=None, text=text, html="", filetype='HTML')

```

10.8 【框架封装】客户端自动化-封装图片定位方法

一、学习目标

1. 图片定位方法封装

二、知识点

1. 【图片定位方法封装】

新建py文件：TestFramework_po\Base\baseAutoClient.py

```

import time
import pyautogui
import pyperclip
from Base.baseData import DataBase
from Base.utils import *
from Base.baseLogger import Logger
from Base.basePath import BasePath as BP

logger = Logger('baseAutoClient.py').getLogger()

class GuiBase(DataBase):

    def __init__(self):
        super().__init__()
        self.duration = float(self.config['客户端自动化配置']['duration'])
        self.interval = float(self.config['客户端自动化配置']['interval']) # 每次点击间隔时间；0为立即执行

        self.minSearchTime = float(self.config['客户端自动化配置']['minSearchTime']) # 隐试等待时间
        self.confidence = float(self.config['客户端自动化配置']['confidence']) # 设置图片识别信任度
        self.grayscale = bool(self.config['客户端自动化配置']['grayscale'])

    def _is_file_exist(self, e1):
        '''判断文件是否存在'''
        abs_path = self.api_path.get(e1)
        if not abs_path:
            raise FileNotFoundError("e1:{}不存在，请检查文件名或者配置文件")
        return abs_path

    def isexist(self, e1, searchTime=None):
        '''检查图片是否出现在屏幕'''
        picPath = self._is_file_exist(e1)
        if not searchTime:
            searchTime = self.minSearchTime
        coordinates = pyautogui.locateOnScreen(picPath,
        minSearchTime=searchTime, confidence=self.confidence, grayscale=True)
        if coordinates:
            logger.debug('查找对象{}存在'.format(e1.split('.')[0]))
            return pyautogui.center(coordinates)
        logger.debug('查找对象{}不存在'.format(e1.split('.')[0]))
        return None

if __name__ == '__main__':
    gui = GuiBase()

```

```
res = gui.isexist('qq1',searchTime=5)
print(res)
```

10.9 【框架封装】客户端自动化-封装图片点击方法

一、学习目标

1.封装图片点击方法

二、知识点

1.【封装图片点击方法】

我们可以调用上节课封装的方法，先判断图片是否存在，存在则点击，不存在则报错并截取当前屏幕的全屏截图。

```
def _error_record(self, name, type):
    '''错误处理'''
    pyautogui.screenshot(os.path.join(BP.SCREENSHOT_DIR, name))
    logger.error('类型: {}, 查找图片 {} 位置, 当前屏幕无此内容, 已截图'.format(type, name))
    raise pyautogui.ImageNotFoundException

def click_picture(self, el, clicks=1, button='left', isclick=True):
    '''图片点击'''
    pos_x_y = self.isexist(el)
    if not pos_x_y:
        self._error_record(el, 'click_picture')
    pyautogui.moveTo(*pos_x_y)
    if isclick:
        pyautogui.click(*pos_x_y, clicks=clicks, button=button, duration=self.duration,
            interval=self.interval)
    logger.debug('移动到图片 {} 位置{}, 点击:{} 成功'.format(el, isclick, pos_x_y))
```

其实点击图片，在项目中还用一个场景，如：输入文本框时，所有文本框很多且都长一个样，都是一个白框，大小也一样，这种如何定位？我这里给出了一个解决方法就是相对位置点击，即像定位文本框旁边的某个位置，然后相对偏移一段距离到文本框的位置，再进行点击或输入。

```
def rel_click_picture(self, el, rel_x=0, rel_y=0, clicks=1, button='left', isclick=True):
    '''相对位置点击'''
    pos_x_y = self.isexist(el)
    if not pos_x_y:
        self._error_record(el, 'rel_click_picture')
    pyautogui.moveTo(*pos_x_y, duration=self.duration)
    pyautogui.moveRel(rel_x, rel_y, duration=self.duration)
    if isclick:
        pyautogui.click(clicks=clicks, button=button, duration=self.duration,
            interval=self.interval)
    logger.debug('查找图片{}, 位置{}, 偏移{}, 点击{}, 成功'.format(el, pos_x_y, (rel_x, rel_y), isclick))
```

10.10 【框架封装】客户端自动化-封装鼠标操作方法

一、学习目标

- 1.封装鼠标点击方法
- 2.封装鼠标移动方法
- 3.封装鼠标拖拽方法
- 4.封装鼠标滑轮滚动

二、知识点

1. 【封装鼠标点击方法】

我们前面基础讲解中已经知道鼠标的点击分为绝对位置点击和相对位置点击，我们将这两个方法二次封装成项目中通用的方法：

```
def click(self, posx=None, posy=None, clicks=1, button='left'):
    ''' 鼠标的绝对位置点击'''
    pyautogui.click(posx, posy, clicks=clicks, button=button, duration=self.duration,
interval=self.interval)
    logger.debug('鼠标在坐标{}, {} 点击{}键 {}次'.format(posx, posy, button, clicks))

def rel_click(self, rel_x=0, rel_y=0, clicks=1, button='left'):
    ''' 鼠标相对位置点击'''
    pyautogui.move(rel_x, rel_y)
    pyautogui.click(clicks=clicks, button=button, duration=self.duration,
interval=self.interval)
    logger.debug('鼠标在相对坐标{}, {} 点击{}键 {}次'.format(rel_x, rel_y, button, clicks))
```

2. 【封装鼠标移动方法】

```
def moveto(self, posx, posy, rel=False):
    ''' 鼠标移动'''
    if rel:
        pyautogui.moveRel(posx, posy, duration=self.duration)
        logger.debug('鼠标偏移{}, {}'.format(posx, posy))
    else:
        pyautogui.moveTo(posx, posy, duration=self.duration)
        logger.debug('鼠标移动到{}, {}'.format(posx, posy))
```

3. 【封装鼠标拖拽方法】

```
def dragto(self, posx, posy, button='left', rel=False):
    ''' 鼠标的拖拽'''
    if rel:
        pyautogui.dragRel(posx, posy, button=button, duration=self.duration)
        logger.debug('鼠标相对拖拽{}, {}'.format(posx, posy))
    else:
        pyautogui.dragTo(posx, posy, button=button, duration=self.duration)
        logger.debug('鼠标拖拽{}, {}'.format(posx, posy))
```

4. 【封装鼠标滑轮滚动】

```
def scroll(self, amount_to_scroll, moveToX=None, moveToY=None):  
    ''' 鼠标的滑轮滚动 '''  
    pyautogui.scroll(clicks=amount_to_scroll, x=moveToX, y=moveToY)  
    logger.debug('鼠标在{}位置滚动{}值'.format(moveToX, moveToY), amount_to_scroll)
```

10.11 【框架封装】客户端自动化-封装键盘输入方法

一、学习目标

- 1.封装键盘输入长文本
- 2.封装键盘单个按键操作
- 3.封装键盘组合按键操作

二、知识点

1. 【封装键盘输入长文本】

```
def type(self, *args):  
    ''' 键盘输入长文本 '''  
    pyautogui.write(*args)  
    logger.debug('文本框输入{}'.format(*args))  
  
def input_string(self, text, clear=False):  
    ''' 输入中文 '''  
    pyperclip.copy(text)  
    if not clear:  
        pyautogui.hotkey('ctrl', 'v')
```

2. 【封装键盘单个按键操作】

```
def press(self, key):  
    ''' 键盘单个按键操作 '''  
    pyautogui.press(key)  
    logger.debug("按下键盘按键: {}".format(key))
```

3. 【封装键盘组合按键操作】

```
def hotkey(self, *keys):  
    ''' 键盘组合键操作 '''  
    pyautogui.hotkey(*keys)  
    logger.debug("执行快捷键{}".format(keys))
```

到目前为止，我们将Pyautogui界面自动化所有操作已经封装完了，这些封装足以满足我们的日常和工作需求！

10.12 【框架封装】web自动化-封装元素定位方法

一、学习目标

1.元素定位方法封装

二、知识点

1.【元素定位方法封装】

新建py文件：TestFramework_po\Base\baseAutoWeb.py

```

from Base.baseData import DataBase
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.support.select import Select
from selenium.webdriver.common.action_chains import ActionChains
from selenium.common.exceptions import TimeoutException
from Base.baseLogger import Logger

logger = Logger("baseAutoWeb.py").getLogger()

class WebBase(DataBase):
    '''web自动化基类'''

    def __init__(self, yamlName):
        super().__init__(yamlName)
        self.driver = self.gm.get_value('driver')
        self.t = 0.5
        self.timeout = 10

    def get_locator_data(self, locator, change_data=None):
        '''获取元素数据,login/password'''
        res = self.get_element_data(change_data)
        items = locator.split('/')
        locator_data = tuple(res[items[0]][items[1]])
        return locator_data

    def findElement(self, locator, change_data=None):
        '''元素定位方法, 返回元素对象'''
        try:
            locator = self.get_locator_data(locator, change_data)
            if not isinstance(locator, tuple):
                logger.error('locator参数类型错误, 必须传列表或元组类型: loc = ["id", "value1"]')
            logger.debug("正在定位元素信息: 定位方式->%s, value值->%s" % (locator[0], locator[1]))
            ele = WebDriverWait(self.driver, self.timeout,
self.t).until(EC.presence_of_element_located(locator))
            logger.debug("定位元素{}成功".format(locator))
            return ele
        except Exception as e:
            logger.error('未定位到元素', locator)
            raise e

    def findElements(self, locator, change_data=None):
        '''元素定位方法, 返回元素对象'''

```

```
try:
    locator = self.get_locator_data(locator, change_data)
    if not isinstance(locator, tuple):
        logger.error('locator参数类型错误，必须传列表或元组类型：loc = ["id", "value1"]')
        logger.debug("正在定位元素信息：定位方式->%s, value值->%s" % (locator[0], locator[1]))
        ele = WebDriverWait(self.driver, self.timeout,
self.t).until(EC.presence_of_all_elements_located(locator))
        logger.debug("定位元素{}成功".format(locator))
        return ele
except Exception as e:
    logger.error('未定位到元素', locator)
    raise e
```

10.13 【框架封装】web自动化-封装元素操作方法

一、学习目标

- 1.封装元素操作方法
- 2.封装获取元素信息方法
- 3.封装元素判断方法

二、知识点

1. 【封装元素操作方法】

```
def get_url(self, url):
    """打开url，最大化窗口"""
    self.driver.get(url)
    self.driver.maximize_window()
    logger.debug("浏览器访问请求地址： %s" % url)

def click(self, locator, change_data=None):
    '''按钮点击'''
    try:
        ele = self.findElement(locator, change_data)
        ele.click()
        logger.debug('按钮点击 {} 成功'.format(locator))
    except Exception as e:
        logger.error('按钮点击 {} 失败'.format(locator))
        raise e

def clear(self, locator, change_data=None):
    '''清空输入框'''
    try:
        ele = self.findElement(locator, change_data)
        ele.clear()
        logger.debug('清空输入框 {} 成功'.format(locator))
    except Exception as e:
        logger.error('清空输入框 {} 失败'.format(locator))
        raise e

def sendkeys(self, locator, text='', change_data=None):
    '''输入文本'''
```



```

try:
    ele = self.findElement(locator, change_data)
    ele.send_keys(text)
    logger.debug('输入文本 {} 成功'.format(text))
except Exception as e:
    logger.error('输入文本 {} 失败'.format(text))
    raise e

```

2. 【封装获取元素信息方法】

```

def get_title(self):
    """获取title"""
    try:
        title = self.driver.title
        logger.debug('获取title {} 成功'.format(title))
        return title
    except Exception as e:
        logger.error('获取title 失败')
        return ''

def get_text(self, locator, change_data=None):
    """获取文本"""
    try:
        content = self.findElement(locator, change_data).text
        logger.debug('获取文本 {} 成功'.format(content))
        return content
    except:
        logger.error('获取文本 失败')
        return ''

def get_attribute(self, locator, name, change_data=None):
    """获取属性"""
    try:
        element = self.findElement(locator, change_data)
        attr = element.get_attribute(name)
        logger.debug('获取属性 {} 成功'.format(attr))
        return attr
    except:
        logger.error('获取属性 失败')
        return ''

```

3. 【封装元素判断方法】

```

def isSelected(self, locator, change_data=None):
    """判断元素是否被选中，返回bool值"""
    ele = self.findElement(locator, change_data)
    r = ele.is_selected()
    return r

def is_title(self, _title=''):
    """返回bool值"""
    try:
        result = WebDriverWait(self.driver, self.timeout, self.t).until(EC.title_is(_title))
        print(result.get_attribute("innerHTML"))

```

```

        return result
    except:
        return False

    def is_title_contains(self, _title=''):
        """返回bool值"""
        try:
            result = WebDriverWait(self.driver, self.timeout,
self.t).until(EC.title_contains(_title))
            return result
        except:
            return False

    def is_text_in_element(self, locator, _text='', change_data=None):
        """返回bool值"""
        try:
            locator = self.get_locator_data(locator, change_data)
            result = WebDriverWait(self.driver, self.timeout,
self.t).until(EC.text_to_be_present_in_element(locator, _text))
            return result
        except:
            return False

    def is_value_in_element(self, locator, _value='', change_data=None):
        """返回bool值，value为空字符串，返回False"""
        try:
            locator = self.get_locator_data(locator, change_data)
            result = WebDriverWait(self.driver, self.timeout,
self.t).until(EC.text_to_be_present_in_element_value(locator, _value))
            return result
        except:
            return False

    def is_alert(self, timeout=3):
        """判断alert,存在返回alert实例，不存在，返回false"""
        try:
            result = WebDriverWait(self.driver, timeout, self.t).until(EC.alert_is_present())
            return result
        except:
            return False

```

10.14 【框架封装】web自动化-封装鼠标和键盘操作方法

一、学习目标

- 1.封装鼠标操作
- 2.封装滑轮滚动
- 3.封装键盘操作

二、知识点

1.【封装鼠标操作】

```
def mouse_move_to(self, locator, change_data=None):
    """鼠标悬停"""
    element = self.findElement(locator, change_data)
    ActionChains(self.driver).move_to_element(element).perform()
    logger.info('鼠标在{}元素悬停'.format(locator))

def mouse_drag_to(self, locator, xoffset, yoffset, change_data=None):
    """鼠标拖拽到某一坐标"""
    element = self.findElement(locator, change_data)
    ActionChains(self.driver).drag_and_drop_by_offset(element, xoffset, yoffset)
    logger.info('鼠标在{}元素拖拽{}, {}'.format(locator, xoffset, yoffset))
```

2. 【封装滑轮滚动】

```
def js_focus_element(self, locator, change_data=None):
    """聚焦元素"""
    target = self.findElement(locator, change_data)
    self.driver.execute_script("arguments[0].scrollIntoView();", target)
    logger.info('聚焦元素{}'.format(locator))

def js_scroll_end(self, x=0):
    """滚动到底部"""
    js = "window.scrollTo(%s, document.body.scrollHeight)"%x
    self.driver.execute_script(js)
    logger.info('鼠标滑轮滚动到底部')

def js_scroll_top(self):
    """滚动到顶部"""
    js = "window.scrollTo(0,0)"
    self.driver.execute_script(js)
    logger.info('鼠标滑轮滚动到顶部')
```

3. 【封装键盘操作】

```
def keyboard_send_keys_to(self, locator, text, change_data=None):
    """模拟键盘输入"""
    element = self.findElement(locator, change_data)
    ActionChains(self.driver).send_keys_to_element(element, text).perform()
    logger.info('键盘在{}位置输入: {}'.format(locator, text))
```

10.15 【框架封装】web自动化-封装弹窗方法

一、学习目标

1. 封装弹窗方法

二、知识点

1. 【封装弹窗方法】

```
def get_alert_text(self):
    '''获取弹窗文本'''
```

```
confirm = self.driver.switch_to.alert
logger.info('获取弹窗文本信息: {}'.format(confirm.text))
return confirm.text

def alert_accept(self):
    '''弹窗点击接受'''
    confirm = self.driver.switch_to.alert
    confirm.accept()
    logger.info('弹窗点击接收')

def alert_dismiss(self):
    '''弹窗点击取消'''
    confirm = self.driver.switch_to.alert
    confirm.dismiss()
    logger.info('弹窗点击取消')

def input_alert(self, text):
    '''弹窗输入'''
    prompt = self.driver.switch_to.alert
    prompt.send_keys(text)
    logger.info('弹窗输入文本值: {}'.format(text))
```

10.16 【框架封装】web自动化-封装下拉框操作方法

一、学习目标

1.封装下拉框操作

二、知识点

1.【封装下拉框操作】

```
def select_by_index(self, locator, index=0, change_data=None):
    """通过索引,index是索引第几个,从0开始,默认选第一个"""
    element = self.findElement(locator, change_data) # 定位select这一栏
    Select(element).select_by_index(index)
    logger.info('选择{}位置的下拉列表的下拉项索引: {}'.format(locator, index))

def select_by_value(self, locator, value, change_data=None):
    """通过value属性"""
    element = self.findElement(locator, change_data)
    Select(element).select_by_value(value)
    logger.info('选择{}位置的下拉列表的下拉项值: {}'.format(locator, value))

def select_by_text(self, locator, text, change_data=None):
    """通过文本值定位"""
    element = self.findElement(locator, change_data)
    Select(element).select_by_visible_text(text)
    logger.info('选择{}位置的下拉列表的下拉项文本: {}'.format(locator, text))
```

10.17【框架封装】web自动化-封装iframe、handle切换方法

一、学习目标

- 1.封装iframe切换
- 2.封装handles切换

二、知识点

1.封装iframe切换

```
def switch_iframe(self, locator, change_data=None):
    """切换iframe"""
    try:
        id_index_locator = self.get_locator_data(locator, change_data)
        if isinstance(id_index_locator, int):
            self.driver.switch_to.frame(id_index_locator)
        elif isinstance(id_index_locator, str):
            self.driver.switch_to.frame(id_index_locator)
        elif isinstance(id_index_locator, tuple) or isinstance(id_index_locator, list):
            ele = self.findElement(locator)
            self.driver.switch_to.frame(ele)
        logger.info("iframe 切换为{}".format(id_index_locator))
    except Exception as e:
        logger.error("iframe{}切换异常,{}".format(locator,e))

def switch_iframe_out(self):
    """切换iframe"""
    try:
        self.driver.switch_to.default_content()
        logger.info('iframe切换到最外层成功')
    except Exception as e:
        logger.error('iframe切换到最外层失败{}'.format(e))

def switch_iframe_up(self):
    """切换iframe"""
    try:
        self.driver.switch_to.parent_frame()
        logger.info('iframe切换到上一层成功')
    except Exception as e:
        logger.error('iframe切换到上一层失败{}'.format(e))
```

2.封装handles切换

```
def get_handles(self):
    '''获取当前所有窗口'''
    try:
        handles = self.driver.window_handles
        logger.info('获取所有的handle: {}'.format(handles))
        return handles
    except Exception as e:
        logger.error('获取所有的handle失败{}'.format(e))

def switch_handle(self, index=-1): # 第几个, -1是最后一个
```

```
'''切换窗口'''
try:
    handle_list = self.driver.window_handles
    self.driver.switch_to.window(handle_list[index])
    logger.info('切换handle成功: {}'.format(index))
except Exception as e:
    logger.error('切换handle失败{}'.format(e))
```

10.18 【框架封装】接口自动化-接口请求模拟通用封装

一、学习目标

1.接口自动化基类封装

二、知识点

1.【接口自动化基类封装】

前面我们封装的 DataBase 类可以读取 yaml 文件中的接口数据，那么我们可以把接口的请求方式，请求地址，请求数据等接口请求信息单独写入 yaml 文件中，然后封装一个通用的接口请求方法，自动通过 yaml 中的数据来发送请求。

新建py文件：TestFramework_po\Base\baseAutoHttp.py

```
#!/usr/bin/python
# -*- coding:utf-8 -*-
import requests
import urllib3
from Base.baseData import DataBase
from Base.baseLogger import Logger
from urllib3.exceptions import InsecureRequestWarning
from urllib.parse import urljoin

logger = Logger('baseAutoHttp.py').getLogger()

class ApiBase(DataBase):
    '''接口自动化基类'''

    session = requests.session()

    def __init__(self, yamlName):
        super().__init__(yamlName)
        self.timeout = 10

    def request_base(self, apiName, change_data=None, **kwargs):
        '''通用接口请求'''
        try:
            logger.info('【{}:{}]接口调用开始'.format(self.yamlName, apiName))
            yaml_data = self.get_element_data(change_data)[apiName]
            yaml_data['url'] = urljoin(self.run_config['TEST_URL'], yaml_data['url'])
            logger.info('获取【{}】文件【{}】接口请求数据: {}'.format(self.yamlName, apiName, yaml_data))
            logger.info('接口的请求方式: {}'.format(yaml_data['method']))
            logger.info('接口的请求地址: {}'.format(yaml_data['url']))
            if 'data' in yaml_data.keys():
                logger.info('接口的请求体: {}'.format(yaml_data['data']))
            elif 'json' in yaml_data.keys():
```

```

        logger.info('接口的请求体: {}'.format(yaml_data['json']))
        urllib3.disable_warnings(InsecureRequestWarning)
        res = ApiBase.session.request(**yaml_data,**kwargs)
        logger.info('接口的响应时间: {}'.format(res.elapsed.total_seconds()))
        logger.debug('POST-接口的响应码: {}'.format(res.status_code))
        logger.debug('POST-接口的响应体: {}'.format(res.text))
        logger.info('【{}:{}]接口调用结束'.format(self.yamlName, apiName))
        return res
    except Exception as e:
        logger.error('接口请求失败! : {}'.format(e))

```

10.19 【框架封装】Pytest集成-web自动化driver封装成前置

一、学习目标

- 1.自定义运行参数
- 2.将driver封装为fixture前置

二、知识点

先新建py文件：TestFramework_po\confest.py

1. 【自定义运行参数】

目的是为了更方便当我们在 cmd 中执行用例时，可以直接在命令中增加 --browser 和 --host 来直接指定用例执行的浏览器和被测主机，而不用在配置文件中配置。

```

#!/usr/bin/python
# -*- coding:utf-8 -*-
import base64
import pytest
from io import BytesIO
from py.xml import html
from Base.utils import *
from Base.basePath import BasePath as BP
from Base.baseContainer import GlobalManager
from Base.baseYaml import write_yaml

config = read_config_ini(BP.CONFIG_FILE)
gm = GlobalManager()
gm.set_value('CONFIG_INFO',config)

def pytest_addoption(parser):
    '''添加命令行参数--browser、--host'''
    parser.addoption(
        "--browser", action="store", default=config['WEB自动化配置']['browser'], help="browser
option: firefox or chrome or ie"
    )
    # 添加host参数，设置默认测试环境地址
    parser.addoption(
        "--host", action="store", default=config['项目运行设置']['test_url'], help="test host-
>http://10.11.1.171:8888"
    )

```

2. 【将driver封装为fixture前置】

```

@pytest.fixture(scope='function')
def driver(request):
    try:
        from selenium import webdriver
        name = request.config.getoption('--browser')
        _driver = None
        if name == 'ie':
            _driver =
webdriver.Ie(executable_path=os.path.join(BP.DRIVER_DIR, 'IEDriverServer.exe'))
        elif name == 'firefox':
            _driver = webdriver.Firefox(executable_path=os.path.join(BP.DRIVER_DIR,
"geckodriver.exe"))
        elif name == 'chrome':
            _driver = webdriver.Chrome(executable_path=os.path.join(BP.DRIVER_DIR,
"chromedriver.exe"))
        elif name == 'chromeheadless':
            from selenium.webdriver.chrome.options import Options
            chrome_options = Options()
            chrome_options.add_argument('--headless')
            _driver = webdriver.Chrome(executable_path=os.path.join(BP.DRIVER_DIR,
"chromedriver.exe"), chrome_options=chrome_options)
            _driver.set_window_size(width=1920, height=1080)
            GlobalManager().set_value('driver', _driver)
            _driver.implicitly_wait(5)
            print("正在启动浏览器名称: %s" % name)
        def fn():
            print("当全部用例执行完之后: teardown quit driver! ")
            _driver.quit()
        request.addfinalizer(fn)
        return _driver
    except ImportError:
        pytest.exit("未安装selenium")
    except Exception as e:
        pytest.exit("启动webdriver错误: {}".format(e))

```

10.20 【框架封装】Pytest集成-Html、Allure测试报告封装

一、学习目标

1. 自动生成测试报告封装
2. 用例失败自动截图

二、知识点

1. 【自动生成测试报告封装】

```

from py.xml import html

def pytest_html_results_summary(prefix, summary, postfix):
    '''Summary部分在此设置'''

```



```

prefix.extend([html.p("测试开发组：工具人1号")])

def pytest_html_results_table_header(cells):
    cells.insert(1, html.th('Description'))
    cells.pop()

def pytest_html_results_table_row(report, cells):
    if hasattr(report, 'description'):
        cells.insert(1, html.td(report.description))
        cells.pop()
    else:
        print("!!!!!!!", report.longreprtext)

```

2. 【用例失败自动截图】

由于我们的框架支持两种UI层面的自动化测试：web自动化和客户端自动化，接口自动化是没有UI的所以不需要截图功能，我们需要封装两种自动化测试报告的失败自动截图。

```

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item):
    """当测试失败的时候，自动截图，展示到html报告中"""
    outcome = yield
    pytest_html = item.config.pluginmanager.getplugin('html')
    report = outcome.get_result()
    extra = getattr(report, 'extra', [])
    if report.when == 'call' or report.when == "setup":
        xfail = hasattr(report, 'wasxfail')
        if config['项目运行设置']['AUTO_TYPE'] == 'WEB':
            screen_img = _capture_screenshot_sel()
        elif config['项目运行设置']['AUTO_TYPE'] == 'CLIENT':
            screen_img = _capture_screenshot_pil()
        else:
            screen_img = None
        if (report.skipped and xfail) or (report.failed and not xfail) and screen_img:
            file_name = report.nodeid.replace(":", "_") + ".png"
            if config['项目运行设置']['REPORT_TYPE'] == 'HTML':
                if file_name:
                    html = '<div></div>' % screen_img
                    script = '''
                    <script>
                        function looking(str)
                        {
                            var newwin=window.open();
                            newwin.document.write("");
                        }
                    </script>
                    '''
                    extra.append(pytest_html.extras.html(html))
                    if not insert_js_html:
                        extra.append(pytest_html.extras.html(script))
            elif config['项目运行设置']['REPORT_TYPE'] == 'ALLURE':

```

```
import allure
with allure.step('添加失败截图...'):
    allure.attach.file(BP.SCREENSHOT_PIC, "失败截图", allure.attachment_type.PNG)
report.extra = extra
report.description = str(item.function.__doc__)
```

10.21【框架封装】测试用例运行入口封装

一、学习目标

1.封装用例执行入口

二、知识点

1.【封装用例执行入口】

我们前面在全局配置文件中已经设置了需要修改的运行参数：

```
[项目运行设置]
#自动化测试类型：HTTP、WEB、CLIENT
AUTO_TYPE = HTTP
#测试报告类型：ALLURE、HTML、XML
REPORT_TYPE = ALLURE
#数据驱动类型：YamlDriver、ExcelDriver
DATA_DRIVER_TYPE = YamlDriver
#项目名称：空为全部项目,TestSuits下的文件名
TEST_PROJECT = pj02_gjxt_http
#项目测试地址
TEST_URL = http://127.0.0.1
#是否发送邮件,yes,no
IS_EMAIL = no
```

那么，我们可以通过这些参数进行运行用例封装：

新建py文件：TestFramework_po\RunMain\run.py

```
import os
import sys
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) #根目录
sys.path.append(PROJECT_ROOT)
import pytest
from Base.basePath import BasePath as BP
from Base.utils import read_config_ini, file_all_delete
from Base.baseContainer import GlobalManager
from Base.baseSendEmail import HandleEmail

config = read_config_ini(BP.CONFIG_FILE)
gm = GlobalManager()
gm.set_value('CONFIG_INFO', config)
gm.set_value('DATA_DRIVER_PATH', os.path.join(BP.DATA_DRIVER_DIR, config['项目运行设置']
['DATA_DRIVER_TYPE']))

def run_main():
```

```

'''运行入口'''
run_config = gm.get_value('CONFIG_INFO')['项目运行设置']
test_case = os.path.join(BP.TEST_SUIT_DIR, run_config['TEST_PROJECT'])
if run_config['REPORT_TYPE'] == 'ALLURE':
    pytest.main(['-v', '--alluredir={}'.format(BP.ALLURE_RESULT), test_case])
    os.system('allure generate {} -o {} --clean'.format(BP.ALLURE_RESULT, BP.ALLURE_REPORT))
    file_all_delete(BP.ALLURE_RESULT)
elif run_config['REPORT_TYPE'] == 'HTML':
    report_path = os.path.join(BP.HTML_DIR, 'auto_reports.html')
    pytest.main(['-v', '--html={}'.format(report_path), '--self-contained-html', test_case])
elif run_config['REPORT_TYPE'] == 'XML':
    report_path = os.path.join(BP.XML_DIR, 'auto_reports.xml')
    pytest.main(['-v', '--junitxml={}'.format(report_path), test_case])
else:
    print("暂不支持此报告类型:{}".format(run_config['REPORT_TYPE']))
#邮件发送
if run_config['IS_EMAIL'] == 'yes':
    el = HandleEmail()
    text = '本邮件由系统自动发出，无需回复！\n各位同事，大家好，以下附件为本次测试报告！'
    el.send_public_email(text=text, filetype=run_config['REPORT_TYPE'])
    print("邮件发送成功: {}".format(run_config['REPORT_TYPE']))

if __name__ == '__main__':
    run_main()

```

10.22 【框架封装】实现 Pyqt5 图形界面运行用例

一、学习目标

1. 图形化运行自动化测试用例

二、知识点

1. 【图形化运行自动化测试用例】

准备：开始之前别忘了要安装pyqt5的三方库：

```
pip install pyqt5 -i https://pypi.douban.com/simple
```

- 步骤一：Base/basePath.py 文件中增加两个路径

```

TESTCASES = os.path.join(DATA_TEMP_DIR, 'testcases.yaml')
TEMPCASES = os.path.join(DATA_TEMP_DIR, 'tempcases.yaml')

```

- 步骤二：收集所有的自动化用例并记录到临时 yaml 文件

在 conftest.py 中增加 pytest 的钩子函数：

```

def pytest_collection_modifyitems(session, config, items):
    '''收集用例后修改'''
    if '--co' in config.invocation_params.args:
        testcases = {}

```

```

for item in items:
    case_class_name = '::'.join(item.nodeid.split("::")[0:2])
    case_name = item.nodeid.split("::")[-1]
    if not testcases.get(case_class_name, None):
        testcases[case_class_name] = {}
    if not testcases[case_class_name].get('comment', None):
        testcases[case_class_name]['comment'] = item.cls.__doc__
    testcases[case_class_name][case_name] = item.function.__doc__
tempcases_path = BP.TEMPCASES
write_yaml(tempcases_path, testcases)

```

- 步骤三：Pyqt5使用树形组件TreeWidget展示用例

新建文件：Base/baseGuiRun.py

```

import sys
from PyQt5.QtWidgets import *
from Base.baseYaml import read_yaml, write_yaml
from PyQt5.QtCore import Qt
from Base.basePath import BasePath as BP

class MainWindow(QMainWindow):
    def __init__(self, data):
        super(QMainWindow, self).__init__()
        tbar = self.addToolBar('Driver')
        submitAct = QAction('提交', self)
        selectAll = QAction('全选', self)
        selectAllCancel = QAction('取消全选', self)
        submitAct.triggered.connect(self.go)
        selectAll.triggered.connect(self.select_all)
        selectAllCancel.triggered.connect(self.select_all_cancel)
        tbar.addAction([submitAct, selectAll, selectAllCancel])
        self.root_list = []
        w = QWidget()
        self.setCentralWidget(w)
        self.topFiller = QWidget()
        self.tree = QTreeWidget()
        # self.setCentralWidget(self.tree)
        self.tree.setColumnCount(2)
        self.tree.setHeaderLabels(['用例名称', '用例函数'])
        self.tree.setColumnWidth(0, 400)
        self.tree.clicked.connect(self.select_child)
        for i, k in enumerate(data):
            root = QTreeWidgetItem(self.tree)
            root.setText(0, data[k]['comment'])
            root.setCheckState(0, Qt.Checked)
            self.root_list.append((root, k))
            for v in data[k]:
                if v == "comment":
                    continue
                checkbox_child = QTreeWidgetItem(root)
                checkbox_child.setText(0, data[k][v])
                checkbox_child.setText(1, v)
                checkbox_child.setCheckState(0, Qt.Checked)

```

```

self.tree.itemChanged.connect(self.select_child)
##创建一个滚动条
self.scroll = QScrollArea()
self.scroll.setWidget(self.tree)
self.tree.setMinimumSize(800, 800) #####设置滚动条的尺寸
self.vbox = QVBoxLayout()
self.vbox.addWidget(self.scroll)
w.setLayout(self.vbox)
self.statusBar().showMessage("Star*")
self.resize(600, 800)
# self.tree.expandAll()

def select_child(self, *args):
    if len(args) != 2:
        return
    item, column = args
    count = item.childCount()
    if item.checkState(column) == Qt.Checked:
        for f in range(count):
            if item.child(f).checkState(0) != Qt.Checked:
                item.child(f).setCheckState(0, Qt.Checked)
    if item.checkState(column) == Qt.Unchecked:
        for f in range(count):
            if item.child(f).checkState != Qt.Unchecked:
                item.child(f).setCheckState(0, Qt.Unchecked)

def select_all(self):
    for item, value in self.root_list:
        item.setCheckState(0, Qt.Checked)

def select_all_cancel(self):
    for item, value in self.root_list:
        item.setCheckState(0, Qt.Unchecked)

def go(self):
    select = {}
    for item, value in self.root_list:
        child_count = item.childCount()
        select[value] = []
        for f in range(child_count):
            if item.child(f).checkState(0) == Qt.Checked:
                select[value].append(item.child(f).text(1))
        if not select[value]:
            del select[value]
    write_yaml(BP.TESTCASES, select)

def run():
    data = read_yaml(BP.EMPCASES)
    app = QApplication(sys.argv)
    mainwindow = MainWindow(data)
    mainwindow.setWindowTitle('自动化测试用例执行程序')
    mainwindow.show()
    sys.exit(app.exec_())

```

```
if __name__ == "__main__":
    run()
```

- 步骤四：实现图形程序的运行入口

新建文件：RunMain/runClient.py

```
import os
import sys
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) #根目录
sys.path.append(PROJECT_ROOT)
from Base.baseContainer import GlobalManager
from Base.baseSendEmail import HandleEmail
import pytest
import multiprocessing
import ctypes
from Base.baseGuiRun import run
from Base.basePath import BasePath as BP
from Base.utils import read_config_ini, file_all_delete
from contextlib import contextmanager
from Base.baseYaml import read_yaml

config = read_config_ini(BP.CONFIG_FILE)
gm = GlobalManager()
gm.set_value('CONFIG_INFO', config)
run_config = gm.get_value('CONFIG_INFO')['项目运行设置']

@contextmanager
def output_to_null():
    f = open(os.devnull, 'w')
    saved_stdout = sys.stdout
    sys.stdout = f
    yield
    sys.stdout = saved_stdout

def run_collect_testcase(v):
    with output_to_null():
        try:
            test_pj = os.path.join(BP.TEST_SUIT_DIR, run_config['TEST_PROJECT'])
            res = pytest.main(['-s', '-q', '--co', test_pj])
        except Exception as e:
            sys.exit('收集用例时发生错误,{}'.format(e))
    if res == 0:
        v.value = True
        print("收集用例成功，生成用例集testcases.yaml")

def run_main():
    try:
        datas = read_yaml(BP.TESTCASES)
        testcases = []
        if not datas:
            print("未选择任何用例")
```

```

        return
    for module in datas.keys():
        for key in datas[module]:
            if key == "comment":
                continue
            testcase = key
            testcase = os.path.join(BP.PROJECT_ROOT,module) + "::" + testcase
            testcases.append(testcase)
    if run_config['REPORT_TYPE'] == 'ALLURE':
        pytest.main(['-v', '--alluredir={}'.format(BP.ALLURE_RESULT), *testcases])
        os.system('allure generate {} -o {} --clean'.format(BP.ALLURE_RESULT,
BP.ALLURE_REPORT))
        file_all_delete(BP.ALLURE_RESULT)
    elif run_config['REPORT_TYPE'] == 'HTML':
        report_path = os.path.join(BP.HTML_DIR, 'auto_reports.html')
        pytest.main(['-v', '--html={}'.format(report_path), '--self-contained-html',
*testcases])
    elif run_config['REPORT_TYPE'] == 'XML':
        report_path = os.path.join(BP.XML_DIR, 'auto_reports.xml')
        pytest.main(['-v', '--junitxml={}'.format(report_path), *testcases])
    else:
        print("暂不支持此报告类型:{}".format(run_config['REPORT_TYPE']))
    # 邮件发送
    if run_config['IS_EMAIL'] == 'yes':
        el = HandleEmail()
        text = '本邮件由系统自动发出，无需回复！\n各位同事，大家好，以下为本次测试报告！'
        el.send_public_email(text=text, filetype=run_config['REPORT_TYPE'])
        print("邮件发送成功: {}".format(run_config['REPORT_TYPE']))
    except FileNotFoundError as e:
        print("无用例文件,执行testsuits下所有用例")
        test_case = os.path.join(BP.TEST_SUIT_DIR, run_config['TEST_PROJECT'])
        pytest.main(['--html=auto_reports.html', '--self-contained-html', test_case])

def run_app(name, *args):
    app = multiprocessing.Process(target=name, args=args)
    app.start()
    app.join()

if __name__ == "__main__":
    manager = multiprocessing.Manager()
    v = manager.Value(ctypes.c_bool, False)
    print("正在加载用例集，请稍后...")
    run_app(run_collect_testcase, v)
    if not v.value:
        sys.exit()
    run_app(run)
    run_main()

```

10.23 【框架封装】扩展方法-数据库增删改查封装

一、学习目标

1. Mysql 数据库增删改查封装
2. Sqlite 数据库增删改查封装

二、知识点

1. 【数据库增删改查封装】

新建文件：TestFramework_po\ExtTools\dbbase.py

```
#!/usr/bin/python
# -*- coding:utf-8 -*-

import pymysql
import sqlite3

class MysqlHelp(object):
    '''Mysql数据库封装类'''

    def __init__(self, host, user, passwd, port, database):
        self.host = host
        self.user = user
        self.passwd = passwd
        self.port = int(port)
        self.db = database
        self.connection = None

    def create_connection(self):
        '''创建连接'''
        self.connection = pymysql.connect(host=self.host, user=self.user, passwd=self.passwd, port=self.port, db=self.db, charset='utf8')

    def mysql_db_select(self, sql):
        '''数据库查询操作'''
        try:
            self.create_connection()
            with self.connection.cursor() as cursor: # 使用with可以在执行结束后自动关闭游标
                cursor.execute(sql)
                result_set = cursor.fetchall()
            return result_set
        except Exception as e:
            print(u'查询错误...', e)
        finally:
            self.connection.close()

    def mysql_db_operate(self, sql):
        '''增删改数据库操作'''
        try:
            self.create_connection()
            with self.connection.cursor() as cursor:
                cursor.execute(sql)
```



```

        # 提交数据库事务
        self.connection.commit()
    except Exception as e:
        # 数据库事务回滚
        self.connection.rollback()
    finally:
        self.connection.close()

```

2.Sqlite数据库增删改查封装

```

class Sqlite3Tools():
    '''Sqlite3数据库封装类'''

    def __init__(self, database=''):
        self.database = database
        self.connection = None

    def dict_factory(self, cursor, row):
        d = {}
        for idx, col in enumerate(cursor.description):
            d[col[0]] = row[idx]
        return d

    def create_connection(self):
        '''创建连接'''
        self.connection = sqlite3.connect(self.database)
        self.connection.row_factory = self.dict_factory

    def sqlite3_db_select(self, sql):
        '''数据库查询操作'''
        try:
            self.create_connection()
            cur = self.connection.cursor().execute(sql)
            result_set = cur.fetchall()
            return result_set
        except Exception as e:
            print(u'查询错误...', e)
        finally:
            self.connection.close()

    def sqlite3_db_operate(self, sql):
        '''增删改数据库操作'''
        try:
            self.create_connection()
            self.connection.cursor().execute(sql)
            self.connection.commit()
        except Exception as e:
            self.connection.rollback()
        finally:
            self.connection.close()

```

10.24 【框架封装】扩展方法-ssh远程执行shell封装

一、学习目标

1. ssh 远程执行shell封装

二、知识点

1. 【ssh远程执行shell封装】

新建文件：TestFramework_po\ExtTools\shellbase.py

```
import paramiko

class SSH():
    """ssh远程连接"""

    def __init__(self, ip, username, password, port=22):
        self.ip = ip
        self.port = port
        self.username = username
        self.password = password

    def shell_cmd(self, cmd):
        try:
            ssh = paramiko.SSHClient()
            ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
            ssh.connect(self.ip, self.port, self.username, self.password, timeout=5)
            stdin, stdout, stderr = ssh.exec_command(cmd)
            content = stdout.read().decode('utf-8')
            res = content.split('\n')
            ssh.close()
            return res
        except Exception as e:
            print('远程执行shell命令失败!!!')
            return False

    def shell_upload(self, localpath, remotepath):
        try:
            transport = paramiko.Transport((self.ip, self.port))
            transport.connect(username=self.username, password=self.password)
            sftp = paramiko.SFTPClient.from_transport(transport)
            sftp.put(localpath, remotepath)
            transport.close()
            print('文件上传成功, 上传路径: {}'.format(remotepath))
            return True
        except Exception as e:
            print('文件上传失败!!!')
            return False

    def shell_download(self, localpath, remotepath):
        try:
            transport = paramiko.Transport((self.ip, self.port))
            transport.connect(username=self.username, password=self.password)
            sftp = paramiko.SFTPClient.from_transport(transport)
```

```
sftp.get remotepath, localpath)
transport.close()
print('文件下载成功, 下载路径: {}'.format(localpath))
return True
except Exception as e:
    print('文件下载失败!!!')
    return False
```

第十一章 项目实践：客户端桌面自动化实战

11.1 【客户端自动化项目实践】项目需求介绍

一、学习目标

1. 学生管理系统功能划分
2. 学生管理系统的测试用例
3. 自动化测试框架环境部署

二、知识点

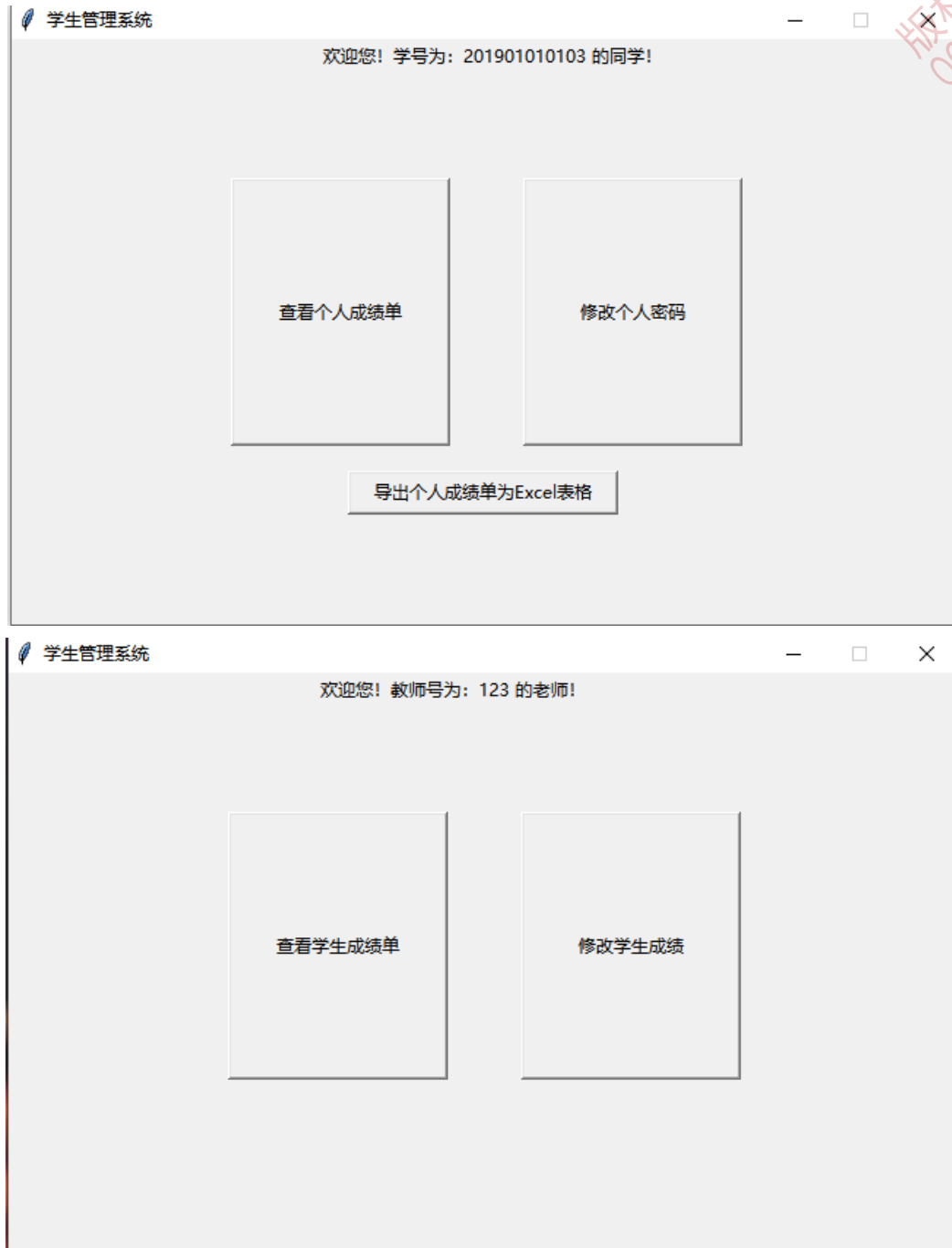
1. 【学生管理系统功能划分】

- 账号信息：

学生：
账号：201901010103 密码：123
账号：201901010104 密码：123
账号：201901010105 密码：123

老师：
账号：123 密码：123

- 功能展示：



2. 【学生管理系统的测试用例】

测试点	用例名称	级别	前置条件	测试步骤	预期结果
学生登录	学生正常登录，验证登录成功	一级	系统未登录	1.输入正确学号：201901010103 2.输入正确密码：123 3.点击学生登录	登录成功，页面展示欢迎信息
老师登录	老师正常登录，验证登录成功	一级	系统未登录	1.输入正确学号：123 2.输入正确密码：123 3.点击教师登录	登录成功，页面展示欢迎信息
学生注册	学生正常注册，验证注册成功	一级	系统未登录	1.输入正确姓名：小明 2.输入正确年龄：18 3.输入正确学号：201901010106 4.输入正确密码：123 5.点击注册	1.界面提示注册成功 2.数据库student_info表新增一条student_name为小明的数据
查看学生成绩	老师查看学生成绩，验证查询成功	一级	老师已登录	1.点击查看学生成绩按钮 2.选择数学 3.点击降序 4.点击查看学生成绩	1.展示成绩单界面 2.数学成绩降序排列
修改学生成绩	老师修改学生成绩，验证修改成功	一级	老师已登录	1.选择科目：物理 2.选择学号：201901010104 3.输入成绩：99 4.点击确认修改	1.界面展示成绩修改成功 2.数据库查询student_achievement表物理成绩修改成功
导出某科成绩单	老师导出学生成绩，验证导出成功	一级	老师已登录	1.点击查看成绩单 2.点击导出某科目成绩	1.界面展示导出成功 2.路径下自动生成文件
查看个人成绩	学生查看学生成绩，验证查询成功	一级	学生已登录	1.点击查看个人成绩单	1.展示成绩单界面 2.成绩单展示正确

测试点	用例名称	级别	预置条件	测试步骤	预期结果
修改个人密码	学生修改个人密码，验证修改成功	一级	学生已登录	1.点击修改个人密码 2.输入新密码 3.输入确认密码 4.点击确认修改	1.界面展示密码修改成功 2.数据库student_info表密码修改成功
导出个人成绩单	学生导出个人成绩，验证导出成功	一级	学生已登录	1.点击导出个人成绩单	1.界面展示导出成功 2.路径下自动生成文件

3.【自动化测试框架环境部署】

• 步骤一：安装python环境和配置环境变量

安装python环境和配置环境变量我们前面的基础课程中已经讲过了，不会的小伙伴可以回顾前面的内容。

• 步骤二：框架环境部署

◦ 方式一：在线部署（有网环境）

框架的根目录下我已经提前准备好了所需要的三方库信息并记录到了 requirements.txt 文件中，我只需要运行一行命令即可自动下载并安装：

```
pip install -i https://pypi.douban.com/simple/ -r requirements.txt
```

◦ 方式二：离线部署（无网环境）

离线安装包分为两种，一种是whl文件，一种是tar压缩包。我已经把框架的所有三方库的包准备好了，我们只需要运行不同的命令来安装这两种包：

■ whl文件

```
pip install --no-index --find-links=./ -r requirements.txt #都放在当前路径下执行
```

■ tar文件

先解压，然后运行：

```
python setup.py install
```

11.2 【客户端自动化项目实践】项目案例实战（一）

一、学习目标

- 1.启动和关闭学生管理系统
- 2.学生登录实战
- 3.教师登录实战

二、知识点

- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
学生登录	学生正常登录, 验证登录成功	一级	系统未登录	1.输入正确学号：201901010103 2.输入正确密码：123 3.点击学生登录	登录成功，页面展示欢迎信息
老师登录	老师正常登录, 验证登录成功	一级	系统未登录	1.输入正确学号：123 2.输入正确密码：123 3.点击教师登录	登录成功，页面展示欢迎信息

- 项目实战：

- 步骤一：新建四个文件夹（注：名字必须一样）
- 步骤二：修改配置文件

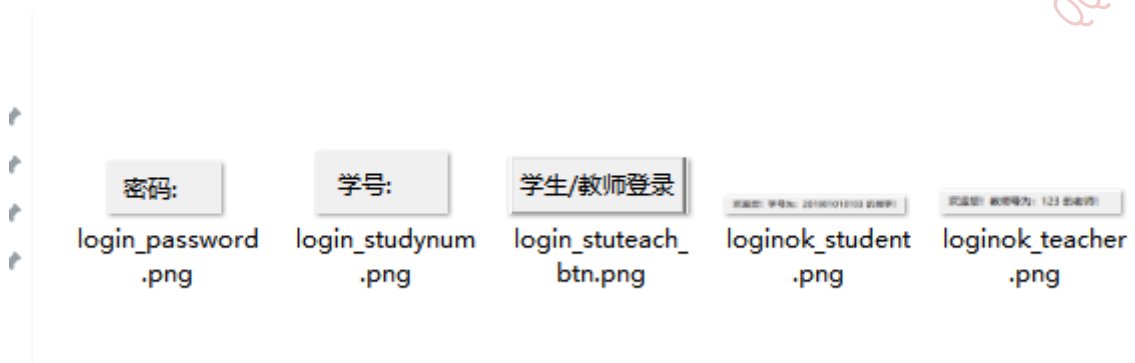
打开Config目录下的配置文件.ini，修改：

```
[项目运行设置]
#自动化测试类型：HTTP、WEB、CLIENT
AUTO_TYPE = CLIENT
#测试报告类型：ALLURE、HTML、XML
REPORT_TYPE = HTML
#数据驱动类型：YamlDriver、ExcelDriver
DATA_DRIVER_TYPE = YamlDriver
#项目名称：空为全部项目,TestSuits下的文件名
TEST_PROJECT = p01_client_xsglxt
#是否发送邮件,yes,no
IS_EMAIL = no
```

- 步骤三：截图需要操作的按钮

将要点击的按钮截图，并放到TestFramework_myproject\Data\DataElement\p01_client_xsglxt 路径下：

用户 > Admin > 桌面 > TestFramework_gjxt > Data > DataElement > pj_xsglxt_client



步骤四：封装功能的业务逻辑

TestFramework_myproject\PageObject\p01_client_xsglxt 目录下新建 client_start_stop.py 文件，封装学生管理系统的登录方法：

```
import os.path
import time

from ExtTools.sysbase import SysOperation
from Base.baseAutoClient import GuiBase
from Base.baseLogger import Logger

logger = Logger('client_start_stop.py').getLogger()

class ClientPage(GuiBase):

    def __init__(self):
        super().__init__()
        self.client_path = r'C:\Users\Admin\Desktop\课程配套资料\02项目实战\学生管理系统\客户端程序'

        self.exe_path = os.path.join(self.client_path, 'main.exe')
        self.db_path = os.path.join(self.client_path, 'student.db')
        self.sys = SysOperation()

    def start_client(self):
        """启动客户端"""
        self.sys.popen_cmd('cd {} & start {}'.format(self.client_path, self.exe_path))
        logger.info("客户端启动成功")

    def close_client(self):
        """关闭客户端"""
        self.sys.popen_cmd('cd {} & taskkill /f /t /im main.exe'.format(self.client_path))
        logger.info("客户端关闭成功")

    def client_login(self, username, password):
        """登录"""
        #1. 输入学号
        self.rel_picture_click('login_studynum', rel_x=100)
        self.type(username)
        logger.info("输入学号成功: {}".format(username))
        #2. 输入密码
```



```
self.rel_picture_click('login_password', rel_x=100)
self.type(password)
logger.info("输入密码成功: {}".format(password))
#3. 点击登录
self.click_picture('login_stuteach_btn')

def assert_login_ok(self, flag):
    """登录断言"""
    if flag == 'student':
        assert self.isexist('loginok_student')
        logger.info("【断言】学生账号登录成功!")
    elif flag == 'teacher':
        assert self.isexist('loginok_teacher')
        logger.info("【断言】老师账号登录成功!")

if __name__ == '__main__':
    client = ClientPage()
    client.start_client()
    time.sleep(3)
    client.client_login('201901010103', '123')
```

○ 步骤五：编写测试用例

将学生管理系统的启动和关闭方法作为测试用例的前置和后置，

TestFramework_myproject\TestSuits\p01_client_xsglxt 目录下新建 conftest.py 文件：

```
import time

import pytest
from PageObject.p01_client_xsglxt.client_start_stop import ClientPage

@pytest.fixture(scope='function')
def init_client():
    """客户端前置和后置"""
    cg = ClientPage()
    cg.start_client()
    yield
    cg.close_client()
    time.sleep(1)
```

新建数据驱动，TestFramework_myproject\Data\DataDriver\YamlDriver\p01_client_xsglxt\01学生管理系统登录.yaml，写入账号信息：

```
- username: '201901010103'
  password: '123'
  flag: 'student'
- username: '123'
  password: '123'
  flag: 'teacher'
```

新建测试用例文件，TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py，编写用例：

```
import pytest
from Base.baseData import DataDriver
from PageObject.p01_client_xsglxt.client_start_stop import ClientPage

class TestClientCase1():
    """客户端自动化-学生管理系统-登录功能模块"""

    @pytest.mark.parametrize('case_data',DataDriver().get_case_data('01学生管理系统登录'))
    @pytest.mark.usefixtures('init_client')
    def test_login_case01(self,case_data):
        """客户端自动化用例-用户登录测试"""
        cp = ClientPage()
        cp.client_login(case_data['username'],case_data['password'])
        cp.assert_login_ok(case_data['flag'])
```

11.3 【客户端自动化项目实践】项目案例实战（二）

一、学习目标

1.学生注册功能自动化脚本

二、知识点

1.【学生管理系统学生注册客户端自动化实战】

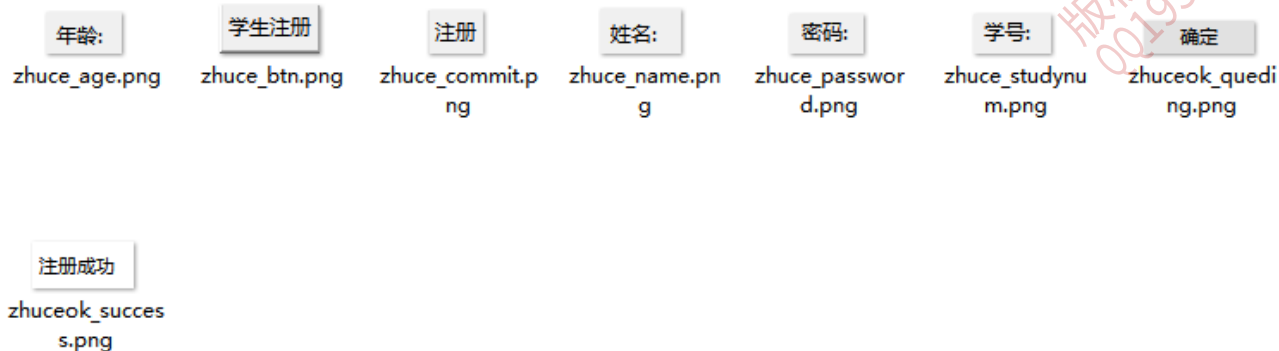
- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
学生注册	学生正常注册，验证注册成功	一级	系统未登录	1.输入正确姓名：小明 2.输入正确年龄：18 3.输入正确学号：201901010106 4.输入正确密码：123 5.点击注册	1.界面提示注册成功 2.数据库student_info表新增一条student_name为小明的数据

- 项目实践：

步骤一：截图需要操作的按钮

将要点击的按钮截图，并放到TestFramework_myproject\Data\DataElement\p01_client_xsglxt 路径下：



步骤二：封装功能的业务逻辑

```
def client_student_register(self, name, age, username, password):
    """学生注册功能"""
    self.click_picture('zhuze_btn')
    time.sleep(1)
    self.rel_picture_click('zhuze_name', rel_x=200)
    self.input_string(name)
    self.rel_picture_click('zhuze_age', rel_x=200)
    self.type(age)
    self.rel_picture_click('zhuze_studynum', rel_x=200)
    self.type(username)
    self.rel_picture_click('zhuze_password', rel_x=200)
    self.type(password)
    self.click_picture('zhuze_commit')

def assert_register_ok(self, name):
    """断言注册成功"""
    # 1. 界面提升注册成功
    assert self.isexist('zhuzeok_success'), '【断言】：学生注册断言失败！'
    logger.info('【断言】：学生注册断言成功！')
    self.click_picture('zhuzeok_queding')
    # 2. 验证数据库
    res = self.sqlite.sqlite3_db_select('select student_name from student_info;')
    all_name = []
    for data in res:
        all_name.append(data['student_name'])
    assert name in all_name, '【断言】：数据库学生注册断言失败！'
    logger.info('【断言】：数据库学生注册断言成功！')
    self.sqlite.sqlite3_db_operate('delete from student_info where student_name = "
    {}".format(name))
```

步骤三：编写测试用例

新建数据驱动，TestFramework_myproject\Data\DataDriver\YamlDriver\p01_client_xsglxt\02学生管理系统注册.yaml：

```
- name: '小明'
  age: '18'
  username: '201901010106'
  password: '123'
```

TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py, 编写用例:

```
@pytest.mark.parametrize('case_data', DataDriver().get_case_data('02学生管理系统注册'))
@pytest.mark.usefixtures('init_client')
def test_register_case02(self, case_data):
    """客户端自动化用例-学生注册测试"""
    cp = ClientPage()

    cp.client_student_register(case_data['name'], case_data['age'], case_data['username'], case_data['password'])
    cp.assert_register_ok(case_data['name'])
```

11.4 【客户端自动化项目实践】项目案例实战（三）

一、学习目标

1.查看学生成绩自动化实战

二、知识点

1.【查看学生成绩客户端自动化实战】

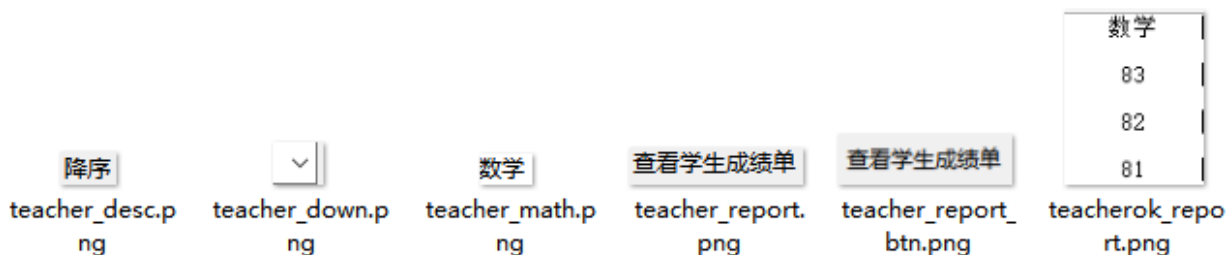
- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
查看学生成绩	老师查看学生成绩，验证查询成功	一级	老师已登录	1.点击查看学生成绩按钮 2.选择数学 3.点击降序 4.点击查看学生成绩	1.展示成绩单界面 2.数学成绩降序排列

- 项目实践：

步骤一：截图需要操作的按钮

将要点击的按钮截图，并放到TestFramework_myproject\Data\DataElement\p01_client_xsglxt路径下：



步骤二：封装功能的业务逻辑

```
import time
from Base.baseAutoClient import GuiBase
from Base.baseLogger import Logger
```

```
from PageObject.p01_client_xsglxt.client_start_stop import ClientPage

logger = Logger('client_teacher_page.py').getLogger()

class TeacherPage(GuiBase):

    def __init__(self):
        super().__init__()

    def client_teacher_report(self):
        """老师查看成绩单"""
        self.click_picture('teacher_report')
        time.sleep(1)
        self.click_picture('teacher_down')
        time.sleep(1)
        self.click_picture('teacher_math')
        self.rel_picture_click('teacher_desc', rel_x=-30)
        self.click_picture('teacher_report_btn')

    def assert_report_ok(self):
        """断言成绩单"""
        assert self.isexist('teacher_title'), '【断言】查看学生成绩单断言失败！'
        logger.info("'【断言】查看学生成绩单断言成功！'")
        assert self.isexist("teacherok_report"), '【断言】查看学生成绩降序排列断言失败！'
        logger.info("'【断言】查看学生成绩降序排列断言成功！'")

if __name__ == '__main__':
    client = ClientPage()
    client.start_client()
    time.sleep(3)
    client.client_login('123', '123')
    teacher = TeacherPage()
    teacher.client_teacher_report()
```

○ 步骤三：编写测试用例

编写前置：TestFramework_myproject\TestSuits\p01_client_xsglxt\conftest.py

```
@pytest.fixture(scope='function')
def teacher_login():
    """老师登录"""
    cp = ClientPage()
    cp.client_login('123', '123')
```

编写测试用例：TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py

```
class TestClientCase2():
    """客户端自动化-学生管理系统-老师功能模块"""

    @pytest.mark.usefixtures('init_client', 'teacher_login')
    def test_report_case01(self):
        """客户端自动化用例-查看学生成绩单"""
        tp = TeacherPage()
        tp.client_teacher_report()
        tp.assert_report_ok()
```

11.5 【客户端自动化项目实践】项目案例实战（四）

一、学习目标

1.修改学生成绩自动化实战

二、知识点

1.【修改学生成绩客户端自动化实战】

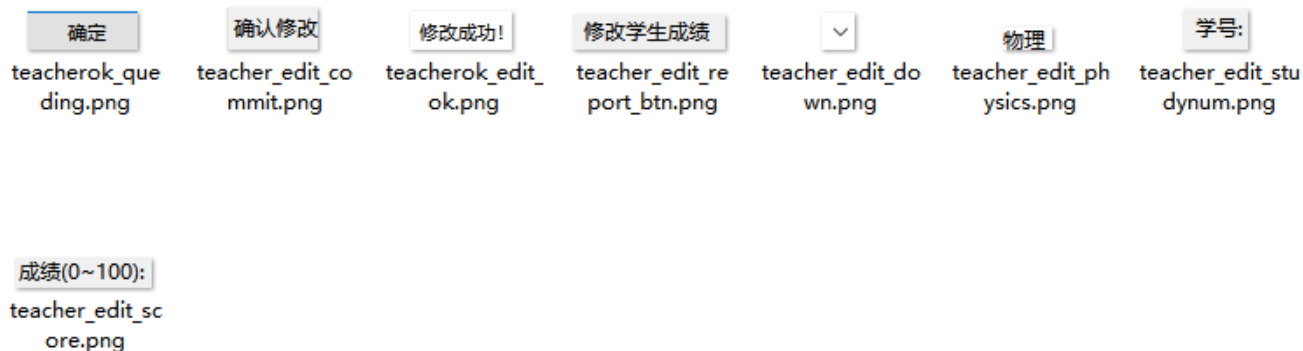
- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
修改学生成绩	老师修改学生成绩，验证修改成功	一级	老师已登录	1.选择科目：物理 2.选择学号：201901010104 3.输入成绩：99 4.点击确认修改	1.界面展示成绩修改成功 2.数据库查询student_achievement表物理成绩修改成功

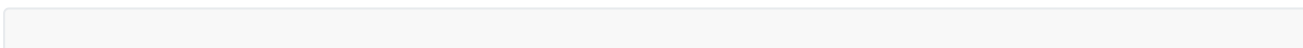
- 项目实践：

○ 步骤一：截图需要操作的按钮

将要点击的按钮截图，并放到TestFramework_myproject\Data\DataElement\p01_client_xsglxt路径下：



○ 步骤二：封装功能的业务逻辑



```

def client_edit_score(self, num, score):
    """修改学生成绩"""
    self.click_picture('teacher_edit_report_btn')
    time.sleep(1)
    self.click_picture('teacher_edit_down')
    time.sleep(1)
    self.click_picture('teacher_edit_physics')
    self.rel_picture_click('teacher_edit_studynum', rel_x=100)
    self.type(num)
    self.rel_picture_click('teacher_edit_score', rel_x=100)
    self.press('backspace') # 删除0
    self.type(score)
    self.click_picture('teacher_edit_commit')

def assert_edit_score_ok(self, num, score):
    """断言修改成绩单"""
    assert self.isexist('teacherok_edit'), '【断言】修改学生成绩单失败！'
    logger.info('【断言】修改学生成绩单成功！')
    res = self.sqlite.sqlite3_db_select('select * from student_achievement where
student_number = "{}".format(num))
    print(res)
    assert str(res[0]['物理']) == score, '【断言】数据库修改学生成绩单失败！'
    logger.info('【断言】数据库修改学生成绩单成功：{}'.format(score))

```

○ 步骤三：编写测试用例

新建数据驱动，TestFramework_myproject\Data\DataDriver\YamlDriver\p01_client_xsglxt\03老师修改成绩单.yaml：

```

- num: '201901010104'
  score: '99'

```

编写测试用例：TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py

```

@pytest.mark.parametrize('case_data', DataDriver().get_case_data('03老师修改成绩单'))
@pytest.mark.usefixtures('init_client', 'teacher_login')
def test_edit_case02(self, case_data):
    """客户端自动化用例-修改学生成绩单"""
    tp = TeacherPage()
    tp.client_edit_score(case_data['num'], case_data['score'])
    tp.assert_edit_score_ok(case_data['num'], case_data['score'])

```

11.6 【客户端自动化项目实践】项目案例实战（五）

一、学习目标

1. 导出某科成绩单自动化实战

二、知识点

1. 【导出某科成绩单客户端自动化实战】

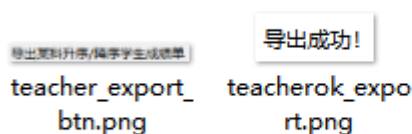
- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
导出某科成绩单	老师导出学生成绩，验证导出成功	一级	老师已登录	1.点击查看成绩单 2.点击导出某科目成绩	1.界面展示导出成功 2.路径下自动生成文件

• 项目实践：

◦ 步骤一：截图需要操作的按钮

将要点击的按钮截图，并放到 `TestFramework_myproject\Data\DataElement\p01_client_xsglxt` 路径下：



◦ 步骤二：封装功能的业务逻辑

`TestFramework_myproject\PageObject\p01_client_xsglxt\client_teacher_page.py`：

```
def client_teacher_export(self):
    """导出成绩单"""
    self.click_picture('teacher_report')
    time.sleep(1)
    self.click_picture('teacher_export_btn')

def assert_teacher_export_ok(self):
    """导出断言"""
    assert self.isexist('teacherok_export'), '【断言】导出成绩单失败！'
    logger.info("【断言】导出成绩单成功！")
    files = os.listdir(self.client_path)
    assert '语文排序成绩.xls' in files, '【断言】文件不存在！'
    logger.info('【断言】文件存在！')
    os.unlink(os.path.join(self.client_path, '语文排序成绩.xls'))
```

◦ 步骤三：编写测试用例

`TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py`：

```
@pytest.mark.usefixtures('init_client', 'teacher_login')
def test_export_case03(self):
    """客户端自动化用例-导出学生成绩单"""
    tp = TeacherPage()
    tp.client_teacher_export()
    tp.assert_teacher_export_ok()
```


11.7 【客户端自动化项目实践】项目案例实战（六）

一、学习目标

1.学生查看个人成绩自动化实战

二、知识点

1.【查看个人成绩客户端自动化实战】

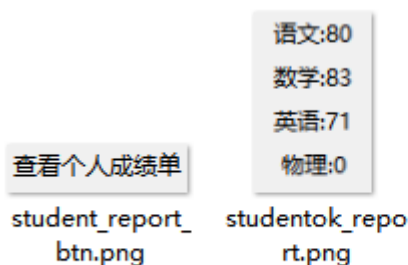
- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
查看个人成绩	学生查看学生成绩，验证查询成功	一级	学生已登录	1.点击查看个人成绩单	1.展示成绩单界面 2.成绩单展示正确

- 项目实践：

步骤一：截图需要操作的按钮

将要点击的按钮截图，并放到 `TestFramework_myproject\Data\DataElement\p01_client_xsglxt` 路径下：



步骤二：封装功能的业务逻辑

`TestFramework_myproject\PageObject\p01_client_xsglxt\client_student_page.py`：

```
import time
from Base.baseLogger import Logger
from PageObject.p01_client_xsglxt.client_start_stop import ClientPage

logger = Logger('client_student_page.py').getLogger()

class StudentPage(ClientPage):

    def __init__(self):
        super().__init__()

    def client_student_report(self):
        """学生查看个人成绩单"""
        self.click_picture('student_report_btn')
        time.sleep(1)
        logger.info('点击个人成绩单')

    def assert_student_ok(self):
        """断言查看个人成绩单"""
```

```

        assert self.isexist('studentok_report'),'【断言】查看个人成绩单失败！'
        logger.info('【断言】查看个人成绩单成功！')

if __name__ == '__main__':
    client = ClientPage()
    client.start_client()
    time.sleep(3)
    client.client_login('201901010104','123')
    sp = StudentPage()
    sp.client_student_report()

```

○ 步骤三：编写测试用例

编写前置：TestFramework_myproject\TestSuits\p01_client_xsglxt\confest.py

```

@pytest.fixture(scope='function')
def student_login():
    """学生登录"""
    sp = ClientPage()
    sp.client_login('201901010104','123')

```

编写用例：TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py

```

class TestClientCase3():
    """客户端自动化-学生管理系统-学生功能模块"""

    @pytest.mark.usefixtures('init_client','student_login')
    def test_stu_report_case01(self):
        """客户端自动化用例-查看个人成绩"""
        sp = StudentPage()
        sp.client_student_report()
        sp.assert_student_ok()

```

11.8 【客户端自动化项目实践】项目案例实战（七）

一、学习目标

1. 学生修改个人密码自动化实战

二、知识点

1. 【修改个人密码客户端自动化实战】

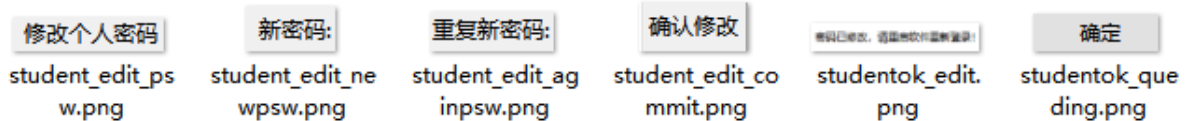
- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
修改个人密码	学生修改个人密码，验证修改成功	一级	学生已登录	1.点击修改个人密码 2.输入新密码 3.输入确认密码 4.点击确认修改	1.界面展示密码修改成功 2.数据库student_info表密码修改成功

• 项目实践：

◦ 步骤一：截图需要操作的按钮

将要点击的按钮截图，并放到 TestFramework_myproject\Data\DataElement\p01_client_xsglxt 路径下：



◦ 步骤二：封装功能的业务逻辑

TestFramework_myproject\PageObject\p01_client_xsglxt\client_student_page.py：

```
def client_student_edit(self, newpsw, againpsw):
    """学生修改密码"""
    self.click_picture('student_edit_psw')
    time.sleep(1)
    self.rel_picture_click('student_edit_newpsw', rel_x=100)
    self.type(newpsw)
    self.rel_picture_click('student_edit_aginpsw', rel_x=100)
    self.type(againpsw)
    self.click_picture('student_edit_commit')

def assert_student_edit_ok(self, newpsw):
    """断言密码修改"""
    # 1. 界面展示密码修改成功
    assert self.isexist('studentok_edit'), '【断言】修改密码失败！'
    logger.info('【断言】修改密码成功！')
    # 2. 数据库student_info表密码修改成功
    res = self.sqlite.sqlite3_db_select('select * from student_info where
student_number = "201901010104"')
    assert res[0]['student_passworld'] == newpsw
    self.sqlite.sqlite3_db_operate('UPDATE student_info SET student_passworld = "123"
where student_number = "201901010104"')
```

◦ 步骤三：编写测试用例

新建数据驱动文件：TestFramework_myproject\Data\DataDriver\YamlDriver\p01_client_xsglxt\04学生修改密码.yaml

```
- newpsw: '103'
  againpsw: '103'
```

编写用例：TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py

```
@pytest.mark.parametrize('case_data',DataDriver().get_case_data('04学生修改密码'))
@pytest.mark.usefixtures('init_client','student_login')
def test_stu_edit_case02(self,case_data):
    """客户端自动化用例-学生修改个人密码"""
    sp = StudentPage()
    sp.client_student_edit(case_data['newpsw'],case_data['againpsw'])
    sp.assert_student_edit_ok(case_data['newpsw'])
```

11.9 【客户端自动化项目实践】项目案例实战（八）

一、学习目标

1.学生导出个人成绩单自动化实战

二、知识点

1.【导出个人成绩单客户端自动化实战】

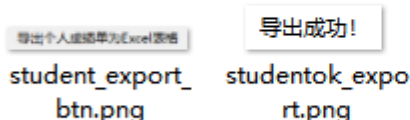
- 需求（功能测试用例）：

测试点	用例名称	级别	预置条件	测试步骤	预期结果
导出个人成绩单	学生导出个人成绩，验证导出成功	一级	学生已登录	1.点击导出个人成绩单	1.界面展示导出成功 2.路径下自动生成文件

- 项目实践：

○ 步骤一：截图需要操作的按钮

将要点击的按钮截图，并放到TestFramework_myproject\Data\DataElement\p01_client_xsglxt路径下：



○ 步骤二：封装功能的业务逻辑

TestFramework_myproject\PageObject\p01_client_xsglxt\client_student_page.py：

```
def client_student_export(self):
```

```

        """学生导出成绩单"""
        self.click_picture('student_export')
        time.sleep(1)
        logger.info("点击导出个人成绩单")

    def assert_student_export_ok(self):
        """断言导出成绩单"""
        assert self.isexist('studentok_export'), '【断言】成绩单导出失败！'
        logger.info('【断言】成绩单导出失败！')
        files = os.listdir(self.client_path)
        assert '201901010104_student_achievement.xls' in files, '【断言】文件不存在！'
        logger.info('【断言】文件不存在！')
        os.unlink(os.path.join(self.client_path, '201901010104_student_achievement.xls'))

```

○ 步骤三：编写测试用例

编写用例：TestFramework_myproject\TestSuits\p01_client_xsglxt\test_client_xsglxt.py

```

@pytest.mark.usefixtures('init_client', 'student_login')
def test_stu_export_case03(self):
    """客户端自动化用例-导出个人成绩单"""
    sp = StudentPage()
    sp.client_student_export()
    sp.assert_student_export_ok()

```

第十二章 项目实践：WEB自动化实战

12.1 【WEB自动化项目实践】项目需求介绍

一、学习目标

- 1.稿件系统一键部署
- 2.稿件系统的功能划分
- 3.稿件系统的测试用例
- 4.自动化测试框架环境部署

二、知识点

1.【稿件系统一键部署】

注：稿件系统的部署环境尽量为windows10环境，如果没有此环境建议在虚拟机里安装一个；浏览器建议用ie浏览器，非ie浏览器会出现稿件新增功能不显示的情况。

• 步骤一：关闭本机的服务

由于稿件系统已经集成了mysql, redis, tomcat等数据库和中间件，所以为了不和本机已经安装的数据库有端口冲突，需要先关闭本机的数据库和中间件的服务。

任务管理器

文件(F) 选项(O) 查看(V)

进程 性能 应用历史记录 启动 用户 详细信息 服务

名称	PID	描述	状态	组
UserDataSvc_571e6		UserDataSvc_571e6	已停止	UnistackSvcG...
UnistoreSvc_571e6		UnistoreSvc_571e6	已停止	UnistackSvcG...
UdkUserSvc_571e6		UdkUserSvc_571e6	已停止	UdkSvcGroup
PrintWorkflowUserSvc_5...		PrintWorkflowUserSvc_571e6	已停止	PrintWorkflow
PimIndexMaintenanceSv...		PimIndexMaintenanceSvc_571e6	已停止	UnistackSvcG...
MessagingService_571e6		MessagingService_571e6	已停止	UnistackSvcG...
DevicesFlowUserSvc_571...		DevicesFlowUserSvc_571e6	已停止	DevicesFlow
DevicePickerUserSvc_571...		DevicePickerUserSvc_571e6	已停止	DevicesFlow
DeviceAssociationBroker...		DeviceAssociationBrokerSvc_571...	已停止	DevicesFlow
CredentialEnrollmentMa...		CredentialEnrollmentManagerUs...	已停止	
ConsentUxUserSvc_571e6		ConsentUxUserSvc_571e6	已停止	DevicesFlow
CaptureService_571e6		CaptureService_571e6	已停止	LocalService
BluetoothUserService_57...		BluetoothUserService_571e6	已停止	RthAppGroup

• 步骤二：运行start.bat程序，一键启动项目

liferay

文件 主页 共享 查看

固定到快速访问 复制 粘贴 复制路径 粘贴快捷方式 移动到 复制到 删除 重命名 新建项目 轻松访问 新建文件夹 新建 打开 属性 编辑 历史记录 全部选择 全部取消 反向选择 选择

教学资料 > 稿件管理项目 > liferay >

名称	修改日期	类型	大小
bin	2021/6/26 20:56	文件夹	
conf	2021/12/5 16:48	文件夹	
jdk1.6.0_07	2021/6/26 20:56	文件夹	
lib	2021/6/26 20:57	文件夹	
logs	2021/12/5 16:48	文件夹	
mysql5	2021/6/26 20:57	文件夹	
resource	2021/6/26 20:57	文件夹	
temp	2021/9/25 14:40	文件夹	
webapps	2021/6/26 20:58	文件夹	
work	2021/6/26 20:58	文件夹	
LICENSE	2008/1/28 23:39	文件	38 KB
liferay-portal-tomcat-6.0-5.1.1.txt	2008/8/26 11:11	文本文档	1 KB
lportal.lck	2008/8/26 17:26	LCK 文件	1 KB
lportal.log	2008/8/26 17:25	文本文档	6 KB
lportal.properties	2008/8/26 16:58	PROPERTIES 文件	1 KB
lportal.script	2008/8/26 16:58	SCRIPT 文件	97 KB
NOTICE	2008/1/28 23:39	文件	1 KB
RELEASE-NOTES	2008/1/28 23:39	文件	8 KB
RUNNING.txt	2008/1/28 23:39	文本文档	7 KB
startall.bat	2008/9/22 16:30	Windows 批处理...	1 KB
stopall.bat	2008/9/24 17:18	Windows 批处理...	1 KB

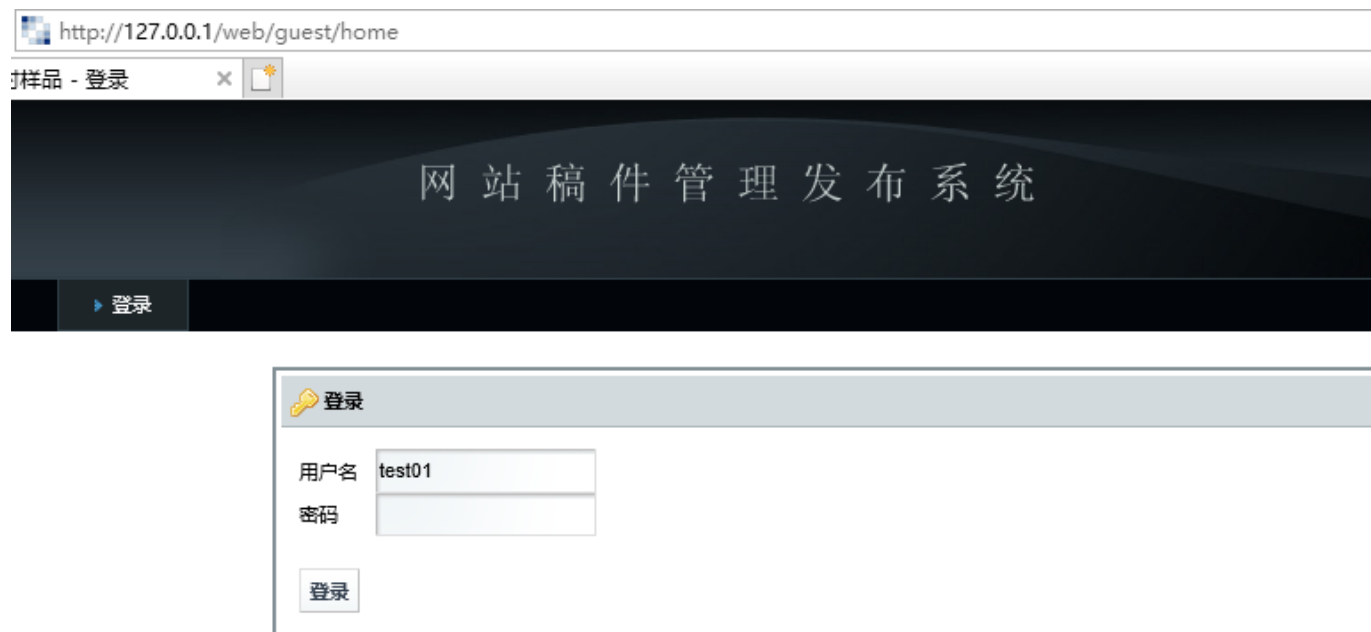
```

Tomcat
Loading jar:file:/C:/Users/Admin/Desktop/鐳鐳 璧栫拏/緋夸欢纒$愬稜圭汨/liferay/webapps/ROOT/WEB-INF/lib/portal-impl.jar!portal.properties
Loading file:/C:/Users/Admin/Desktop/鐳鐳 璧栫拏/緋夸欢纒$愬稜圭汨/liferay/webapps/ROOT/WEB-INF/classes/portal-ext.properties
08:00:58,015 INFO [DynamicDialect:81] Determining dialect for MySQL 5
08:00:58,030 INFO [DynamicDialect:109] Using dialect org.hibernate.dialect.MySQLDialect
08:00:58,702 INFO [PortalImpl:228] Portal lib url /C:/Users/Admin/Desktop/鐳鐳 璧栫拏/緋夸欢纒$愬稜圭汨/liferay/webapps/ROOT/WEB-INF/lib/portal-impl.jar!/com/liferay/portal/util/PortalImpl.class
08:00:58,702 INFO [PortalImpl:263] Portal lib directory /C:/Users/Admin/Desktop/鐳鐳 璧栫拏/緋夸欢纒$愬稜圭汨/liferay/webapps/ROOT/WEB-INF/lib/
Starting Liferay Portal 5.1.1 (Calvin / Build 5101 / August 11, 2008)
(閿嶅 浠嶅 鐳鐳 璧栫拏)org.apache.xml.utils.URI$MalformedURLException: 璽 緞鎶 吧 憫 爇 砑 瀛 惓 铸 氩 咼
08:01:02,173 INFO [ServerDetector:76] Detected server tomcat
08:01:02,205 INFO [HotDeployUtil:76] Initializing hot deploy manager 20622431
08:01:02,252 INFO [AutoDeployDir:76] Auto deploy scanner started for .\resource\deploy
2021-12-11 8:01:02 org.apache.catalina.core.StandardContext addApplicationListener
信息: The listener "com.liferay.portal.kernel.servlet.PortletContextListener" is already configured for this context. The duplicate definition has been ignored.
2021-12-11 8:01:02 org.apache.catalina.core.StandardContext addApplicationListener
信息: The listener "com.liferay.portal.kernel.servlet.ThemeContextListener" is already configured for this context. The duplicate definition has been ignored.
08:01:02,486 INFO [ThemeHotDeployListener:92] Registering themes for liferay-jedi-theme
08:01:02,674 INFO [ThemeHotDeployListener:104] Themes for liferay-jedi-theme registered successfully
08:01:02,676 INFO [ThemeHotDeployListener:92] Registering themes for liferay-jedi-theme
08:01:02,678 INFO [ThemeHotDeployListener:104] Themes for liferay-jedi-theme registered successfully
2021-12-11 8:01:03 org.apache.coyote.http11.Http11Protocol start
信息: Starting Coyote HTTP/1.1 on http-80
2021-12-11 8:01:03 org.apache.catalina.startup.Catalina start
信息: Server startup in 7735 ms

```

• 步骤三：登录并查看

项目启动后直接通过IE浏览器访问，地址栏输入<http://127.0.0.1>即可。



出现这个页面就说明稿件系统一键部署完成了，是不是很方便。

◦ 登录账号密码：

用户名：test01

密码：1111

◦ 数据库账号密码 (mysql)：

用户名：root

密码：1111

库名：lportal

2. 【稿件系统的功能划分】

• 功能展示：

网站稿件管理发布系统

稿件管理 文档上传下载

稿件管理

您的请求执行成功。

稿件

查询高级

查看稿件 增加稿件

到期 删除

显示12条结果。

	编号	标题	版本	状态	修改过的日期	显示日期	作者
☐	13501	金鹰机身 最新GPS导航手机M5卖899元	1.0	批准	08-9-18 下午2:38	08-9-17 下午5:00	User test01
☐	13417	国产型生企业输出核心技术费失论	1.0	批准	08-9-17 下午5:21	08-9-17 下午5:00	User test01
☐	13023	美国科学家首次拍到非洲牧角兽(图)	1.0	批准	08-9-12 上午10:02	08-9-12 上午10:01	User test11
☐	12890	昆明市	1.0	批准	08-9-11 下午5:10	08-9-11 下午5:09	User test10
☐	12847	云南省高院试行法官责任终身追究制	1.0	批准	08-9-11 下午4:49	08-9-11 下午4:49	User test10
☐	12842	德国外长可能因伊拉克问题蒙斯德政治前罪	1.0	批准	08-9-11 下午4:40	08-9-11 下午4:39	User test10
☐	10868	传中国电信CDMA设备规格标准体系已出	1.0	到期	08-9-2 下午2:32	08-8-30 下午3:34	Test User1
☐	11006	台湾副统承认将推动武器与微软E80竞争	1.0	到期	08-9-1 上午10:20	08-9-1 上午10:15	Test User1
☐	11001	电信重拟在鄂“潜”进	1.0	批准	08-9-1 上午9:44	08-9-1 上午9:40	Test User1
☐	10929	张敬伟全民健身非要大建体育场馆吗？	1.0	批准	08-8-30 下午5:11	08-8-30 下午5:10	Test User1
☐	10863	中国电信确定0网将向3G升级 4G网络则向LTE	1.0	批准	08-8-30 下午3:35	08-8-30 下午3:33	Test User1
☐	10839	铁道总经理张永平调任天津移动老总	1.0	到期	08-8-30 下午3:12	08-8-30 下午3:10	Test User1

显示12条结果。

● 功能划分：

一、登录、退出功能

二、稿件管理，可以执行查询、增加、删除、修改、显示以及批准稿件和稿件到期操作，批准后的稿件即可在网站上发布。

三、文件上传下载，用户可以将本地文档上传到平台，也可将平台中的文档下载到本地位置，并对平台目标文件夹和文档执行增加、删除等操作。

3. 【稿件系统的测试用例】

通过以上的功能，我们可以得到以下的测试用例：

测试点	用例名称	级别	前置条件	测试步骤	预期结果
正常登录	正常输入用户名，正确密码，登录	一级	系统未登录	1.输入正确用户名 2.输入正确密码 3.发起登录请求	登录成功，验证当前页面标题：测试比对样品 - 稿件管理
用户名错误	输入错误用户名，正确密码，登录	二级	系统未登录	1.输入错误用户名 2.输入正确密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录
密码错误	输入正确用户名，错误密码，登录	二级	系统未登录	1.输入正确用户名 2.输入错误密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录
稿件增加	增加稿件，保存并返回	一级	系统已登录	1.点击增加稿件按钮 2.输入标题 3.输入内容 4.点击保存并返回	1.页面：新增了一条记录，标题为输入标题，状态为不批准 2.数据库：journalarticle表，title为输入标题，content为输入内容，approved（是否批准）为0
稿件删除	选择一条稿件，删除	一级	系统已登录	1.先新增一个稿件 2.点击删除按钮	1.页面：按标题查询，稿件不存在 2.数据库：journalarticle表，按标题查询，稿件不存在
稿件查询	输入标题，查询稿件	一级	系统已登录	1.先新增一个稿件 2.输入已存在稿件的标题 3.点击查询稿件按钮	1.页面：查询稿件的标题都为查询输入的标题 2.数据库：查询条件为输入的标题，查询信息的编号与页面一致
稿件修改	选择一条稿件，修改标题及内容	一级	系统已登录	1.先新增一个稿件 2.修改标题 3.修改内容 4.点击保持并返回	1.页面：标题为修改后标题，内容为修改后内容，状态为不批准 2.数据库：journalarticle表，title为修改后标题，content为修改后内容，approved（是否批准）为0

测试点	用例名称	级别	前置条件	测试步骤	预期结果
新增文件夹	新增文件夹, 保存并返回	一级	系统已登录	1.点击新增文件夹按钮 2.输入名称 3.输入描述 4.点击保存并返回	1.页面：新增了一条记录，文件夹名称为输入名称 2.数据库：dlfolder表，name为输入文件夹名称，description为输入文件夹描述
删除文件夹	选择一个文件夹, 删除	一级	文件夹已存在	1.选择一条稿件 2.点击删除按钮	1.页面：页面文件夹不存在
文档上传	选择文件夹, 上传	一级	文件夹已存在	1.选择一个文件夹并点击进入 2.点击上传文档按钮 3.选择文档路径 4.文档重命名 5.输入说明 6.点击上传文档按钮	1.页面：文件命名显示正确 2.数据库：dlfileentry表，title为文件重命名，description为文件说明
文档下载	选择文档, 下载	一级	文件已上传	1.选择一个文档并点击 2.点击下载保存	文档下载成功

3.【自动化测试框架环境部署】

• 步骤一：安装python环境和配置环境变量

安装python环境和配置环境变量我们前面的基础课程中已经讲过了，不会的小伙伴可以回顾前面的内容。

• 步骤二：框架环境部署

◦ 方式一：在线部署（有网环境）

框架的根目录下我已经提前准备好了所需要的三方库信息并记录到了 requirements.txt 文件中，我只需要运行一行命令即可自动下载并安装：

```
pip install -i https://pypi.douban.com/simple/ -r requirements.txt
```

◦ 方式二：离线部署（无网环境）

离线安装包分为两种，一种是whl文件，一种是tar压缩包。我已经把框架的所有三方库的包准备好了，我们只需要运行不同的命令来安装这两种包：

▪ whl文件

```
pip install --no-index --find-links=./ -r requirements.txt #都放在当前路径下执行
```

- tar文件

先解压，然后运行：

```
python setup.py install
```

12.2 【WEB自动化项目实践】项目案例实战（一）

一、学习目标

1.登录功能WEB自动化实战

二、知识点

1.【登录功能WEB自动化实战】

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
正常登录	正常输入用户名，正确密码，登录	一级	系统未登录	1.输入正确用户名 2.输入正确密码 3.发起登录请求	登录成功，验证当前页面标题：测试比对样品 - 稿件管理
用户名错误	输入错误用户名，正确密码，登录	二级	系统未登录	1.输入错误用户名 2.输入正确密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录
密码错误	输入正确用户名，错误密码，登录	二级	系统未登录	1.输入正确用户名 2.输入错误密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录

- 项目实践：
 - 步骤一：新建四个文件夹（注：名字必须一样）

说明：创建四个文件夹名字自己定义，但是要保证四个文件夹的名字必须一样。
 - 步骤二：修改配置文件
- 打开 Config 目录下的 配置文件.ini，修改：

```
[WEB自动化配置]
#选择浏览器类型ie、firefox、chrome,chromeheadless
browser = ie

[数据库连接配置]
dbtype = mysql
host = 127.0.0.1
user = root
passwd = 1111
port = 3306
database = lportal

[项目运行设置]
```

```
#自动化测试类型：HTTP、WEB、CLIENT
AUTO_TYPE = WEB
#测试报告类型：ALLURE、HTML、XML
REPORT_TYPE = HTML
#数据驱动类型：YamlDriver、ExcelDriver
DATA_DRIVER_TYPE = YamlDriver
#项目名称：空为全部项目,TestSuits下的文件名
TEST_PROJECT = p02_web_gjxt
#项目测试地址
TEST_URL = http://127.0.0.1
#是否发送邮件,yes,no
IS_EMAIL = no
```

○ 步骤三：定位登录功能的元素信息

在对应的项目文件夹下新建 yaml 文件：Data/DataElement/p02_web_gjxt/01登录页面元素信息.yaml

```
login:
  username: ["name", "_58_login"]
  password: ["name", "_58_password"]
  loginbtn: ["xpath", ".*//input[@type='submit' and @value='登录']"]
  welcome: ["xpath", ".*//div[@id='banner']/div/h2/span"]
  quietbtn: ["xpath", ".*//div[@class='lfr-dock-list-container']/ul/li/a"]
  error1: ["xpath", ".*//div[@class='portlet-content-container']/div/form/span"]
```

说明：我们必须按照两层字典的形式，第一层是功能的名称，第二层是功能的所有元素信息。

○ 步骤四：封装功能的业务逻辑

PageObject 层新建 py 文件：PageObject/p02_web_gjxt/web_login_page.py

```
import time

from Base.baseAutoweb import WebBase
from Base.baseLogger import Logger
from Base.basePath import BasePath
from Base.utils import read_config_ini

logger = Logger('web_login_page.py').getLogger()

class LoginPage(WebBase):

    def __init__(self):
        super().__init__('01登录页面元素信息')

    def login(self, username, password):
        """稿件系统登录"""
        logger.info("稿件系统登录开始")
        self.get_url(read_config_ini(BasePath.CONFIG_FILE)['项目运行设置']['TEST_URL'])
        time.sleep(1)
        # 1.输入用户名
        self.clear("login/username")
        self.sendKeys("login/username", username)
        time.sleep(1)
```

```

self.click("login/password")
# 2.输入密码
self.clear("login/password")
self.sendkeys("login/password", password)
# 3.点击登录
self.click("login/loginbtn")
logger.info("稿件系统登录结束")

def assert_login_ok(self, flag):
    """登录成功断言"""
    if flag == "1":
        assert self.get_title() == "测试比对样品 - 稿件管理", "【断言】断言失败！"
        logger.info("登录成功！")
        assert self.get_text("login/welcome") == "welcome test01!", "【断言】断言失败！"
        logger.info("登录成功！")
    elif flag == "2":
        assert self.get_title() == "测试比对样品 - 登录", "【断言】断言失败！"
        logger.info("验证登录失败成功！")
    elif flag == "3":
        assert self.get_title() == "测试比对样品 - 登录", "【断言】断言失败！"
        logger.info("验证登录失败成功！")

if __name__ == '__main__':
    from selenium import webdriver
    from Base.baseContainer import GlobalManager
    driver =
webdriver.Ie(executable_path=r"C:\Users\Admin\Desktop\stuCode\TestFramework_myproject\Driver\IEDriverServer.exe")
    GlobalManager().set_value('driver', driver)
    lp = LoginPage()
    lp.login("test01", "1111")

```

○ 步骤五：编写测试用例

TestSuits 层新建 py 文件：TestSuits/p02_web_gjxt/test_web_gjxt.py

```

from Base.baseData import DataDriver
from PageObject.p02_web_gjxt.web_login_page import LoginPage
import pytest

class TestCase1():
    """WEB自动化-稿件管理系统-登录功能模块"""

    @pytest.mark.parametrize('case_data', DataDriver().get_case_data('01登录功能'))
    def test_login_case01(self, driver, case_data):
        """WEB自动化用例-用户登录测试"""
        lp = LoginPage()
        lp.login(case_data['username'], case_data['password'])
        lp.assert_login_ok(case_data['flag'])

```

12.3 【WEB自动化项目实践】项目案例实战（二）

一、学习目标

1.稿件新增功能WEB自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件增加	增加稿件, 保存并返回	一级	系统已登录	1.点击增加稿件按钮 2.输入标题 3.输入内容 4.点击保存并返回	1.页面：新增了一条记录，标题为输入标题，状态为不批准 2.数据库：journalarticle 表，title为输入标题，content为输入内容，approved（是否批准）为0

- 项目实践：

○ 步骤一：定位稿件新增功能的元素信息

在对应的项目文件夹下新建 yaml 文件：Data/DataElement/p02_web_gjxt/02稿件管理元素信息.yaml

```
article: #稿件管理
  add_article_btn: ["xpath", ".//input[@type='button' and @value='增加稿件']"]
  title: ["id", "_15_title"]
  add_iframe: ["id", "_15_editor"]
  content: ["id", "textArea"]
  save: ["xpath", ".//input[@type='submit' and @value='保存并返回']"]
  select_btn: ["xpath", ".//input[@type='submit' and @value='查询稿件']"]
  first: ["xpath", ".//div[@class='results-grid']/table/tbody/tr[2]/td[3]/a"]
  state: ["xpath", ".//div[@class='results-grid']/table/tbody/tr[2]/td[5]/a"]
```

○ 步骤二：封装稿件新增的业务逻辑

```
import time

from Base.baseAutoweb import WebBase
from Base.baseLogger import Logger
from ExtTools.dbbase import MySQLHelp
from PageObject.p02_web_gjxt.web_login_page import LoginPage

logger = Logger('web_article_page.py').getLogger()

class ArticlePage(WebBase):

    def __init__(self):
        super().__init__("02稿件管理元素信息")
```

```

def add_article(self, title, content):
    """稿件新增"""
    logger.info("稿件新增开始")
    self.click("article/add_article_btn")
    time.sleep(1)
    self.sendKeys("article/title", title)
    self.switch_iframe("article/add_iframe")
    self.sendKeys("article/content", content)
    time.sleep(1)
    self.switch_iframe_out()
    self.click("article/save")
    time.sleep(1)
    self.click("article/select_btn")
    logger.info("稿件新增结束")

def assert_article_add(self, title):
    """断言稿件增加成功"""
    assert self.get_text("article/first") == title, "【断言】稿件新增验证失败！"
    logger.info("【断言】稿件新增验证成功！")
    assert self.get_text("article/state") == "不批准", "【断言】稿件新增验证失败！"
    logger.info("【断言】稿件新增验证成功！")

def assert_add_database(self, title, content):
    """稿件新增-数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db =
MysqlHelp(host=dbInfo['host'],user=dbInfo['user'],passwd=dbInfo['passwd'],port=dbInfo['po
rt'],database=dbInfo['database'])
    res = db.mysql_db_select("select title,content,approved from journalarticle order
by createDate desc limit 1;")[0]
    assert res[0] == title, "【断言】稿件新增验证失败！"
    assert content in res[1], "【断言】稿件新增验证失败！"
    assert res[2] == 0, "【断言】稿件新增验证失败！"
    logger.info("【断言】稿件新增验证成功！")

if __name__ == '__main__':
    # from selenium import webdriver
    # from Base.baseContainer import GlobalManager
    # driver =
webdriver.Ie(executable_path=r"C:\Users\Admin\Desktop\stuCode\TestFramework_myproject\Driv
er\IEDriverServer.exe")
    # GlobalManager().set_value('driver', driver)
    # lp = LoginPage()
    # lp.login("test01", "1111")
    ap = ArticlePage()
    ap.assert_add_database()
    # ap.add_article("这是标题","这是内容")

```

○ 步骤三：编写测试用例

编写用例之前先思考一个问题，web自动化的运行起点是打开浏览器开始，那么我们打开浏览器就是稿件新增页面吗，显然不是。我们必须先进行登录才能进行稿件新增。而登录就是稿件新增功能的前置操作。

编写fixture前置：

```
import pytest
from PageObject.p02_web_gjxt.web_login_page import LoginPage

@pytest.fixture(scope="function")
def init_login():
    """稿件系统登录"""
    lp = LoginPage()
    lp.login("test01", "1111")
```

编写用例：

(1) 新建数据驱动文件：Data/DataDriver/YamlDriver/p02_web_gjxt/02稿件新增.yaml

```
- title: "这是标题1"
  content: "这是内容1"
```

(2) 编写测试用例：TestSuits/p02_web_gjxt/test_web_gjxt.py

```
class TestCase2():
    """WEB自动化-稿件管理系统-稿件管理功能模块"""

    @pytest.mark.parametrize('case_data',DataDriver().get_case_data('02稿件新增'))
    def test_article_case01(self, driver, init_login, case_data):
        """WEB自动化用例-稿件增加"""
        ap = ArticlePage()
        ap.add_article(case_data['title'],case_data['content'])
        ap.assert_article_add(case_data['title'])
        ap.assert_add_database(case_data['title'],case_data['content'])
```

12.4 【WEB自动化项目实践】项目案例实战（三）

一、学习目标

1.稿件删除功能WEB自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件删除	选择一条稿件，删除	一级	已新增稿件	1.选择一条稿件 2.点击删除按钮	1.页面：按标题查询，稿件不存在 2.数据库：journalarticle表，按标题查询，稿件不存在

- 项目实践：

步骤一：定位稿件删除功能的元素信息

在对应的项目文件夹下修改 yaml 文件：Data/DataElement/p02_web_gjxt/02稿件管理元素信息.yaml

```
check: ["xpath", "//*[@class='results-grid']/table/tbody/tr[2]/td/input"]
delete_btn: ["xpath", "//*[@type='button' and @value='删除']"]
```

步骤二：封装稿件删除的业务逻辑

修改 PageObject 下的 py 文件：PageObject/p02_web_gjxt/web_article_page.py

```
def delete_article(self):
    """删除稿件"""
    logger.info("稿件删除开始")
    self.click("article/check")
    self.click("article/delete_btn")
    self.is_alert().accept()
    logger.info("稿件删除结束")

def assert_delete_article(self, title):
    """删除稿件页面断言"""
    assert self.get_text("article/first") != title, "【断言】验证稿件删除失败！"
    logger.info("【断言】验证稿件删除成功！")

def assert_delete_database(self, title):
    """删除稿件数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db =
    MySQLHelp(host=dbInfo['host'],user=dbInfo['user'],passwd=dbInfo['passwd'],port=dbInfo['port'],
    ,database=dbInfo['database'])
    res = db.mysql_db_select("SELECT count(*) FROM journalarticle where title =
'{}'.format(title))[0]
    assert res[0] == 0, "【断言】验证稿件删除失败！"
    logger.info("【断言】验证稿件删除成功！")
```

步骤三：编写测试用例

新建数据驱动文件：Data/DataDriver/YamlDriver/p02_web_gjxt/03稿件删除.yaml

```
- title: "这是标题2"
  content: "这是内容2"
```

修改 TestSuits 下的 py 文件：TestSuits/p02_web_gjxt/test_web_gjxt.py

```
@pytest.mark.parametrize('case_data', DataDriver().get_case_data('03稿件删除'))
def test_article_case02(self, driver, init_login,case_data):
    """WEB自动化用例-稿件删除"""
    ap = ArticlePage()
    ap.add_article(case_data['title'],case_data['content'])
    ap.delete_article()
    ap.assert_delete_article(case_data['title'])
    ap.assert_delete_database(case_data['title'])
```

12.5 【WEB自动化项目实践】项目案例实战（四）

一、学习目标

1.稿件修改功能WEB自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件修改	选择一条稿件，修改标题及内容	一级	已新增稿件	1.选择一条稿件信息，点击进入 2.修改标题 3.修改内容 4.点击保持并返回	1.页面：标题为修改后标题，内容为修改后内容，状态为不批准 2.数据库：journalarticle表，title为修改后标题，content为修改后内容，approved（是否批准）为0

- 项目实践：

步骤一：定位稿件修改功能的元素信息

修改 yaml 文件：Data/DataElement/p02_web_gjxt/02稿件管理元素信息.yaml

```
first_article: ["xpath", "//*[@class='results-grid']/table/tbody/tr[2]/td[2]/a"]
```

步骤二：封装稿件修改的业务逻辑

修改 PageObject 下的 py 文件：PageObject/p02_web_gjxt/web_article_page.py

```
def edit_article(self, title, content):
    """稿件修改"""
    logger.info("稿件修改开始")
    self.click("article/first_article")
    time.sleep(1)
    self.clear("article/title")
    self.sendKeys("article/title", title)
    self.switch_iframe("article/add_iframe")
    self.clear("article/content")
    self.sendKeys("article/content", content)
    self.switch_iframe_out()
    self.click("article/save")
    logger.info("稿件修改结束")
```

步骤三：编写测试用例

同样，登录也是稿件修改的前置，为了保证本次运行的数据不对下次运行造成影响，建议后置增加稿件删除。

编写前置：

```
@pytest.fixture(scope="function")
def delete_article():
    lp = LoginPage()
    lp.login("test01", "1111")
    yield
    ap = ArticlePage()
    ap.delete_article()
```

数据驱动：Data/DataDriver/YamlDriver/p02_web_gjxt/04稿件修改.yaml

```
- title: "这是标题3"
  content: "这是内容3"
```

编写用例：TestSuits/p02_web_gjxt/test_web_gjxt.py

```
@pytest.mark.parametrize('case_data', DataDriver().get_case_data('04稿件修改'))
def test_article_case03(self, driver, delete_article, case_data):
    """WEB自动化用例-稿件修改"""
    ap = ArticlePage()
    ap.add_article(case_data['title'], case_data['content'])
    ap.edit_article(case_data['title'], case_data['content'])
    ap.assert_article_add(case_data['title'])
    ap.assert_add_database(case_data['title'], case_data['content'])
```

12.6 【WEB自动化项目实践】项目案例实战（五）

一、学习目标

1.稿件查询功能WEB自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件查询	输入标题，查询稿件	一级	已新增稿件	1.输入已存在稿件的标题 2.点击查询稿件按钮	1.页面：查询稿件的标题都为查询输入的标题 2.数据库：查询条件为输入的标题，查询信息的编号与页面一致

- 项目实践：

步骤一：定位稿件查询功能的元素信息

修改yaml文件：Data/DataElement/p02_web_gjxt/02稿件管理元素信息.yaml

```
select_input: ["id", "toggle_id_journal_article_searchkeywords"]
```

步骤二：封装稿件查询的业务逻辑

修改 PageObject 下的 py 文件：PageObject/p02_web_gjxt/web_article_page.py

```

def select_article(self, title):
    """稿件查询"""
    logger.info("稿件查询开始")
    self.clear("article/select_input")
    self.sendKeys("article/select_input", title)
    self.click("article/select_btn")
    logger.info("稿件查询结束")

def assert_select_article(self, title):
    """稿件查询页面断言"""
    assert self.get_text("article/first") == title, "【断言】验证稿件查询失败！"
    logger.info("【断言】验证稿件查询成功！")

def assert_select_database(self, title):
    """稿件查询数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db =
    MysqlHelp(host=dbInfo['host'], user=dbInfo['user'], passwd=dbInfo['passwd'], port=dbInfo['port'],
    , database=dbInfo['database'])
    res = db.mysql_db_select("SELECT count(*) FROM journalarticle where title =
'{}'.format(title))[0]
    assert res[0] == 1, "【断言】数据库验证稿件查询失败！"
    logger.info("【断言】数据库验证稿件查询成功！")

```

步骤三：编写测试用例

数据驱动：Data/DataDriver/YamlDriver/p02_web_gjxt/05稿件查询.yaml

```

- title: "这是标题4"
  content: "这是内容4"

```

编写用例：TestSuits/p02_web_gjxt/test_web_gjxt.py

```

@pytest.mark.parametrize('case_data', DataDriver().get_case_data('05稿件查询'))
def test_article_case04(self, driver, delete_article, case_data):
    """WEB自动化用例-稿件查询"""
    ap = ArticlePage()
    ap.add_article(case_data['title'], case_data['content'])
    ap.select_article(case_data['title'])
    ap.assert_select_article(case_data['title'])
    ap.assert_select_database(case_data['title'])

```

12.7 【WEB自动化项目实践】项目案例实战（六）

一、学习目标

1.新增,删除文件夹功能WEB自动化实战

二、知识点

• 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
新增文件夹	新增文件夹，保存并返回	一级	系统已登录	1.点击新增文件夹按钮 2.输入名称 3.输入描述 4.点击保存并返回	1.页面：新增了一条记录，文件夹名称为输入名称 2.数据库：d1folder 表，name为输入文件夹名称，description为输入文件夹描述
删除文件夹	选择一个文件夹，删除	一级	文件夹已存在	1.选择一条数据 2.点击删除按钮	1.页面：页面文件夹不存在

• 项目实践：

步骤一：定位新增文件夹功能的元素信息

DataElement 目录下新建 yaml 文件：Data/DataElement/p02_web_gjxt/03文件上传下载元素信息.yaml

```
file: #文档上传下载功能模块
file_page: ["xpath", ".//span[text()='文档上传下载']"]
add_btn: ["xpath", ".//input[@value='新建文件夹']"]
folder_name: ["id", "_20_name"]
folder_desc: ["id", "_20_description"]
save_folder: ["xpath", ".//input[@type='submit' and @value='保存并返回']"]
first_name: ["xpath", ".//ul[@class='lfr-component']/li[2]/a"]
first_del: ["xpath", ".//ul[@class='lfr-component']/li[2]/noabr/a[2]"]
msg_sucess: ["xpath", ".//div[@class='portlet-content']/div/div/div"]
```

步骤二：封装新增文件夹功能的业务逻辑

PageObject 目录下新建 py 文件：PageObject/p02_web_gjxt/web_file_page.py

```
import time
from Base.baseAutoWeb import WebBase
from Base.baseLogger import Logger
from ExtTools.dbbase import MysqlHelp
from PageObject.p02_web_gjxt.web_login_page import LoginPage

logger = Logger("web_file_page.py").getLogger()

class FilePage(WebBase):

    def __init__(self):
        super().__init__("03文件上传下载元素信息")
```

```

def add_folder(self, name, desc):
    """文件夹新增"""
    logger.info("文件新增开始")
    self.click("file/file_page")
    time.sleep(1)
    self.click("file/add_btn")
    time.sleep(1)
    self.sendKeys("file/folder_name", name)
    self.sendKeys("file/folder_desc", desc)
    self.click("file/save_folder")
    logger.info("文件新增结束")

def assert_addfolder_page(self, name):
    """文件夹新增页面断言"""
    assert self.get_text("file/first_name") == name, "【断言】验证文件夹新增失败！"
    logger.info("【断言】验证文件夹新增成功！")

def assert__addfolder_database(self, name, desc):
    """文件夹新增数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db =
    MysqlHelp(host=dbInfo['host'], user=dbInfo['user'], passwd=dbInfo['passwd'], port=dbInfo['port'],
    , database=dbInfo['database'])
    res = db.mysql_db_select("select name,description from dlfolder order by createDate
    desc limit 1;")[0]
    assert res[0] == name, "【断言】数据库验证文件夹新增失败！"
    assert res[1] == desc, "【断言】数据库验证文件夹新增失败！"
    logger.info("【断言】数据库验证文件夹新增成功！")

def delete_folder(self):
    """删除文件夹"""
    logger.info("删除文件夹开始")
    self.click("file/file_page")
    time.sleep(1)
    self.click("file/first_del")
    self.is_alert().accept()
    logger.info("删除文件夹结束")

def assert_delfolder_page(self):
    """断言文件夹删除页面"""
    assert self.get_text("file/msg_sucess") == "您的请求执行成功。", "【断言】数据库验证文件夹删除失败！"
    logger.info("【断言】数据库验证文件夹删除成功！")

if __name__ == '__main__':
    from selenium import webdriver
    from Base.baseContainer import GlobalManager
    driver =
    webdriver.Ie(executable_path=r"C:\Users\Admin\Desktop\stuCode\TestFramework_myproject\Driver\
    IEDriverServer.exe")
    GlobalManager().set_value('driver', driver)
    lp = LoginPage()
    lp.login("test01", "1111")
    fp = FilePage()

```

fp.delete_folder()

步骤三：编写测试用例

数据驱动：Data/DataDriver/YamlDriver/p02_web_gjxt/06文件夹新增和删除.yaml

```
- name: "文件夹名称"
  desc: "文件夹描述"
```

测试用例：TestSuits/p02_web_gjxt/test_web_gjxt.py

```
class TestCase3():
    """WEB自动化-稿件管理系统-文档上传下载功能模块"""

    @pytest.mark.parametrize('case_data', DataDriver().get_case_data('06文件夹新增和删除'))
    def test_file_case01(self, driver, init_login, case_data):
        """WEB自动化用例-文件夹新增和删除"""
        fp = FilePage()
        fp.add_folder(case_data['name'], case_data['desc'])
        fp.assert_addfolder_page(case_data['name'])
        fp.assert__addfolder_database(case_data['name'], case_data['desc'])
        fp.delete_folder()
        fp.assert_delfolder_page()
```

12.8 【WEB自动化项目实践】项目案例实战（七）

一、学习目标

1.文件上传功能WEB自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
文档上传	选择文件夹，上传	一级	文件夹已存在	1.选择一个文件夹并点击进入 2.点击上传文档按钮 3.选择文档路径 4.文档重命名 5.输入说明 6.点击上传文档按钮	1.页面：文件命名显示正确 2.数据库：dlfileentry表，title为文件重命名，description为文件说明

- 项目实践：

步骤一：定位文件上传功能的元素信息

DataElement 目录下修改 yaml 文件：Data/DataElement/p02_web_gjxt/03文件上传下载元素信息.yaml

```

upload_btn: ["xpath", "//*[@type='button' and @value='上传文档']"]
iframe_file: ["id", "dlFileEntryUploadProgress-iframe"]
input_file: ["name", "_20_file"]
rename_file: ["id", "_20_title"]
desc_file: ["id", "_20_description"]
sub_file: ["xpath", "//*[@type='submit' and @value='上传文档']"]
first_file: ["xpath", "//*[@class='results-grid']/table/tbody/tr[2]/td/a"]

```

步骤二：封装文件上传功能的业务逻辑

PageObject 目录下修改 py 文件：PageObject/p02_web_gjxt/web_file_page.py

```

def upload_file(self, rename, desc):
    """上传文件"""
    logger.info("上传文件开始")
    self.click("file/first_name")
    time.sleep(1)
    self.click("file/upload_btn")
    time.sleep(1)
    self.switch_iframe("file/iframe_file")
    file_path = os.path.join(BasePath.DATA_TEMP_DIR, "upload_file.txt")
    self.sendKeys("file/input_file", file_path)
    self.sendKeys("file/rename_file", rename)
    self.sendKeys("file/desc_file", desc)
    self.click("file/sub_file")
    self.switch_iframe_out()
    logger.info("上传文件结束")

def assert_upload_page(self, rename, desc):
    """文件上传页面断言"""
    file_info = self.get_text("file/first_file")
    assert file_info.split('\n')[0] == rename, "【断言】验证文件上传失败！"
    assert file_info.split('\n')[1] == desc, "【断言】验证文件上传失败！"
    logger.info("【断言】验证文件上传成功！")

def assert_upload_database(self, rename, desc):
    """文件上传数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db =
    MySQLHelp(host=dbInfo['host'], user=dbInfo['user'], passwd=dbInfo['passwd'], port=dbInfo['port'],
    database=dbInfo['database'])
    res = db.mysql_db_select("select title,description from dlfileentry order by
    createDate desc limit 1;")[0]
    assert res[0] == rename.split('.')[0], "【断言】数据库验证文件上传失败！"
    assert res[1] == desc, "【断言】数据库验证文件上传失败！"
    logger.info("【断言】数据库验证文件上传成功！")

```

步骤三：编写测试用例

文件上传也是有前置操作的，必须先创建文件夹，所有先编写前置和后置：


```
@pytest.fixture(scope="function")
def add_del_folder():
    lp = LoginPage()
    lp.login("test01", "1111")
    fp = FilePage()
    fp.add_folder("文件夹名称", "文件夹描述")
    yield
    fp.delete_folder()
```

数据驱动：Data/DataDriver/YamlDriver/p02_web_gjxt/07文件上传.yaml

```
- rename: "rename.txt"
  desc: "这是描述"
```

测试用例：TestSuits/p02_web_gjxt/test_web_gjxt.py

```
@pytest.mark.parametrize('case_data', DataDriver().get_case_data('07文件上传'))
def test_file_case02(self, driver, add_del_folder, case_data):
    """WEB自动化用例-文件上传"""
    fp = FilePage()
    fp.upload_file(case_data['rename'], case_data['desc'])
    fp.assert_upload_page(case_data['rename'], case_data['desc'])
    fp.assert_upload_database(case_data['rename'], case_data['desc'])
```

12.9 【WEB自动化项目实践】项目案例实战（八）

一、学习目标

1.文件下载功能WEB自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
文档下载	选择文档，下载	一级	文件已上传	1.选择一个文档并点击2.点击下载保存	文档下载成功

- 项目实践：

步骤一：定位文件下载功能的元素信息

DataElement 目录下修改 yaml 文件：Data/DataElement/p02_web_gjxt/03文件上传下载元素信息.yaml

略过

步骤二：封装文件下载功能的业务逻辑

PageObject 目录下修改 py 文件：PageObject/p02_web_gjxt/web_file_page.py

```
def download_file(self):
```

```

"""文件下载"""
logger.info("文件下载开始")
# self.click("file/first_file")
url = self.get_attribute("file/first_file", "href")
self.get_url(url)
time.sleep(1)
pyautogui.hotkey('alt', 's')
time.sleep(1)
logger.info("文件下载结束")

def assert_download_file(self, rename):
    """文件下载断言"""
    down_path = r'C:\Users\Admin\Downloads'
    path = os.path.join(down_path, rename)
    is_ext = os.path.exists(path)
    assert is_ext, "【断言】验证文件下载失败！"
    logger.info("【断言】验证文件下载成功！")
    for file in os.listdir(down_path):
        path_all = os.path.join(down_path, file)
        os.unlink(path_all)

```

步骤三：编写测试用例

数据驱动：Data/DataDriver/YamlDriver/p02_web_gjxt/08文件下载.yaml

```

- rename: "rename.txt"
  desc: "这是描述"

```

测试用例：TestSuits/p02_web_gjxt/test_web_gjxt.py

```

@pytest.mark.parametrize('case_data', DataDriver().get_case_data('08文件下载'))
def test_file_case03(self, driver, add_del_folder, case_data):
    """WEB自动化用例-文件下载"""
    fp = FilePage()
    fp.upload_file(case_data['rename'], case_data['desc'])
    fp.download_file()
    fp.assert_download_file(case_data['rename'])

```

第十三章 项目实践：接口自动化实战

13.1 【接口自动化项目实践】项目需求介绍

一、学习目标

- 1.稿件系统一键部署
- 2.稿件系统的功能划分
- 3.稿件系统的测试用例
- 4.自动化测试框架环境部署

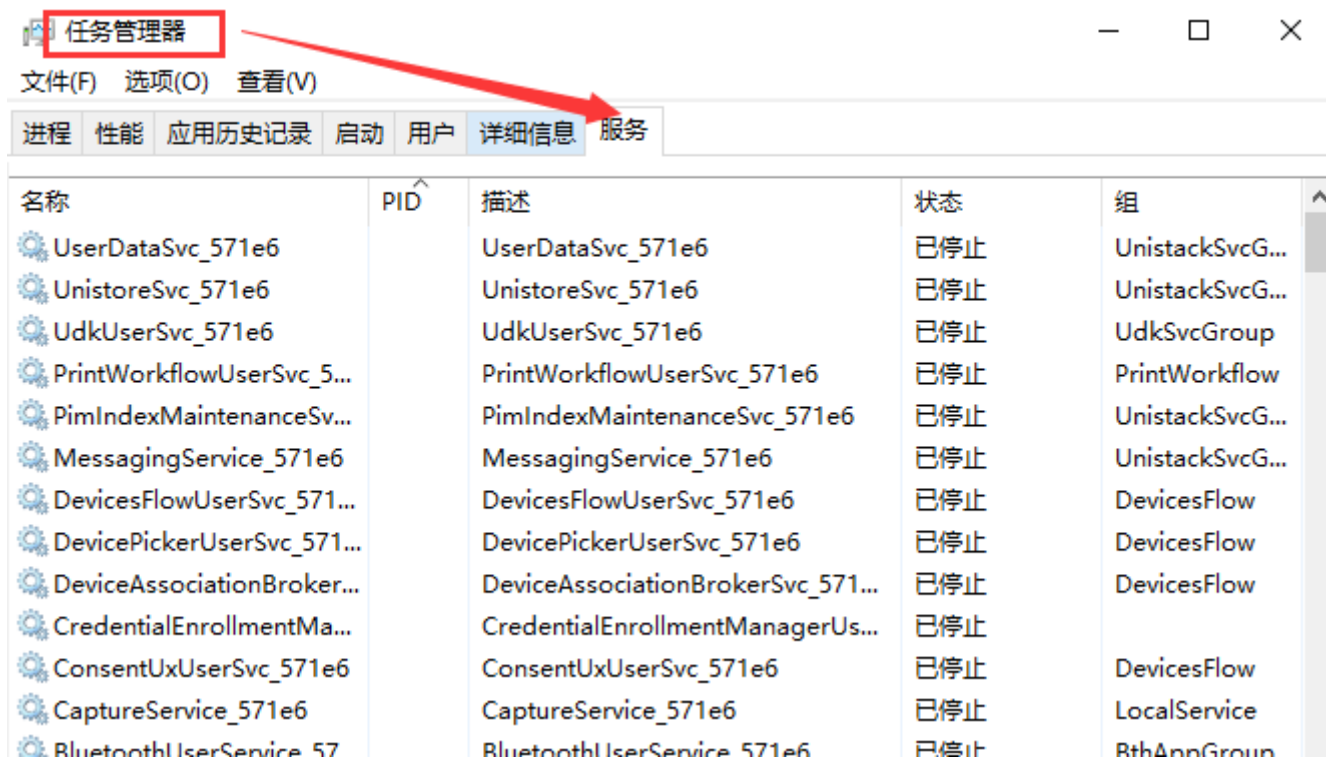
二、知识点

1.【稿件系统一键部署】

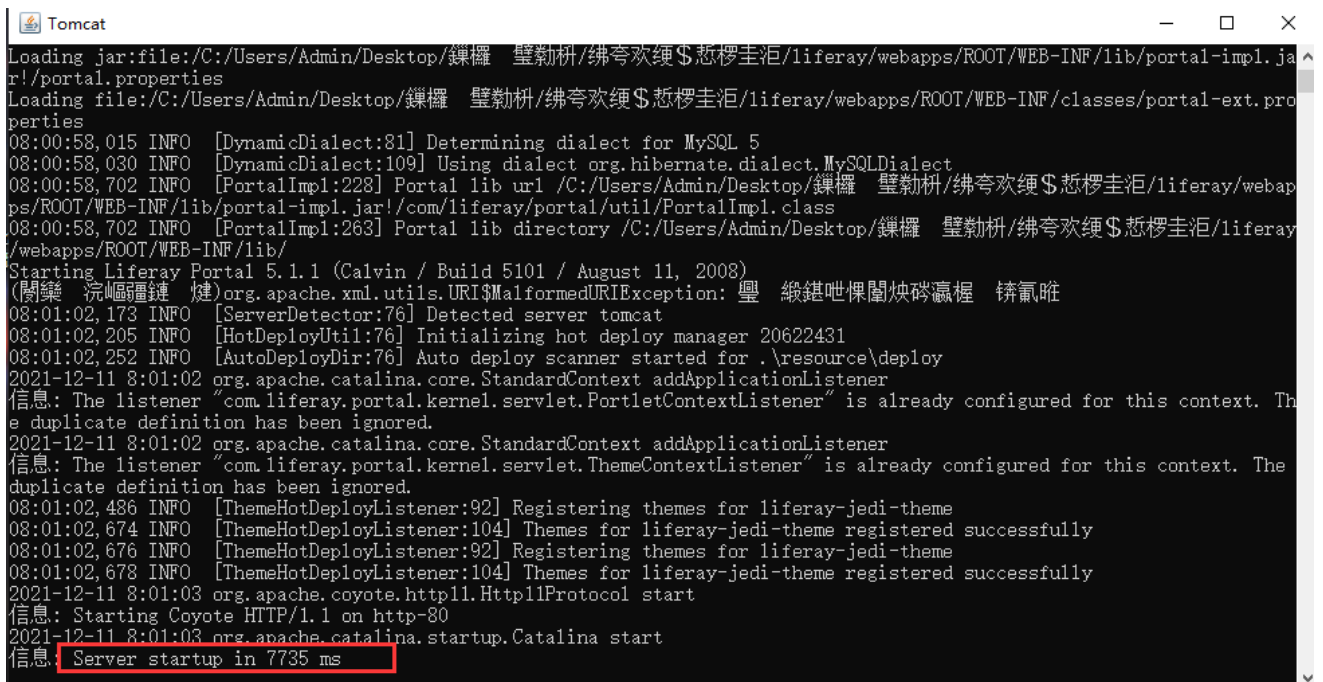
注：稿件系统的部署环境尽量为windows10环境，如果没有此环境建议在虚拟机里安装一个；浏览器建议用ie浏览器，非ie浏览器会出现稿件新增功能不显示的情况。

• 步骤一：关闭本机的服务

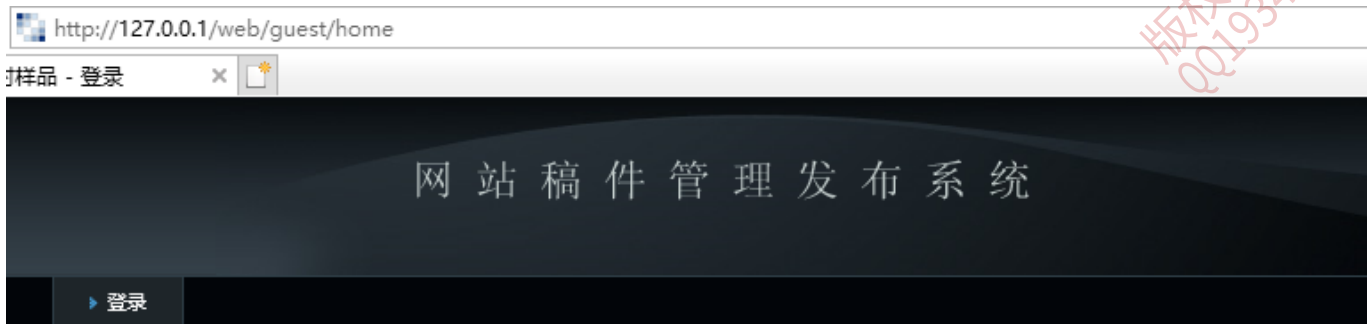
由于稿件系统已经集成了mysql，redis，tomcat等数据库和中间件，所以为了不和本机已经安装的数据库有端口冲突，需要先关闭本机的数据库和中间件的服务。



• 步骤二：运行start.bat程序，一键启动项目



项目启动后直接通过IE浏览器访问，地址栏输入<http://127.0.0.1>即可。



登录

用户名

密码

出现这个页面就说明稿件系统一键部署完成了，是不是很方便。

◦ **登录账号密码：**

用户名：test01

密码：1111

◦ **数据库账号密码（mysql）：**

用户名：root

密码：1111

库名：lportal

2.【稿件系统的功能划分】

• 功能展示：

网站稿件管理发布系统

Welcome test01!

稿件管理

文档上传下载

稿件管理

您的请求执行成功。

稿件

查询

高级

查询稿件

增加稿件

到期

删除

显示12条结果。

	编号	标题	版本	状态	修改过的日期	显示日期	作者
☐	13501	金鹰机身 最新GPS导航手机MS899元	1.0	批准	08-9-18 下午2:38	08-9-17 下午5:00	User test01
☐	13417	国产彩电企业输出核心技术将诞生	1.0	批准	08-9-17 下午5:21	08-9-17 下午5:00	User test01
☐	13023	美国科学家首次拍到非洲象偷蜜(图)	1.0	批准	08-9-12 上午10:02	08-9-12 上午10:01	User test11
☐	12890	昆明市	1.0	批准	08-9-11 下午5:10	08-9-11 下午5:09	User test10
☐	12847	云南高院试行法官责任终身追究制	1.0	批准	08-9-11 下午4:49	08-9-11 下午4:48	User test10
☐	12842	德国外长可能回伊拉克访问谈安斯达政治前景	1.0	批准	08-9-11 下午4:40	08-9-11 下午4:39	User test10
☐	10868	传中国电信CDMA设备招投标整体结果已出	1.0	到期	08-9-2 下午2:32	08-8-30 下午3:34	Test User1
☐	11006	谷歌浏览器将推出浏览器与微软IE8竞争	1.0	到期	08-9-1 上午10:20	08-9-1 上午10:15	Test User1
☐	11001	电信重拟在鄂“增”进	1.0	批准	08-9-1 上午9:44	08-9-1 上午9:40	Test User1
☐	10929	张敬伟全民健身非要大建体育场馆吗?	1.0	批准	08-8-30 下午5:11	08-8-30 下午5:10	Test User1
☐	10863	中国电信确定C网将向3G升级 4G网络则向LTE	1.0	批准	08-8-30 下午3:35	08-8-30 下午3:33	Test User1
☐	10839	铁道总经理张永平调任天津移动副总	1.0	到期	08-8-30 下午3:12	08-8-30 下午3:10	Test User1

显示12条结果。

• 功能划分：

一、登录、退出功能

二、稿件管理，可以执行查询、增加、删除、修改、显示以及批准稿件和稿件到期操作，批准后的稿件即可在网站上发布。

三、文件上传下载，用户可以将本地文档上传到平台，也可将平台中的文档下载到本地位置，并对平台目标文件夹和文档执行增加、删除等操作。

3.【稿件系统的测试用例】

通过以上的功能，我们可以得到以下的测试用例：

测试点	用例名称	级别	前置条件	测试步骤	预期结果
正常登录	正常输入用户名，正确密码，登录	一级	系统未登录	1.输入正确用户名 2.输入正确密码 3.发起登录请求	登录成功，验证当前页面标题：测试比对样品 - 稿件管理
用户名错误	输入错误用户名，正确密码，登录	二级	系统未登录	1.输入错误用户名 2.输入正确密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录
密码错误	输入正确用户名，错误密码，登录	二级	系统未登录	1.输入正确用户名 2.输入错误密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录
稿件增加	增加稿件，保存并返回	一级	系统已登录	1.点击增加稿件按钮 2.输入标题 3.输入内容 4.点击保存并返回	1.页面：新增了一条记录，标题为输入标题，状态为不批准 2.数据库：journalarticle表，title为输入标题，content为输入内容，approved（是否批准）为0
稿件删除	选择一条稿件，删除	一级	系统已登录	1.先新增一个稿件 2.点击删除按钮	1.页面：按标题查询，稿件不存在 2.数据库：journalarticle表，按标题查询，稿件不存在
稿件查询	输入标题，查询稿件	一级	系统已登录	1.先新增一个稿件 2.输入已存在稿件的标题 3.点击查询稿件按钮	1.页面：查询稿件的标题都为查询输入的标题 2.数据库：查询条件为输入的标题，查询信息的编号与页面一致
稿件修改	选择一条稿件，修改标题及内容	一级	系统已登录	1.先新增一个稿件 2.修改标题 3.修改内容 4.点击保持并返回	1.页面：标题为修改后标题，内容为修改后内容，状态为不批准 2.数据库：journalarticle表，title为修改后标题，content为修改后内容，approved（是否批准）为0

测试点	用例名称	级别	前置条件	测试步骤	预期结果
新增文件夹	新增文件夹, 保存并返回	一级	系统已登录	1.点击新增文件夹按钮 2.输入名称 3.输入描述 4.点击保存并返回	1.页面：新增了一条记录，文件夹名称为输入名称 2.数据库：dlfolder表，name为输入文件夹名称，description为输入文件夹描述
删除文件夹	选择一个文件夹, 删除	一级	文件夹已存在	1.选择一条稿件 2.点击删除按钮	1.页面：页面文件夹不存在
文档上传	选择文件夹, 上传	一级	文件夹已存在	1.选择一个文件夹并点击进入 2.点击上传文档按钮 3.选择文档路径 4.文档重命名 5.输入说明 6.点击上传文档按钮	1.页面：文件命名显示正确 2.数据库：dlfileentry表，title为文件重命名，description为文件说明
文档下载	选择文档, 下载	一级	文件已上传	1.选择一个文档并点击 2.点击下载保存	文档下载成功

3.【自动化测试框架环境部署】

• 步骤一：安装python环境和配置环境变量

安装python环境和配置环境变量我们前面的基础课程中已经讲过了，不会的小伙伴可以回顾前面的内容。

• 步骤二：框架环境部署

◦ 方式一：在线部署（有网环境）

框架的根目录下我已经提前准备好了所需要的三方库信息并记录到了 requirements.txt 文件中，我只需要运行一行命令即可自动下载并安装：

```
pip install -i https://pypi.douban.com/simple/ -r requirements.txt
```

◦ 方式二：离线部署（无网环境）

离线安装包分为两种，一种是whl文件，一种是tar压缩包。我已经把框架的所有三方库的包准备好了，我们只需要运行不同的命令来安装这两种包：

▪ whl文件

```
pip install --no-index --find-links=./ -r requirements.txt #都放在当前路径下执行
```


- tar文件

先解压，然后运行：

```
python setup.py install
```

13.2 【接口自动化项目实践】项目案例实战（一）

一、学习目标

1.登录功能接口自动化实战

二、知识点

1.【登录功能接口自动化实战】

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
正常登录	正常输入用户名，正确密码，登录	一级	系统未登录	1.输入正确用户名 2.输入正确密码 3.发起登录请求	登录成功，验证当前页面标题：测试比对样品 - 稿件管理
用户名错误	输入错误用户名，正确密码，登录	二级	系统未登录	1.输入错误用户名 2.输入正确密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录
密码错误	输入正确用户名，错误密码，登录	二级	系统未登录	1.输入正确用户名 2.输入错误密码 3.发起登录请求	登录失败，验证当前页面标题：测试比对样品 - 登录

- 项目实践：

- 步骤一：新建四个文件夹（注：名字必须一样）

说明：创建四个文件夹名字自己定义，但是要保证四个文件夹的名字必须一样。

- 步骤二：修改配置文件

打开 Config 目录下的 配置文件.ini，修改：

```
[数据库连接配置]
dbtype = mysql
host = 127.0.0.1
user = root
passwd = 1111
port = 3306
database = lportal

[项目运行设置]
#自动化测试类型：HTTP、WEB、CLIENT
AUTO_TYPE = HTTP
#测试报告类型：ALLURE、HTML、XML
REPORT_TYPE = HTML
```

```
#数据驱动类型: YamlDriver、ExcelDriver
DATA_DRIVER_TYPE = YamlDriver
#项目名称: 空为全部项目, TestSuits下的文件名
TEST_PROJECT = p03_http_gjxt
#项目测试地址
TEST_URL = http://127.0.0.1
#是否发送邮件, yes, no
IS_EMAIL = no
```

○ 步骤三：抓取接口信息

DataElement 目录下新建 yaml 文件：Data/DataElement/p03_http_gjxt/01登录页面接口信息.yaml

```
home_api:
  method: get
  url: /web/guest/home
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko

login_api:
  method: post
  url: /web/guest/home
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
    Content-Type: application/x-www-form-urlencoded
  params:
    "p_p_id": "58"
    "p_p_lifecycle": "1"
    "p_p_state": "normal"
    "p_p_mode": "view"
    "p_p_col_id": "column-1"
    "p_p_col_count": "1"
    "saveLastPath": "0"
    "_58_struts_action": "/login/view"
    "_58_cmd": "update"
  data:
    "_58_redirect": ""
    "_58_login": "${_58_login}"
    "_58_password": "${_58_password}"
```

说明：我们必须按照两层字典的形式，第一层是功能的名称，第二层是功能的所有元素信息。

○ 步骤四：封装功能的业务逻辑

PageObject 目录下新建 py 文件：PageObject/p03_http_gjxt/api_login_page.py

```
import re

from Base.baseAutoHttp import ApiBase
from Base.baseLogger import Logger

logger = Logger("api_login_page.py").getLogger()

class LoginPage(ApiBase):
```

```

def __init__(self):
    super().__init__("01登录页面接口信息")

def login(self, user, psw):
    """登录功能"""
    self.request_base("home_api")
    change_data = {
        "_58_login": user,
        "_58_password": psw
    }
    res = self.request_base("login_api", change_data=change_data)
    return res.text

def assert_login_ok(self, res, title):
    """登录断言"""
    page_title = re.findall("<title>(.*?)</title>", res)[0]
    assert page_title == title, "【断言】登录验证失败！"
    logger.info("【断言】登录验证成功！")

if __name__ == '__main__':
    lp = LoginPage()
    res = lp.login("test01", "1111")
    print(res)

```

步骤五：编写测试用例

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/01稿件系统登录.yaml

```

- username: "test01"
  password: "1111"
  title: "测试比对样品 - 稿件管理"
- username: "test02"
  password: "1111"
  title: "测试比对样品 - 登录"
- username: "test01"
  password: "2222"
  title: "测试比对样品 - 登录"

```

用例代码：

TestSuits 目录下新建 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```

import pytest

from Base.baseData import DataDriver
from PageObject.p03_http_gjxt.api_login_page import LoginPage

class TestApiCase1():
    """接口自动化-稿件管理系统-登录功能模块"""

```

```
@pytest.mark.parametrize('case_data',DataDriver().get_case_data("01稿件系统登录"))
def test_login_case01(self,case_data):
    """接口自动化用例-用户登录测试"""
    lp = LoginPage()
    res = lp.login(case_data['username'],case_data['password'])
    lp.assert_login_ok(res, case_data['title'])
```

13.3 【接口自动化项目实践】项目案例实战（二）

一、学习目标

1.稿件新增功能接口自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件增加	增加稿件,保存并返回	一级	系统已登录	1.点击增加稿件按钮 2.输入标题 3.输入内容 4.点击保存并返回	1.页面：新增了一条记录，标题为输入标题，状态为不批准 2.数据库：journalarticle表，title为输入标题，content为输入内容，approved（是否批准）为0

- 项目实践：

步骤一：抓取稿件新增功能的接口信息

DataElement 目录下新建 yaml 文件：Data/DataElement/p03_http_gjxt/02稿件管理接口信息.yaml

```
add_api:
  method: post
  url: /group/10779/articles
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    _15_struts_action: "/journal/edit_article"
    p_p_id: "15"
    p_p_lifecycle: "1"
    p_p_mode: "view"
    p_p_state: "maximized"
  data:
    "_15_portletResource": ""
    "_15_cmd": "add"
    "_15_tabs2": ""
    "_15_redirect": "/group/10779/articles?
p_p_id=15&p_p_lifecycle=0&p_p_state=maximized&p_p_mode=view&_15_struts_action=%2Fjournal%
2Fview&_15_tabs1=articles"
```

```
"_15_originalRedirect": "/group/10779/articles?
p_p_id=15&p_p_lifecycle=0&p_p_state=maximized&p_p_mode=view&_15_struts_action=%2Fjournal%
2Fview&_15_tabs1=articles"
"_15_referringPortletResource": ""
"_15_groupId": "10780"
"_15_articleId": ""
"_15_version": "1.0"
"_15_content": "${content}"
"_15_articleURL": "http://127.0.0.1/group/10779/articles?
p_p_id=15&p_p_lifecycle=0&p_p_state=maximized&p_p_mode=view&_15_struts_action=%2Fjournal%
2Fedit_article"
"_15_approve": ""
"_15_saveAndContinue": ""
"_15_deleteArticleIds": "_version_1.0"
"_15_expireArticleIds": "_version_1.0"
"_15_tabs1TabsScroll": ""
"_15_newArticleId": ""
"_15_autoArticleId": "true"
"_15_title": "${title}"
"_15_structureId": ""
"_15_templateId": ""
"_15_type": "general"
"_15_tagsEntries": ""
"_15_indexable": "true"
"_15_indexableCheckbox": "on"
"_15_displayDateMonth": "6"
"_15_displayDateDay": "23"
"_15_displayDateYear": "2022"
"_15_displayDateHour": "9"
"_15_displayDateMinute": "19"
"_15_displayDateAmPm": "1"
"_15_neverExpire": "true"
"_15_neverExpireCheckbox": "on"
"_15_neverReview": "true"
"_15_neverReviewCheckbox": "on"

select_api:
  method: post
  url: /group/10779/articles
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    _15_struts_action: "/journal/view"
    _15_tabs1: "articles"
    p_p_id: "15"
    p_p_lifecycle: "0"
    p_p_mode: "view"
    p_p_state: "maximized"
  data:
    _15_cmd: ""
    _15_tabs1TabsScroll: ""
    _15_groupId: ""
    _15_deleteArticleIds: ""
    _15_expireArticleIds: ""
    _15_advancedSearch: "false"
```

```
_15_keywords: "${title}"
_15_andOperator: "1"
_15_searchArticleId: ""
_15_version: ""
_15_title: ""
_15_content: ""
```

○ 步骤二：封装稿件新增的业务逻辑

PageObject 目录下新建 py 文件：PageObject/p03_http_gjxt/api_article_page.py

```
import re

from Base.baseAutoHttp import ApiBase
from Base.baseLogger import Logger
from ExtTools.dbbase import MysqlHelp
from PageObject.p03_http_gjxt.api_login_page import LoginPage

logger = Logger("api_article_page.py").getLogger()

class ApiArticle(ApiBase):

    def __init__(self):
        super().__init__("02稿件管理接口信息")

    def add_article(self, title, content):
        """稿件新增"""
        change_data = {
            "title": title,
            "content": content
        }
        res = self.request_base('add_api', change_data=change_data)
        return res.text

    def select_article(self, title=''):
        """查询稿件"""
        change_data = {
            "title": title
        }
        res = self.request_base('select_api', change_data=change_data)
        re_info = re.findall('_15_version=1.0">(.*?)</a>', res.text)[:7]
        return re_info

    def assert_add_article(self, title):
        """稿件系统页面断言"""
        re_info = self.select_article(title)
        assert re_info[1] == title, "【断言】稿件新增页面断言失败！"
        logger.info("【断言】稿件新增页面断言成功！")
        assert re_info[3] == "不批准", "【断言】稿件新增页面断言失败！"
        logger.info("【断言】稿件新增页面断言成功！")

    def assert_add_database(self, title, content):
        """高级新增数据断言"""
        dbInfo = self.config['数据库连接配置']
```

```
db = MySQLHelp(host=dbInfo['host'],user=dbInfo['user'],passwd=dbInfo['passwd'],
                port=dbInfo['port'],database=dbInfo['database'])
res = db.mysql_db_select("SELECT title,content,approved FROM journalarticle order
by createDate desc limit 1;")[0]
assert res[0] == title, "【断言】稿件新增数据库断言失败：{}".format(title)
logger.info("【断言】稿件新增数据库断言成功：{}".format(title))
assert content in res[1], "【断言】稿件新增数据库断言失败：{}".format(content)
logger.info("【断言】稿件新增数据库断言成功：{}".format(content))
assert res[2] == 0, "【断言】稿件新增数据库断言失败：0！"
logger.info("【断言】稿件新增数据库断言成功：0！")

if __name__ == '__main__':
    l = LoginPage()
    l.login("test01", "1111")
    a = ApiArticle()
    # res = a.add_article('1111','aaaaa')
    # print(res)
    a.assert_add_database()
```

○ 步骤三：编写测试用例

编写用例之前先思考一个问题，接口自动化的运行起点是打开浏览器开始，那么我们打开浏览器就是稿件新增页面吗，显然不是。我们必须先进行登录才能进行稿件新增。而登录就是稿件新增功能的前置操作。

fixture前置：TestSuits/p03_http_gjxt/conftest.py

```
import pytest
from PageObject.p03_http_gjxt.api_login_page import LoginPage

@pytest.fixture(scope="function")
def init_login():
    """登录"""
    lp = LoginPage()
    lp.login("test01", "1111")
```

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/02稿件新增.yaml

```
- title: "新增：这是标题1"
  content: "新增：这是内容1"
```

编写用例：

TestSuits 目录下修改 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```
class TestApiCase2():
    """接口自动化-稿件管理系统-稿件管理功能模块"""

    @pytest.mark.usefixtures('init_login')
    @pytest.mark.parametrize('case_data', DataDriver().get_case_data("02稿件新增"))
    def test_article_case01(self, case_data):
        """接口自动化用例-稿件增加测试"""
        art = ApiArticle()
        art.add_article(case_data['title'], case_data['content'])
        art.assert_add_article(case_data['title'])
        art.assert_add_database(case_data['title'], case_data['content'])
```

13.4 【接口自动化项目实践】项目案例实战（三）

一、学习目标

1.稿件删除功能接口自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件删除	选择一条稿件，删除	一级	已新增稿件	1.选择一条稿件 2.点击删除按钮	1.页面：按标题查询，稿件不存在 2.数据库： journalarticle 表，按标题查询，稿件不存在

- 项目实践:

步骤一：抓取稿件删除功能的接口信息

DataElement 目录下修改 yaml 文件: Data/DataElement/p03_http_gjxt/02稿件管理接口信息.yaml

```
delete_api:
  method: post
  url: /group/10779/articles
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    p_p_id: "15"
    p_p_lifecycle: "1"
    p_p_state: "maximized"
    p_p_mode: "view"
    _15_struts_action: "/journal/edit_article"
    _15_redirect: "/group/10779/articles?
p_p_id=15&p_p_lifecycle=0&p_p_state=maximized&p_p_mode=view&p_p_col_id=column-
1&p_p_col_count=1&_15_struts_action=%2Fjournal%2Fview&_15_tabs1=articles"
  data:
    _15_cmd: "delete"
    _15_tabs1TabsScroll: ""
```



```

    _15_groupId: "10780"
    _15_deleteArticleIds: "${_15_deleteArticleIds}"
    _15_expireArticleIds: ""
    _15_advancedSearch: "false"
    _15_keywords: ""
    _15_andOperator: "1"
    _15_searchArticleId: ""
    _15_version: ""
    _15_title: ""
    _15_content: ""
    _15_rowIds: "${_15_rowIds}"

```

步骤二：封装稿件删除的业务逻辑

PageObject 目录下修改 py 文件：PageObject/p03_http_gjxt/api_article_page.py

```

def delete_article(self, title):
    """稿件删除"""
    id = self.select_article(title)[0]
    change_data = {
        "_15_deleteArticleIds": "{}_version_1.0".format(id),
        "_15_rowIds": "{}_version_1.0".format(id)
    }
    res = self.request_base('delete_api', change_data=change_data)
    return res.text

def assert_delete_article(self, title):
    """删除页面断言"""
    first_info = self.select_article(title)
    assert first_info == [], "【断言】验证稿件删除页面断言失败！"
    logger.info("【断言】验证稿件删除页面断言成功！")

def assert_delete_database(self, title):
    """删除数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db = MySQLHelp(host=dbInfo['host'], user=dbInfo['user'], passwd=dbInfo['passwd'],
                    port=dbInfo['port'], database=dbInfo['database'])
    res = db.mysql_db_select("select count(*) from journalarticle where title='{}'".format(title))[0]
    assert res[0] == 0, "【断言】验证稿件删除数据库断言失败！"
    logger.info("【断言】验证稿件删除数据库断言成功！")

```

步骤三：编写测试用例

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/03稿件删除.yaml

```

- title: "这是标题2"
  content: "这是内容2"

```

测试用例：

TestSuits 目录下修改 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```
@pytest.mark.usefixtures('init_login')
@pytest.mark.parametrize('case_data', DataDriver().get_case_data("03稿件删除"))
def test_article_case02(self, case_data):
    """接口自动化用例-稿件删除"""
    art = ApiArticle()
    art.add_article(case_data['title'], case_data['content'])
    art.delete_article(case_data['title'])
    art.assert_delete_article(case_data['title'])
    art.assert_delete_database(case_data['title'])
```

13.5 【接口自动化项目实践】项目案例实战（四）

一、学习目标

1.稿件修改功能接口自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件修改	选择一条稿件，修改标题及内容	一级	已新增稿件	1.选择一条稿件信息，点击进入 2.修改标题 3.修改内容 4.点击保持并返回	1.页面：标题为修改后标题，内容为修改后内容，状态为不批准 2.数据库：journalarticle表，title为修改后标题，content为修改后内容，approved（是否批准）为0

- 项目实践：

步骤一：抓取稿件修改功能的接口信息

DataElement 目录下修改 yam1 文件：Data/DataElement/p03_http_gjxt/02稿件管理接口信息.yam1

```
edit_api:
  method: post
  url: /group/10779/articles
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    p_p_id: "15"
    p_p_lifecycle: "1"
    p_p_state: "maximized"
    p_p_mode: "view"
    _15_struts_action: "/journal/edit_article"
  data:
    _15_portletResource: ""
    _15_cmd: "update"
    _15_tabs2: ""
```

```

        _15_redirect: "/group/10779/articles?
p_p_id=15&p_p_lifecycle=0&p_p_state=maximized&p_p_mode=view&p_p_col_id=column-
1&p_p_col_count=1&_15_struts_action=%2Fjournal%2Fview&_15_tabs1=articles"
        _15_originalRedirect: "/group/10779/articles?
p_p_id=15&p_p_lifecycle=0&p_p_state=maximized&p_p_mode=view&p_p_col_id=column-
1&p_p_col_count=1&_15_struts_action=%2Fjournal%2Fview&_15_tabs1=articles"
        _15_referringPortletResource: ""
        _15_groupId: "10780"
        _15_articleId: "${articleId}"
        _15_version: "1.0"
        _15_content: "${content}"
        _15_articleURL: "http://127.0.0.1/group/10779/articles?
p_p_id=15&p_p_lifecycle=0&p_p_state=maximized&p_p_mode=view&_15_struts_action=%2Fjournal%2Fed
it_article"
        _15_approve: ""
        _15_saveAndContinue: ""
        _15_deleteArticleIds: "${deleteArticleIds}"
        _15_expireArticleIds: "${expireArticleIds}"
        _15_tabs1TabsScroll: ""
        _15_title: "${title}"
        _15_incrementVersion: "false"
        _15_structureId: ""
        _15_templateId: ""
        _15_type: "general"
        _15_tagsEntries: ""
        _15_indexable: "true"
        _15_indexableCheckbox: "on"
        _15_displayDateMonth: "6"
        _15_displayDateDay: "17"
        _15_displayDateYear: "2022"
        _15_displayDateHour: "2"
        _15_displayDateMinute: "12"
        _15_displayDateAmPm: "1"
        _15_neverExpire: "true"
        _15_neverExpireCheckbox: "on"
        _15_neverReview: "true"
        _15_neverReviewCheckbox: "on"
    
```

步骤二：封装稿件修改的业务逻辑

PageObject 目录下修改 py 文件：PageObject/p03_http_gjxt/api_article_page.py

```

def edit_article(self, title, content):
    """稿件编辑"""
    id = self.select_article(title)[0]
    change_data = {
        "title": title,
        "content": content,
        "articleId": id,
        "deleteArticleIds": "{}_version_1.0".format(id),
        "expireArticleIds": "{}_version_1.0".format(id)
    }
    res = self.request_base('edit_api', change_data=change_data)
    return res.text
    
```

步骤三：编写测试用例

同样，登录也是稿件修改的前置，为了保证本次运行的数据不对下次运行造成影响，建议后置增加稿件删除。

编写前置：TestSuits/p03_http_gjxt/conftest.py

```
@pytest.fixture(scope="function")
def article_add_delete(request):
    """稿件新增删除"""
    case_data = request.param
    lp = LoginPage()
    lp.login("test01", "1111")
    at = ApiArticle()
    at.add_article(case_data['title'], case_data['content'])
    yield case_data
    at.delete_article(case_data['title'])
```

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/04稿件修改.yaml

```
- title: "这是标题3"
  content: "这是内容3"
```

测试用例：

TestSuits 目录下修改 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```
@pytest.mark.parametrize('article_add_delete', DataDriver().get_case_data("04稿件修改"), indirect=True)
def test_article_case03(self, article_add_delete):
    """接口自动化用例-稿件编辑"""
    art = ApiArticle()
    art.edit_article(article_add_delete['title'], article_add_delete['content'])
    art.assert_add_article(article_add_delete['title'])
    art.assert_add_database(article_add_delete['title'], article_add_delete['content'])
```

13.6 【接口自动化项目实践】项目案例实战（五）

一、学习目标

1. 稿件查询功能接口自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
稿件查询	输入标题，查询稿件	一级	已新增稿件	1.输入已存在稿件的标题 2.点击查询稿件按钮	1.页面：查询稿件的标题都为查询输入的标题 2.数据库：查询条件为输入的标题，查询信息的编号与页面一致

• 项目实践：

步骤一：抓取稿件查询功能的接口信息

DataElement 目录下修改 yaml 文件：Data/DataElement/p03_http_gjxt/02稿件管理接口信息.yaml

略见稿件新增部分

步骤二：封装稿件查询的业务逻辑

PageObject 目录下修改 py 文件：PageObject/p03_http_gjxt/api_article_page.py

```
def assert_select_article(self,res,title):
    """查询页面断言"""
    assert res[1] == title, "【断言】验证稿件查询断言失败！"
    logger.info("【断言】验证稿件查询断言成功！")

def assert_select_database(self,title):
    """查询数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db = MysqlHelp(host=dbInfo['host'],user=dbInfo['user'],passwd=dbInfo['passwd'],
                    port=dbInfo['port'],database=dbInfo['database'])
    res = db.mysql_db_select("select count(*) from journalarticle where title = '{}'".format(title))[0]
    assert res[0] == 1, "【断言】验证稿件查询断言失败！"
    logger.info("【断言】验证稿件查询断言成功！")
```

步骤三：编写测试用例

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/05稿件查询.yaml

```
- title: "这是标题4"
  content: "这是内容4"
```

测试用例：

TestSuits 目录下修改 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```
@pytest.mark.parametrize('article_add_delete', DataDriver().get_case_data("05稿件查询"),
indirect=True)
def test_article_case04(self, article_add_delete):
    """接口自动化用例-稿件查询"""
    art = ApiArticle()
    res = art.select_article(article_add_delete['title'])
    art.assert_select_article(res, article_add_delete['title'])
    art.assert_select_database(article_add_delete['title'])
```

13.7 【接口自动化项目实践】项目案例实战（六）

一、学习目标

1.文件夹新增,删除功能接口自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
新增文件夹	新增文件夹, 保存并返回	一级	系统已登录	1.点击新增文件夹按钮 2.输入名称 3.输入描述 4.点击保存并返回	1.页面：新增了一条记录，文件夹名称为输入名称 2.数据库：dlfolder 表，name为输入文件夹名称，description为输入文件夹描述
删除文件夹	选择一个文件夹，删除	一级	文件夹已存在	1.选择一条稿件 2.点击删除按钮	1.页面：页面文件夹不存在

- 项目实践：

步骤一：抓取新增和删除文件夹功能的接口信息

DataElement 目录下新建 yaml 文件：Data/DataElement/p03_http_gjxt/03文件上传下载接口信息.yaml

```
add_foler:
  method: post
  url: /group/10779/upload
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    p_p_id: "20"
    p_p_lifecycle: "1"
    p_p_state: "maximized"
    p_p_mode: "view"
```

```

        _20_struts_action: "/document_library/edit_folder"
data:
    _20_cmd: "add"
    _20_redirect: "/group/10779/upload"
    _20_folderId: "0"
    _20_parentFolderId: "0"
    _20_tabs1TabsScroll: ""
    _20_name: "${name}"
    _20_description: "${description}"

select_folder:
    method: get
    url: /group/10779/upload
    headers:
        User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko

delete_folder:
    method: post
    url: /group/10779/upload
    headers:
        User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
    params:
        p_p_id: "20"
        p_p_lifecycle: "1"
        p_p_state: "maximized"
        p_p_mode: "view"
        p_p_col_id: "column-1"
        p_p_col_count: "1"
        _20_struts_action: "/document_library/edit_folder"
        _20_cmd: "delete"
        _20_redirect: "/group/10779/upload"
        _20_folderId: "${folderId}"
    
```

步骤二：封装新增文件夹功能的业务逻辑

PageObject 目录下新建 py 文件：PageObject/p03_http_gjxt/api_file_page.py

```

import re

from Base.baseAutoHttp import ApiBase
from Base.baseLogger import Logger
from ExtTools.dbbase import MysqlHelp
from PageObject.p03_http_gjxt.api_login_page import LoginPage

logger = Logger("api_file_page.py").getLogger()

class ApiFile(ApiBase):

    def __init__(self):
        super().__init__("03文件上传下载接口信息")

    def add_folder(self, name, desc):
        """文件夹新增"""
        change_data = {
    
```

```

        "name": name,
        "description": desc
    }
    res = self.request_base("add_foler", change_data=change_data)
    return res.text

def select_folder(self):
    """文件夹查询"""
    res = self.request_base("select_folder")
    re_info = re.findall('2Fdocument_library%2Fview%20_folderId=(.*?)">(.*?)</a>', res.text)
    return re_info

def assert_add_folder(self, name):
    """文件夹新增页面断言"""
    re_info = self.select_folder()[0]
    assert name in re_info, "【断言】验证文件夹新增失败！"
    logger.info("【断言】验证文件夹新增成功！")

def assert_add_database(self, name, desc):
    """文件夹新增数据库断言"""
    dbInfo = self.config['数据库连接配置']
    db = MysqlHelp(host=dbInfo['host'], user=dbInfo['user'], passwd=dbInfo['passwd'],
                    port=dbInfo['port'], database=dbInfo['database'])
    res = db.mysql_db_select("SELECT name,description FROM dlfolder ORDER BY createDate
DESC LIMIT 1;")[0]
    assert res[0] == name, "【断言】数据库验证文件夹新增失败！"
    assert res[1] == desc, "【断言】数据库验证文件夹新增失败！"
    logger.info("【断言】数据库验证文件夹新增成功！")

def delete_folder(self, name):
    """删除文件夹"""
    res = self.select_folder()
    print(res)
    id = None
    for i in res:
        if name in i:
            id = i[0]
    change_data = {
        "folderId": id
    }
    res = self.request_base("delete_folder", change_data=change_data)
    return res.text

def assert_delete_folder(self, name):
    """删除文件夹断言"""
    res = self.select_folder()
    assert name not in str(res), "【断言】验证文件夹删除失败！"
    logger.info("【断言】验证文件夹删除成功！")

if __name__ == '__main__':
    l = LoginPage()
    l.login("test01", "1111")
    af = ApiFile()

```



```
af.assert_delete_folder('111')
```

步骤三：编写测试用例

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/06文件夹新增和删除.yaml

```
- name: "文件夹名称"
  desc: "文件夹描述"
```

测试用例：

TestSuits 目录下修改 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```
class TestApiCase3():
    """接口自动化-稿件管理系统-文件上传下载功能模块"""

    @pytest.mark.usefixtures('init_login')
    @pytest.mark.parametrize('case_data', DataDriver().get_case_data("06文件夹新增和删除"))
    def test_file_case01(self, case_data):
        """接口自动化用例-文件夹新增与删除"""
        af = ApiFile()
        af.add_folder(case_data['name'], case_data['desc'])
        af.assert_add_folder(case_data['name'])
        af.assert_add_database(case_data['name'], case_data['desc'])
        af.delete_folder(case_data['name'])
        af.assert_delete_folder(case_data['name'])
```

13.8 【接口自动化项目实践】项目案例实战（七）

一、学习目标

1.文件上传功能接口自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
文档上传	选择文件夹，上传	一级	文件夹已存在	1.选择一个文件夹并点击进入 2.点击上传文档按钮 3.选择文档路径 4.文档重命名 5.输入说明 6.点击上传文档按钮	1.页面：文件命名显示正确 2.数据库：dlfileentry 表，title为文件重命名，description为文件说明

- 项目实践：

步骤一：定位文件上传功能的元素信息

DataElement 目录下修改 yaml 文件：Data/DataElement/p03_http_gjxt/03文件上传下载接口信息.yaml

```
file_upload:
  method: post
  url: /group/10779/upload
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    p_p_id: "20"
    p_p_lifecycle: "1"
    p_p_state: "pop-up"
    p_p_mode: "view"
    _20_struts_action: "/document_library/edit_file_entry"
  data:
    _20_cmd: "add"
    _20_redirect: "/group/10779/upload?
p_p_id=20&p_p_lifecycle=0&p_p_state=normal&p_p_mode=view&p_p_col_id=column-
1&p_p_col_count=1&_20_struts_action=%2Fdocument_library%2Fview&_20_folderId=${folderId}"
    _20_referringPortletResource: ""
    _20_uploadProgressId: "dlFileEntryUploadProgress"
    _20_folderId: "${folderId}"
    _20_newFolderId: ""
    _20_name: ""
    _20_title: "${title}"
    _20_description: "${description}"
```

步骤二：封装文件上传功能的业务逻辑

PageObject 目录下修改 py 文件：PageObject/p03_http_gjxt/api_file_page.py

```
def upload_file(self, rename, desc):
    """文件上传"""
    folder_id = self.select_folder()[0][0]
    change_data = {
        "title": rename,
        "description": desc,
        "folderId": folder_id
    }
    file_path = os.path.join(BasePath.DATA_TEMP_DIR, "upload_file.txt")
    files = {
        "_20_file": ("upload_file.txt", open(file_path, 'r', encoding='utf-8'),
"text/plain")
    }
    res = self.request_base('file_upload', change_data=change_data, files=files)
    return res.text

def assert_upload_page(self, res, rename):
    """文件上传页面验证"""
    name = re.findall('id="_20_title" name="_20_title" style="width: 350px; " type="text"
value="(.*?)"', res)[0]
    assert name == rename, "【断言】验证文件上传页面验证失败！"
    logger.info("【断言】验证文件上传页面验证成功！")

def assert_upload_database(self, rename, desc):
```

```

        """文件上传数据库断言"""
        dbInfo = self.config['数据库连接配置']
        db = MysqlHelp(host=dbInfo['host'],user=dbInfo['user'],passwd=dbInfo['passwd'],
                        port=dbInfo['port'],database=dbInfo['database'])
        res = db.mysql_db_select("SELECT title,description FROM dlfileentry ORDER BY
createDate DESC LIMIT 1;")[0]
        assert res[0] == rename.split('.')[0], "【断言】验证文件上传数据库验证失败！"
        assert res[1] == desc, "【断言】验证文件上传数据库验证失败！"
        logger.info("【断言】验证文件上传数据库验证成功！")
    
```

步骤三：编写测试用例

测试前置：

```

@pytest.fixture(scope="function")
def add_delete_folder():
    """文件夹新增与删除"""
    lp = LoginPage()
    lp.login("test01", "1111")
    af = ApiFile()
    af.add_folder("文件夹名称", "文件夹描述")
    yield
    af.delete_folder("文件夹名称")
    
```

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/07文件上传.yaml

```

- rename: "rename.txt"
  desc: "这是上传描述"
    
```

测试用例：

TestSuits 目录下修改 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```

@pytest.mark.usefixtures("add_delete_folder")
@pytest.mark.parametrize('case_data', DataDriver().get_case_data("07文件上传"))
def test_file_case02(self, case_data):
    """接口自动化用例-文件上传"""
    af = ApiFile()
    res = af.upload_file(case_data['rename'], case_data['desc'])
    af.assert_upload_page(res, case_data['rename'])
    af.assert_upload_database(case_data['rename'], case_data['desc'])
    
```

13.9 【接口自动化项目实践】项目案例实战（八）

一、学习目标

1.文件下载功能接口自动化实战

二、知识点

- 需求（功能测试用例）

测试点	用例名称	级别	预置条件	测试步骤	预期结果
文档下载	选择文档，下载	一级	文档已上传	1.选择一个文档并点击2.点击下载保存	文档下载成功，文件夹内文件存在

• 项目实践：

步骤一：抓取文件下载功能的接口信息

DataElement 目录下修改 yaml 文件：Data/DataElement/p03_http_gjxt/03文件上传下载接口信息.yaml

```
select_file:
  method: get
  url: /group/10779/upload
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    p_p_id: "20"
    p_p_lifecycle: "0"
    p_p_state: "maximized"
    p_p_mode: "view"
    p_p_col_id: "column-1"
    p_p_col_count: "1"
    _20_struts_action: "/document_library/search"
    _20_redirect: "/group/10779/upload?"
    p_p_id=20&p_p_lifecycle=0&p_p_state=normal&p_p_mode=view&p_p_col_id=column-1&p_p_col_count=1&_20_struts_action=%2Fdocument_library%2Fview&_20_folderId=${FolderId}"
    _20_breadcrumbsFolderId: "${FolderId}"
    _20_searchFolderIds: "${FolderId}"
    _20_tabs1TabsScroll: ""
    _20_keywords: "${keywords}"

down_file:
  method: get
  url: /group/10779/upload
  headers:
    User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko
  params:
    p_p_id: "20"
    p_p_lifecycle: "1"
    p_p_state: "exclusive"
    p_p_mode: "view"
    _20_struts_action: "/document_library/get_file"
    _20_folderId: "${FolderId}"
    _20_name: "${name}"
```

步骤二：封装文件下载功能的业务逻辑

PageObject 目录下修改 py 文件：PageObject/p03_http_gjxt/api_file_page.py

```
def select_file(self, rename):
    """文件查询"""
    folder_id = self.select_folder()[0][0]
    change_data = {
```

```

        "keywords": rename.split('.')[0],
        "FolderId": folder_id
    }
    res = self.request_base("select_file", change_data=change_data)
    return res.text

def download_file(self, res, rename):
    """文档下载"""
    res = re.findall('&_20_folderId=(.*?)&_20_name=(.*?)>', res)[0]
    change_data = {
        "FolderId": res[0],
        "name": res[1]
    }
    res = self.request_base("down_file", change_data=change_data)
    file_path = os.path.join(BasePath.DATA_TEMP_DIR, rename)
    with open(file_path, 'wb') as f:
        f.write(res.content)

def assert_download_file(self, rename):
    """下载断言"""
    file_path = os.path.join(BasePath.DATA_TEMP_DIR, rename)
    assert os.path.exists(file_path), "【断言】文档下载失败！"
    logger.info("【断言】文档下载成功！")

```

步骤三：编写测试用例

数据驱动：

DataDriver 目录下新建 yaml 文件：Data/DataDriver/YamlDriver/p03_http_gjxt/08文件下载.yaml

```

- rename: "rename.txt"
  desc: "文件下载描述"

```

测试用例：

TestSuits 目录下修改 py 文件：TestSuits/p03_http_gjxt/test_http_gjxt.py

```

@pytest.mark.usefixtures("add_delete_folder")
@pytest.mark.parametrize('case_data', DataDriver().get_case_data("08文件下载"))
def test_file_case03(self, case_data):
    """接口自动化用例-文件下载"""
    af = ApiFile()
    af.upload_file(case_data['rename'], case_data['desc'])
    res = af.select_file(case_data['rename'])
    af.download_file(res, case_data['rename'])
    af.assert_download_file(case_data['rename'])

```

第十四章 AI自动化测试：基于yolov5深度学习模型的ai自动化测试实战

14.1 【AI自动化测试】yolov5 深度学习算法介绍

一、学习目标

1. YOLOV5 目标检测算法介绍
2. 基于 YOLOV5 自动化测试实现过程

二、知识点

1. 【YOLOV5目标检测算法介绍】

我们本章节课程的目的是为了通过 yolov5 目标检测算法来实现自动化测试，我们重点是如何应用yolov5,所以不需要深究 yolov5算法原理。

- YOLOV5 的优势
 1. 使用 PyTorch 进行编写。
 2. 速度极快，每秒 140FPS。
 3. 精度超高。
 4. 体积很小。
 5. 集成了 YOLOv3-spp 和 YOLOv4 部分特性。

- YOLOV5 的网络模型

yolov5 官方给出了几个常用的模型，他们的大小和识别精度各有不同：

Model	size (pixels)	mAP ^{val} 0.5:0.95	mAP ^{val} 0.5	Speed CPU b1 (ms)	Speed V100 b1 (ms)	Speed V100 b32 (ms)	params (M)	FLOPs @640 (B)
YOLOv5n	640	28.0	45.7	45	6.3	0.6	1.9	4.5
YOLOv5s	640	37.4	56.8	98	6.4	0.9	7.2	16.5
YOLOv5m	640	45.4	64.1	224	8.2	1.7	21.2	49.0
YOLOv5l	640	49.0	67.3	430	10.1	2.7	46.5	109.1
YOLOv5x	640	50.7	68.9	766	12.1	4.8	86.7	205.7

YOLOv5 的官方代码中，给出了几种版本的目标检测网络，分别是 YOLOv5n、YOLOv5s、YOLOv5m、YOLOv5l、YOLOv5x 这几种模型。

这几种模型文件的大小逐渐增加，精度逐渐提高，但识别速度逐渐变慢，其中 YOLOv5n 模型速度最快，但精度也是最低的。

- YOLOV5 的源码结构

github 源码：<https://github.com/ultralytics/YOLOv5>

 .github	Temporarily remove <code>macos-latest</code> from CI (#9049)
 classify	Default <code>--data imagenette160</code> training (fastest) (#9033)
 data	VOC <code>names</code> dictionary fix (#9034)
 models	<code>torch.empty()</code> for speed improvements (#9025)
 utils	Daemon <code>plot_labels()</code> for faster start (#9057)
 .dockerignore	Add <code>.git</code> to <code>.dockerignore</code> (#8815)
 .gitattributes	git attrib
 .gitignore	Ignore <code>*_opencv_model/</code> dir (#6180)
 .pre-commit-config.yaml	[pre-commit.ci] pre-commit suggestions (#8828)
 CONTRIBUTING.md	Add <code>mdformat</code> to precommit checks and update other version (#7529)
 LICENSE	Add pre-commit CI actions (#4982)
 README.md	Update <code>attempt_download(release='v6.2')</code> (#8998)
 detect.py	Improved <code>Profile()</code> inference timing (#9024)
 export.py	Refactor for simplification 2 (#9055)
 hubconf.py	Add PyTorch Hub classification CI checks (#9027)
 requirements.txt	ClearML experiment tracking integration (#8620)
 setup.cfg	Update <code>setup.cfg</code> to <code>description_file</code> field (#7248)
 train.py	Daemon <code>plot_labels()</code> for faster start (#9057)
 tutorial.ipynb	Created using Colaboratory
 val.py	Add <code>--save-hybrid</code> mAP warning (#9050)

作为小白而言，主要先关注两个文件，首先是 `train.py` 和 `detect.py` 文件。`train.py` 是训练自己的数据集用的，`detect.py` 是用来识别用的；使用 `train.py` 提前训练好的数据权重文件（`.pt` 文件），然后通过调用 `detect.py` 提供的命令进行图像的识别并标注。

2. 【基于YOLOV5 自动化测试实现过程】

到这里有的小伙伴可能会问，通过 `pyautogui` 不也能通过图像识别实现自动化测试吗？其实 `pyautogui` 的图像识别是针对图片的像素点进行匹配的，那么这将需要运行环境的分辨率必须一致，图片的一致度要求也很苛刻，背景色的改变也会造成识别失败。那么就需要有一种不需依赖分辨率的图像识别技术，通过AI图像识别技术就不需要考虑分辨率的问题。

AI自动化测试的实现过程：

1. `Anconda` 环境搭建
2. `YOLOV5` 图像训练 `.pt`
3. `YOLOV5` 图像识别及模拟操作

后面的课程中将针对每个步骤做详细介绍。

14.2 【AI自动化测试】 windows 系统 yolo v5 环境部署

一、学习目标

1. 安装 Anaconda
2. 安装 pytorch
3. 安装 yolo v5 依赖包
4. 安装 nvidia 显卡驱动
5. 安装 cuda
6. 安装 cuDNN

二、知识点

1. 【安装Anaconda】

Anaconda 包括 Conda、Python 以及一大堆安装好的工具包，比如：numpy、pandas 等，支持 Linux, Mac, Windows, 包含了众多流行的科学计算、数据分析的 Python 包。

- 一、下载安装包

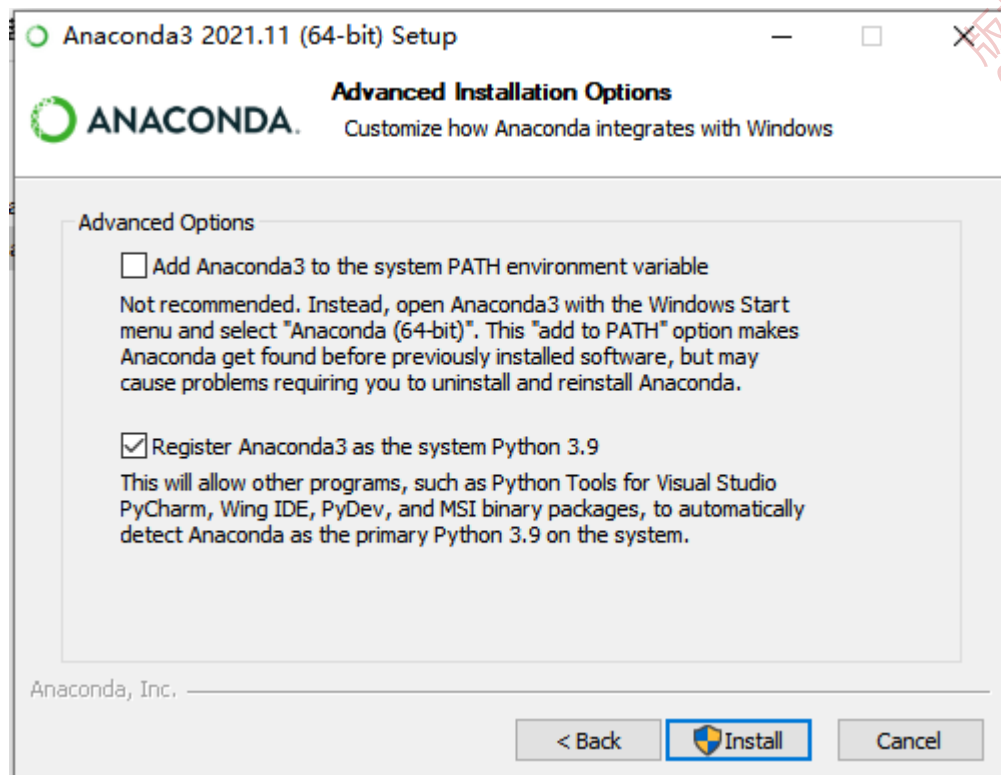
Anaconda 下载 Windows 版: <https://www.anaconda.com/products/individual>



我们下载最新的64位版本的Anaconda即可。

- 二、然后安装anaconda

双击exe执行程序，选择D盘目录安装（尽量别装在C盘）。



• 三、添加Anaconda国内镜像配置

清华TUNA提供了 Anaconda 仓库的镜像，运行以下命令：

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
conda config --set show_channel_urls yes
```

另外为了保险起见，建议同时添加第三方conda源：

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/msys2/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/bioconda/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/menpo/
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/pytorch/
```

2. 【安装 pytorch】

• 查看cuda版本

CUDA 是显卡运算平台。只有安装这个框架才能够进行复杂的并行计算。主流的深度学习框架也都是基于 CUDA 进行 GPU 并行加速的，几乎无一例外。还有一个叫做 cudnn，是针对深度卷积神经网络的加速库。

也就是说如果电脑没有独立显卡，则没有 CUDA，那么就只能使用 CPU 进行运算。

```
nvidia-smi
```

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.19043.1348]
(c) Microsoft Corporation。保留所有权利。

C:\Users\Admin>nvidia-smi
Sat Dec 18 16:55:06 2021

+-----+
| NVIDIA-SMI 457.51                Driver Version: 457.51          CUDA Version: 11.1     |
+-----+-----+
| GPU Name               TCC/WDDM | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                               |                    |    MIG M. |
+-----+-----+
| 0  GeForce GTX 1650   WDDM      | 00000000:01:00.0 On  |          N/A         |
| 35%   28C    P8      N/A / 75W | 617MiB / 4096MiB |      2%    Default  |
|                               |                    |    N/A         |
+-----+-----+

Processes:
+-----+
| GPU  GI  CI           PID  Type  Process name                        GPU Memory |
|   ID  ID  ID                    |                    |      Usage |
+-----+
|  0   N/A N/A         1208   C+G   Insufficient Permissions            N/A        |
|  0   N/A N/A         1252   C+G   ...lPanel\SystemSettings.exe       N/A        |
|  0   N/A N/A         1388   C+G   C:\Windows\explorer.exe            N/A        |
|  0   N/A N/A         1864   C+G   ...me\Application\chrome.exe       N/A        |
|  0   N/A N/A         2116   C+G   ...y\ShellExperienceHost.exe       N/A        |
|  0   N/A N/A         7120   C+G   ...artMenuExperienceHost.exe       N/A        |
|  0   N/A N/A         7488   C+G   ...5nlh2txyewy\SearchApp.exe       N/A        |
|  0   N/A N/A         8168   C+G   ...nputApp\TextInputHost.exe       N/A        |
|  0   N/A N/A         9120   C+G   ...wekyb3d8bbwe\Video.UI.exe       N/A        |
|  0   N/A N/A         9496   C+G   D:\Typora\Typora\Typora.exe        N/A        |
|  0   N/A N/A         9608   C+G   ...ekyb3d8bbwe\YourPhone.exe       N/A        |
|  0   N/A N/A        11372   C+G   ...e\vmaz\x64\vmware-vmx.exe       N/A        |
+-----+

C:\Users\Admin>

```

通过命令可以看到我的cuda版本是11.5。

• 在线安装：

◦ 创建环境

注意：需要安装 pytorch 1.7 以上的版本 创建虚拟环境，环境名字可自己确定，这里本人使用 pytorch1.9 作为环境名：

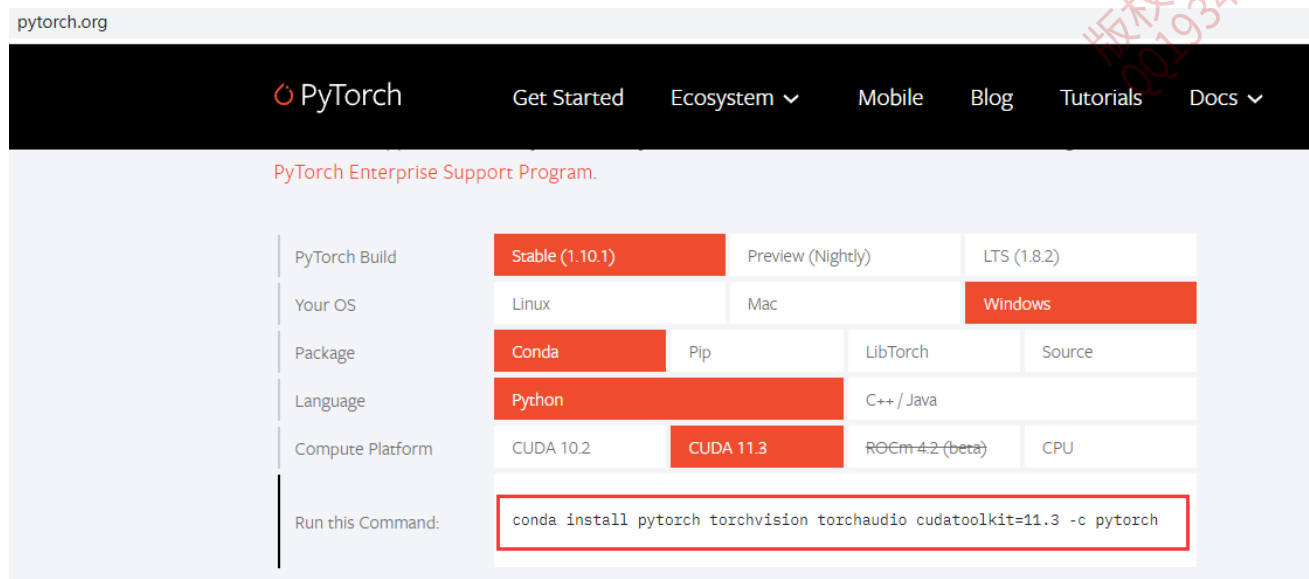
```
conda create -n pytorch1.9 python=3.9
```

◦ 激活环境

安装成功后激活pytorch1.8环境：

```
conda activate pytorch1.9
```

◦ 安装pytorch



在所创建的 `pytorch` 环境下安装 `pytorch` 的 1.9 版本, 执行命令:

但是这里一定要注意, 去掉 `-c pytorch`, 安装的时候才会默认从清华源下载相应的包, 因此这里用命令:

```
conda install pytorch==1.9.0 torchvision==0.9.0 torchaudio==0.8.0 cudatoolkit=11.1
```

- 离线安装:

下载网址: <https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/pytorch/win-64/>

版本: `pytorch-1.9.1-py3.9_cuda11.1_cudnn8_0.tar.bz2` (有独显)

或版本: `pytorch-1.9.1-py3.9_cpu_0.tar.bz2` (无独显)

Anaconda 的安装包存放在安装目录下的 `pkgs/` 目录下, 删除之前因为下载失败的安装包。

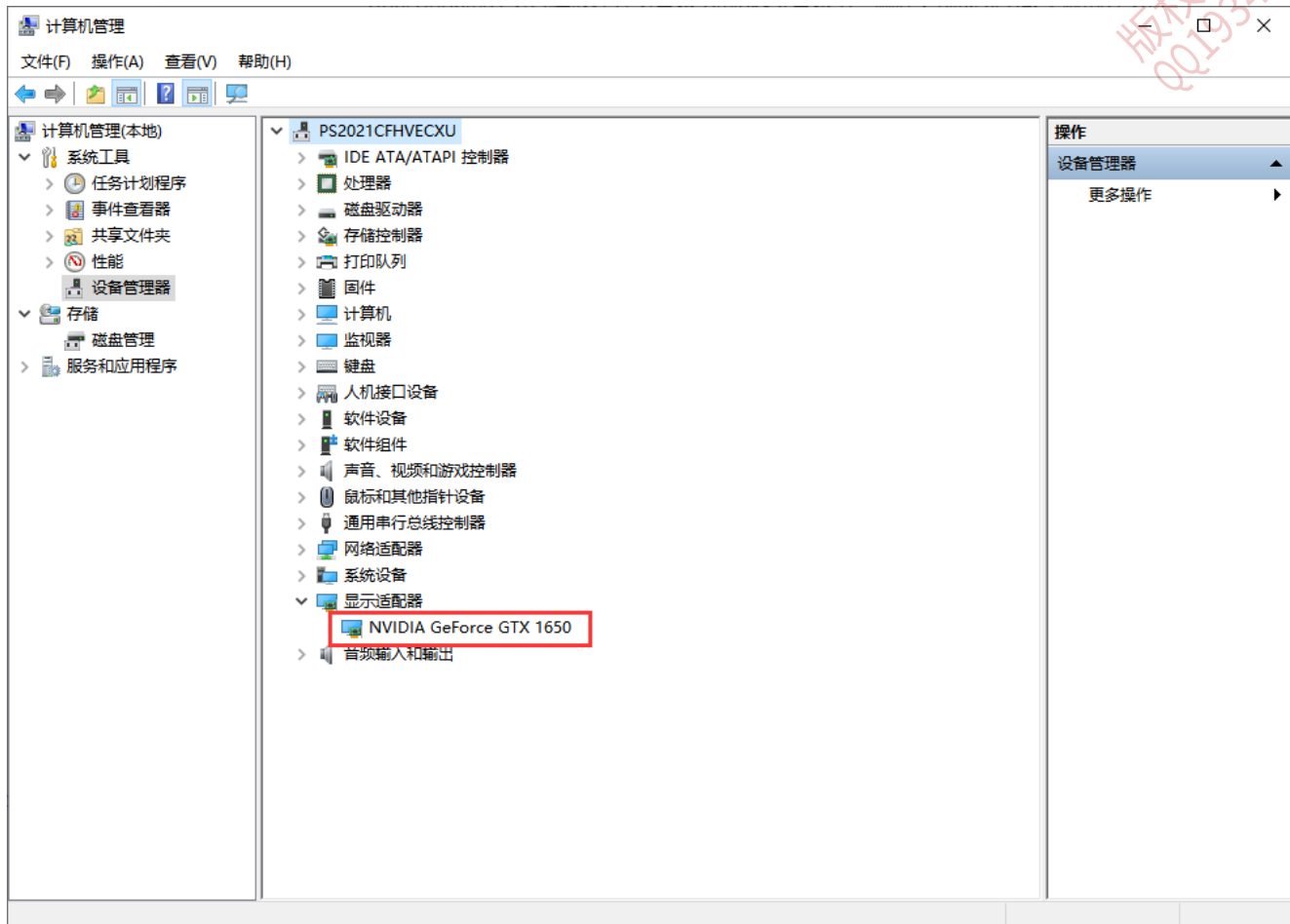
```
conda install --use-local pytorch-1.9.0-py3.9_cuda11.1_cudnn8_0.tar.bz2
```

3. 【安装yolov5 依赖包】

```
cd yolov5-master
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple -r requirements.txt
```

4. 【安装nvidia显卡驱动】 (无nvidia显卡忽略此步)

首先要在设备管理器中查看你的显卡型号, 比如在这里可以看到我的显卡型号为 GTX 1650。



NVIDIA 驱动下载: <https://www.nvidia.cn/Download/index.aspx?lang=cn>

驱动程序下载

NVIDIA > 驱动程序下载



**NVIDIA 深度学习
培训中心 (DLI) 公开课**

立即报名

NVIDIA 驱动程序下载

在下方的下拉列表中进行选择，针对您的 NVIDIA 产品确定合适的驱动。

产品类型: GeForce

产品系列: GeForce GTX 16 Series (Notebooks)

产品家族: GeForce GTX 1650

操作系统: Windows 10 64-bit

下载类型: Studio 驱动程序 (SD)

语言: Chinese (Simplified)

搜索

5.【安装 cuda】（无 nvidia 显卡忽略此步）

- 下载:

cuda 下载链接: https://developer.nvidia.cn/cuda-downloads?target_os=Windows&target_arch=x86_64&target_version=11&target_type=exe_local

Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [CUDA EULA](#).

Operating System	Linux	Windows			
Architecture	x86_64				
Version	10	11	Server 2016	Server 2019	Server 2022
Installer Type	exe (local)	exe (network)			

Download Installer for Windows 11 x86_64

The base installer is available for download below.

> Base Installer Installation Instructions: 1. Double click cuda_11.5.1_496.13_windows.exe 2. Follow on-screen prompts	Download [2.4 GB]
--	--------------------------

• 安装：





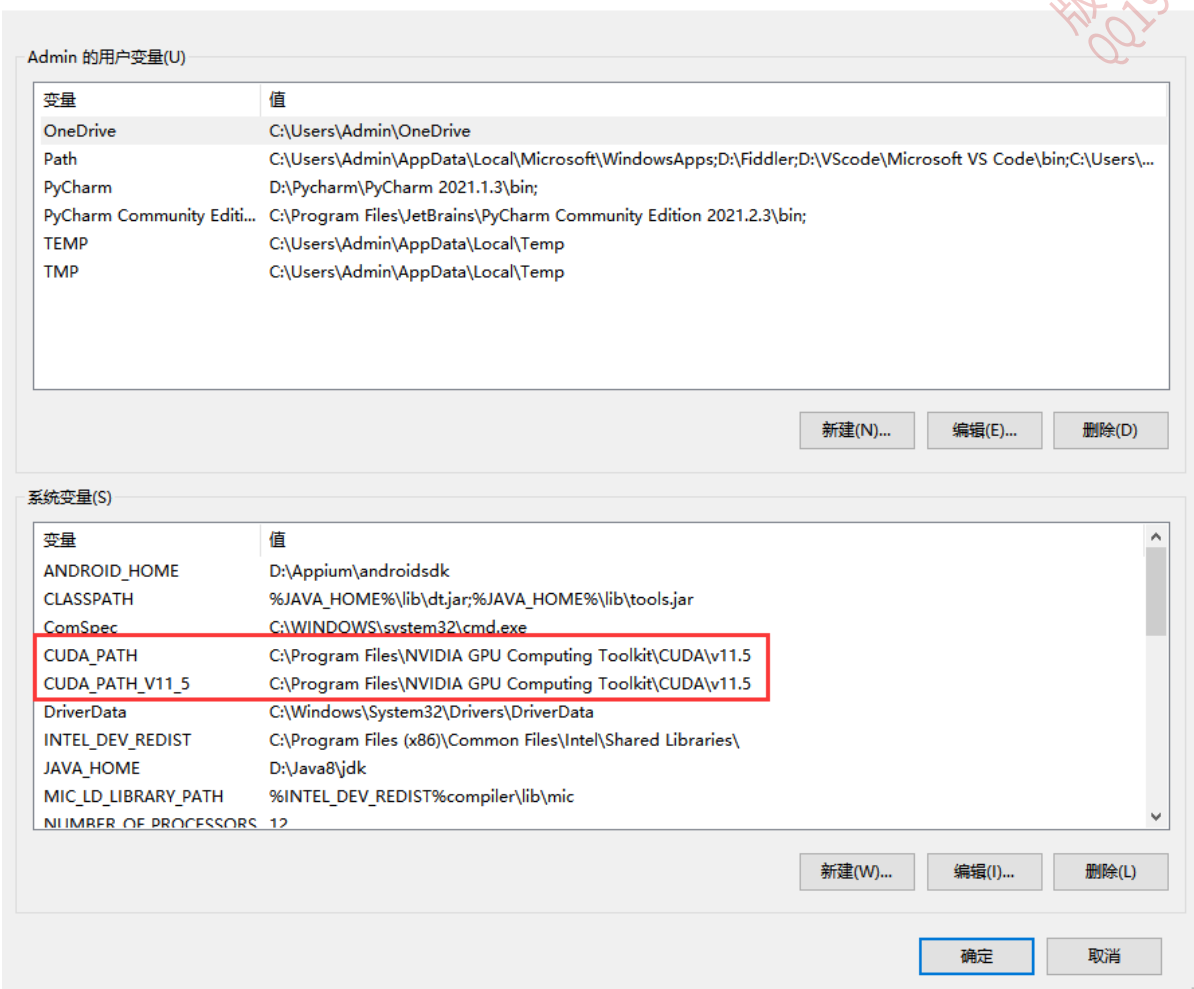
注：如果 visual studio inegration 安装失败，请先安装 vscode2017。

- 设置环境变量

(1) 计算机上点右键，打开属性->高级系统设置->环境变量，可以看到系统中多了 `CUDA_PATH` 和 `CUDA_PATH_V11_5` 两个环境变量。

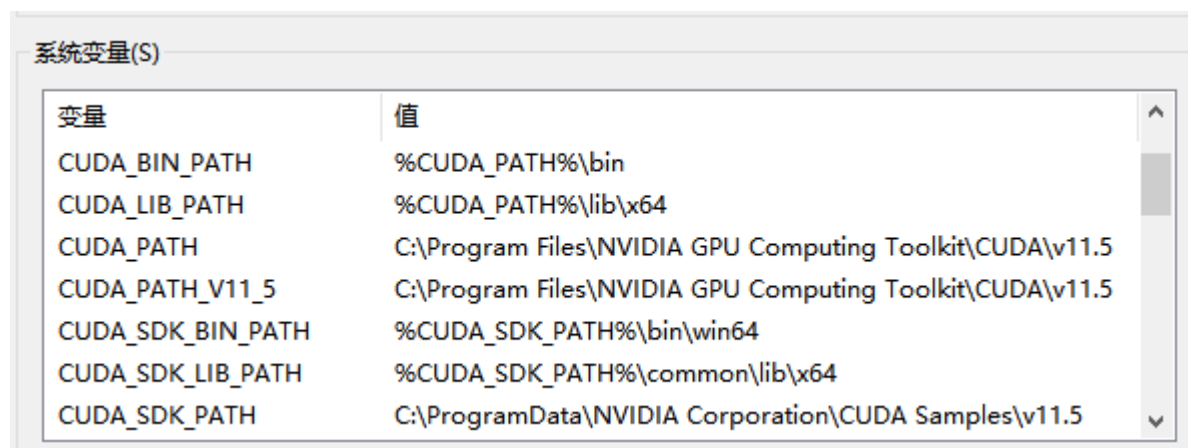
```
CUDA_PATH = C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.5
CUDA_PATH_V11_5 = C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.5
```

环境变量



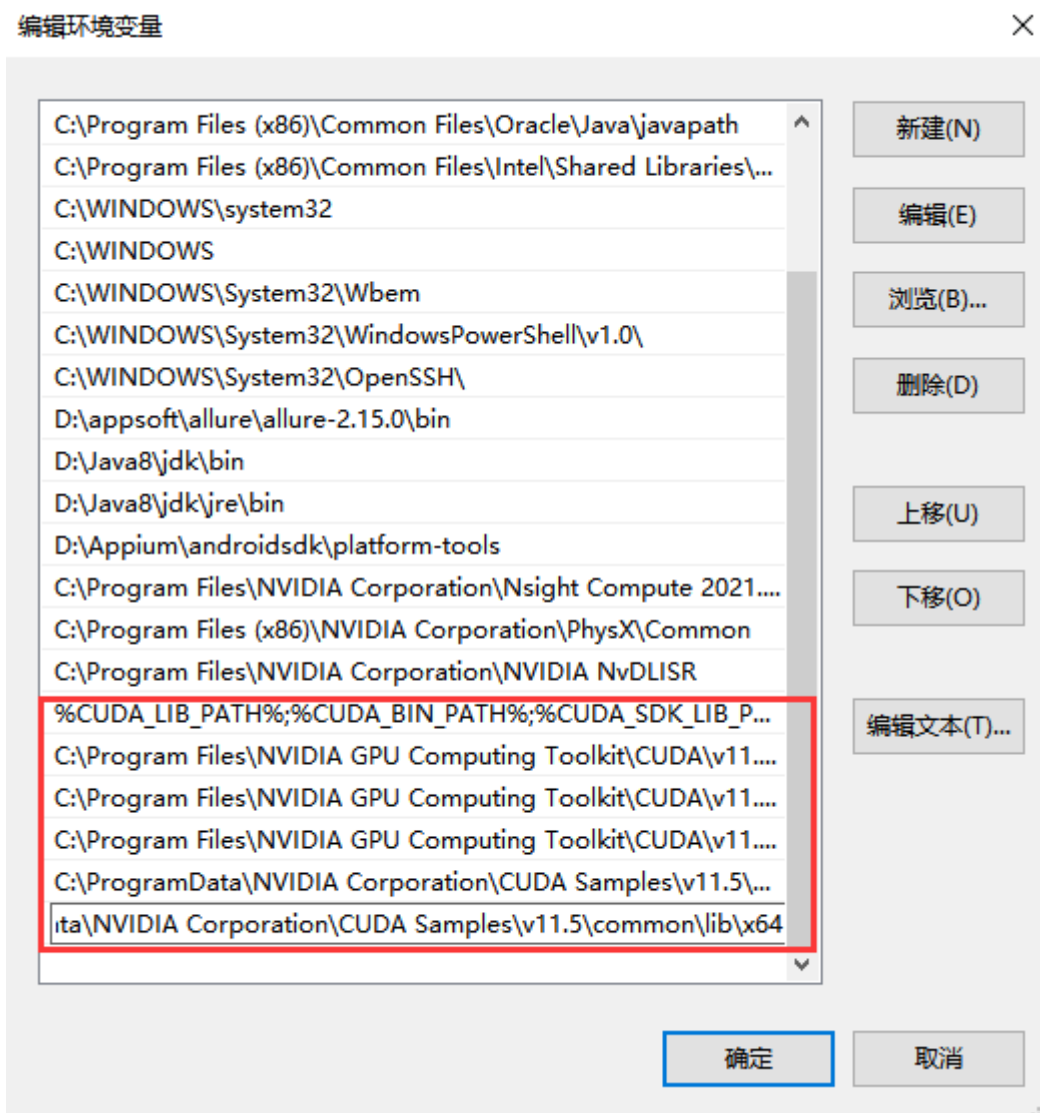
(2) 接下来，还要在系统中添加以下几个环境变量：

```
CUDA_SDK_PATH = C:\ProgramData\NVIDIA Corporation\CUDA Samples\v11.5
CUDA_LIB_PATH = %CUDA_PATH%\lib\x64
CUDA_BIN_PATH = %CUDA_PATH%\bin
CUDA_SDK_BIN_PATH = %CUDA_SDK_PATH%\bin\win64
CUDA_SDK_LIB_PATH = %CUDA_SDK_PATH%\common\lib\x64
```



(3) 在系统变量 Path中添加：

```
%CUDA_LIB_PATH%;%CUDA_BIN_PATH%;%CUDA_SDK_LIB_PATH%;%CUDA_SDK_BIN_PATH%;
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.5\lib\x64
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.5\include
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.5\extras\CUPTI\lib64
C:\ProgramData\NVIDIA Corporation\CUDA Samples\v11.5\bin\win64
C:\ProgramData\NVIDIA Corporation\CUDA Samples\v11.5\common\lib\x64
```



- 最后测试 cuda 是否配置成功：

```
nvcc -V #打开CMD执行
```

6. 【安装 cudnn】（无 nvidia 显卡忽略此步）

- 下载：

cudnn 下载地址：<https://developer.nvidia.com/rdp/cudnn-archive#a-collapse742-10>

cuDNN Download

NVIDIA cuDNN is a GPU-accelerated library of primitives for deep neural networks.

☒ I Agree To the Terms of the [cuDNN Software License Agreement](#)

Note: Please refer to the [Installation Guide](#) for release prerequisites, including supported GPU architectures and compute capabilities, before downloading. For more information, refer to the cuDNN Developer Guide, Installation Guide and Release Notes on the [Deep Learning SDK Documentation](#) web page.

[Download cuDNN v8.3.1 \[November 22nd, 2021\], for CUDA 11.5](#)

Local Installers for Windows and Linux, Ubuntu(x86_64, armsbsa)

[Local Installer for Windows \(Zip\)](#)

[Local Installer for Linux x86_64 \(Tar\)](#)

[Local Installer for Linux PPC \(Tar\)](#)

[Local Installer for Linux SBSA \(Tar\)](#)

[Local Installer for Ubuntu18.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu18.04 aarch64sbsa \(Deb\)](#)

[Local Installer for Ubuntu18.04 cross-sbsa \(Deb\)](#)

[Local Installer for Ubuntu20.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu20.04 aarch64sbsa \(Deb\)](#)

[Local Installer for Ubuntu20.04 cross-sbsa \(Deb\)](#)

• 安装:

对于 cudnn 直接将其解开压缩包，然后需要将bin,include,lib中的文件复制粘贴到 cuda 的文件夹下:

```
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.5
```

14.3 【AI自动化测试】Ubuntu 系统 yolov5 环境部署

一、学习目标

- 1.安装 Anaconda
- 2.安装 pytorch
- 3.安装 yolov5 依赖包

二、知识点

我的虚拟机里没有独立显卡，所以就不需要考虑 CUDA 环境的安装，用系统自身的 CPU 运算。

1. 【安装Anaconda】

Anaconda 是一个用于科学计算的 Python 发行版，支持 Linux, Mac, Windows, 包含了众多流行的科学计算、数据分析的 Python 包。

- (1) 先去官方地址下载好对应的安装包 下载地址:<https://www.anaconda.com/products/distribution>
- (2) 然后安装anaconda

```
bash Anaconda3-2022.05-Linux-x86_64.sh
```

anaconda会自动将环境变量添加到PATH里面，如果后面你发现输出 conda 提示没有该命令，那么你需要执行命令 `source ~/.bashrc` 更新环境变量，就可以正常使用了。如果发现这样还是没用，那么需要添加环境变量。

编辑 `~/.bashrc` 文件，在最后面加上：

```
export PATH=/home/lcy/anaconda3/bin:$PATH
```

注意：路径应改为自己机器上的路径

保存退出后执行：`source ~/.bashrc` 再次输入 `conda list` 测试看看，应该没有问题。

(3) 添加 Anaconda 国内镜像配置

清华TUNA提供了 Anaconda 仓库的镜像，运行以下三个命令：

```
conda config --add channels http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
conda config --add channels http://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
conda config --set show_channel_urls yes
```

恢复默认镜像地址：

```
conda config --remove-key channels
```

2. 【安装pytorch】

(1) 首先为 pytorch 创建一个 anaconda 虚拟环境，环境名字可自己确定，这里本人使用 `pytorch1.9` 作为环境名：

```
conda create -n pytorch1.9 python=3.9
```

(2) 安装成功后激活 `pytorch1.9` 环境：

```
source activate          #进入anconda
source deactivate        #退出anconda
conda activate pytorch1.9 #激活创建的环境
```

(3) 在所创建的 `pytorch` 环境下安装 `pytorch` 的1.9版本, 执行命令：

```
conda install --use-local ./pytorch-1.9.1-py3.9_cpu_0.tar.bz2
```

如果上面报错：`specified in the package manifest cannot be found.`，则运行：

```
conda clean --packages --tarballs #删除缓存的包重新下载
```

编辑 `~/.bashrc` 文件，设置使用 `pytorch1.9` 环境下的 `python3.9`：

```
alias python='/home/lcy/anaconda3/envs/pytorch1.9/bin/python3.9'
```

注意：python路径应改为自己机器上的路径

保存退出后执行：`source ~/.bashrc`

该命令将自动回到base环境，再执 `conda activate pytorch1.9` 到 pytorch 环境。

3. 【安装yolov5 依赖包】

```
cd yolov5-master  
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple -r requirements.txt
```

测试：

```
python detect.py --source ./data/images/bus.jpg --weights weights/yolov5s.pt --conf 0.4
```

14.4 【AI自动化测试】制作数据集

一、学习目标

1.制作数据集

二、知识点

1.【制作数据集】

- 安装图像标注工具 labelImg

- (1) 下载图像标注工具

网址：<https://github.com/tzutalin/labelImg>，下载后得到文件 `labelImg-master.zip`，解压：`D:\labelImg-master`

- (2) labelImg 运行环境安装

pytorch1.9 环境激活：

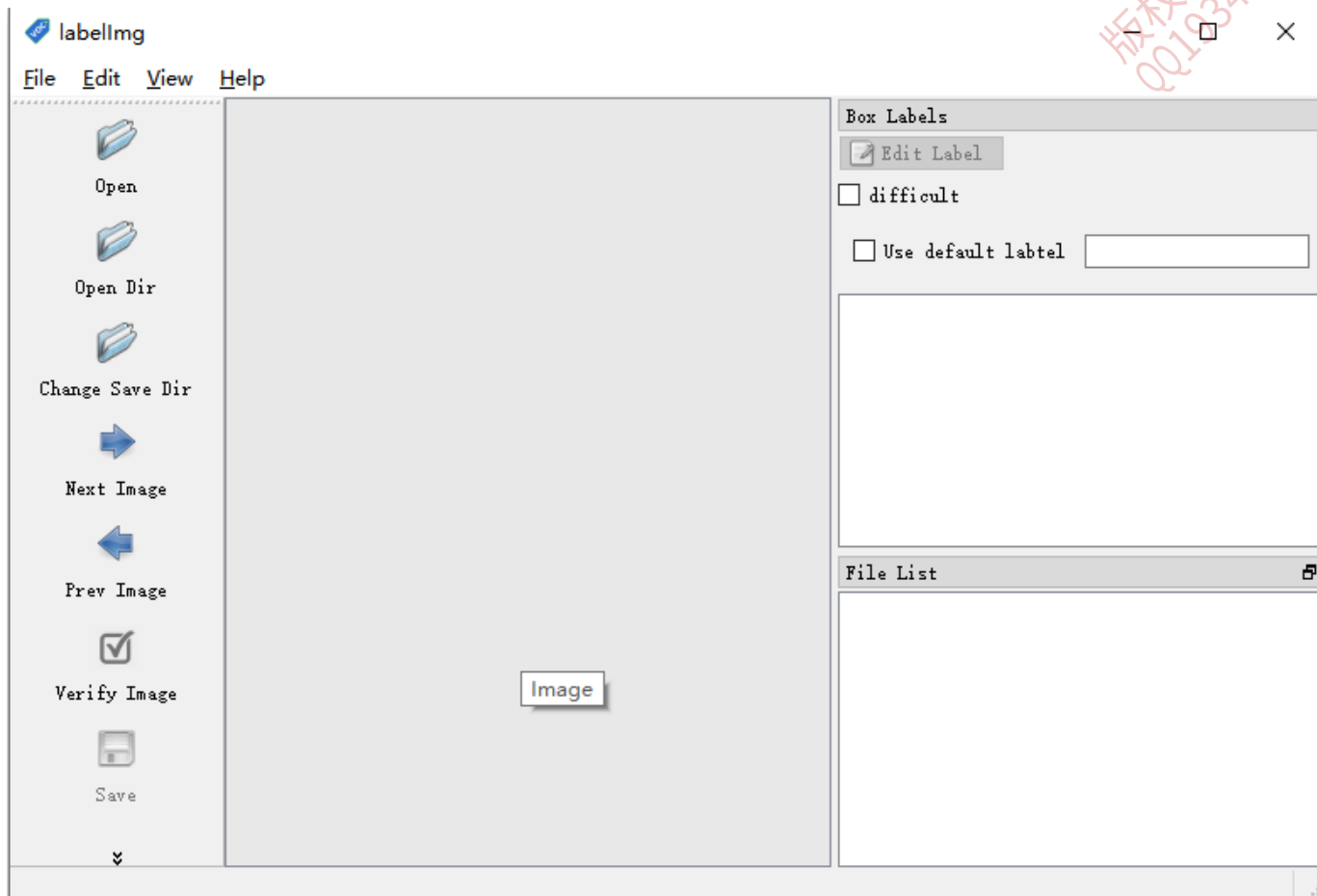
```
conda activate pytorch1.9      #激活创建的环境
```

安装工具依赖库：

```
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple pyqt5  
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple lxml
```

- (3) 运行 labelImg 图像标注工具

```
pyrcc5 -o libs/resources.py resources.qrc  
python labelImg.py
```



- 图像标注

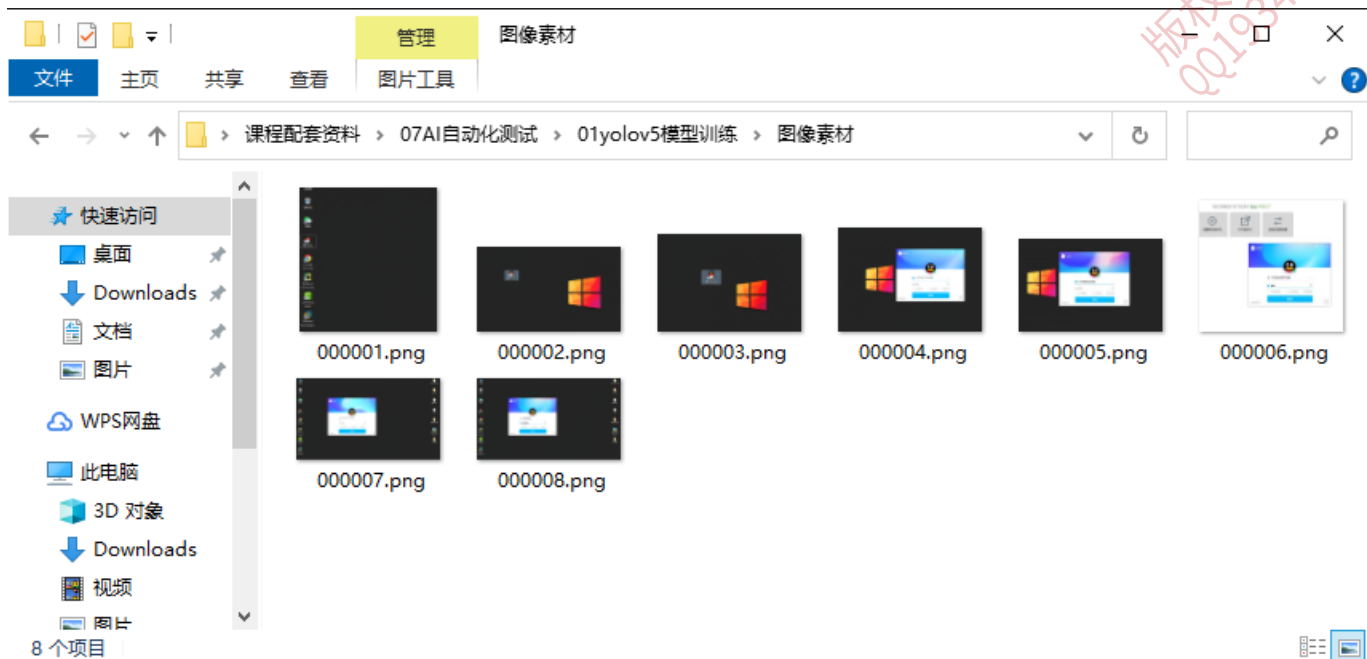
我们本章节演示一下通过 yolov5 的图像识别来自动完成 QQ 登录，所以我们需要训练登录操作的相关按钮。

(1) 修改 labelImg-master\data\predefined_classes.txt

```
qqicon
username
password
loginbtn
```

(2) 图片素材准备（素材尽可能多）

注：图片命名为六位数字，格式为 png，如 000001.png



(3) 构造文件夹目录

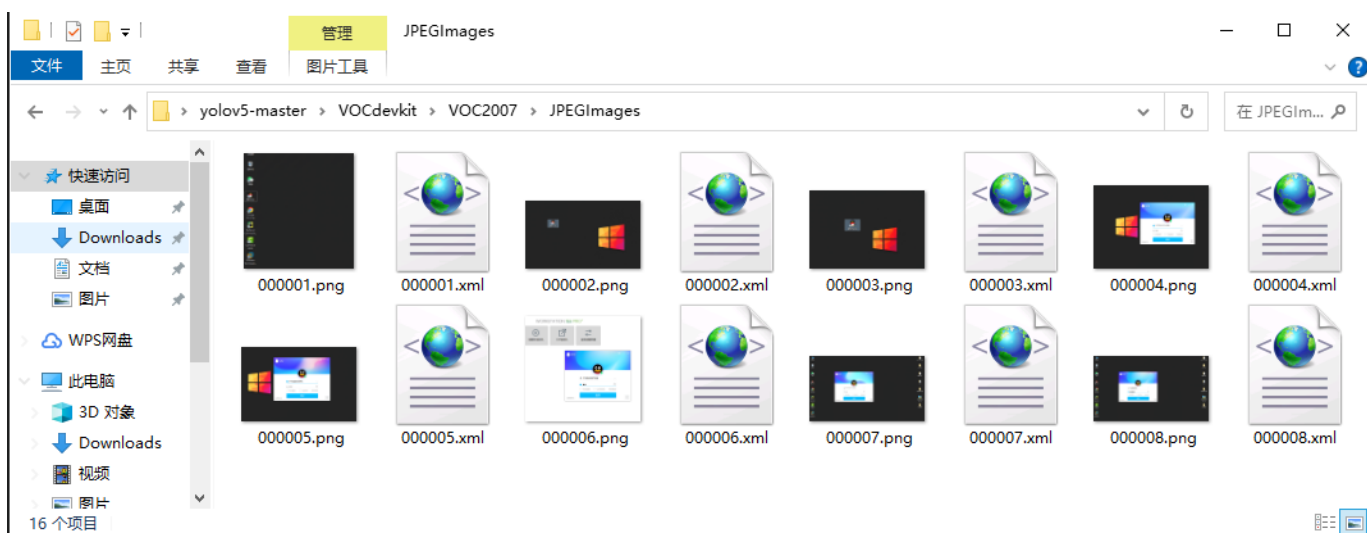
```
VOCdevkit
--VOC2007
--Annotations #用来存放上面标注的xml文件
--JPEGImages #用来存放上面标注的png文件
```

将图像素材放入 JPEGImages 目录，将 VOCdevkit 文件夹放入 yolov5 项目工程的根目录下

(4) 打开图像标注工具 LabelImg 进行按钮类别标注

注：我们在标注之前一定要将图片存放到一个全英文路径下，如果带有中文，在后续图像增强会报错！！

标注完成并保存后，每个图片都会生成一个对应的 xml 文件，用来记录图片的标注信息。



14.5 【AI自动化测试】数据增强

一、学习目标

1.数据增强

三、知识点

1.【数据增强】

📁 > 课程配套资料 > 07AI自动化测试 > 01yolov5模型训练 > Data-Augment-master				
	名称	修改日期	类型	大小
ads	📁 .idea	2022/8/27 19:11	文件夹	
	📁 Annotations	2022/8/27 19:07	文件夹	
	📁 JPEGImages	2022/8/27 19:07	文件夹	
	📁 VOC2007	2022/8/27 19:07	文件夹	
	📄 augmentation.py	2021/1/28 9:48	Python File	8 KB
ads	📄 README.md	2019/4/10 16:20	Markdown File	7 KB

两个文件夹一个是需要增强的图像数据（JPEGImages），一个是对应的xml文件（Annotations）。注意：图像文件名需和xml文件名相对应！

安装依赖包：

```
pip install six numpy scipy matplotliblib scikit-image opencv-python imageio
pip install imgaug
```

(1) 设置文件路径

```
IMG_DIR = "./VOC2007/JPEGImages"
XML_DIR = "./VOC2007/Annotations"

AUG_XML_DIR = "./Annotations" # 存储增强后的XML文件夹路径
try:
    shutil.rmtree(AUG_XML_DIR)
except FileNotFoundError as e:
    a = 1
mkdir(AUG_XML_DIR)

AUG_IMG_DIR = "./JPEGImages" # 存储增强后的影像文件夹路径
try:
    shutil.rmtree(AUG_IMG_DIR)
except FileNotFoundError as e:
    a = 1
mkdir(AUG_IMG_DIR)
```

(2) 设置增强次数

AUGLOOP = 20 # 每张影像增强的数量

(3) 设置增强参数

```
# 图像增强
seq = iaa.Sequential([
    iaa.Flipud(0.5), # 翻转
    iaa.Fliplr(0.5), # 镜像
    iaa.Multiply((1.2, 1.5)), # 改变明亮度
    iaa.GaussianBlur(sigma=(0, 3.0)), # 高斯噪声
    iaa.Affine(
        translate_px={"x": 15, "y": 15},
        scale=(0.8, 0.95),
        rotate=(-30, 30)
    ) # translate by 40/60px on x/y axis, and scale to 50-70%, affects BBs
])
```

(4) 输出

1.分别将图像文件以及 xml 文件放入文件夹：VOC2007/JPEGImages、VOC2007/Annotations

2.运行 augmentation.py，运行结束后即可得到增强的图像和对应的 xml 文件夹

报错：UnicodeDecodeError: 'gbk' codec can't [decode](https://so.csdn.net/so/search?q=decode&spm=1001.2101.3001.7020) byte 0xa2 in position 100: illegal multibyte sequence

解决：将数据集放到英文路径下，如桌面。将我们的 xml 文件中的中文改为英文路径。

14.6 【AI自动化测试】图像训练

一、学习目标

1.图像训练

二、知识点

1.【图像训练】

(1) 生成训练集和测试集文件

这里需要用到我提前准备好的脚本 prepare_data.py，在运行前需要修改这个 py 文件的两个配置：

```
classes=["qqicon","username","password","loginbtn"] # 训练的所有类别
#classes=["qqicon"]

TRAIN_RATIO = 80 # 训练集和测试集划分比例，8：2
```

修改完成后，运行：

```
python prepare_data.py
```

运行完成后会在 yolov5 项目目录下新增文件：

yolov5_train.txt：按比例划分后的训练图片文件的所有路径。

yo1ov5_val.txt：按比例划分后的验证图片文件的所有路径。

vocdevkit/VOC2007/YOLOLabels:是images文件夹下每一个图像的 yo1o 格式的标注文件,这是由annotations的 xml 标注文件转换来的。

在 vocdevkit 目录下生成了images和labels文件夹：

images文件夹下有train和val文件夹，分别放置训练集和验证集图片；

labels文件夹有train和val文件夹，分别放置训练集和验证集标签（yo1o 格式）

（2）修改训练参数配置

1.新建文件 data/VOC_qq.yaml

可以复制 data/VOC.yaml 再根据自己情况的修改；可以重新命名如： data/VOC_qq.yaml

修改配置：

```
# Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list:
[path/to/imgs1, path/to/imgs2, ..]
path: ./VOCdevkit
train: # train images (relative to 'path') 16551 images
  - images/train

val: # val images (relative to 'path') 4952 images
  - images/val
test: # test images (optional)

# Classes
names:
  0: qqicon
  1: username
  2: password
  3: loginbtn
```

2.新建文件 models/yo1ov5s_qq.yaml

可以复制 models/yo1ov5s.yaml 再根据自己情况的修改；可以重新命名如： models/yo1ov5s_qq.yaml

修改配置：

```
nc: 4 # number of classes
```

（3）图像训练

命令：

```
conda activate pytorch1.9 #激活创建的环境
```

```
python train.py --data data/VOC_qq.yaml --cfg models/yo1ov5s_qq.yaml --weights weights/yo1ov5s.pt
--batch-size 6 --epochs 100
```

常用参数介绍：

- `--weights`: 指的是训练好的网络模型，用来初始化网络权重
- `--cfg`: 为configuration的缩写，指的是网络结构，一般对应models文件夹下的 `xxx.yaml` 文件
- `--data`: 训练数据路径，一般为data文件夹下的 `xxx.yaml` 文件
- `--hyp`: 训练网络的一些超参数设置，(一般用不到)
- `--epochs`: 设置训练的轮数
- `--batch-size`: 每次输出给神经网络的图片数
- `--img-size`: 用于分别设置训练集和测试集的大小。两个数字前者为训练集大小，后者为测试集
- `--device`: 训练网络的设备 `cpu` 还是 `gpu`

常见报错: `OSError: [WinError 1455] 页面文件太小，无法完成操作`的问题

这是由于训练时电脑的虚拟内存太小导致，可以修改文件 `yolo5-master\utils\dataloaders.py`:

```
num_workers=0    #大约在145行左右
```

14.7 【AI自动化测试】图像测试

一、学习目标

1.图像测试

二、知识点

1.【图像测试】

命令:

```
conda activate pytorch1.9    #激活创建的环境
```

```
python detect.py --source ./testfiles/qq_icon.png --weights runs/train/exp/weights/last.pt --conf 0.4
```

参数介绍:

- `--weights`: 训练的权重
- `--source`: 测试数据，可以是图片/视频路径，也可以是'0'(电脑自带摄像头),也可以是 `rtsp` 等视频流
- `--output`: 网络预测之后的图片/视频的保存路径
- `--img-size`: 网络输入图片大小
- `--conf-thres`: 置信度阈值 (检测精度，作者是设置的0.25)
- `--iou-thres`: 做 nms 的 iou 阈值 ()
- `--device`: 设置设备
- `--view-img`: 是否展示预测之后的图片/视频，默认False
- `--save-txt`: 是否将预测的框坐标以 `txt` 文件形式保存，默认False
- `--classes`: 设置只保留某一部分类别，形如0或者0 2 3
- `--agnostic-nms`: 进行 nms 是否也去除不同类别之间的框，默认False
- `--augment`: 推理的时候进行多尺度，翻转等操作(TTA)推理
- `--update`: 如果为True，则对所有模型进行 `strip_optimizer` 操作，去除pt文件中的优化器等信息，默认为False

14.8 【AI自动化测试】AI自动化测试实现（一）

一、学习目标

1.部署 `yo1ov5_serve`

2. `yo1ov5_serve` 接口介绍

二、知识点

1. 【部署 `yo1ov5_serve`】

(1) 环境安装

前面的小节中我们已经部署了 `yo1ov5` 的运行环境，由于 `yo1ov5_serve` 是我们用 `flask web` 后端框架实现的服务，我们还需要额外安装 `flask` 环境：

```
pip install flask
```

(2) 启动服务

```
python flask_app.py          # 正常运行
nohup python flask_app.py &   # linux后台运行
```

(3) 关闭服务

```
netstat -tunlp                # 查询进程号
kill -9 5000端口上的进程号    # 杀进程
```

2. 【`yo1ov5_serve` 接口介绍】

`yo1ov5_serve` 我已经提前实现了三个接口，我们将 `yo1ov5_serve` 部署到服务器之后可以远程调用这三个接口实现对图像的识别，并返回识别类型的坐标信息。

(1) 上传权重pt文件接口

该接口上传之前训练好的权重pt文件，用于识别。

- 请求方式： `post`
- 请求URL： `/uploadPt/`

如果服务部署在本地，则完整 `url` 为： <http://127.0.0.1:5000/uploadPt/>

- 请求头： `Content-Type: multipart/form-data`
- 请求体：

```
{
  file: <file>
}
```

- 响应体：

```
{
  "msg": "权重pt文件上传成功！",
  "result": "YES"
}
```

(2) 上传权重yaml文件接口

该接口上传权重yaml文件，yaml文件中记录类别编号和名称，用于识别。

- 请求方式：post
- 请求URL：/uploadId2name/
- 请求头：Content-Type: multipart/form-data
- 请求体：

```
{
  file: <file>
}
```

- 响应体：

```
{
  "msg": "权重yaml文件上传成功！",
  "result": "YES"
}
```

(3) 识别接口

该接口是远程识别接口，返回识别元素的坐标信息及识别信任度。

- 请求方式：post
- 请求URL：/predict/
- 请求头：Content-Type: application/json
- 请求体：

```
{
  "image": "",
  "weights": "yolov5s",
  "device": "cpu"
}
```

- 响应体：

```
{
  "results": [
    {
      "bbox": [
        226,
        15,
        380,
        228
      ]
    }
  ]
}
```

```
    ],
    "conf": "0.28627973794937134",
    "name": "person"
  },
  {
    "bbox": [
      134,
      175,
      335,
      313
    ],
    "conf": "0.33037129044532776",
    "name": "dog"
  },
  {
    "bbox": [
      355,
      32,
      594,
      314
    ],
    "conf": "0.4839382469654083",
    "name": "person"
  },
  {
    "bbox": [
      341,
      183,
      592,
      323
    ],
    "conf": "0.6084736585617065",
    "name": "dog"
  },
  {
    "bbox": [
      25,
      31,
      318,
      329
    ],
    "conf": "0.6486324071884155",
    "name": "person"
  }
]
```

14.9 【AI自动化测试】AI自动化测试实现（二）

一、学习目标

1.ai 自动化基类介绍

二、知识点

1.【ai 自动化基类介绍】

准备：先读取 配置文件.ini 中的AI自动化配置信息：

```
class YOLOv5GuiBase(GuiBase):

    def __init__(self):
        super(YOLOv5GuiBase, self).__init__()
        self.ai_server = self.config['AI自动化配置']['ai_server']           # yolov5远程
识别地址
        self.ai_confidence = float(self.config['AI自动化配置']['ai_confidence']) # 识别信任度
        self.device = self.config['AI自动化配置']['device']                 # cpu or gpu
        self.ai_minSearchTime = float(self.config['AI自动化配置']['minSearchTime']) # 识别超时时间
```

实现 ai 自动化基类，我们直接继承 GuiBase 客户端自动化基类，因为我们需要用到该类中已经封装好的鼠标键盘的模拟操作。

- 获取元素识别结果

```
@staticmethod
def _current_screen_base64():
    """图片转base64格式"""
    try:
        buffer = io.BytesIO()
        img = ImageGrab.grab()
        img.save(buffer, format='PNG')
        img.close()
        img_bytes = base64.b64encode(buffer.getvalue())
        img_base64 = 'Data:image/png;base64,%s' % img_bytes.decode()
        return img_base64
    except Exception as e:
        logger.error("截取当前屏幕图片并转base64失败: {}".format(e))

def yolov5_serve_post(self, weight, timeout=None):
    """获取识别结果"""
    el_list = None
    start = time.time()
    if not timeout:
        timeout = self.ai_minSearchTime
    try:
        while True:
            pic_base64 = self._current_screen_base64()
            header = {
                'Content-Type': 'application/json'
            }
            json = {
                "image": pic_base64,
```

```

        "weights": weight,
        "device": self.device
    }
    res = requests.post(url=self.ai_server, headers=header, json=json)
    if (res.status_code == 200) and (res.json()['results']):
        el_list = res.json()['results']
        logger.info("访问yolov5服务获取图像识别结果成功: {}".format(el_list))
        break
    if time.time() - start > timeout:
        logger.error("访问yolov5服务获取图像识别结果超时!")
        break
    return el_list
except KeyError as e:
    logger.error('配置文件中缺少配置: {}'.format(e))
except Exception as e:
    logger.error("访问yolov5服务获取图像识别结果失败: {}".format(e))

```

- 判断元素是否存在

```

@staticmethod
def _get_bbox_center(bbox):
    """获取按钮坐标"""
    x = int(bbox[0] + (bbox[2] - bbox[0]) / 2)
    y = int(bbox[1] + (bbox[3] - bbox[1]) / 2)
    return x, y

def yolov5_is_el_exist(self, el, el_list, el_index=0):
    """判断元素是否存在"""
    try:
        el_pos_list = []
        for i in el_list:
            if i['name'] == el:
                if float(i['conf']) >= self.ai_confidence:
                    el_pos_list.append(self._get_bbox_center(i['bbox']))
        if el_pos_list:
            el_pos = el_pos_list[el_index]
            logger.info("元素存在: {},元素坐标: {}".format(el, el_pos))
            return el_pos
        else:
            logger.error("元素不存在: {}".format(el))
            return None
    except Exception as e:
        logger.error('【判断元素是否存在】运行失败: {}'.format(e))

```

- 元素点击方法

```

def yolov5_click_picture(self, el, el_list, clicks=1, button='left', isclick=True,
el_index=0):
    """点击图片方法"""
    try:
        import pyautogui
        pos_x_y = self.yolov5_is_el_exist(el, el_list, int(el_index))
        if not pos_x_y:
            raise Exception("图片识别失败: {}".format(el))

```

```

        pyautogui.moveTo(*pos_x_y)
        if isclick:
            pyautogui.click(*pos_x_y, duration=self.duration, interval=self.interval,
clicks=clicks, button=button)
            logger.info('移动到图片 {} 位置{}, 点击:{} 成功'.format(e1, isclick, pos_x_y))
        except KeyError as e:
            logger.error('配置文件中缺少配置: {}'.format(e))
        except Exception as e:
            logger.error('【yolov5点击图片方法】运行失败: {}'.format(e))
    
```

• 元素相对位置点击方法

```

def yolov5_rel_picture_click(self, e1, el_list, rel_x=0, rel_y=0, clicks=1,
button='left',
                                isclick=True, el_index=0):
    """图像的相对位置点击"""
    try:
        import pyautogui
        pos_x_y = self.yolov5_is_el_exist(e1, el_list, el_index)
        if not pos_x_y:
            raise Exception("图片识别失败: {}".format(e1))
        pyautogui.moveTo(*pos_x_y, duration=self.duration) # 移动到 (100,100)
        pyautogui.move(rel_x, rel_y, duration=self.duration) # 从当前位置右移100像素
        if isclick:
            pyautogui.click(clicks=clicks, button=button, duration=self.duration)
            logger.info('查找图片{}, 位置{}, 偏移{}, 点击{}, 成功'.format(e1, pos_x_y, (rel_x,
rel_y), isclick))
        except KeyError as e:
            logger.error('配置文件中缺少配置: {}'.format(e))
        except Exception as e:
            logger.error('【yolov5图像的相对位置点击】运行失败: {}'.format(e))
    
```

14.10 【AI自动化测试】AI自动化测试实现（三）

一、学习目标

1. QQ 登录权重文件上传
2. QQ 登录 ai 自动化脚本实现

二、知识点

1. 【QQ 登录权重文件上传】

- yaml 权重信息文件

文件名为训练好的pt文件的文件名前缀。

```
qqlogin:  
0: 'qqicon'  
1: 'username'  
2: 'password'  
3: 'loginbtn'
```

使用postman等接口发送工具，请求接口 /uploadId2name/，实现 yam1 文件上传到 yolov5 远程服务端。

- pt 权重文件

使用postman等接口发送工具，请求接口 /uploadPt/，实现 pt 文件上传到 yolov5 远程服务端。

2. 【QQ 登录 ai 自动化脚本实现】

```
gui = Yolov5GuiBase()  
gui.hotkey('win', 'm')  
time.sleep(2)  
# 第一步：双击QQ图标，打开登录界面  
res = gui.yolov5_serve_post(weight='qqlogin')  
print(res)  
gui.yolov5_click_picture(el='qqicon', el_list=res, clicks=2, el_index=0)  
# 第二步：用户名输入  
time.sleep(3)  
res = gui.yolov5_serve_post(weight='qqlogin')  
print(res)  
gui.yolov5_click_picture(el='username', el_list=res, clicks=1, el_index=0)  
gui.type('')  
# 第三步：输入密码  
gui.yolov5_click_picture(el='password', el_list=res, clicks=1, el_index=0)  
gui.type('')  
# 第四步：点击登录  
gui.yolov5_click_picture(el='loginbtn', el_list=res, clicks=1, el_index=0)  
# 第五步：断言，登录按钮不出现  
time.sleep(3)  
res = gui.yolov5_serve_post(weight='qqlogin')  
result = gui.yolov5_is_el_exist(el='loginbtn', el_list=res, el_index=0)  
assert not result, '【断言】界面还停留在登录！'
```